

Manual de Python

Do primeiro script ao código de produção — a linguagem, a biblioteca padrão e o ecossistema

Vitor Mattos

11/08/2026

Índice

Apresentação	1
Para quem é este manual	1
Como o manual está organizado	2
Seções fixas de cada capítulo	3
Sobre o código deste livro	3
Licença	3
I. Volume 1 — Primeiros Passos com Python	5
1. O que é Python: história, filosofia e o Zen da linguagem	7
1.1. Seis linhas e vinte e quatro	7
1.2. De um projeto de fim de ano a uma linguagem de infraestrutura	8
1.3. A ruptura do Python 3 e a lição que ficou	9
1.4. Quem decide o que entra na linguagem	9
1.5. Python 3.13, a versão de referência deste manual	10
1.6. O Zen do Python	11
1.7. Onde Python é forte e onde não é	13
1.8. Jeito Pythonico	13
1.9. Laboratório	14
1.10. Resumo	17
2. Como o Python executa código: interpretador, bytecode e CPython	19
2.1. O erro que chega antes da primeira linha	19
2.2. As quatro etapas	20
2.3. O bytecode por dentro	21
2.4. <code>__pycache__</code> , o cache que aparece sozinho	24
2.5. CPython e as outras implementações	25
2.6. Jeito Pythonico	26
2.7. Laboratório	27
2.8. Resumo	29
3. Instalando o Python: versões, gerenciadores e o Python do sistema	31
3.1. O Python do sistema não é seu	31
3.2. As três camadas	33
3.3. O ambiente virtual, com precisão	33
3.4. Quando você precisa de outra versão	36
3.5. Windows e macOS em duas páginas	37
3.6. Jeito Pythonico	37
3.7. Laboratório	38
3.8. Resumo	41
4. O REPL, os scripts e o primeiro programa	43
4.1. O REPL: onde se descobre	43
4.2. O script: onde se repete	46

4.3.	<code>if __name__ == "__main__"</code>	47
4.4.	As cinco formas de rodar código Python	49
4.5.	Jeito Pythônico	50
4.6.	Laboratório	51
4.7.	Resumo	54
5.	Ambiente de trabalho: editor, VS Code, Jupyter e o terminal	55
5.1.	Uma regra que resolve a maioria dos problemas	55
5.2.	O terminal	56
5.3.	O editor	56
5.4.	Linters e formatadores: <code>ruff</code>	57
5.5.	O notebook	60
5.6.	Outras opções	62
5.7.	Jeito Pythônico	62
5.8.	Laboratório	63
5.9.	Resumo	66
6.	Sintaxe essencial: indentação, linhas lógicas e comentários	67
6.1.	O bloco é a indentação	67
6.2.	Linha física e linha lógica	69
6.3.	Comando e expressão	71
6.4.	Nomes	72
6.5.	Comentários e documentação	73
6.6.	Uma nota sobre codificação	74
6.7.	Jeito Pythônico	74
6.8.	Laboratório	75
6.9.	Resumo	78
7.	Erros e tracebacks: como ler o que o Python está dizendo	81
7.1.	O traceback do começo ao fim	81
7.2.	As exceções que você vai ver mais	83
7.3.	Exceções encadeadas	85
7.4.	Quando o traceback é gigante	87
7.5.	<code>SyntaxError</code> é diferente de todos os outros	87
7.6.	Jeito Pythônico	88
7.7.	Laboratório	90
7.8.	Resumo	93
II.	Volume 2 — Tipos, Operadores e Expressões	95
8.	Objetos, variáveis e o modelo de referências	97
8.1.	Tudo é objeto, e todo objeto tem três coisas	97
8.2.	O nome é uma etiqueta, não uma caixa	98
8.3.	Contagem de referências	99
8.4.	Mutável e imutável	100
8.5.	<code>is</code> e <code>==</code> não são a mesma pergunta	101
8.6.	Cópia rasa e cópia profunda	103
8.7.	Argumentos de função	105
8.8.	Jeito Pythônico	106
8.9.	Laboratório	107
8.10.	Resumo	110

9. Números inteiros, de ponto flutuante e complexos	113
9.1. <code>int</code> não tem teto	113
9.2. Literais e bases	114
9.3. As duas divisões	115
9.4. Operadores bit a bit	116
9.5. <code>float</code> é IEEE 754 de 64 bits	117
9.6. <code>round</code> e a regra do banqueiro	118
9.7. A torre numérica	119
9.8. Complexos	120
9.9. Jeito Pythônico	120
9.10. Laboratório	122
9.11. Resumo	126
10. Precisão numérica: decimal, fractions e as armadilhas do float	127
10.1. Por que 0,1 não é 0,1	127
10.2. O erro se acumula (e a soma de Python não deixa mais)	128
10.3. Comparar <code>float</code> com tolerância	129
10.4. <code>decimal</code> : aritmética como a de um contador	130
10.5. <code>fractions</code> : exatidão sem arredondamento nenhum	132
10.6. Centavos em <code>int</code> : a resposta que quase sempre serve	133
10.7. Jeito Pythônico	134
10.8. Laboratório	135
10.9. Resumo	140
11. Booleanos, valor-verdade e os operadores lógicos	141
11.1. <code>bool</code> é uma subclasse de <code>int</code>	141
11.2. Todo objeto tem valor-verdade	142
11.3. <code>and</code> e <code>or</code> não devolvem booleanos	144
11.4. Precedência	146
11.5. Comparação encadeada	146
11.6. <code>all</code> e <code>any</code>	147
11.7. Jeito Pythônico	147
11.8. Laboratório	149
11.9. Resumo	153
12. Strings: criação, indexação e fatiamento	155
12.1. Escrevendo uma string	155
12.2. Imutável, como todo <code>str</code>	157
12.3. Índices e fronteiras	157
12.4. O terceiro número: o passo	158
12.5. <code>slice</code> é um objeto	159
12.6. Concatenar, repetir, comparar	160
12.7. Construir texto: a armadilha do <code>+=</code>	161
12.8. Jeito Pythônico	162
12.9. Laboratório	163
12.10. Resumo	167
13. Métodos de string, formatação e f-strings	169
13.1. <code>strip</code> é um conjunto, não um prefixo	169
13.2. Os métodos, por família	170
13.3. Três formas de formatar	173
13.4. A mini-linguagem de formatação	174

13.5. Detalhes de f-string que valem saber	175
13.6. Quando o molde não é código	176
13.7. Jeito Pythônico	177
13.8. Laboratório	178
13.9. Resumo	182
14. Unicode, bytes e codificação de caracteres	185
14.1. Duas coisas diferentes: <code>str</code> e <code>bytes</code>	185
14.2. O sanduíche Unicode	187
14.3. UTF-8 e as outras	187
14.4. Os tratadores de erro	188
14.5. <code>len()</code> conta pontos de código, não o que você vê	189
14.6. Normalização	190
14.7. A codificação padrão, e por que não confiar nela	191
14.8. O BOM	192
14.9. Jeito Pythônico	192
14.10. Laboratório	193
14.11. Resumo	199
 III. Volume 3 — Controle de Fluxo e Funções	 201
15. Condicionais: <code>if</code>, <code>elif</code>, <code>else</code> e a expressão condicional	203
15.1. <code>if</code> , <code>elif</code> , <code>else</code>	204
15.2. O que o <code>if</code> considera verdadeiro	205
15.3. Comparações encadeadas	207
15.4. Cláusulas de guarda em vez de pirâmide	208
15.5. A expressão condicional	209
15.6. O operador morsa em condições	211
15.7. Escada de <code>elif</code> ou tabela?	211
15.8. Jeito Pythônico	212
15.9. Laboratório	213
15.10. Resumo	219
16. Laços: <code>while</code>, <code>for</code> e o protocolo de iteração	221
16.1. <code>while</code> : repetir enquanto uma condição valer	222
16.2. <code>for</code> : percorrer um iterável	223
16.3. O protocolo de iteração	223
16.4. <code>range</code> não é uma lista	226
16.5. <code>zip</code> : percorrer em paralelo	227
16.6. Mudar o que se está percorrendo	227
16.7. Laços aninhados	229
16.8. Jeito Pythônico	229
16.9. Laboratório	230
16.10. Resumo	236
17. <code>break</code>, <code>continue</code>, <code>else</code> em laços e o casamento de padrões	237
17.1. <code>break</code> : abandonar o laço	238
17.2. <code>continue</code> : pular para a próxima volta	238
17.3. <code>else</code> em laço: “terminou sem <code>break</code> ”	239
17.4. <code>match</code> : casar com a forma do dado	241
17.5. Jeito Pythônico	246

17.6. Laboratório	247
17.7. Resumo	253
18. Definindo funções: parâmetros, argumentos e retorno	255
18.1. Anatomia de uma definição	256
18.2. Parâmetro e argumento	257
18.3. <code>return</code>	257
18.4. O que a chamada faz	258
18.5. Como os argumentos chegam	259
18.6. Docstring	261
18.7. Anotações de tipo	261
18.8. Funções que devolvem <code>None</code> de propósito	261
18.9. Recursão, em uma seção curta	262
18.10. Jeito Pythônico	263
18.11. Laboratório	264
18.12. Resumo	270
19. Argumentos nomeados, valores padrão, <code>*args</code> e <code>**kwargs</code>	273
19.1. Valores padrão são avaliados uma vez , na definição	273
19.2. Argumentos por nome	275
19.3. <code>*args</code> : os posicionais que sobraram	276
19.4. <code>**kwargs</code> : os nomeados que sobraram	276
19.5. A ordem canônica dos parâmetros	277
19.6. <code>*</code> e <code>**</code> na chamada	279
19.7. Jeito Pythônico	280
19.8. Laboratório	281
19.9. Resumo	288
20. Escopo, closures e a regra LEGB	289
20.1. LEGB: a ordem da busca	290
20.2. <code>global</code> e <code>nonlocal</code>	291
20.3. Sombrear um nome embutido	292
20.4. Closures	293
20.5. A armadilha da captura tardia	293
20.6. Compreensões têm escopo próprio; laços, não	295
20.7. <code>locals()</code> e <code>globals()</code>	295
20.8. Jeito Pythônico	296
20.9. Laboratório	296
20.10. Resumo	303
21. Funções como objetos: <code>lambda</code>, <code>map</code>, <code>filter</code> e ordem superior	305
21.1. O nome refere, os parênteses chamam	306
21.2. Funções de ordem superior	306
21.3. <code>lambda</code>	308
21.4. <code>map</code> e <code>filter</code>	309
21.5. <code>operator</code> : as funções que você ia escrever	310
21.6. <code>functools.partial</code>	310
21.7. <code>functools.reduce</code> , e por que quase nunca	311
21.8. Um pipeline montado com funções	311
21.9. Jeito Pythônico	313
21.10. Laboratório	314
21.11. Resumo	320

IV. Volume 4 — Estruturas de Dados Nativas	323
22.Listas: operações, mutabilidade e cópia	325
23.Tuplas, empacotamento e desempacotamento	327
24.Dicionários: chaves, ordem e os métodos essenciais	329
25.Conjuntos e operações de conjunto	331
26.Compreensões de lista, dicionário e conjunto	333
27.Ordenação, chaves de ordenação e comparação de objetos	335
28.Escolhendo a estrutura certa: custo das operações	337
 V. Volume 5 — Módulos, Pacotes e o Ambiente	 339
29.Módulos, import e o caminho de busca	341
30.Pacotes, <code>__init__.py</code> e a organização de um projeto	343
31.Ambientes virtuais: venv, pip e o isolamento de dependências	345
32.Empacotamento moderno: pyproject.toml, build e publicação	347
33.uv, pipx e Poetry: o ecossistema de gerenciamento	349
34.Ponto de entrada: <code>__main__</code> , scripts de console e a linha de comando	351
 VI. Volume 6 — Arquivos, Erros e Contextos	 353
35.Leitura e escrita de arquivos de texto	355
36.Arquivos binários, buffers e o módulo io	357
37.Caminhos e o sistema de arquivos com pathlib	359
38.Exceções: hierarquia e o bloco try, except, else e finally	361
39.Levantando exceções e criando exceções próprias	363
40.Gerenciadores de contexto e a instrução with	365
 VII. Volume 7 — Programação Orientada a Objetos	 367
41.Classes, instâncias e o self	369
42.Atributos, propriedades e encapsulamento	371
43.Herança, MRO e a função super	373
44.Composição, delegação e quando não herdar	375

45. Métodos estáticos, métodos de classe e o desenho da API	377
46. Dataclasses, NamedTuple e enumerações	379
47. Classes abstratas, protocolos e duck typing	381
VIII. Volume 8 — O Modelo de Dados de Python	383
48. Métodos especiais: o protocolo por trás da sintaxe	385
49. Iteradores e o protocolo de iteração	387
50. Geradores, yield e a avaliação preguiçosa	389
51. Decoradores de função	391
52. Decoradores de classe, functools e metadados preservados	393
53. Descritores, <code>__slots__</code> e o acesso a atributos	395
54. Metaclasses e <code>__init_subclass__</code> : quando (não) usar	397
IX. Volume 9 — A Biblioteca Padrão Essencial	399
55. <code>os</code> , <code>sys</code> e a interação com o sistema	401
56. <code>datetime</code> , <code>zoneinfo</code> e o tratamento de tempo	403
57. <code>collections</code> : <code>deque</code> , <code>Counter</code> , <code>defaultdict</code> e <code>namedtuple</code>	405
58. <code>itertools</code> e <code>functools</code> : a caixa de ferramentas funcional	407
59. <code>math</code> , <code>statistics</code> e <code>random</code>	409
60. <code>logging</code> : registrando o que o programa faz	411
X. Volume 10 — Texto, Dados e Serialização	413
61. Expressões regulares com o módulo <code>re</code>	415
62. JSON, CSV e formatos tabulares	417
63. YAML, TOML e arquivos de configuração	419
64. Serialização binária: <code>pickle</code> , <code>struct</code> e os riscos envolvidos	421
65. SQLite e a DB-API: banco de dados sem servidor	423
66. <code>hashlib</code> , <code>secrets</code> e o tratamento de dados sensíveis	425

XI. Volume 11 — Qualidade: Testes, Tipagem e Ferramentas	427
67. Estilo, PEP 8 e formatação automática com Ruff e Black	429
68. Anotações de tipo: fundamentos e o módulo typing	431
69. Tipagem avançada: genéricos, protocolos e mypy	433
70. Testes com pytest: fundamentos, asserções e fixtures	435
71. Parametrização, dublês de teste e cobertura	437
72. Depuração: pdb, breakpoint e o traceback difícil	439
73. Documentação: docstrings, doctest e Sphinx	441
 XII. Volume 12 — Automação, Sistema e Redes	 443
74. Scripts de automação: arquivos, planilhas e tarefas repetitivas	445
75. subprocess: chamando programas externos com segurança	447
76. Interfaces de linha de comando com argparse, Click e Typer	449
77. Redes com sockets: TCP, UDP e o modelo cliente-servidor	451
78. Requisições HTTP com requests e httpx	453
79. Raspagem de dados responsável: HTML, seletores e limites éticos	455
 XIII. Volume 13 — Dados, Análise e Visualização	 457
80. NumPy: arrays, vetorização e broadcasting	459
81. pandas: Series, DataFrame e a carga de dados	461
82. Limpeza, transformação e agregação com pandas	463
83. Visualização de dados com Matplotlib	465
84. SciPy e a computação científica aplicada	467
85. Introdução ao aprendizado de máquina com scikit-learn	469
 XIV. Volume 14 — Web, APIs e Bancos de Dados	 471
86. Como funciona uma aplicação web: HTTP, WSGI e ASGI	473
87. Validação de dados com Pydantic	475
88. APIs REST com FastAPI	477
89. Flask e Django: quando usar cada um	479

90. Bancos de dados relacionais com SQLAlchemy	481
91. Autenticação, autorização e segurança em aplicações Python	483
92. Implantação: contêineres, servidores de aplicação e produção	485
 XV. Volume 15 — Concorrência, Desempenho e Internos	 487
93. Concorrência, paralelismo e o GIL	489
94. Threads e o módulo threading	491
95. Multiprocessing e o paralelismo real	493
96. Programação assíncrona: async, await e asyncio	495
97. Medindo desempenho: timeit, profiling e otimização	497
98. Memória, contagem de referências e o coletor de lixo	499
 XVI. Volume 16 — Fronteira e Prática Contínua	 501
99. Estendendo Python: C, Cython e ligações nativas	503
100. Python acelerado: PyPy, Numba e o interpretador sem GIL	505
101. Arquitetura de projetos e código sustentável	507
102. Git, pre-commit e integração contínua para projetos Python	509
103. Python e inteligência artificial: LLMs, APIs e agentes	511
104. Para onde ir depois: PEPs, comunidade e prática contínua	513
Referências	515

Apresentação

Este é um manual de **Python** escrito em português do Brasil, pensado para levar o leitor do primeiro `print` até o código que roda em produção — passando pela linguagem a fundo, pela biblioteca padrão, pelas ferramentas que separam script de software e pelos domínios em que Python se tornou padrão de fato.

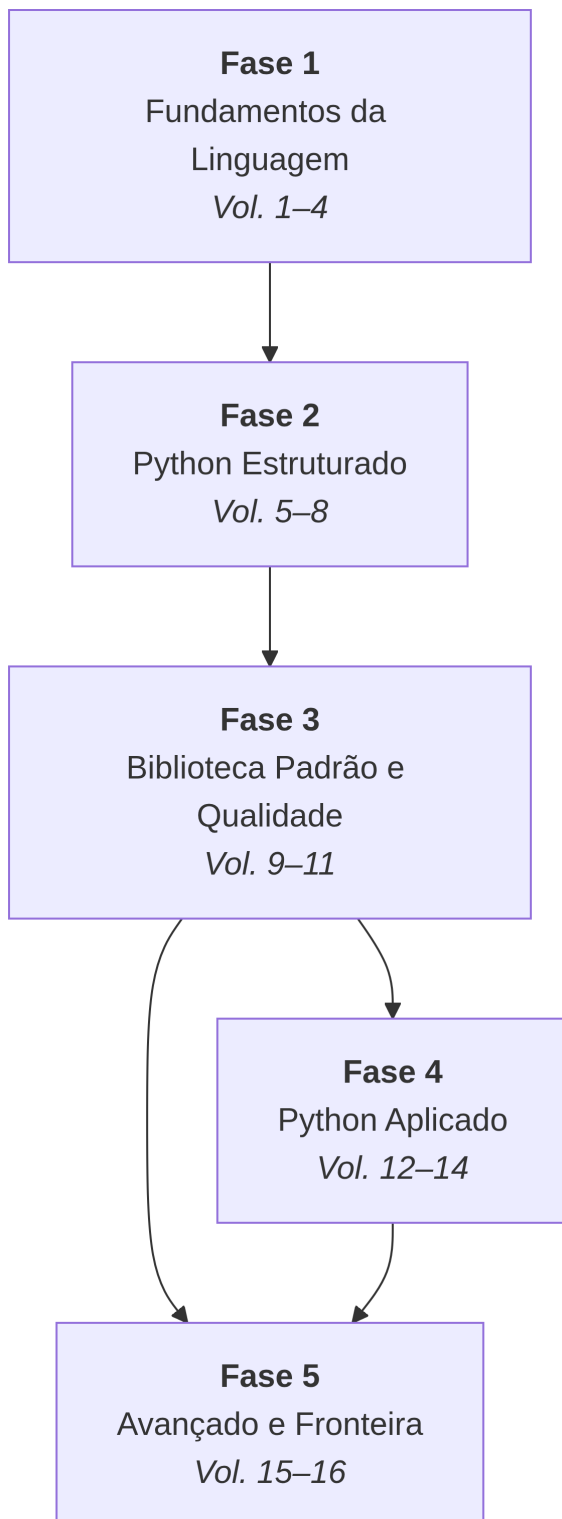
A versão de referência é o **Python 3.13**. O código é moderno: `pathlib` no lugar de `os.path`, f-strings no lugar do operador `%`, `list[int]` no lugar de `typing.List[int]`. Onde um recurso é recente, o texto diz a partir de qual versão ele existe; onde o código antigo ainda é comum em produção, explica o que se fazia antes e por que mudou. O ambiente de referência é Linux, com notas para Windows e macOS onde a diferença importa.

Para quem é este manual

Para quem quer aprender Python a sério e não parar no básico. Serve a quem nunca programou — o Volume 1 começa do zero, sem pressupor outra linguagem —, a quem já programa em outra linguagem e quer entender o que é específico de Python em vez de escrever Java com sintaxe de Python, e a quem já usa Python há anos de forma intuitiva e quer fechar as lacunas: descritores, o modelo de dados, o GIL, empacotamento, tipagem.

Não é preciso conhecimento prévio de programação. É preciso, sim, um interpretador aberto ao lado enquanto se lê.

Como o manual está organizado



Dependências entre as fases do manual

São **104 capítulos** distribuídos em **16 volumes** e **5 fases**, escritos para serem lidos em sequência por quem está começando, e consultados por assunto por quem já atua na área.

A **Fase 1** cobre a linguagem no nível da expressão e da função: tipos, operadores, controle de

fluxo e as estruturas de dados nativas. A **Fase 2** sobe um nível e trata da organização do código — módulos, pacotes, ambientes, exceções, orientação a objetos e o modelo de dados que faz a sintaxe de Python funcionar como funciona. A **Fase 3** é sobre não ficar sozinho: a biblioteca padrão e as ferramentas de qualidade (testes, tipagem, formatação, documentação). A **Fase 4** aplica tudo isso a quatro domínios concretos — automação e redes, análise de dados, web e bancos de dados. A **Fase 5** vai ao que está embaixo: concorrência, o GIL, memória, desempenho, extensão em C e para onde seguir depois.

As fases 4 e 5 são independentes entre si: quem quer o caminho aplicado pode ir direto da 3 para a 4, e quem quer entender os internos pode pular para a 5, aceitando algumas referências para frente.

Seções fixas de cada capítulo

Todo capítulo traz duas seções obrigatórias ao final:

i **Jeito Pythônico** — a forma idiomática de fazer o que o capítulo ensinou: o que a comunidade escreve de verdade, qual PEP fundamenta a escolha, qual é o antipadrão equivalente e por que ele machuca.

💡 **Laboratório** — roteiro prático reproduzível: passos numerados que você executa no seu próprio ambiente, com o resultado esperado de cada um. Termina em algo que funciona.

Sobre o código deste livro

! Todo exemplo roda

Os exemplos foram executados num ambiente virtual limpo antes de entrar no texto, e a saída mostrada é a saída real do interpretador — inclusive nos exemplos de erro, onde o traceback vai completo, porque ler traceback é parte de saber Python.

Onde a saída varia entre execuções (endereço de objeto, ordem de um `set`, data e hora, caminho absoluto), o texto avisa. Onde o exemplo é do que **não** fazer, isso está dito antes do bloco, não depois.

Código que apaga arquivo, executa comando externo ou desserializa dado de origem desconhecida vem sempre precedido do aviso e, quando possível, de um ensaio seguro em diretório descartável.

Licença

Conteúdo sob licença [CC BY-SA 4.0](#). Código e exemplos sob licença MIT. Contribuições, correções e sugestões são bem-vindas via issues no repositório.

Bom proveito — e mantenha um REPL aberto ao lado enquanto lê.

Parte I.

**Volume 1 — Primeiros Passos com
Python**

1. O que é Python: história, filosofia e o Zen da linguagem

Segunda-feira, oito da manhã. O suporte quer saber quais clientes caíram mais vezes durante o fim de semana, porque três deles ligaram reclamando e ninguém sabe se o problema é da rede ou do roteador da casa. O arquivo com os eventos de conexão está ali, com uma linha por evento, e tem alguns milhares delas. Abrir isso numa planilha é possível, mas você vai fazer de novo na semana que vem, e na outra.

Com Python, a resposta cabe em seis linhas:

```
from collections import Counter
from pathlib import Path

linhas = Path("conexoes.log").read_text(encoding="utf-8").splitlines()
quedas = Counter(l.split()[2] for l in linhas if l.endswith("DOWN"))

for cliente, total in quedas.most_common(3):
    print(f"{cliente:<20} {total} quedas")
```

```
sitio-mangueira      4 quedas
fazenda-boa-vista    3 quedas
chacara-do-ze        2 quedas
```

Este capítulo é sobre a linguagem que permite escrever essas seis linhas — de onde ela veio, quem decide como ela cresce e qual é o conjunto de valores que faz um programa Python parecer com esse e não com outra coisa.

1.1. Seis linhas e vinte e quatro

O trecho acima não é curto por esperteza. Ele é curto porque a linguagem já tem palavras para as três coisas que o problema pede: *ler um arquivo de texto* (`Path.read_text`), *contar ocorrências* (`Counter`) e *pegar as maiores* (`most_common`). Escrito como quem transliterou o programa de outra linguagem, o mesmo resultado sai assim:

```
arquivo = open("conexoes.log")
linhas = arquivo.readlines()
arquivo.close()

quedas = {}
i = 0
while i < len(linhas):
    linha = linhas[i]
```

1. O que é Python: história, filosofia e o Zen da linguagem

```
partes = linha.split()
if len(partes) == 4 and partes[3] == "DOWN":
    cliente = partes[2]
    if cliente in quedas:
        quedas[cliente] = quedas[cliente] + 1
    else:
        quedas[cliente] = 1
i = i + 1

pares = []
for cliente in quedas:
    pares.append((quedas[cliente], cliente))
pares.sort()
pares.reverse()

j = 0
while j < 3 and j < len(pares):
    print(pares[j][1] + " " * (21 - len(pares[j][1])) + str(pares[j][0]) + "
↪ quedas")
    j = j + 1
```

São 24 linhas de código contra 6, e a saída é byte a byte a mesma. As duas versões rodam; a segunda apenas gasta o leitor. Ela também esconde três bugs em potencial que a primeira nem tem chance de cometer: o arquivo que fica aberto se algo estourar no meio, o índice `i` que alguém esquece de incrementar num `if`, e o 21 mágico da formatação manual.

A diferença entre as duas não é uma questão de gosto. É a consequência prática de uma decisão de projeto tomada em 1989 e defendida com teimosia desde então: **o código é lido muito mais vezes do que é escrito**, e a linguagem deve otimizar para a leitura.

1.2. De um projeto de fim de ano a uma linguagem de infraestrutura

Em dezembro de 1989, Guido van Rossum trabalhava no CWI — o Centrum voor Wiskunde en Informatica, o instituto nacional holandês de matemática e computação, em Amsterdã. A equipe dele mantinha o Amoeba, um sistema operacional distribuído, e enfrentava um problema chato: escrever utilitários de administração em C era lento demais para o tamanho da tarefa, e o shell não dava conta de nada que exigisse estrutura de dados de verdade.

Van Rossum tinha acabado de sair de outro projeto do mesmo instituto: a linguagem ABC, pensada para ensinar programação a não programadores. O ABC acertou em coisas que ninguém mais fazia — blocos delimitados por indentação, tipos de alto nível embutidos, sintaxe legível em voz alta — e errou em outras, de forma fatal: era monolítica, praticamente impossível de estender, e não conversava com o sistema operacional. Você não conseguia abrir um arquivo do jeito que o sistema queria, nem chamar uma biblioteca em C.

O projeto que ele começou naquele fim de ano era, em essência, o ABC com as arestas corrigidas: a mesma legibilidade, mas extensível, capaz de chamar código C e de falar com o sistema. O nome não veio da cobra. Van Rossum era fã do grupo de humor britânico Monty Python, e “Python” entrou como título provisório de um arquivo. Nunca foi trocado.

A primeira versão pública, a 0.9.0, foi postada em fevereiro de 1991 no grupo `alt.sources` da Usenet — já com classes, herança, tratamento de exceções, funções e os tipos `list` e `dict`. A 1.0 saiu em janeiro de 1994. A 2.0, em outubro de 2000, trouxe as compreensões de lista e um coletor de lixo capaz de resolver ciclos de referência. Em 2001 foi criada a Python Software Foundation, que passou a deter a propriedade intelectual do projeto — uma decisão burocrática e sem glamour que provavelmente é uma das razões de a linguagem ainda existir. A Figura 1.1 situa esses marcos.

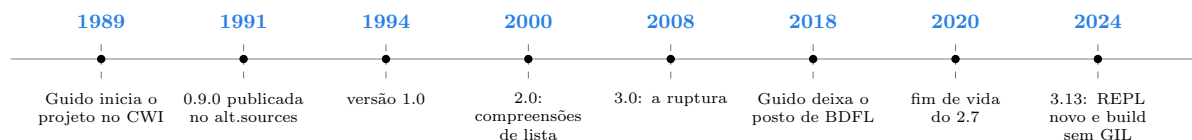


Figura 1.1.: Trinta e cinco anos de Python em oito marcos.

1.3. A ruptura do Python 3 e a lição que ficou

Em dezembro de 2008 saiu o Python 3.0, e ele quebrou compatibilidade de propósito. Foi a decisão mais cara da história do projeto e a mais defensável.

O motivo central era texto. No Python 2, o tipo `str` era uma sequência de bytes que fingia ser texto, e o texto de verdade morava num tipo separado, `unicode`. Enquanto todo mundo escrevia em inglês ASCII, dava para não notar. Assim que entrava um acento — um nome de cliente como “Fazenda Boa Vista” vindo de um sistema em ISO-8859-1, ou um log gerado por um equipamento que fala UTF-8 — o programa passava a misturar bytes e texto sem avisar, e o resultado eram aqueles caracteres embaralhados que todo mundo já viu, ou uma exceção estourando em produção três meses depois, na única linha em que o acento aparecia.

O Python 3 separou os dois de forma inegociável: `str` é texto, `bytes` é byte, e a conversão entre eles exige que você diga qual é a codificação. É por isso que o exemplo da abertura escreve `encoding="utf-8"` em vez de deixar o padrão decidir. Parece rigor excessivo até o dia em que salva o seu relatório.

A travessia foi longa e dolorosa. Por mais de uma década existiu código de produção nas duas versões, bibliotecas que só funcionavam numa delas e distribuições Linux que traziam `python` apontando para o 2.7 por medo de quebrar scripts de sistema. O suporte ao Python 2.7 terminou em 1º de janeiro de 2020, onze anos depois do lançamento do 3.0.

Para quem começa hoje, isso é história — mas história com uma consequência prática: **ao procurar solução na internet, você ainda vai esbarrar em respostas escritas para o Python 2**. O sinal mais rápido é o `print` sem parênteses: `print "olá"` é Python 2 e é erro de sintaxe hoje. Nas distribuições atuais, incluindo o Kali, o comando `python` já aponta para o Python 3, e `python2` simplesmente não vem instalado.

1.4. Quem decide o que entra na linguagem

Mudanças no Python não acontecem por decisão isolada de quem tem acesso ao repositório. Elas passam por uma PEP — *Python Enhancement Proposal*, proposta de melhoria do Python —, um documento numerado que descreve a mudança, a motivação, a especificação técnica, as

1. O que é Python: história, filosofia e o Zen da linguagem

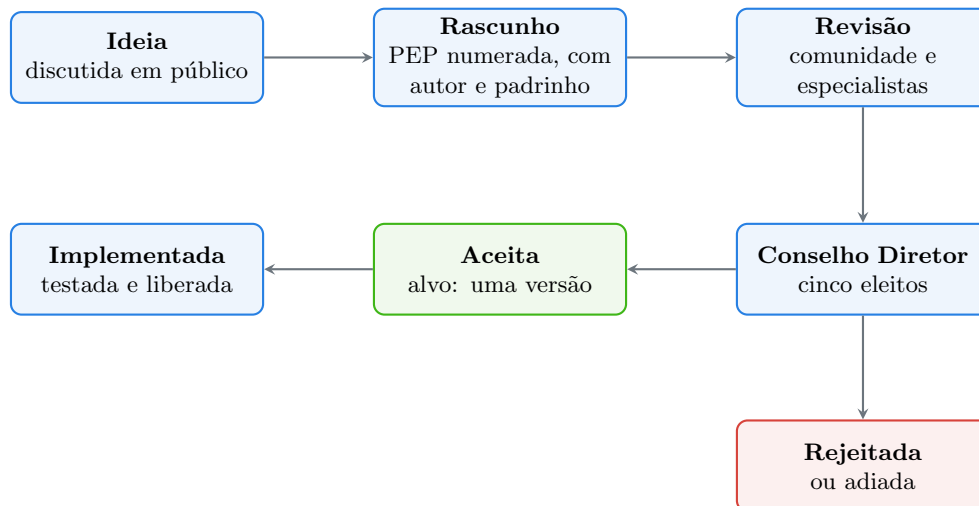


Figura 1.2.: O caminho de uma mudança na linguagem, da ideia à liberação.

alternativas consideradas e o impacto sobre código existente. O processo é definido pela PEP 1, e o fluxo está em Figura 1.2.

Por quase trinta anos, a palavra final era de Guido van Rossum, com o título semi-brincalhão de BDFL — *Benevolent Dictator For Life*, ditador benevolente vitalício. O arranjo acabou em julho de 2018. A PEP 572, que introduziu o operador morsa (`:=`), gerou uma discussão longa e áspera; Van Rossum aprovou a proposta e, dias depois, anunciou que estava se afastando do papel de árbitro. A comunidade respondeu com a PEP 13, que criou o Conselho Diretor: cinco pessoas eleitas anualmente pelos desenvolvedores do núcleo, com mandato coletivo sobre as decisões técnicas.

Vale registrar que o operador morsa, motivo de tanto barulho, hoje é usado sem drama — e que Van Rossum continuou contribuindo com o projeto depois de sair do posto. A instituição sobreviveu à saída do fundador, que é exatamente o que se espera de uma linguagem em que as pessoas vão apostar a infraestrutura delas.

A PEP 602 estabeleceu o ritmo atual de lançamentos: uma versão nova por ano, sempre em outubro. Cada versão recebe cerca de dois anos de correções de bugs e mais três de correções apenas de segurança — cinco anos de vida útil, previsíveis de antemão. Isso permite planejar migração em vez de reagir a ela.

1.5. Python 3.13, a versão de referência deste manual

Este manual usa o **Python 3.13**, lançado em outubro de 2024. É a versão em que todos os exemplos foram executados antes de a saída ser colada aqui. Especificamente:

```
$ python --version
Python 3.13.15
```

O que o 3.13 trouxe de mais visível:

- **Um REPL novo.** O modo interativo ganhou histórico multilinha de verdade, colagem de blocos inteiros sem bagunçar a indentação, cores na saída de erro e comandos como

`help`, `exit` e `clear` funcionando sem parênteses. Quem usava o IPython só por causa disso talvez não precise mais.

- **Mensagens de erro melhores.** A linha do traceback agora aponta com `~` e `^` exatamente qual pedaço da expressão falhou, e não apenas em qual linha.
- **Um build experimental sem GIL** (PEP 703), instalado à parte como `python3.13t`, em que threads de Python realmente rodam em paralelo. É o começo de um trabalho de vários anos; nas versões seguintes ele deixou de ser experimental. O assunto tem um volume inteiro reservado neste manual.
- **Um compilador JIT experimental** (PEP 744), desligado por padrão.
- **A remoção das “pilhas mortas”** (PEP 594): dezenove módulos obsoletos da biblioteca padrão saíram de vez, entre eles `telnetlib`, `cgi` e `crypt`.

Nada disso muda a linguagem que você vai aprender nos próximos capítulos. Muda o conforto de usá-la.

1.6. O Zen do Python

Em 1999, Tim Peters — um dos desenvolvedores mais influentes do núcleo, autor do algoritmo de ordenação que o Python usa até hoje — resumiu numa lista de aforismos os critérios que a comunidade usava para decidir se uma proposta “parecia Python”. A lista virou a PEP 20 e está dentro do interpretador. Digite `import this` em qualquer instalação:

```
import this
```

The Zen of Python, by Tim Peters

```
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

São 19 aforismos. A PEP 20 diz que são 20, e que o vigésimo ficou reservado para Guido preencher — ele nunca preencheu, e a piada permaneceu no documento.

Quatro deles mudam código no dia a dia com mais frequência que os outros.

1. O que é Python: história, filosofia e o Zen da linguagem

“Legibilidade conta.” É a régua para desempatar qualquer discussão de estilo. Se uma linha economiza três caracteres e custa dez segundos de leitura, ela perdeu.

“Explícito é melhor que implícito.” É a razão de o `self` aparecer escrito na assinatura dos métodos, de o `encoding` ser recomendado ao abrir um arquivo de texto e de não existir conversão automática entre número e texto: `"3" + 4` é um erro, não um palpite. Linguagens que adivinham economizam digitação e cobram em depuração.

“Erros nunca devem passar em silêncio — a menos que silenciados explicitamente.” Quando algo dá errado, o Python para e diz onde. Considere um script que lê níveis de sinal óptico e encontra um campo estragado no meio do arquivo:

```
leituras = ["-18.4", "-21.7", "sem leitura", "-19.2"]

for leitura in leituras:
    print(float(leitura))

-18.4
-21.7
Traceback (most recent call last):
  File "C:\Users\Usuario\lab01\sinal.py", line 4, in <module>
    print(float(leitura))
    ~~~~~^~~~~~
ValueError: could not convert string to float: 'sem leitura'
```

O caminho da primeira linha é o do arquivo na máquina onde o exemplo rodou — no seu, vai aparecer o seu. O que importa é o resto: a linha exata, a expressão exata marcada por `~~~~~^~~~~~`, o tipo do erro e o valor que causou o problema. Nenhum NaN silencioso, nenhum zero inventado no lugar do dado ruim. O programa parou porque continuar seria pior — e é você quem decide o que fazer com o campo estragado, não o interpretador.

“Deve haver uma — e de preferência só uma — maneira óbvia de fazer.” É o aforismo mais citado e o mais mal-entendido. Não quer dizer que exista uma única forma possível; quer dizer que, entre as formas possíveis, a comunidade converge para uma, e essa uma vira a que todo mundo reconhece de imediato. Na prática é isso que permite ler código Python de um projeto que você nunca viu e entender o que ele faz.

O contrapeso está logo acima: **“embora a praticidade vença a pureza”**. O Python aceita romper a regra quando o custo de mantê-la é alto demais. O próprio módulo `this` é a prova:

```
import this

print(this.c, this.i, len(this.d))
```

97 25 52

O texto do Zen está guardado em `this.s` cifrado em ROT13, e o módulo monta a tabela de decifração com dois laços aninhados — deixando as variáveis `c` e `i` vazando no espaço de nomes do módulo. É um deslize que qualquer revisor apontaria, dentro da biblioteca padrão, no arquivo que descreve as boas práticas. Está lá há mais de vinte anos porque não faz mal a ninguém e a piada é boa. Praticidade venceu a pureza.

Já a rigidez sobre a sintaxe, essa é levada a sério, com humor:


```
from __future__ import braces
```

```
File "<string>", line 1
SyntaxError: not a chance
```

Chaves para delimitar blocos, o pedido mais repetido por quem vem de C e de Java, tem uma resposta oficial no interpretador: *nem a pau*.

1.7. Onde Python é forte e onde não é

Ser honesto sobre os limites poupa frustração. O Python é lento em laço numérico puro comparado a C, Rust ou Go — uma ordem de grandeza ou mais. O bloqueio global do interpretador, o GIL, historicamente impediu que threads de Python usassem vários núcleos ao mesmo tempo para trabalho de CPU. Distribuir um programa como um único executável dá trabalho, e a linguagem não é uma escolha natural para aplicativo de celular nem para código embarcado com poucos kilobytes de memória.

Em compensação, ela é imbatível em três situações que cobrem boa parte do trabalho real: **colar sistemas que não foram feitos para conversar** (um log aqui, uma API ali, um banco acolá), **explorar dados** e **automatizar o que hoje é feito na mão**. Nesses casos o gargalo não é o processador — é o disco, a rede ou o tempo de quem escreve o programa. E as partes que precisam de velocidade quase sempre já existem escritas em C por baixo de uma interface Python: é assim que funcionam NumPy, pandas e praticamente todo o ecossistema científico.

Uma distinção que vale desde já: **Python é a linguagem; CPython é o programa que a executa**. O CPython, escrito em C, é a implementação de referência — é ele que você instala e é sobre ele que este manual fala. Existem outras: o PyPy, com compilação em tempo de execução e desempenho muito superior em código de laço; o MicroPython, para microcontroladores; o GraalPy, sobre a máquina virtual da JVM. Todas implementam a mesma linguagem, com graus diferentes de compatibilidade com as extensões em C.

1.8. Jeito Pythônico

“Pythônico” é o adjetivo que a comunidade usa para o código que resolve o problema com o vocabulário da linguagem, e não com o vocabulário de outra linguagem escrito com sintaxe de Python. Ele tem duas fontes escritas: a **PEP 20**, que dá os valores, e a **PEP 8**, o guia de estilo, que dá as convenções concretas — quatro espaços de indentação, **snake_case** para funções e variáveis, **CamelCase** para classes, linhas curtas.

O antipadrão mais comum de quem chega de outra linguagem é percorrer uma sequência pelo índice:

```
# antipadrão: índice manual, herdado de C
i = 0
while i < len(linhas):
    processar(linhas[i])
    i = i + 1
```

1. O que é Python: história, filosofia e o Zen da linguagem

```
# pythônico: iterar sobre o objeto
for linha in linhas:
    processar(linha)
```

A segunda versão não é só mais curta. Ela não tem como errar o limite do laço, funciona igual para lista, arquivo aberto, conjunto ou gerador — e comunica a intenção em vez do mecanismo. Quando o índice é realmente necessário, existe a forma canônica para isso:

```
for numero, linha in enumerate(linhas, start=1):
    if linha.endswith("DOWN"):
        print(f"linha {numero}: {linha}")
```

O mesmo raciocínio vale para “acumular contagem num dicionário” (é `collections.Counter`), “juntar textos com separador” (é `"; ".join(partes)`, não concatenação em laço) e “montar texto com valores dentro” (é a f-string, `f"{cliente}: {total}"`).

Sobre versões, quatro marcos que aparecem em código moderno e que não existiam antes — vale saber a partir de quando cada um pode ser usado:

- **Python 3.8** — o operador morsa, que atribui dentro de uma expressão: `if (n := len(fila)) > 10:`.
- **Python 3.10** — o casamento de padrões, com `match` e `case`, e a barra vertical como união de tipos na anotação, no lugar de `typing.Optional` e `typing.Union`.
- **Python 3.11** — `tomllib` na biblioteca padrão, para ler arquivos TOML sem instalar nada.
- **Python 3.12** — a sintaxe nova de genéricos e de alias de tipo, com `type` e colchetes na assinatura.

Se o seu código precisa rodar num servidor com Python 3.9, `match` está fora — e é para isso que a versão mínima serve. Cada um desses recursos tem seu capítulo mais adiante.

Por fim, uma regra que resume o resto: **escreva para quem vai ler o código daqui a seis meses, que provavelmente é você e não vai lembrar de nada**. Um nome de variável de dez letras que diz o que a variável é vale mais do que um comentário explicando um nome de uma letra.

1.9. Laboratório

Objetivo: montar um ambiente isolado com Python 3.13, confirmar a versão, explorar o Zen por dentro e rodar o script da abertura com dados próprios. Tudo com a biblioteca padrão — nada a instalar.

1. Crie o diretório e o ambiente virtual. No Linux (Kali, Debian, Ubuntu):

```
mkdir -p ~/lab01 && cd ~/lab01
python3 -m venv .venv
source .venv/bin/activate
```

No Windows, no PowerShell, as duas últimas linhas viram:

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

O prompt passa a exibir (.venv) na frente. A partir daqui, python é o do ambiente.

2. Confirme a versão. Esperado: 3.13 alguma coisa. O número após o segundo ponto varia conforme a data.

```
python --version
python -c "import sys; print(sys.version_info)"
```

```
Python 3.13.15
sys.version_info(major=3, minor=13, micro=15, releaselevel='final', serial=0)
```

Se aparecer 3.9 ou 2.7, o ambiente não foi ativado ou a versão instalada é outra. O capítulo sobre instalação, mais adiante no Volume 1, trata dos gerenciadores de versão.

3. Leia o Zen. Ainda no terminal:

```
python -c "import this"
```

A saída é a lista de 19 aforismos mostrada neste capítulo.

4. Olhe o Zen por dentro. Crie o arquivo zen.py:

```
import this

print(this.c, this.i, len(this.d))
print(this.s.splitlines()[13])
print("".join(this.d.get(ch, ch) for ch in this.s.splitlines()[13]))
```

Rode com `python zen.py`. Depois das 19 linhas impressas pelo `import`, sai isto:

```
97 25 52
Va gur snpr bs nzovthvgl, ershfr gur grzcgngvba gb thrff.
In the face of ambiguity, refuse the temptation to guess.
```

Três coisas para notar. As variáveis `c` e `i` sobreviveram aos laços do módulo — é o vazamento comentado acima. O dicionário `d` tem 52 entradas: 26 letras maiúsculas mais 26 minúsculas. E a última linha decifra o aforismo aplicando a mesma tabela: ROT13 é a sua própria inversa.

5. Gere um log de exemplo. Crie `conexoes.log` com este conteúdo — são pares de queda e retorno de quatro clientes ao longo de um dia:

1. O que é Python: história, filosofia e o Zen da linguagem

```
2026-03-11 03:14:02 sitio-mangueira DOWN
2026-03-11 03:14:41 sitio-mangueira UP
2026-03-11 05:02:19 fazenda-boavista DOWN
2026-03-11 05:03:07 fazenda-boavista UP
2026-03-11 06:47:55 sitio-mangueira DOWN
2026-03-11 06:48:30 sitio-mangueira UP
2026-03-11 09:21:10 chacara-do-ze DOWN
2026-03-11 09:22:02 chacara-do-ze UP
2026-03-11 11:05:44 sitio-mangueira DOWN
2026-03-11 11:06:21 sitio-mangueira UP
2026-03-11 12:44:38 fazenda-boavista DOWN
2026-03-11 12:45:26 fazenda-boavista UP
2026-03-11 14:38:09 chacara-do-ze DOWN
2026-03-11 14:39:00 chacara-do-ze UP
2026-03-11 17:12:47 fazenda-boavista DOWN
2026-03-11 17:13:35 fazenda-boavista UP
2026-03-11 18:52:31 sitio-mangueira DOWN
2026-03-11 18:53:12 sitio-mangueira UP
2026-03-11 21:30:05 vila-sao-joao DOWN
2026-03-11 21:30:52 vila-sao-joao UP
```

6. Rode o script da abertura. Salve como `quedas.py` o trecho de seis linhas do começo do capítulo e execute:

```
python quedas.py
```

```
sitio-mangueira      4 quedas
fazenda-boavista     3 quedas
chacara-do-ze        2 quedas
```

7. Provoque um erro de propósito. Crie `sinal.py` com o exemplo dos níveis de sinal e rode. Você deve ver o `ValueError` com o traceback apontando a linha 4 e a expressão `float(leitura)`. Leia a última linha do traceback antes de qualquer outra coisa: é ela que diz o que aconteceu. Ler traceback é a habilidade que mais economiza tempo em Python, e tem um capítulo só para isso ainda no Volume 1.

8. Peça as chaves. Para fechar:

```
python -c "from __future__ import braces"
```

```
File "<string>", line 1
SyntaxError: not a chance
```

O número da linha muda conforme a forma de execução — dentro de um arquivo, ele aponta a linha do arquivo.

9. Saia do ambiente com `deactivate`. O diretório `~/lab01` fica para os próximos capítulos.

1.10. Resumo

- Python nasceu em dezembro de 1989, no CWI, em Amsterdã, como um ABC corrigido: a mesma legibilidade, mas extensível e capaz de falar com o sistema operacional. O nome vem do Monty Python.
- A ruptura do Python 3, em 2008, existiu principalmente para separar texto de bytes de forma inegociável. Custou uma década de transição e terminou em 2020, com o fim de vida do 2.7.
- Mudanças na linguagem passam por PEPs. O poder de decisão saiu do BDFL em 2018 e hoje está com um Conselho Diretor de cinco pessoas eleitas; desde a PEP 602 sai uma versão por ano, em outubro, com cinco anos de vida útil.
- A versão de referência deste manual é a 3.13, com REPL novo, mensagens de erro mais precisas e os primeiros passos do interpretador sem GIL.
- O Zen do Python (PEP 20, `import this`) resume os valores: legibilidade conta, explícito vence implícito, erro não passa em silêncio, e há uma maneira óbvia — embora a praticidade vença a pureza quando o custo da pureza é alto.
- Código pythônico usa o vocabulário da linguagem: iterar sobre o objeto em vez de indexar, `Counter` em vez de contar na mão, f-string em vez de concatenar. Seis linhas legíveis valem mais que vinte e quatro literais.

2. Como o Python executa código: interpretador, bytecode e CPython

Você entrega um script para um colega e ele volta com duas perguntas. A primeira: “apareceu uma pasta `__pycache__` no meio do projeto, posso apagar?”. A segunda é mais interessante — ele digitou o script errado, com um `+` sobrando no fim de uma função que o programa nem chega a chamar, e nada rodou. Nem a primeira linha, que era um `print`. Se o Python é interpretado linha por linha, por que a primeira linha não executou?

Porque ele não é interpretado linha por linha. Entre o arquivo `.py` e o resultado na tela existem quatro etapas, e saber quais são explica a pasta misteriosa, o erro que aparece antes de qualquer saída, o motivo de o segundo `import` de um módulo ser mais rápido que o primeiro e boa parte do vocabulário que você vai encontrar em discussão sobre desempenho de Python.

2.1. O erro que chega antes da primeira linha

Vamos reproduzir a queixa do colega. Salve isto como `sintaxe.py`:

```
print("esta linha nunca vai aparecer")

def nunca_chamada():
    return 1 +
```

A função com o `+` solto nunca é chamada. Ainda assim:

```
File "C:\Users\Usuario\lab02\sintaxe.py", line 5
    return 1 +
            ^
SyntaxError: invalid syntax
```

O `print` não rodou. Agora troque o erro de sintaxe por um erro de nome — uma variável que não existe, também dentro da função que ninguém chama. Salve como `nome.py`:

```
print("esta linha aparece")

def nunca_chamada():
    return taxa_de_juros * 2
```

```
esta linha aparece
```

2. Como o Python executa código: interpretador, bytecode e CPython

Nenhum erro. A variável `taxa_de_juros` não existe em lugar nenhum, e o programa terminou com sucesso.

A diferença entre os dois casos é a diferença entre as duas fases da execução. Antes de rodar qualquer coisa, o Python lê o arquivo **inteiro** e o traduz para uma forma interna. Um + sobrando impede a tradução: não há como traduzir o que não é Python válido, então nada roda. Já `taxa_de_juros` é sintaticamente perfeito — é um nome, e nomes são resolvidos no momento em que a linha executa. Como aquela linha nunca executou, ninguém foi procurar a variável.

Essa é a regra prática que vale guardar: **erro de sintaxe é detectado antes de o programa começar; quase todo o resto só aparece quando a linha é alcançada**. É por isso que um `if` com um trecho errado num galho raro pode ficar meses em produção sem ninguém notar, e por isso existem verificadores estáticos e testes — assunto do Volume 11.

2.2. As quatro etapas

O caminho do arquivo até o resultado está em Figura 2.1.

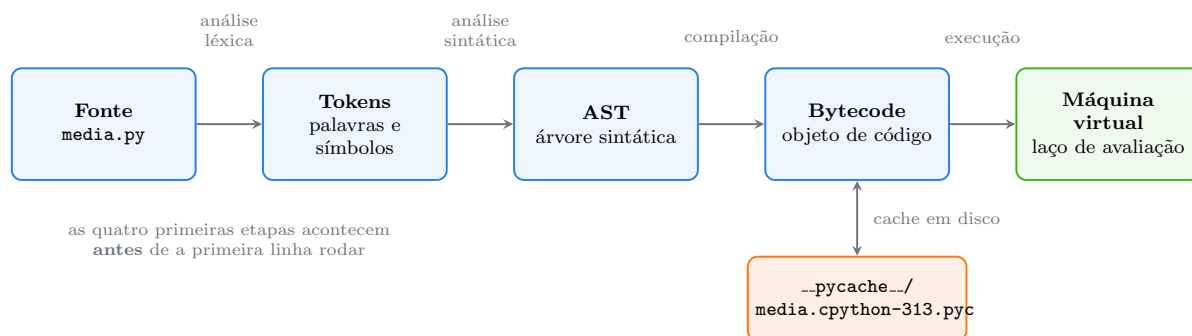


Figura 2.1.: Do arquivo `.py` ao resultado: quatro etapas e um cache.

Análise léxica. O texto do arquivo é quebrado em *tokens* — as menores unidades com significado. A linha `total = preco * 3` vira seis tokens: o nome `total`, o sinal de igual, o nome `preco`, o asterisco, o número 3 e o fim de linha. É nesta etapa que a indentação é convertida em tokens de abertura e fechamento de bloco, o que faz do espaço em branco uma parte da gramática — tema do Capítulo 6.

Análise sintática. Os tokens são organizados numa árvore que representa a estrutura do programa: a *AST*, de *Abstract Syntax Tree*, árvore sintática abstrata. É aqui que o + solto do exemplo anterior derruba tudo: a gramática do Python não tem nenhuma forma válida em que um + seja seguido de fim de linha dentro de um `return`. Você pode ver a árvore com o módulo `ast`:

```
import ast

arvore = ast.parse("total = preco * 3")
print(ast.dump(arvore, indent=2))
```

```
Module(
  body=[
    Assign(
```



```

targets=[
    Name(id='total', ctx=Store())],
value=BinOp(
    left=Name(id='preco', ctx=Load()),
    op=Mult(),
    right=Constant(value=3))))

```

Leia de dentro para fora: uma operação binária (**BinOp**) de multiplicação (**Mult**) entre o nome **preco**, lido (**Load**), e a constante 3; o resultado é atribuído (**Assign**) ao nome **total**, agora escrito (**Store**). A distinção entre **Load** e **Store** no mesmo tipo de nó é o que diferencia ler de uma variável e escrever nela.

Compilação. A árvore é traduzida para *bytecode*: uma sequência de instruções simples para uma máquina imaginária. O bytecode não é código de máquina — nenhum processador real sabe executá-lo. Ele fica guardado num **objeto de código**, que é um valor de primeira classe em Python, como um número ou uma lista:

```

codigo = compile("preco = 19.9\ntotal = preco * 3", "<exemplo>", "exec")
print(type(codigo))
print(codigo.co_consts)
print(codigo.co_names)

```

```

<class 'code'>
(19.9, 3, None)
('preco', 'total')

```

O objeto de código carrega as constantes que aparecem no trecho (**co_consts**) e os nomes que ele usa (**co_names**), separados. É o formato que a máquina virtual consome.

Execução. A máquina virtual — na prática, um laço em C que lê uma instrução, faz o que ela manda e passa para a seguinte — avalia o bytecode. É esse laço que as pessoas chamam de “o interpretador”, e é dele que vem a fama de lentidão do Python: cada operação que em C seria uma instrução de processador aqui custa uma volta no laço, com desvio, verificação de tipo e contagem de referências.

Então o Python é compilado ou interpretado? As duas coisas, e a pergunta é mal colocada. Ele é **compilado para bytecode** e **o bytecode é interpretado**. Java faz o mesmo. A diferença prática em relação a C é que a compilação acontece automaticamente, na hora de rodar, e não numa etapa separada que produz um executável.

2.3. O bytecode por dentro

O módulo **dis** — de *disassemble*, desmontar — mostra o bytecode em forma legível. Crie **media.py**:

```

def media(valores):
    total = 0
    for v in valores:
        total += v
    return total / len(valores)

```

2. Como o Python executa código: interpretador, bytecode e CPython

E `ver.py`:

```
import dis
from media import media

dis.dis(media)
```

1	RESUME	0
2	LOAD_CONST	1 (0)
	STORE_FAST	1 (total)
3	LOAD_FAST	0 (valores)
	GET_ITER	
L1:	FOR_ITER	7 (to L2)
	STORE_FAST	2 (v)
4	LOAD_FAST_LOAD_FAST	18 (total, v)
	BINARY_OP	13 (+=)
	STORE_FAST	1 (total)
	JUMP_BACKWARD	9 (to L1)
3	L2: END_FOR	
	POP_TOP	
5	LOAD_FAST	1 (total)
	LOAD_GLOBAL	1 (len + NULL)
	LOAD_FAST	0 (valores)
	CALL	1
	BINARY_OP	11 (/)
	RETURN_VALUE	

A primeira coluna é a linha do arquivo original — é assim que o traceback consegue dizer em que linha o erro aconteceu, assunto do Capítulo 7. Depois vem o nome da instrução, o argumento numérico e, entre parênteses, o que aquele número significa.

Duas coisas para notar. A primeira é `LOAD_FAST` contra `LOAD_GLOBAL`: variáveis locais de uma função moram num vetor de tamanho fixo, acessado por posição, enquanto `len` é procurada num dicionário de nomes globais e depois nos embutidos. É a razão técnica de a busca de local ser mais rápida que a de global, e de o truque antigo de copiar uma função global para uma variável local em laço apertado às vezes ainda render alguma coisa.

A segunda é `LOAD_FAST_LOAD_FAST`, que carrega duas variáveis locais numa única instrução. Não existe nada disso no código-fonte: é o compilador juntando duas operações frequentes numa só para gastar menos voltas no laço de avaliação. Instruções combinadas assim vêm crescendo desde o Python 3.11, e é boa parte do ganho de velocidade dessas versões.

2.3.1. A máquina é uma pilha

O detalhe que torna o bytecode legível é que a máquina virtual do CPython é uma **máquina de pilha**: não existem registradores nomeados, apenas uma pilha de valores. Instrução que começa

com `LOAD` empilha; operação consome do topo e devolve o resultado ao topo; `STORE` retira do topo e guarda num nome. Figura 2.2 acompanha a avaliação de `total = preco * 3`.

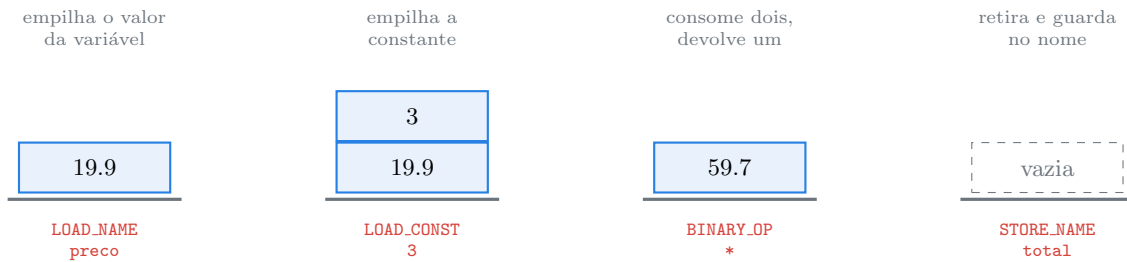


Figura 2.2.: A pilha de avaliação durante `total = preco * 3`, com `preco` valendo 19.9. A pilha cresce para cima; a linha grossa é o fundo.

Quatro instruções, quatro estados da pilha, e no fim ela volta a ficar vazia. Toda expressão Python, por complicada que seja, é decomposta nesse vaivém.

2.3.2. O compilador faz pouca coisa — mas faz

O compilador do CPython não é um otimizador agressivo, e não pretende ser. Ainda assim, algumas simplificações óbvias ele resolve na compilação, de graça:

```
import dis
dis.dis("segundos = 2 * 60 * 60")
```

0	RESUME	0
1	LOAD_CONST	0 (7200)
	STORE_NAME	0 (segundos)
	RETURN_CONST	1 (None)

Não há multiplicação nenhuma no bytecode: o 7200 já está calculado. Isso se chama *constant folding*, dobramento de constantes, e a consequência prática é boa notícia para a legibilidade — escrever `2 * 60 * 60` em vez de 7200 documenta a intenção e **não custa nada em tempo de execução**. Vale para expressões entre literais. Não vale se qualquer operando for uma variável, porque aí o valor só é conhecido na hora.

O que o compilador **não** faz é reorganizar o seu algoritmo. Compare montar um texto concatenando em laço com montar usando `join`:

```
python -m timeit -s "partes=['abc']*2000" "s="
for p in partes: s+=p
python -m timeit -s "partes=['abc']*2000" "s=''.join(partes)"
```

```
2000 loops, best of 5: 259 usec per loop
50000 loops, best of 5: 6.77 usec per loop
```

2. Como o Python executa código: interpretador, bytecode e CPython

Quase quarenta vezes de diferença, e os números na sua máquina vão ser outros — o que se mantém é a ordem de grandeza. Nenhuma otimização de bytecode salva o primeiro caso, porque o problema não está nas instruções: está em criar dois mil textos intermediários em vez de um. Escolher a estrutura e o algoritmo certos vale mais do que qualquer coisa que se faça no nível da instrução, e é por isso que este manual gasta um volume inteiro em estruturas de dados antes de falar de desempenho.

2.4. `__pycache__`, o cache que aparece sozinho

Compilar custa tempo. Fazer isso a cada execução de um módulo que não mudou seria desperdício, então o CPython guarda o resultado. Depois de rodar `ver.py`, o diretório tem uma pasta nova:

```
ls __pycache__
```

```
media.cpython-313.pyc
```

Observe **o que não está ali**: não há `ver.cpython-313.pyc`. O arquivo executado diretamente não é cacheado; só os módulos **importados** são. Faz sentido — o script de entrada roda uma vez, o módulo importado tende a ser reaproveitado por vários programas.

O nome do arquivo traz a implementação e a versão (`cpython-313`) justamente para que instalações diferentes de Python convivam no mesmo diretório sem se atropelar. O formato do bytecode muda entre versões menores, e um `.pyc` do 3.12 é ilegível para o 3.13.

O cabeçalho do `.pyc` tem 16 bytes e explica como a invalidação funciona:

```
from pathlib import Path
import struct

dados = Path("__pycache__/media.cpython-313.pyc").read_bytes()
magic, flags, mtime, tam = struct.unpack("<4sIII", dados[:16])
print("magic", magic)
print("flags", flags)
print("mtime no pyc ", mtime)
print("mtime do fonte", int(Path("media.py").stat().st_mtime))
print("tamanho fonte no pyc", tam, "| real", Path("media.py").stat().st_size)
```

```
magic b'\xf3\r\r\n'
flags 0
mtime no pyc 1786393562
mtime do fonte 1786393562
tamanho fonte no pyc 107 | real 107
```

São quatro campos. O *número mágico* identifica a versão do formato de bytecode — se não bater com o do interpretador, o cache é descartado. Os **flags** dizem qual estratégia de invalidação está em uso. E os dois últimos campos são a data de modificação e o tamanho do arquivo-fonte no momento em que o cache foi escrito: a cada **import**, o Python compara esses dois valores com os do `.py` em disco e recompila se algum diferir.

Duas consequências práticas. A primeira é que **você pode apagar `__pycache__` sem medo** — ela é recriada. Responda isso ao colega, e acrescente `__pycache__/` ao `.gitignore`, porque cache não entra no controle de versão. A segunda é mais sutil: a invalidação depende do relógio. Se um processo de build restaura arquivos com data antiga, ou se um contêiner tem relógio deslocado, é possível ficar com bytecode velho rodando fonte novo. Para esses casos existe o modo baseado em hash do conteúdo (PEP 552), ativado por um bit nos `flags`.

Duas variáveis de ambiente e duas opções de linha de comando aparecem em torno disso. `python -B`, ou `PYTHONDONTWRITEBYTECODE=1`, desliga a escrita do cache — comum em imagem de contêiner, onde o `.pyc` só engorda a camada. E `python -O` produz um arquivo com sufixo diferente:

```
python -O -c "import media"
ls __pycache__
```

```
media.cpython-313.opt-1.pyc
media.cpython-313.pyc
```

O que o `-O` faz merece atenção porque o nome engana. Ele não otimiza nada de verdade: apenas remove as instruções `assert` e trecho protegido por `if __debug__`. Compare:

```
python -c "assert False, 'estourou'"
python -O -c "assert False, 'estourou'; print('passou reto')"
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    assert False, 'estourou'
    ~~~~~
AssertionError: estourou
passou reto
```

Com `-O`, a asserção desaparece. Essa é a razão de nunca usar `assert` para validar entrada de usuário ou verificar permissão: alguém rodando com `-O` some com a sua checagem. `assert` é para condição que você acredita ser sempre verdadeira, como rede de segurança durante o desenvolvimento e nos testes.

2.5. CPython e as outras implementações

Vale repetir a distinção do Capítulo 1, agora com conteúdo técnico: **Python é a linguagem, CPython é o programa que a executa**. O interpretador sabe se apresentar:

```
python -c "import sys; print(sys.implementation.name); print(sys.version)"
```

```
cpython
3.13.15 (main, Aug 7 2026, 02:25:50) [MSC v.1944 64 bit (AMD64)]
```

Tudo que descrevemos até aqui — bytecode de pilha, `__pycache__`, contagem de referências, GIL — é decisão de projeto do CPython, não exigência da linguagem. Outras implementações fazem diferente:

2. Como o Python executa código: interpretador, bytecode e CPython

- **PyPy** compila em tempo de execução, com JIT: ele observa quais trechos rodam muito e gera código de máquina de verdade para eles. Em laço numérico puro chega a ser uma ordem de grandeza mais rápido que o CPython. Em troca, a interface com extensões escritas em C é mais frágil e o consumo de memória é maior.
- **MicroPython** e **CircuitPython** rodam em microcontrolador, com biblioteca padrão reduzida ao que cabe em poucas centenas de kilobytes.
- **GraalPy** roda sobre a máquina virtual da JVM, o que interessa a quem precisa conviver com código Java.

O CPython também está mudando por dentro. O 3.11 trouxe o *interpretador especializado* (PEP 659): a máquina virtual observa os tipos que passam por cada instrução e a substitui por uma versão especializada — um `BINARY_OP` genérico vira uma soma de inteiros direta quando os dois operandos vêm sendo inteiros. É otimização em tempo de execução dentro do interpretador, sem JIT. O 3.13 acrescentou um compilador JIT experimental (PEP 744), desligado por padrão, e o build sem GIL (PEP 703). Nada disso muda o que você escreve; muda quanto tempo leva para rodar. O Volume 15 trata do assunto com o cuidado que ele merece.

Um efeito colateral útil de saber que o `import` custa: quando um programa de linha de comando demora a responder, muitas vezes o culpado é o que ele importa, não o que ele faz. A opção `-X importtime` cobra o tempo de cada `import`, em microssegundos:

```
python -X importtime -c "import json"
```

import time:	1646	4792	re._compiler
import time:	758	758	copyreg
import time:	1864	10631	re
import time:	305	305	_json
import time:	7730	8034	json.scanner
import time:	11253	29918	json.decoder
import time:	7461	7461	json.encoder
import time:	10180	47558	json

A primeira coluna é o tempo do módulo em si; a segunda, o acumulado com tudo que ele arrasta; a indentação mostra quem importou quem. Aqui, importar `json` custou 47 milissegundos, e quase dois terços disso foram gastos em `json.decoder` e no `re` que ele carrega. Numa ferramenta de linha de comando que precisa responder rápido, esse é o tipo de conta que compensa fazer.

2.6. Jeito Pythônico

O erro pythônico mais comum em torno deste assunto é usar o `dis` como árbitro de desempenho. Contar instruções não mede tempo: `LOAD_FAST` e `CALL` aparecem iguais na listagem e custam ordens de grandeza diferentes, e o interpretador especializado troca instruções por outras durante a execução, então a listagem estática nem descreve o que rodou. A forma correta de comparar é medir:

```
# antipadrão: decidir por contagem de instrução no dis
# "a versão A tem 12 instruções e a B tem 14, então A é mais rápida"

# pythônico: medir, com a mesma entrada, no mesmo ambiente
python -m timeit -s "setup" "trecho"
```

O `dis` serve para **entender semântica**, não velocidade: descobrir por que um `for` sobre um dicionário percorre chaves, o que exatamente um decorador acrescentou, por que uma comparação com `is` deu um resultado inesperado. Para velocidade existem o `timeit` e os *profilers*, assunto reservado ao Volume 15.

Três regras que caem direto no dia a dia:

`__pycache__`/ no `.gitignore`, sempre. E jamais distribua `.pyc` no lugar do `.py` como proteção de código: o bytecode é trivial de desmontar com o próprio `dis` da biblioteca padrão, e você amarra o pacote a uma versão exata do interpretador.

`assert` não é validação. Como `-O` remove as asserções, entrada de usuário e verificação de permissão pedem `if` com `raise` explícito:

```
# antipadrão: some com -O
assert idade >= 18, "menor de idade"

# pythônico: sobrevive a qualquer modo de execução
if idade < 18:
    raise ValueError("menor de idade")
```

Prefira legibilidade a pré-cálculo. Como o dobramento de constantes resolve expressão entre literais na compilação, escreva `60 * 60 * 24` e não `86400`; `1024 ** 2` e não `1048576`. Custa zero e explica o número.

Sobre versões, o que muda por aqui:

- **Python 3.11** — interpretador especializante (PEP 659) e `tracebacks` com marcação de coluna. O formato do bytecode mudou bastante; qualquer código que dependa de instruções específicas precisa ser revisto a cada versão.
- **Python 3.12** — `dis` passou a mostrar as instruções combinadas e os saltos com rótulos (`L1`, `L2`), como na saída deste capítulo.
- **Python 3.13** — JIT experimental (PEP 744) e build sem GIL (PEP 703), ambos opcionais.

A moral: **bytecode é detalhe de implementação**. É legítimo e útil olhar para ele para entender o que a linguagem está fazendo. Não é lugar para construir código que dependa de instrução específica, porque isso quebra na próxima versão sem aviso e sem culpa de ninguém.

2.7. Laboratório

Objetivo: observar cada etapa da execução no seu próprio ambiente — ver o erro de compilação chegar antes da execução, desmontar bytecode, decodificar o cabeçalho de um `.pyc` na mão e provocar a invalidação do cache. Só biblioteca padrão.

1. **Prepare o diretório e o ambiente.** No Linux:

```
mkdir -p ~/lab02 && cd ~/lab02
python3 -m venv .venv
source .venv/bin/activate
python --version
```

Python 3.13.15

No Windows, `py -3 -m venv .venv e .\.venv\Scripts\Activate.ps1`.

2. Veja a compilação acontecer antes da execução. Crie `sintaxe.py` e `nome.py` com o conteúdo mostrado no começo do capítulo e rode os dois. O primeiro não imprime nada e falha com `SyntaxError` apontando a linha 5; o segundo imprime a mensagem e termina normalmente, apesar de conter uma variável inexistente. Se você inverter — chamando `nunca_chamada()` no fim do `nome.py` — aí sim aparece o `NameError`, e apenas então.

3. Desmonte uma função. Crie `media.py` e `ver.py` como no capítulo e rode `python ver.py`. Compare a saída com a listagem mostrada aqui. Localize na sua saída: onde o laço volta (`JUMP_BACKWARD`), onde `len` é buscada (`LOAD_GLOBAL`) e onde as locais são lidas (`LOAD_FAST`).

4. Escreva uma variante e compare. Troque o corpo de `media.py` pela versão com `sum`:

```
def media(valores):  
    return sum(valores) / len(valores)
```

Rode `python ver.py` de novo:

1	RESUME	0
2	LOAD_GLOBAL	1 (sum + NULL)
	LOAD_FAST	0 (valores)
	CALL	1
	LOAD_GLOBAL	3 (len + NULL)
	LOAD_FAST	0 (valores)
	CALL	1
	BINARY_OP	11 (/)
	RETURN_VALUE	

Nove instruções contra as dezenove da versão com laço, e o laço em si desapareceu — ele agora acontece dentro de `sum`, escrita em C. Este é o padrão que se repete em Python: empurrar a repetição para dentro de uma função da biblioteca padrão.

5. Encontre o cache. Rode `ls __pycache__` (ou `dir __pycache__` no `cmd`). Deve haver `media.cpython-313.pyc` e **não** deve haver nada referente a `ver.py`. Se a sua versão for outra, o número no nome muda.

6. Decodifique o cabeçalho. Crie `cabecalho.py` com o trecho do `struct.unpack` mostrado no capítulo e rode. Os valores de `mtime` na sua máquina serão outros, mas os dois têm de ser **iguais entre si**, e o tamanho no cabeçalho tem de bater com o tamanho real do `media.py`.

7. Provoque a invalidação. Acrescente uma linha em branco no fim de `media.py`, salve e rode `python cabecalho.py` de novo. Os dois `mtime` agora divergem — o do cache é o antigo. Rode `python ver.py`, que importa o módulo e força a recompilação, e depois `python cabecalho.py` mais uma vez: voltaram a bater, e o tamanho subiu um byte (ou dois, no Windows, por causa do fim de linha).

8. Desligue o cache. Apague a pasta e rode com `-B`:


```
rm -rf __pycache__
python -B ver.py
ls __pycache__
```

A listagem falha porque a pasta não foi criada. Rode sem o `-B` e ela volta.

9. Meça em vez de contar instruções. Reproduza a comparação de concatenação:

```
python -m timeit -s "partes=['abc']*2000" "s="
for p in partes: s+=p"
python -m timeit -s "partes=['abc']*2000" "s=''.join(partes)"
```

Os números absolutos vão diferir dos deste capítulo; a diferença entre eles, não muito.

10. Veja o custo dos imports. Rode `python -X importtime -c "import json"` e depois com `import decimal` e `import asyncio`. O `asyncio` é o mais caro dos três com folga — é bom saber disso antes de importá-lo num script que precisa responder em milissegundos.

11. Feche com deactivate. O diretório `~/lab02` fica para consulta.

2.8. Resumo

- Python não é interpretado linha por linha: o arquivo inteiro é lido, analisado e compilado para bytecode **antes** de qualquer linha rodar. Por isso erro de sintaxe aparece antes do primeiro `print`, e erro de nome só aparece quando a linha é alcançada.
- São quatro etapas: análise léxica (tokens), análise sintática (AST), compilação (bytecode em um objeto de código) e execução na máquina virtual, que é o laço de avaliação escrito em C.
- A máquina virtual do CPython é uma máquina de pilha: `LOAD` empilha, operação consome do topo, `STORE` retira e guarda num nome. O módulo `dis` mostra tudo isso, com a linha do fonte na primeira coluna.
- `__pycache__` guarda o bytecode dos módulos **importados**, com a versão no nome do arquivo. A invalidação compara data e tamanho do fonte gravados no cabeçalho de 16 bytes. Pode apagar, pode ignorar no Git, não serve para esconder código.
- `-O` não otimiza: remove `assert` e `if __debug__`. Logo, `assert` nunca é validação de entrada — para isso, `if` com `raise`.
- Bytecode é detalhe de implementação do CPython, não da linguagem, e muda a cada versão. Use `dis` para entender semântica e `timeit` para medir velocidade; nunca conte instruções para prever desempenho.

3. Instalando o Python: versões, gerenciadores e o Python do sistema

Você precisa de uma biblioteca para consumir uma API. Abre o terminal do Kali, digita `pip install requests` e leva um tapa na cara:

```
error: externally-managed-environment

× This environment is externally managed
> To install Python packages system-wide, try apt install
  python3-xyz, where xyz is the package you are trying to
  install.

  If you wish to install a non-Kali-packaged Python package,
  create a virtual environment using python3 -m venv path/to/venv.
  Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
  sure you have python3-full installed.

  If you wish to install a non-Kali-packaged Python application,
  it may be easiest to use pipx install xyz, which will manage a
  virtual environment for you. Make sure you have pipx installed.

  For more information, refer to the following:
  * https://www.kali.org/docs/general-use/python3-external-packages/
  * /usr/share/doc/python3.13/README.venv

note: If you believe this is a mistake, please contact your Python installation
↪ or OS distribution provider. You can override this, at the risk of breaking
↪ your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

A tentação é óbvia: a última linha oferece uma saída. `--break-system-packages` resolve na hora, o `requests` instala e você volta ao trabalho. Também é a forma mais rápida conhecida de estragar uma instalação de Linux, e a mensagem se dá o trabalho de dizer isso no nome da opção.

Este capítulo explica por que essa barreira existe, o que fazer em vez de derrubá-la, e como ter a versão do Python que você quer sem entrar em conflito com a que o sistema operacional precisa.

3.1. O Python do sistema não é seu

Numa distribuição Linux moderna, o Python não é um programa que alguém instalou para programar. É uma **peça de infraestrutura**. Ferramentas de administração, o gerenciador de

3. Instalando o Python: versões, gerenciadores e o Python do sistema

pacotes, utilitários de rede e, no caso do Kali, uma parte considerável do arsenal de segurança são escritos em Python e dependem de que o interpretador e as bibliotecas do sistema estejam exatamente na versão que o pacote declarou.

Dá para ver quem está na fila:

```
apt-cache rdepends --installed python3 | head -8
```

```
python3
Reverse Depends:
  python3-dev
  weeveily
  theharvester
  texlive-pictures
  texlive-latex-extra
  |sublist3r
```

`weeveily`, `theHarvester` e `sublist3r` são ferramentas do Kali. Elas não têm ambiente próprio: usam o Python do sistema e as bibliotecas que o `apt` instalou. E são muitas:

```
dpkg -l 'python3-*' | grep -c '^ii'
```

529

Quinhentos e vinte e nove pacotes Python gerenciados pelo `apt`. Quando você roda `pip install requests` com privilégio no interpretador do sistema, o `pip` sobrescreve arquivos que o `apt` considera seus, sem avisar o `apt`. O banco de dados de pacotes passa a mentir: ele diz que a versão instalada é a que veio da distribuição, e o disco tem outra. A partir daí, uma atualização de sistema pode reverter a sua biblioteca, ou a sua biblioteca pode quebrar uma ferramenta que esperava a versão antiga — e o sintoma aparece semanas depois, num programa que você não tocou.

É contra isso que a **PEP 668** existe. Ela padronizou um arquivo marcador que a distribuição coloca ao lado da biblioteca padrão:

```
ls -l /usr/lib/python3.13/EXTERNALLY-MANAGED
```

```
-rw-r--r-- 1 root root 734 Jun  3 04:36 /usr/lib/python3.13/EXTERNALLY-MANAGED
```

O arquivo é um `.ini` cujo campo `Error` é justamente o texto que o `pip` imprimiu:

```
head -4 /usr/lib/python3.13/EXTERNALLY-MANAGED
```

```
[externally-managed]
Error=To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.
```

Nenhuma mágica: o `pip` procura esse arquivo e, se o encontra, recusa instalar naquele interpretador. É um acordo entre o empacotador da distribuição e a ferramenta do Python, e a mensagem de erro personalizada é escrita por quem monta a distro. A sua existência não é um bug nem excesso de zelo — é a distribuição dizendo “este interpretador tem dono, e não é você”.

3.2. As três camadas

A saída não é lutar com a barreira, é entender que existem três lugares diferentes onde um Python pode morar, com donos diferentes. Figura 3.1 organiza isso.

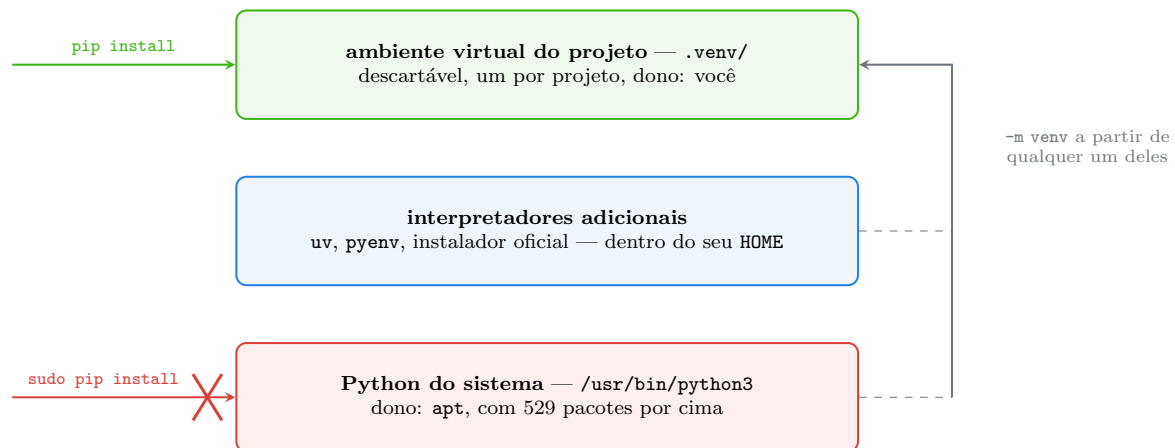


Figura 3.1.: Três camadas, três donos. O `pip` só entra sem pedir licença na de cima.

A camada de baixo é do `apt`. Você lê, executa e não escreve. A do meio é opcional e existe quando você precisa de uma versão que a distribuição não oferece — mora no seu diretório pessoal e não conflita com nada. A de cima é o ambiente virtual, criado a partir de qualquer interpretador das duas camadas de baixo, e é onde o `pip` trabalha à vontade.

3.3. O ambiente virtual, com precisão

Um ambiente virtual é bem menos mágico do que o nome sugere. Crie um e olhe por dentro:

```
mkdir -p /tmp/lab03 && cd /tmp/lab03
python3 -m venv .venv
cat .venv/pyvenv.cfg
```

```
home = /usr/bin
include-system-site-packages = false
version = 3.13.12
executable = /usr/bin/python3.13
command = /usr/bin/python3 -m venv /tmp/lab03/.venv
```

É isso: um arquivo de texto de cinco linhas, um diretório `bin/` com alguns links e um `lib/python3.13/site-packages/` vazio. Não há cópia do interpretador — `home` e `executable` apontam para o Python de verdade, lá na camada de baixo. O que muda é onde ele procura pacotes:

3. Instalando o Python: versões, gerenciadores e o Python do sistema

```
.venv/bin/python -c "import sys; print('prefix      ', sys.prefix);  
↪ print('base_prefix', sys.base_prefix)"
```

```
prefix      /tmp/lab03/.venv  
base_prefix /usr
```

`sys.base_prefix` é o interpretador real; `sys.prefix` é o ambiente. Quando os dois diferem, você está num venv — e é exatamente assim que ferramentas detectam isso. O efeito prático aparece no caminho de busca:

```
.venv/bin/python -c "import sys; [print(p) for p in sys.path if p]"
```

```
/usr/lib/python313.zip  
/usr/lib/python3.13  
/usr/lib/python3.13/lib-dynload  
/tmp/lab03/.venv/lib/python3.13/site-packages
```

A biblioteca padrão continua vindo de `/usr/lib/python3.13` — ela é compartilhada, e não faria sentido duplicá-la. O que sumiu da lista é o `/usr/lib/python3/dist-packages`, onde moram os 529 pacotes do `apt`. No lugar dele entrou o `site-packages` do projeto. Por isso o `pip` pode trabalhar aqui sem incomodar ninguém:

```
.venv/bin/python -m pip install requests cowsay  
.venv/bin/python -m pip list
```

Package	Version
certifi	2026.7.22
charset-normalizer	3.4.9
cowsay	6.1
humanize	4.16.0
idna	3.18
pip	26.1.1
requests	2.34.2
urllib3	2.7.0

E o isolamento é verificável nos dois sentidos. Um pacote que só existe no ambiente não vaza para o sistema:

```
.venv/bin/python -c "import cowsay; print(cowsay.__file__)"  
python3 -c "import cowsay"
```

```
/tmp/lab03/.venv/lib/python3.13/site-packages/cowsay/__init__.py  
import cowsay  
ModuleNotFoundError: No module named 'cowsay'
```

E um pacote que existe nos dois lados fica em duas versões independentes, cada interpretador vendo a sua:

```
python3 -c "import requests; print(requests.__version__, requests.__file__)"
.venv/bin/python -c "import requests; print(requests.__version__,
↪ requests.__file__)"
```

```
2.32.5 /usr/lib/python3/dist-packages/requests/__init__.py
2.34.2 /tmp/lab03/.venv/lib/python3.13/site-packages/requests/__init__.py
```

O Kali traz `requests` 2.32.5 porque é a versão que as ferramentas dele foram testadas contra. O seu projeto usa a 2.34.2. As duas convivem, ninguém quebra, e você não precisou pedir licença. **Este é o argumento inteiro a favor do ambiente virtual**, e ele vale muito mais do que a economia de espaço em disco que às vezes se cita.

3.3.1. O que `activate` faz de verdade

Chamar `.venv/bin/python` pelo caminho completo funciona sempre e é o que scripts e ferramentas fazem. Para trabalho interativo existe a ativação, que costuma ser descrita como um ritual obscuro e não é nada disso:

```
which python3
source .venv/bin/activate
which python
python --version
```

```
/usr/bin/python3
/tmp/lab03/.venv/bin/python
Python 3.13.12
```

A única coisa relevante que aconteceu está em Figura 3.2: o diretório `bin/` do ambiente foi colocado **na frente** do `PATH`.

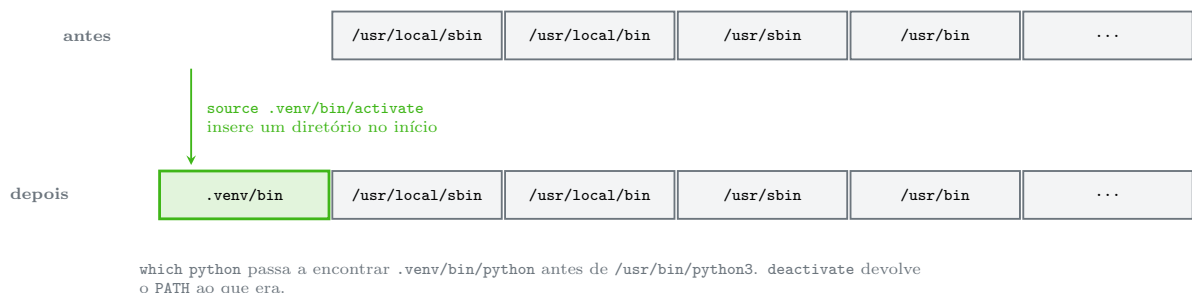


Figura 3.2.: Ativar um ambiente virtual é inserir um diretório no começo do `PATH`.

Nada mais acontece de essencial. O script também ajusta o prompt para mostrar `(.venv)` e define uma função `deactivate` que desfaz a mudança. Não há processo em segundo plano, nada é gravado fora do diretório, e fechar o terminal já desativa. Uma consequência prática útil: como é só `PATH`, ativação **não** é herdada por um processo que você lançou antes, e `sudo` não a preserva — `sudo pip install` dentro de um `venv` ativado instala no Python do sistema, que é o oposto do que você queria.

3.4. Quando você precisa de outra versão

Até aqui, o venv resolveu o problema dos **pacotes**. Ele não resolve o problema da **versão do interpretador**: o `pyvenv.cfg` acima diz `version = 3.13.12` porque foi criado a partir do Python do sistema, e não há como um venv oferecer um 3.11 se a máquina só tem 3.13.

Três situações pedem outra versão. Um projeto de trabalho preso a um servidor com 3.9. Uma biblioteca que ainda não suporta a versão mais nova. E o inverso: você quer testar um recurso de 3.14 numa máquina cuja distribuição parou no 3.11. Para todos os casos, a resposta é a camada do meio da Figura 3.1 — interpretadores instalados no seu diretório pessoal, sem `sudo` e sem tocar no sistema.

uv é hoje o caminho mais direto. Ele é um gerenciador de pacotes e de versões escrito em Rust, resolve dependências rápido e baixa interpretadores prontos:

```
uv python list
```

```
cpython-3.15.0rc1-windows-x86_64-none          <download available>
cpython-3.15.0rc1+freethreaded-windows-x86_64-none  <download available>
cpython-3.14.7-windows-x86_64-none             <download available>
cpython-3.14.7+freethreaded-windows-x86_64-none  <download available>
cpython-3.14.4-windows-x86_64-none             C:\Python314\python.exe
cpython-3.13.15-windows-x86_64-none
  ↳ C:\Users\Usuario\AppData\Roaming\uv\python\cpython-3.13.15-windows-x86_64-none\python.exe
cpython-3.13.15+freethreaded-windows-x86_64-none  <download available>
cpython-3.12.13-windows-x86_64-none             <download available>
cpython-3.11.15-windows-x86_64-none             <download available>
cpython-3.10.20-windows-x86_64-none             <download available>
cpython-3.9.25-windows-x86_64-none              <download available>
pypy-3.11.15-windows-x86_64-none                <download available>
graalpy-3.12.0-windows-x86_64-none              <download available>
```

A listagem foi tirada de uma máquina Windows, e é por isso que aparece `windows-x86_64`; no Linux os nomes trazem `linux-x86_64-gnu`. O que interessa é a estrutura: cada linha é uma combinação de implementação, versão e plataforma, e a coluna da direita diz se já está em disco ou se é download. Repare que estão ali o **freethreaded** — o build sem GIL de que falamos no Capítulo 2 — e também PyPy e GraalPy, as implementações alternativas do Capítulo 1. Criar um ambiente com uma versão específica é uma linha:

```
uv venv --python 3.13 .venv
```

Se a versão pedida não estiver em disco, o **uv** baixa o interpretador e segue. Ele não pede `sudo`, não escreve em `/usr` e não conhece o `EXTERNALLY-MANAGED` porque nunca chega perto do Python do sistema.

pyenv é a alternativa mais antiga e ainda muito usada. A diferença de filosofia importa: o **pyenv** **compila** o interpretador na sua máquina, o que exige as bibliotecas de desenvolvimento instaladas e leva alguns minutos, mas produz um binário adaptado ao seu sistema. Ele também oferece um recurso que o **uv** trata de outra forma: o arquivo `.python-version` no diretório do projeto, que faz o **pyenv** trocar de versão automaticamente quando você entra na pasta.

Os pacotes da distribuição resolvem o caso em que a versão que você quer está no repositório. No Debian e no Kali, versões adicionais aparecem como pacotes próprios quando existem, e a instalação é a de sempre. No Ubuntu, o repositório extra **deadsnakes** é a fonte tradicional de versões que não vieram no sistema. Vale a advertência: instalar um **python3.12** pelo **apt** ao lado do **python3.13** do sistema é seguro, porque são pacotes distintos com arquivos distintos; o que não é seguro é usar **pip** com privilégio em qualquer um dos dois.

O **instalador oficial** de **python.org** é a rota natural no Windows e no macOS, e a única que este manual recomenda no Windows para quem não vai usar **uv**.

3.5. Windows e macOS em duas páginas

No **Windows**, o instalador oficial traz um programa que não existe no Linux e que resolve o problema de conviver com várias versões: o *launcher*, chamado **py**. Ele lê as instalações registradas e escolhe uma:

```
py -0p
```

```
-V:3.14 *          C:\Python314\python.exe
-V:Astral/CPython3.13.15
↪ C:\Users\Usuario\AppData\Roaming\uv\python\cpython-3.13.15-windows-x86_64-none\python.exe
```

O asterisco marca o padrão. **py -3.13** chama aquela versão específica, **py -3** chama o 3 mais recente, e **py -0p** lista tudo com o caminho. Como o **py** sabe de todas, ele é a forma mais confiável de escolher o interpretador no Windows — mais do que mexer no **PATH**.

Duas armadilhas específicas do Windows valem aviso. A primeira: se **python** abrir a Microsoft Store em vez de rodar, você encontrou o *stub* que a Microsoft instala por padrão. Ele não é um Python; é um atalho para a loja. A solução é usar **py -3** ou instalar de verdade e marcar a opção “**Add python.exe to PATH**” no instalador. A segunda: dentro de um ambiente virtual, prefira sempre **python -m pip** a **pip** solto, porque um **pip.exe** velho em outro diretório do **PATH** pode vencer a disputa e instalar no interpretador errado.

No **macOS**, o Python que vem com o sistema é ainda mais restrito do que no Linux — a Apple o marca como componente interno e já anunciou sua remoção. Use o instalador oficial ou o Homebrew (**brew install python@3.13**), e note que o Homebrew também aplica a PEP 668: **pip install** fora de um **venv** falha do mesmo jeito.

3.6. Jeito Pythônico

A convenção mais estabelecida do ecossistema é curta: **um ambiente virtual por projeto, dentro do projeto, chamado .venv, fora do controle de versão.**

```
# pythônico
cd ~/projetos/relatorios
python3 -m venv .venv
source .venv/bin/activate
python -m pip install requests
echo ".venv/" >> .gitignore
```

3. Instalando o Python: versões, gerenciadores e o Python do sistema

O nome `.venv` é reconhecido automaticamente pelo VS Code, pelo PyCharm, pelo `uv` e por várias outras ferramentas — usar outro nome só cria trabalho. E `.venv/` entra no `.gitignore` sempre: o ambiente é derivado, tem caminhos absolutos dentro dele e não é portátil entre máquinas. O que se versiona é a **lista** de dependências, tema do Volume 5.

O antipadrão a evitar tem várias formas, todas com a mesma raiz:

```
# antipadrão: escrever no Python do sistema
sudo pip install requests
pip install --break-system-packages requests
sudo python3 -m pip install --upgrade pip

# antipadrão: um ambiente só para tudo
source ~/.venv-geral/bin/activate # usado em quinze projetos diferentes
```

Os três primeiros brigam com o `apt` pelos mesmos arquivos. O quarto não quebra o sistema, mas devolve o problema que o `venv` resolvia: dois projetos que precisam de versões diferentes da mesma biblioteca voltam a se atropelar, e você descobre isso no pior momento.

Sobre `--break-system-packages`, uma nota honesta: ele existe para casos legítimos, quase todos em contêiner. Numa imagem Docker cujo único propósito é rodar um programa Python, não há conflito possível com ferramentas de distribuição, e a alternativa (criar um `venv` dentro da imagem) só acrescenta uma camada. Fora desse contexto, a opção faz exatamente o que o nome diz.

Para **instalar aplicativos** escritos em Python — `ruff`, `httpie`, `yt-dlp`, coisas que você chama pelo nome no terminal e não importa no seu código — nem `pip` nem `venv` manual são a resposta certa. O `pipx` cria um ambiente isolado por aplicativo e coloca só o executável no seu `PATH`:

```
sudo apt install pipx
pipx install ruff
```

O `uv` faz a mesma coisa com `uv tool install`. A regra mental é limpa: **biblioteca que você importa vai no `venv` do projeto; programa que você executa vai no `pipx`.**

Sobre versões, o que muda por aqui:

- **PEP 668** — o marcador `EXTERNALLY-MANAGED`. Chegou às distribuições ao longo de 2023; Debian 12, Ubuntu 23.04 e o Kali de então em diante. É a razão de tutoriais antigos com `sudo pip install` não funcionarem mais.
- **PEP 405** — o módulo `venv` na biblioteca padrão, desde o Python 3.3. Antes dele, `virtualenv` era um pacote externo obrigatório; hoje é opcional e serve sobretudo a quem precisa de compatibilidade mais ampla.
- **Python 3.11** — `python3 -m venv` passou a ser bem mais rápido, e `--upgrade-deps` ficou prático para já criar o ambiente com `pip` atualizado.

3.7. Laboratório

Objetivo: reproduzir a barreira da PEP 668, montar um ambiente virtual, provar o isolamento nos dois sentidos e inspecionar o que a ativação faz. Nada é instalado no sistema, e nada exige `sudo`.

O roteiro está em Linux (Kali/Debian). As diferenças de Windows estão no passo 9.

1. Confirme quem é o Python do sistema.

```
python3 --version
which -a python3
```

```
Python 3.13.12
/usr/bin/python3
/bin/python3
```

`/bin` é um link para `/usr/bin` nas distribuições atuais, então os dois caminhos são o mesmo arquivo.

2. Veja quem depende dele. Não pule este passo — é o que dá peso ao resto:

```
dpkg -l 'python3-*' | grep -c '^ii'
apt-cache rdepends --installed python3 | head -8
```

O primeiro comando conta os pacotes Python que o `apt` administra (529 na máquina deste capítulo; na sua, outro número). O segundo mostra alguns dos programas que quebrariam se o interpretador mudasse por baixo deles.

3. Provoque a PEP 668. Este comando **falha de propósito** e não altera nada no sistema:

```
pip install requests
```

Você deve ver o `error: externally-managed-environment` da abertura do capítulo, com o texto que a sua distribuição escreveu. **Não** execute a sugestão do `--break-system-packages`.

4. Encontre o marcador.

```
ls -l /usr/lib/python3.13/EXTERNALLY-MANAGED
head -4 /usr/lib/python3.13/EXTERNALLY-MANAGED
```

Compare o campo `Error` com a mensagem que o `pip` imprimiu no passo 3. É a mesma.

5. Crie o ambiente e leia o `pyvenv.cfg`.

```
mkdir -p ~/lab03 && cd ~/lab03
python3 -m venv .venv
cat .venv/pyvenv.cfg
```

Se der erro dizendo que `ensurepip` não está disponível, falta o pacote `python3-venv` (ou `python3-full`): instale com `sudo apt install python3-full` e repita. É a única vez neste laboratório em que `sudo` aparece, e é para instalar um pacote da distribuição pelo `apt` — não para mexer em pacote Python.

6. Compare os prefixos.

3. Instalando o Python: versões, gerenciadores e o Python do sistema

```
.venv/bin/python -c "import sys; print(sys.prefix); print(sys.base_prefix)"
```

Os dois têm de ser diferentes. Se forem iguais, você chamou o Python do sistema por engano.

7. Prove o isolamento nos dois sentidos.

```
.venv/bin/python -m pip install requests cowsay  
.venv/bin/python -c "import cowsay; print(cowsay.__file__)"  
python3 -c "import cowsay"
```

O ambiente encontra o `cowsay`; o sistema responde `ModuleNotFoundError`. Agora o caso mais interessante:

```
python3 -c "import requests; print(requests.__version__)"  
.venv/bin/python -c "import requests; print(requests.__version__)"
```

Dois números diferentes, dois arquivos diferentes, nenhum conflito. Se o seu Kali não trouxer `requests` do `apt`, o primeiro comando dá `ModuleNotFoundError` — o que também serve, só é menos ilustrativo.

8. Observe a ativação.

```
which python3  
source .venv/bin/activate  
which python  
echo "$VIRTUAL_ENV"  
deactivate  
which python3
```

A variável `VIRTUAL_ENV` é definida pela ativação e apagada pela desativação; é o que o seu prompt e o seu editor leem para mostrar qual ambiente está em uso. Confirme que, depois do `deactivate`, `which python3` volta a apontar para `/usr/bin/python3`.

9. No Windows. Os passos 3 e 4 não se aplicam — o Python oficial do Windows não é gerenciado externamente e o `pip` instala sem reclamar, o que é justamente por que ali o cuidado com ambientes é responsabilidade sua. O equivalente aos passos 1, 5 e 8:

```
py -Op  
py -3 -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install requests cowsay  
deactivate
```

Se o `Activate.ps1` for bloqueado pela política de execução do PowerShell, a saída sem baixar a guarda do sistema é chamar o interpretador pelo caminho: `.\.venv\Scripts\python.exe -m pip install ...` funciona sem ativação alguma.

10. Descarte e recrie. Para fechar, apague o ambiente e faça de novo:

```
rm -rf .venv
python3 -m venv .venv
```

Levou poucos segundos e nada foi perdido — porque não havia nada de valor lá dentro. Essa é a propriedade mais útil de um ambiente virtual: ele é descartável. Quando algo der errado nas dependências de um projeto, apagar o `.venv` e recriar é uma solução legítima, não uma gambiarra.

3.8. Resumo

- O Python de `/usr/bin` é infraestrutura da distribuição, não ferramenta de desenvolvimento. No Kali deste capítulo, 529 pacotes do `apt` e ferramentas como `theHarvester` e `weevely` dependem dele.
- A PEP 668 formalizou o arquivo `EXTERNALLY-MANAGED`, que faz o `pip` recusar instalação no interpretador do sistema. Não é bug: é a distribuição declarando que aquele Python tem dono.
- Um ambiente virtual é um `pyvenv.cfg`, um `bin/` e um `site-packages/` vazio. Ele não copia o interpretador: substitui o caminho de busca de pacotes, deixando a biblioteca padrão compartilhada e trocando `dist-packages` pelo `site-packages` do projeto.
- `sys.prefix` diferente de `sys.base_prefix` é a maneira canônica de saber que se está num `venv`. `activate` apenas insere um diretório no início do `PATH` e define `VIRTUAL_ENV`; `sudo` não herda nada disso.
- Para outra **versão** do interpretador, use `uv` (baixa binários prontos) ou `pyenv` (compila), ambos dentro do seu diretório pessoal. No Windows, o launcher `py` escolhe entre as versões instaladas.
- Convenção: um `.venv` por projeto, no projeto, no `.gitignore`. Biblioteca que se importa vai no `venv`; programa que se executa vai no `pipx`. `sudo pip install` não tem caso de uso legítimo numa máquina de trabalho.

4. O REPL, os scripts e o primeiro programa

Um cliente pede um orçamento: sete horas e meia de trabalho, a oitenta e cinco reais a hora. Você abre a calculadora do sistema, digita, anota o resultado. Meia hora depois vem outro cliente, e outro depois dele. No fim do dia você fez a mesma conta onze vezes e errou uma.

Existem duas ferramentas para isso, e a diferença entre elas é o assunto deste capítulo. Uma serve para **descobrir** qual é a conta; a outra, para **nunca mais** ter que fazê-la. Quem usa só a primeira repete trabalho para sempre. Quem usa só a segunda escreve arquivo para responder perguntas de dez segundos.

4.1. O REPL: onde se descobre

Digite `python3` no terminal, sem argumento nenhum:

```
Python 3.13.12 (main, Feb  4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3
5
>>> _ * 10
50
>>> horas = 7.5
>>> valor = 85
>>> horas * valor
637.5
>>> nome = "manual"
>>> nome.upper()
'MANUAL'
>>> len(nome)
6
>>> type(horas)
<class 'float'>
>>> for letra in "py":
...     print(letra, letra.upper())
...
p P
y Y
>>>
```

O nome **REPL** descreve o que aquele `>>>` faz, e está em Figura 4.1: *Read, Eval, Print, Loop* — lê a linha, avalia, imprime o resultado, volta a esperar.

Quatro detalhes dessa sessão valem atenção, porque são a diferença entre usar o REPL e brigar com ele.

4. O REPL, os scripts e o primeiro programa

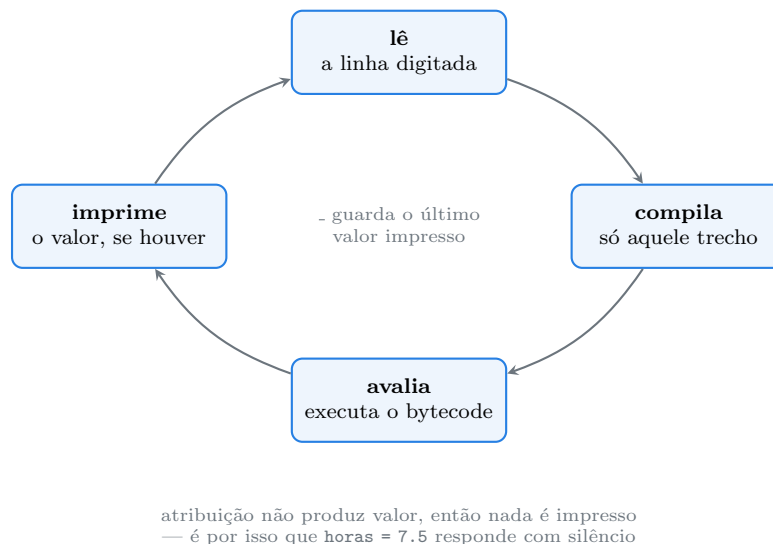


Figura 4.1.: O ciclo do REPL. A etapa de compilação é a mesma do Capítulo 2, só que sobre uma linha em vez de um arquivo.

Expressão imprime, atribuição não. `horas * valor` respondeu 637.5; `horas = 7.5` não respondeu nada. Não é economia de tinta: atribuição é um comando e não produz valor, então não há o que imprimir. Quando você digita algo e o REPL fica calado, ele está dizendo “isto não é uma expressão”.

O `_` guarda o último resultado. `2 + 3` deu 5, e `_ * 10` deu 50 sem que eu tivesse dado nome a nada. É útil para encadear exploração — calcule, veja, use o resultado no passo seguinte. E é uma armadilha em script: `_` só existe no modo interativo, e por convenção significa outra coisa em código (o valor que se descarta).

A resposta mostra o repr, não o texto. `nome.upper()` respondeu `'MANUAL'` com aspas, porque o REPL exibe a *representação* do objeto — a forma que você digitaria para recriá-lo. Já `print` mostra o texto sem aspas. É a razão de `<class 'float'>` aparecer com esse formato em `type(horas)`.

Bloco pede linha em branco. Quando você digita `for letra in "py":` e aperta Enter, o prompt muda para `...`, indicando que o comando não terminou. Para fechar o bloco, uma linha vazia. Esquecer isso e ficar preso no `...` é a primeira frustração de quase todo mundo.

4.1.1. O REPL do 3.13

O modo interativo foi reescrito no Python 3.13, e a diferença é grande o suficiente para valer nota. O REPL novo tem histórico que atravessa comandos de várias linhas, colagem de bloco inteiro sem que a indentação automática estrague o texto, prompt colorido e — o que mais economiza irritação — comandos que funcionam **sem parênteses**.

Isso não é um detalhe cosmético; é uma lista fixa dentro da implementação. Ela está em `_pyrepl/simple_interact.py`, no interpretador que você tem instalado:

```
grep -n -A 7 "^REPL_COMMANDS" /usr/lib/python3.13/_pyrepl/simple_interact.py
```



```

75:REPL_COMMANDS = {
76-     "exit": _sitebuiltins.Quitter('exit', ''),
77-     "quit": _sitebuiltins.Quitter('quit', ''),
78-     "copyright": _sitebuiltins._Printer('copyright', sys.copyright),
79-     "help": _sitebuiltins._Helper(),
80-     "clear": _clear_screen,
81-     "\x1a": _sitebuiltins.Quitter('\x1a', ''),
82-}

```

São cinco nomes — `exit`, `quit`, `copyright`, `help`, `clear` — mais o caractere de fim de arquivo do Windows. Antes do 3.13, digitar `exit` sozinho respondia com a dica irritante `Use exit() or Ctrl-D (i.e. EOF) to exit`, e você tinha que digitar de novo com os parênteses. Se quiser ver o comportamento antigo, force o REPL básico:

```
PYTHON_BASIC_REPL=1 python3
```

```

>>> exit
Use exit() or Ctrl-D (i.e. EOF) to exit
>>>

```

Uma sutileza que a implementação revela: a interceptação só vale se o nome **não** for uma variável sua. A linha que decide é `if statement in console.locals or statement not in REPL_COMMANDS`, ou seja, o seu `exit = 3` vence o comando. Faz sentido e evita surpresa.

Para pedir informação sem sair do REPL, dois embutidos resolvem quase tudo. O `help` mostra a documentação:

```

from orcamento import total
help(total)

```

```
Help on function total in module orcamento:
```

```

total(horas: float, valor_hora: float) -> float
    Devolve o valor total de `horas` trabalhadas a `valor_hora` por hora.

```

Note que o `help` mostrou a assinatura com as anotações de tipo e a primeira linha da documentação da função. E o `dir` lista o que um objeto sabe fazer:

```
print([m for m in dir("py") if not m.startswith("_")][:12])
```

```

['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith',
↪  'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum']

```

Filtrei os nomes que começam com underscore para deixar visíveis os métodos de uso comum — os outros são o protocolo interno, assunto do Volume 8. Este par, `dir` para descobrir o que existe e `help` para descobrir como se usa, resolve a maior parte das dúvidas sem abrir o navegador.

4.2. O script: onde se repete

O REPL é excelente para descobrir e péssimo para guardar: feche o terminal e tudo se perde. O orçamento do cliente é uma tarefa recorrente, então ele merece um arquivo. Este é o primeiro programa completo do manual, `orcamento.py`:

```
#!/usr/bin/env python3
"""Calcula o valor de um serviço a partir de horas e valor por hora."""

import sys

def total(horas: float, valor_hora: float) -> float:
    """Devolve o valor total de `horas` trabalhadas a `valor_hora` por hora."""
    return horas * valor_hora

def main() -> int:
    if len(sys.argv) != 3:
        print(f"uso: {sys.argv[0]} HORAS VALOR_HORA", file=sys.stderr)
        return 2
    horas = float(sys.argv[1])
    valor_hora = float(sys.argv[2])
    print(f"{horas} h x R$ {valor_hora:.2f} = R$ {total(horas,
        ↪ valor_hora):.2f}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

```
python3 orcamento.py 7.5 85
```

```
7.5 h x R$ 85.00 = R$ 637.50
```

São vinte linhas e cada bloco tem uma razão de ser. Vamos por partes, porque este esqueleto vai reaparecer em todo programa de linha de comando deste manual.

A primeira linha, o *shebang*. `#!/usr/bin/env python3` só tem efeito em sistemas Unix e só quando o arquivo é executável. Ela diz ao sistema operacional qual interpretador usar:

```
chmod +x orcamento.py
./orcamento.py 3 120
```

```
3.0 h x R$ 120.00 = R$ 360.00
```

Repare que se escreve `#!/usr/bin/env python3` e não `#!/usr/bin/python3`. A forma com `env` procura o `python3` no `PATH`, o que faz o script respeitar o ambiente virtual ativo — exatamente

o mecanismo do Capítulo 3. A forma com caminho fixo ignora o `venv` e vai sempre no Python do sistema, que quase nunca é o que você quer.

A segunda linha, a *docstring* do módulo. Um texto entre três aspas no começo do arquivo não é comentário: é um valor, guardado no atributo `__doc__` e legível por ferramentas.

```
python3 -c "import orcamento; print(orcamento.__doc__)"
```

```
Calcula o valor de um servico a partir de horas e valor por hora.
```

`sys.argv`, os argumentos da linha de comando. É uma lista de textos em que a posição zero é o nome do programa e o resto são os argumentos. São **textos**, sempre: `float(sys.argv[1])` é obrigatório, e é por isso que passar `abc` no lugar de um número derruba o programa com `ValueError`. Tratar isso bem é assunto do Capítulo 7 e, com ferramenta apropriada, do Volume 12.

Erro vai para `stderr`, e o programa devolve um código. Quando faltam argumentos, a mensagem sai por `sys.stderr` e o programa termina com 2:

```
python3 orcamento.py 2>/dev/null      # so o stdout
python3 orcamento.py 2>&1 >/dev/null  # so o stderr
python3 orcamento.py; echo $?
```

```
uso: orcamento.py HORAS VALOR_HORA
2
```

O primeiro comando não imprimiu nada — o `stdout` estava vazio. O segundo mostrou a mensagem de uso. Essa separação é o que permite `python3 orcamento.py 7.5 85 > planilha.txt` gravar só o resultado, com os erros continuando visíveis na tela. E o código de saída — 0 para sucesso, diferente de zero para falha — é como o shell sabe se deu certo, o que faz `&&` e `||` funcionarem.

Funções em vez de código solto. `total` faz a conta e `main` cuida da conversa com o mundo. A separação parece exagero em vinte linhas, mas é o que permite reaproveitar a conta sem executar o programa:

```
python3 -c "from orcamento import total; print(total(2, 50))"
```

```
100
```

Isso funciona por causa da última peça, que merece seção própria.

4.3. `if __name__ == "__main__"`

Todo módulo Python tem uma variável `__name__`. O valor dela depende de **como** o arquivo foi carregado, e é a única diferença entre “programa” e “biblioteca” na linguagem. Dois arquivos mostram isso. Primeiro `quemsou.py`:

4. O REPL, os scripts e o primeiro programa

```
print("modulo quemsou, __name__ =", __name__)

if __name__ == "__main__":
    print("  -> rodando como programa principal")
else:
    print("  -> fui importado por outro modulo")
```

E `usa.py`, que só o importa:

```
import quemsou

print("modulo usa, __name__ =", __name__)
```

Rodando o primeiro direto:

```
python3 quemsou.py
```

```
modulo quemsou, __name__ = __main__
-> rodando como programa principal
```

Rodando o segundo, que importa o primeiro:

```
python3 usa.py
```

```
modulo quemsou, __name__ = quemsou
-> fui importado por outro modulo
modulo usa, __name__ = __main__
```

Três coisas de uma vez. O módulo importado recebeu o próprio nome, `quemsou`. O arquivo que foi chamado na linha de comando recebeu `__main__` — e note que agora é o `usa.py` que tem esse valor. E o `print` do topo do `quemsou.py` **executou durante o import**: importar um módulo roda o código do nível superior dele, inteiro.

Esse último ponto é a razão de o guarda existir. Figura 4.2 mostra os dois caminhos.

Sem aquele `if`, o `from orcamento import total` do exemplo anterior teria rodado o `main()`, que olharia o `sys.argv` do processo que importou, não acharia dois argumentos e mataria o programa com código 2. Com o guarda, o mesmo arquivo é as duas coisas: um programa quando chamado e uma biblioteca quando importado.

Vale desfazer uma confusão comum: `if __name__ == "__main__"` não é uma declaração de “função principal” como o `main` de C. É uma condição normal, avaliada quando o módulo carrega, sobre uma variável normal. Não há nada de especial nela além da convenção — e a convenção é universal.

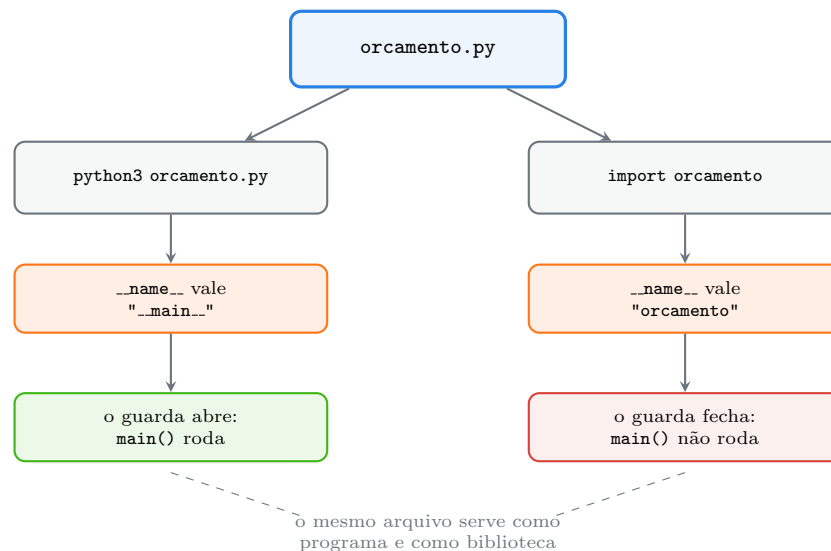


Figura 4.2.: Sem o guarda, importar o módulo executaria o programa.

4.4. As cinco formas de rodar código Python

Além do REPL e do script, existem outras. Cada uma tem um uso próprio:

Forma	Para quê
<code>python3</code>	REPL: explorar, testar ideia, inspecionar objeto
<code>python3 arquivo.py</code>	rodar um programa que você escreveu
<code>python3 -c "código"</code>	uma linha só, direto no shell ou num script de shell
<code>python3 -m modulo</code>	rodar um módulo instalado como se fosse programa
<code>python3 -i arquivo.py</code>	rodar o arquivo e cair no REPL com o estado dele

O `-m` merece destaque porque resolve um problema real. Ele procura o módulo no caminho de importação e o executa com `__name__` valendo `__main__`:

```
python3 -m quemsou
```

```
modulo quemsou, __name__ = __main__
-> rodando como programa principal
```

Muitos módulos da biblioteca padrão são úteis assim, sem que você escreva nada:

```
python3 -m json.tool <<< '{"cliente":"sitio","quedas":4}'
```

4. O REPL, os scripts e o primeiro programa

```
{  
    "cliente": "sitio",  
    "quedas": 4  
}
```

O `-m` também é a forma correta de chamar o `pip` (`python3 -m pip`, como no Capítulo 3), porque garante que o `pip` executado é o do interpretador que você invocou, e não outro que esteja no `PATH`.

Cuidado com uma diferença sutil no `-c`: o `sys.argv` começa com a string `'-c'`, não com um nome de arquivo:

```
python3 -c "import sys; print(sys.argv)" um dois tres
```

```
['-c', 'um', 'dois', 'tres']
```

Já o `-i` é a ponte entre os dois mundos deste capítulo, e a ferramenta de depuração mais subestimada que existe. Ele roda o arquivo e, em vez de terminar, abre o REPL com tudo que o arquivo definiu ainda vivo — inclusive quando o arquivo morreu com uma exceção. Você cai no prompt com as variáveis no estado em que estavam e pode olhar uma por uma.

4.5. Jeito Pythônico

O esqueleto de `orcamento.py` é a forma canônica de escrever um programa de linha de comando em Python, e cada peça dele tem um antipadrão correspondente.

```
# antipadrão: código solto no nível do módulo  
import sys  
  
horas = float(sys.argv[1])  
valor = float(sys.argv[2])  
print(horas * valor)
```

```
# pythônico: função + guarda  
def main() -> int:  
    ...  
    return 0  
  
if __name__ == "__main__":  
    sys.exit(main())
```

A versão de cima funciona e é o que quase todo mundo escreve primeiro. Ela tem três problemas que só aparecem depois: não pode ser importada sem executar, não pode ser testada (o Volume 11 vai querer chamar `total` num teste), e não tem como devolver código de saída diferente de zero sem espalhar `sys.exit` pelo meio.

Sobre o guarda, a forma idiomática é `sys.exit(main())` e não só `main()`. Assim o valor devolvido pela função vira o código de saída do processo, e você escreve um único `return` em vez de chamar `sys.exit` em cada ramo.

Três outros pontos de estilo:

Mensagem de erro vai para `stderr`. `print(..., file=sys.stderr)` e não `print(...)`. Quem usa o seu programa num pipe agradece.

`_` não sai do REPL. No modo interativo ele é o último resultado; em código, a convenção é o oposto — `_` marca o valor que você está descartando de propósito, como em `for _ in range(3):` quando o índice não importa.

Duas aspas triplas no topo, não um comentário. Docstring é dado, comentário é ruído para a máquina. Ferramentas leem `__doc__`; nenhuma lê o `#`.

Sobre versões:

- **Python 3.13** — REPL novo (PEP 762), com `exit`, `quit`, `help`, `clear` e `copyright` sem parênteses, histórico multilinha e colagem de bloco. `PYTHON_BASIC_REPL=1` volta ao antigo.
- **Python 3.11** — `-P` desliga a inclusão automática do diretório do script no caminho de importação, e `PYTHONSAFEPATH` faz o mesmo por variável de ambiente. Interessa a quem já foi mordido por um arquivo local chamado `json.py` sombreando o módulo da biblioteca padrão.
- **Python 3.12** — o `sys.last_exc` guarda a última exceção no modo interativo, mais conveniente que os três atributos antigos.

Por fim, a divisão de trabalho que resume o capítulo: **use o REPL para perguntas e o arquivo para respostas**. Se você digitou a mesma coisa no REPL duas vezes, é hora de abrir um arquivo. Se você está editando um arquivo para descobrir o nome de um método, é hora de abrir o REPL.

4.6. Laboratório

Objetivo: usar o REPL para descobrir, transformar a descoberta num programa completo e verificar as duas identidades do mesmo arquivo. Só biblioteca padrão.

1. Ambiente.

```
mkdir -p ~/lab04 && cd ~/lab04
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. Explore no REPL. Abra `python` e reproduza a sessão da abertura do capítulo. Depois experimente três coisas fora do roteiro:

```
>>> 10 / 3
>>> 10 // 3
>>> 10 % 3
```

4. O REPL, os scripts e o primeiro programa

Anote os três resultados. Divisão em Python devolve `float` mesmo entre inteiros — e isso derruba quem chega de C. O Volume 2 trata do assunto.

3. Confirme que atribuição não imprime. Digite `x = 5` e depois `x`. A primeira linha responde com silêncio, a segunda com 5. Agora digite `x == 5`: isso é uma expressão, e responde `True`.

4. Use `dir` e `help` para descobrir sozinho. Sem consultar nada externo, descubra o que faz o método `zfill` de um texto:

```
>>> help("".zfill)
>>> "42".zfill(5)
```

Este é o hábito a construir: a documentação está dentro do interpretador.

5. Teste os comandos sem parênteses. Digite `help` sozinho e aperte Enter — no 3.13 entra no modo de ajuda interativo (saia com Enter em branco). Depois compare com o REPL antigo:

```
PYTHON_BASIC_REPL=1 python
```

Digite `exit` sem parênteses. Você deve ver `Use exit() or Ctrl-D (i.e. EOF) to exit`. Saia com `exit()` e reabra o `python` normal, onde `exit` sozinho funciona.

6. Escreva o programa. Crie `orcamento.py` com o conteúdo mostrado no capítulo e rode:

```
python orcamento.py 7.5 85
```

```
7.5 h x R$ 85.00 = R$ 637.50
```

7. Exercite as bordas. Três execuções que **falham de propósito**, para você ver o que cada uma faz:

```
python orcamento.py           # faltam argumentos
python orcamento.py 7.5       # falta um
python orcamento.py 7.5 abc   # o segundo não é número
```

As duas primeiras devem imprimir a mensagem de uso e sair com código 2 — confirme com `echo $?`. A terceira estoura um `ValueError` com traceback, porque `float("abc")` não tem resposta. Guarde essa diferença: erro previsto é mensagem, erro imprevisto é traceback. O Capítulo 7 é sobre isso.

8. Separe os fluxos.

```
python orcamento.py 7.5 85 > saida.txt
cat saida.txt
python orcamento.py > vazio.txt
cat vazio.txt
```

O primeiro `cat` mostra o resultado; o segundo mostra um arquivo vazio, porque a mensagem de uso foi para `stderr` e continuou aparecendo na tela.

9. Torne executável. Em Linux ou macOS:


```
chmod +x orcamento.py
./orcamento.py 3 120
```

```
3.0 h x R$ 120.00 = R$ 360.00
```

No Windows não há `chmod`; o equivalente é a associação de extensão feita pelo instalador, e `py orcamento.py 3 120` sempre funciona.

10. Prove as duas identidades. Crie `quemsou.py` e `usa.py` como no capítulo e rode os dois, mais o `-m`:

```
python quemsou.py
python usa.py
python -m quemsou
```

Confirme que `__name__` vale `__main__` no primeiro e no terceiro, e `quemsou` no segundo.

11. Importe sem executar.

```
python -c "from orcamento import total; print(total(2, 50))"
```

```
100
```

Agora **apague** as duas últimas linhas do `orcamento.py` (o `if` e o `sys.exit`), acrescente `main()` solto no lugar e repita o comando acima. Você deve ver a mensagem de uso e o programa morrer — porque o `main` rodou durante o `import` e não achou os argumentos. Desfaça a mudança. Este é o passo que faz o guarda parar de ser superstição.

12. Caia no REPL com o estado.

```
python -i orcamento.py 7.5 85
```

Não se assuste com o que aparece antes do prompt:

```
File "/tmp/lab04/orcamento.py", line 23, in <module>
    sys.exit(main())
    ~~~~~~^~~~~~
SystemExit: 0
```

Isso **não** é um erro. O `sys.exit` funciona levantando a exceção `SystemExit`, e normalmente o interpretador a intercepta em silêncio para encerrar o processo. Com `-i`, como o processo não vai terminar, ela é exibida como qualquer outra exceção — e o 0 no fim é o código de saída de sucesso. No seu terminal o traceback sai colorido, novidade do 3.13.

Já no prompt, tudo do arquivo está carregado. Experimente:

4. O REPL, os scripts e o primeiro programa

```
>>> total(1, 1)
1
>>> sys.argv
['orcamento.py', '7.5', '85']
>>> horas
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    horas
NameError: name 'horas' is not defined
```

`total` e `sys` existem porque são do nível superior do módulo; `horas` não, porque era variável local do `main`. É exatamente a distinção de escopo que o Volume 3 vai formalizar. Saia com `exit`.

13. Use um módulo da biblioteca padrão como programa.

```
echo '{"cliente":"sitio","quedas":4}' | python -m json.tool
python -m this
python -m timeit "sum(range(100))"
```

14. Feche com `deactivate`.

4.7. Resumo

- O REPL é para descobrir; o arquivo é para repetir. Digitou a mesma coisa duas vezes no REPL: abra um arquivo.
- No REPL, expressão imprime e atribuição não; `_` guarda o último valor impresso; a resposta é o `repr` do objeto, não o texto; bloco só fecha com linha em branco.
- O REPL do 3.13 aceita `exit`, `quit`, `help`, `clear` e `copyright` sem parênteses — uma lista fixa em `_pyrepl/simple_interact.py`, que perde para uma variável sua de mesmo nome. `PYTHON_BASIC_REPL=1` volta ao comportamento antigo.
- Um programa de linha de comando tem shebang com `env`, docstring de módulo, a lógica em funções, `sys.argv` convertido explicitamente, erro em `sys.stderr` e código de saída via `sys.exit(main())`.
- `__name__` vale `"__main__"` no arquivo que foi chamado e o nome do módulo quando importado. O guarda `if __name__ == "__main__"` é o que permite ao mesmo arquivo ser programa e biblioteca — sem ele, importar executa.
- Cinco formas de rodar: REPL, arquivo, `-c` para uma linha, `-m` para módulo instalado (é assim que se chama o `pip`), `-i` para cair no REPL com o estado do arquivo.

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

O código roda no terminal. No editor, a mesma linha aparece sublinhada de amarelo com “import não pode ser resolvido”. Você roda de novo no terminal: funciona. Aperta o botão de executar do editor: `ModuleNotFoundError`. Abre um notebook para testar e lá o import funciona, mas a função que você acabou de corrigir no arquivo continua devolvendo o valor antigo.

Nada disso é bug de ferramenta. São três programas diferentes perguntando a três interpretadores diferentes, e a solução para todos os casos é a mesma. Este capítulo monta o ambiente de trabalho de forma que essa classe de problema deixe de existir.

5.1. Uma regra que resolve a maioria dos problemas

O ambiente virtual do Capítulo 3 é um interpretador com um conjunto de pacotes. O terminal sabe qual usar porque a ativação mexeu no `PATH`. O editor não vê o seu `PATH`, e o kernel de um notebook é um terceiro processo, iniciado por quem sabe qual Python. Figura 5.1 mostra o arranjo que funciona.

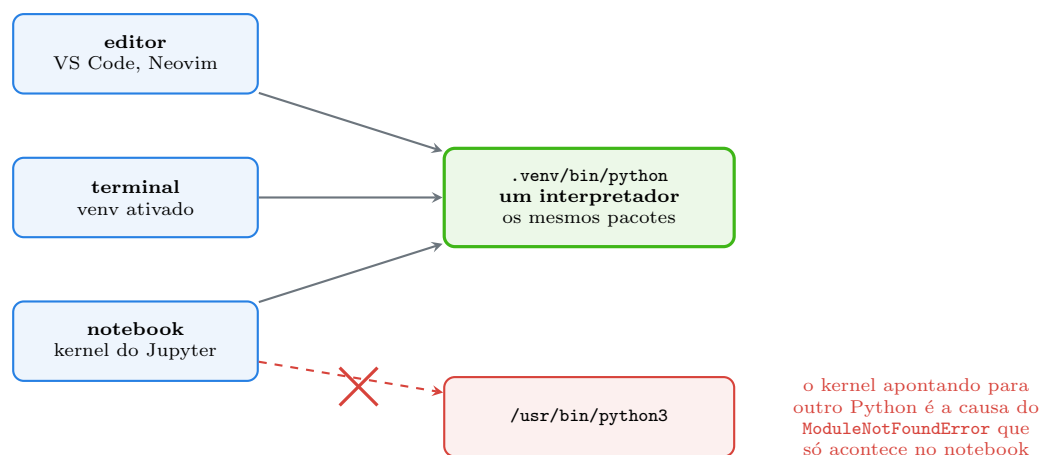


Figura 5.1.: Editor, terminal e notebook precisam apontar para o mesmo interpretador.

A regra é: **um ambiente virtual por projeto, e as três ferramentas apontando para ele**. Quando alguma coisa funciona num lugar e não no outro, a primeira pergunta não é “qual é o bug”, é *qual Python cada um está usando*. E a resposta é sempre a mesma linha, rodada de dentro de cada ferramenta:

```
import sys
print(sys.executable)
```

Se o terminal responde `/home/voce/projeto/.venv/bin/python` e o notebook responde `/usr/bin/python3`, você achou o problema em dez segundos, sem investigar mais nada.

5.2. O terminal

O terminal não é um detalhe de acompanhamento — é onde o Python é executado de verdade, em servidor, em contêiner e em automação. Vale saber o mínimo bem.

Quatro coisas cobrem o dia a dia. **Navegar:** `cd`, `ls`, `pwd`. **Ativar o ambiente:** `source .venv/bin/activate`, e conferir com `which python`. **Rodar:** `python arquivo.py`, `python -m modulo`, `python -c` — as formas do Capítulo 4. E **redirecionar:** `>` para gravar a saída, `2>` para gravar o erro, `|` para passar a saída de um programa como entrada de outro.

Esse último merece um exemplo, porque é o que torna um script Python uma peça reutilizável em vez de um programa isolado:

```
python orcamento.py 7.5 85 | tee ultimo-orcamento.txt
```

O programa não sabe que está num pipe, e é justamente por isso que funciona: ele escreveu no `stdout`, como manda a convenção do Capítulo 4, e o shell cuidou do resto.

Duas variáveis de ambiente valem conhecer agora. `PYTHONPATH` acrescenta diretórios ao caminho de busca de módulos — útil em emergência, e um cheiro de problema de estrutura quando aparece em uso permanente (o Volume 5 resolve isso direito). E `PYTHONDONTWRITEBYTECODE=1`, que desliga o `__pycache__` do Capítulo 2.

No **Windows**, o equivalente é o PowerShell, com `Activate.ps1` no lugar do `source`. O terminal integrado do VS Code herda a ativação, o que resolve metade dos problemas deste capítulo de graça.

5.3. O editor

Qualquer editor serve para escrever Python. O que muda a produtividade é ter cinco coisas funcionando: realce de sintaxe, autocompletar que conhece os seus objetos, o erro apontado antes de rodar, formatação automática ao salvar e um depurador. Este manual usa o **VS Code** nos exemplos por ser gratuito, multiplataforma e o mais comum — mas nada aqui depende dele.

O essencial da configuração é uma extensão e uma escolha. A extensão é a **Python**, da Microsoft, que traz junto o servidor de linguagem Pylance. A escolha é o interpretador: `Ctrl+Shift+P`, comando **Python: Select Interpreter**, e aponte para `./.venv/bin/python`. Isso é o que faz o sublinhado amarelo desaparecer, porque o editor passa a olhar os pacotes do ambiente certo.

Melhor do que escolher na mão é deixar a escolha registrada no projeto. Crie `.vscode/settings.json`:

```
{
  "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
  "editor.formatOnSave": true,
  "editor.rulers": [88],
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true,
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.codeActionsOnSave": {
      "source.organizeImports": "explicit"
    }
  }
}
```

```
}
}
```

Esse arquivo **vai para o Git**. É a diferença entre “funciona na minha máquina” e “funciona para quem clonar o repositório”: qualquer pessoa que abrir o projeto recebe o mesmo interpretador, a mesma largura de linha e a mesma formatação ao salvar. No Windows, o caminho do interpretador é `${workspaceFolder}/.venv/Scripts/python.exe`.

O 88 dos **rulers** não é arbitrário: é a largura de linha que o formatador usa por padrão, e a régua na tela evita a briga de reformatação a cada salvamento.

5.3.1. O depurador, que substitui o `print`

O recurso mais subutilizado do editor é o depurador. Colocar `print` para descobrir o valor de uma variável funciona, mas é lento: você edita, roda, lê, edita de novo, e no fim precisa lembrar de apagar tudo.

Com o depurador, você clica na margem esquerda para criar um ponto de parada, aperta F5 e o programa congela naquela linha com todas as variáveis visíveis num painel. Dá para avançar linha por linha, entrar dentro de uma função, e — o que mais importa — digitar expressões no console de depuração usando o estado real daquele instante.

O equivalente sem editor nenhum já está na linguagem, desde o Python 3.7:

```
def total(horas, valor_hora):
    breakpoint()
    return horas * valor_hora
```

`breakpoint()` para a execução e abre o `pdb`, o depurador de linha de comando da biblioteca padrão. É o mesmo recurso, disponível em servidor por SSH, onde não há editor gráfico. O Volume 11 tem um capítulo sobre isso; por ora, saiba que existe e que a variável `PYTHONBREAKPOINT=0` desliga todos os `breakpoint()` de uma vez, o que é útil quando você esqueceu um.

5.4. Linter e formatador: *ruff*

Duas ferramentas diferentes com nomes parecidos. O **formatador** arruma a aparência: espaços, quebras de linha, aspas. Ele não muda o que o programa faz e não tem opinião sobre a sua lógica. O **linter** procura problemas: `import` que não é usado, variável definida e nunca lida, nome fora da convenção, construção que tem uma forma melhor.

O **ruff** faz as duas coisas, é escrito em Rust e é rápido o bastante para rodar a cada salvamento. Instale no ambiente do projeto:

```
python -m pip install ruff
ruff --version
```

```
ruff 0.16.2
```

Vamos dar a ele um arquivo com problemas de propósito, `bagunca.py`:

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

```
import os
import sys
from pathlib import Path

def Media( valores ):
    total=0
    for i in range(len(valores)):
        total = total + valores[i]
    return total/len(valores)

x = Media([1,2,3])
print( "media:",x )
```

Primeiro o linter, com as regras que vêm ligadas por padrão:

```
ruff check --output-format concise bagunca.py
```

```
bagunca.py:1:1: I001 [*] Import block is un-sorted or un-formatted
bagunca.py:1:8: F401 [*] `os` imported but unused
bagunca.py:2:8: F401 [*] `sys` imported but unused
bagunca.py:3:21: F401 [*] `pathlib.Path` imported but unused
Found 4 errors.
[*] 4 fixable with the `--fix` option.
```

Cada linha traz arquivo, linha, coluna, código da regra e a explicação. O [*] marca o que a ferramenta sabe corrigir sozinha. As regras padrão são conservadoras; ligando mais famílias, aparece mais:

```
ruff check --output-format concise --select E,F,I,N,UP,B,SIM bagunca.py
```

```
bagunca.py:1:1: I001 [*] Import block is un-sorted or un-formatted
bagunca.py:1:8: F401 [*] `os` imported but unused
bagunca.py:2:8: F401 [*] `sys` imported but unused
bagunca.py:3:21: F401 [*] `pathlib.Path` imported but unused
bagunca.py:5:5: N802 Function name `Media` should be lowercase
Found 5 errors.
[*] 4 fixable with the `--fix` option.
```

Entrou o N802: `Media` deveria ser `media`, porque `CamelCase` é para classes. E note que ele **não** é corrigível automaticamente — renomear uma função exige saber quem a chama, e isso a ferramenta não decide por você.

Agora o formatador, mostrando o que mudaria antes de mudar:

```
ruff format --diff bagunca.py
```

```

--- bagunca.py
+++ bagunca.py
@@ -2,11 +2,13 @@
     import sys
     from pathlib import Path

-def Media( valores ):
-    total=0
+
+def Media(valores):
+    total = 0
+    for i in range(len(valores)):
+        total = total + valores[i]
-    return total/len(valores)
+    return total / len(valores)
+

-x = Media([1,2,3])
-print( "media:",x )
+x = Media([1, 2, 3])
+print("media:", x)

```

Espaços dentro dos parênteses removidos, espaço em volta dos operadores acrescentado, duas linhas em branco antes da função — tudo o que a PEP 8 pede e ninguém quer fazer à mão. Aplicando as duas ferramentas:

```

ruff check --select E,F,I,N,UP,B,SIM --fix --output-format concise bagunca.py
ruff format bagunca.py

```

```

bagunca.py:2:5: N802 Function name `Media` should be lowercase
Found 4 errors (3 fixed, 1 remaining).
1 file reformatted

```

O resultado:

```

def Media(valores):
    total = 0
    for i in range(len(valores)):
        total = total + valores[i]
    return total / len(valores)

x = Media([1, 2, 3])
print("media:", x)

```

Vale olhar com atenção para **o que continua errado**. Os três imports inúteis saíram, o espaçamento está correto, mas o laço percorrendo a lista por índice — o antipadrão do Capítulo 1 — passou intacto, e o nome `Media` só recebeu um aviso. Nenhuma ferramenta escreve `sum(valores) / len(valores)` no seu lugar.

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

Isso define o limite da automação, e é um limite honesto: **linter e formatador cuidam do que é mecânico para liberar a sua atenção para o que não é**. Eles não substituem saber a linguagem. Um arquivo aprovado pelo **ruff** pode ser um programa ruim.

A configuração do **ruff** mora no `pyproject.toml`, na raiz do projeto, e também vai para o Git:

```
[tool.ruff]
line-length = 88
target-version = "py313"

[tool.ruff.lint]
select = ["E", "F", "I", "N", "UP", "B", "SIM"]
```

Com isso, **ruff check** e **ruff format** já saem com as regras do projeto, sem `--select` na linha de comando, e o editor lê o mesmo arquivo. Uma fonte de verdade para você, para a equipe e para a integração contínua — assunto do Volume 16.

5.5. O notebook

Um notebook Jupyter é uma sequência de células que você executa na ordem que quiser, com o resultado — texto, tabela, gráfico — logo abaixo de cada uma. Instale no ambiente do projeto:

```
python -m pip install jupyterlab
jupyter --version
```

```
Selected Jupyter core packages...
IPython          : 9.16.1
ipykernel        : 7.3.0
jupyter_client   : 8.9.1
jupyter_core     : 5.9.1
jupyter_server   : 2.20.0
jupyterlab       : 4.6.3
nbclient         : 0.11.0
nbconvert        : 7.17.1
nbformat         : 5.11.0
```

Abra com **jupyter lab**, que sobe um servidor local e abre o navegador. O VS Code também abre arquivos `.ipynb` direto, o que costuma ser mais confortável.

Notebook é excelente para **explorar dados**: carregar um arquivo, olhar, filtrar, desenhar um gráfico, mudar de ideia. O ciclo é curto e o estado fica vivo entre as células, então você não recarrega o arquivo de 200 MB a cada tentativa. Os volumes de dados e visualização deste manual vivem nesse modo.

E notebook tem uma armadilha que precisa ser entendida antes do primeiro uso, não depois: **o estado depende da ordem em que você executou, não da ordem em que as células aparecem na tela**.

5.5.1. A ordem de execução

Imagine três células, nesta ordem no arquivo:

```
# célula A
taxa = 0.10
print("taxa definida:", taxa)
```

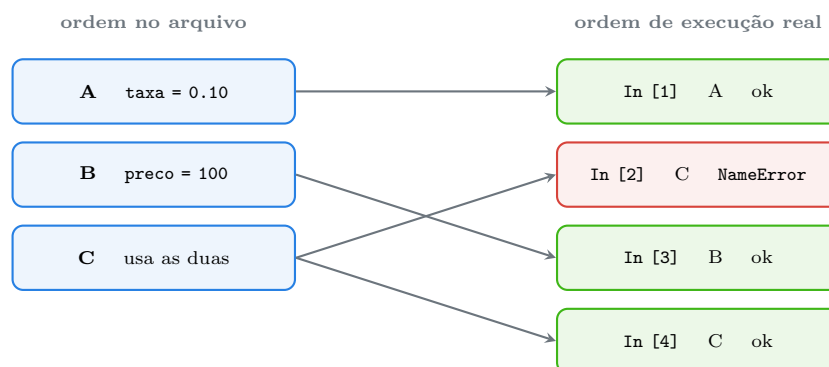
```
# célula B
preco = 100
print("preco definido:", preco)
```

```
# célula C
print("total:", preco * (1 + taxa))
```

Agora suponha que você clicou nelas na ordem A, C, B, C — o que é absolutamente normal quando se está explorando. Foi o que rodamos num kernel de verdade:

```
clique na celula A -> In [1]
    taxa definida: 0.1
clique na celula C -> In [2]
    NameError: name 'preco' is not defined
clique na celula B -> In [3]
    preco definido: 100
clique na celula C -> In [4]
    total: 110.00000000000001
```

O número entre colchetes é o **contador de execução**, e é a informação mais importante da tela. Ele não é o número da célula: é a ordem em que as coisas aconteceram. Figura 5.2 põe as duas ordens lado a lado.



o contador entre colchetes conta **execuções**, não células — um notebook cujos números não sejam 1, 2, 3... em sequência não roda igual para outra pessoa

Figura 5.2.: A mesma célula C falha e funciona, dependendo do que rodou antes.

Duas consequências práticas. A primeira: um notebook com contadores fora de ordem **não é reproduzível**. Ele funciona na sua tela porque o kernel guarda variáveis que não estão mais

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

em nenhuma célula, ou porque uma célula foi executada com um código que você depois editou. Passe para outra pessoa e não roda.

A segunda: o hábito que evita tudo isso é **Restart Kernel and Run All Cells** antes de considerar o trabalho pronto. Isso apaga o estado, executa tudo de cima para baixo e prova que o notebook é o que ele parece ser. Se falhar, era mentira.

Como brinde, aquele 110.000000000000001 no lugar de 110.0 não é erro do Jupyter: é o comportamento do ponto flutuante binário, que tem um capítulo inteiro reservado no Volume 2.

Notebook é ruim para outras coisas, e vale dizer quais. Código que vai para produção, biblioteca reutilizável, qualquer coisa que precise de teste automatizado ou de revisão em *pull request* — nada disso pertence a um `.ipynb`. O formato é JSON com a saída embutida, o que faz o `diff` do Git ficar ilegível e o histórico engordar com imagens. A divisão saudável é: **explore no notebook, mude para arquivo .py assim que o código tiver valor além daquela sessão.**

5.6. Outras opções

PyCharm é um ambiente completo, com o depurador mais confortável do ecossistema e refatoração de verdade. A edição Community é gratuita. Custa mais memória e tem mais curva de aprendizado.

Neovim com `pylsp` ou `pyright` entrega o mesmo autocompletar e diagnóstico dentro do terminal, o que importa em servidor remoto. Exige montar a configuração.

Spyder imita o Matlab e agrada quem vem da computação científica.

E `python -i` do Capítulo 4 continua sendo o ambiente mais rápido de todos para uma pergunta pequena.

5.7. Jeito Pythonico

A convenção sobre ambiente de trabalho é curta: **versione a configuração, nunca o ambiente.**

```
# vai para o Git
pyproject.toml          # regras do ruff, dependências, metadados
.vscode/settings.json   # interpretador, formatação ao salvar
.gitignore

# não vai para o Git
.venv/                  # derivado, com caminhos absolutos dentro
__pycache__/            # cache de bytecode
*.ipynb_checkpoints/    # rascunhos do Jupyter
```

O `.gitignore` mínimo de um projeto Python:

```
.venv/
__pycache__/
*.py[cod]
.ipynb_checkpoints/
.ruff_cache/
```

Três antipadrões comuns:

```
# antipadrão: instalar a ferramenta no ambiente do projeto sem precisar
python -m pip install ruff black isort flake8 pylint mypy

# pythônico: uma ferramenta que faz o trabalho de várias
python -m pip install ruff
```

O `ruff` substitui `flake8`, `isort`, `black` e boa parte do `pylint` com uma configuração só. Empilhar ferramentas com opiniões diferentes sobre a mesma linha gera conflito de formatação a cada salvamento.

```
# antipadrão: depurar por print e esquecer de apagar
print("AQUI 1", valores)
print(">>>", total)

# pythônico: ponto de parada
breakpoint()
```

```
# antipadrão: um notebook de 200 células que é o produto final
analise.ipynb

# pythônico: notebook para explorar, .py para o que ficou de pé
explorar.ipynb -> src/relatorio.py + tests/test_relatorio.py
```

Sobre versões e ferramentas:

- **Python 3.7** — `breakpoint()` embutido (PEP 553), com `PYTHONBREAKPOINT` para desligar ou trocar o depurador.
- **Python 3.11** — `PYTHONSAFEPATH` evita que um arquivo local com nome de módulo da biblioteca padrão sobre escreva o original.
- **ruff** — pede `target-version` no `pyproject.toml` para saber quais modernizações propor. Com `py313`, ele sugere sintaxe que não existia em versões anteriores; com `py39`, não.
- **PEP 8** é a referência de estilo, e a largura de linha de 88 caracteres é convenção de ferramenta, não da PEP (que fala em 79). O Volume 11 trata do assunto.

5.8. Laboratório

Objetivo: montar um projeto com ambiente, configuração versionada, linter, formatador e notebook, e provar que as três ferramentas usam o mesmo interpretador.

1. Estrutura e ambiente.

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

```
mkdir -p ~/lab05 && cd ~/lab05
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -c "import sys; print(sys.executable)"
```

A última linha tem de apontar para dentro de `~/lab05/.venv`. Guarde esse caminho: ele é o gabarito dos passos 7 e 9.

2. Instale as ferramentas.

```
python -m pip install ruff jupyterlab
ruff --version
```

```
ruff 0.16.2
```

A sua versão provavelmente será mais nova; o `ruff` muda rápido, e mensagens podem diferir em detalhe.

3. Crie o arquivo problemático. Salve `bagunca.py` com o conteúdo do capítulo — **exatamente** como está, com os espaços tortos.

4. Rode o linter.

```
ruff check --output-format concise bagunca.py
```

Quatro erros. Agora com mais regras:

```
ruff check --output-format concise --select E,F,I,N,UP,B,SIM bagunca.py
```

Cinco, com o N802 do nome `Media`.

5. Veja o formatador antes de aplicar.

```
ruff format --diff bagunca.py
```

Leia o diff com calma: `-` é o que sai, `+` é o que entra. Este hábito — olhar antes de deixar a ferramenta escrever — vale para o resto da vida profissional.

6. Aplique e confira o que sobrou.

```
ruff check --select E,F,I,N,UP,B,SIM --fix --output-format concise bagunca.py
ruff format bagunca.py
cat bagunca.py
```

Confirme que o laço por índice continua lá. Agora **corrija à mão** o que a ferramenta não corrige:

```
def media(valores):
    return sum(valores) / len(valores)

x = media([1, 2, 3])
print("media:", x)
```

7. Versione a configuração. Crie `pyproject.toml` e `.gitignore` com o conteúdo do capítulo, e `.vscode/settings.json` se você usa VS Code. Depois, sem `--select` — as regras agora vêm do `pyproject.toml`:

```
ruff check bagunca.py
ruff format --check bagunca.py
```

```
All checks passed!
1 file already formatted
```

E o programa ficou melhor por decisão sua, não da ferramenta.

8. Experimente o depurador. Acrescente `breakpoint()` como primeira linha de `media` e rode `python bagunca.py`. Você cai no `pdb`, que mostra o arquivo, a linha e o comando onde parou:

```
> /home/voce/lab05/bagunca.py(2)media()
-> breakpoint()
(Pdb)
```

Digite `valores` para ver o argumento, `n` para avançar uma linha, `c` para continuar até o fim e `q` para sair:

```
(Pdb) valores
[1, 2, 3]
(Pdb) n
> /home/voce/lab05/bagunca.py(3)media()
-> return sum(valores) / len(valores)
(Pdb) c
2.0
```

O caminho na sua máquina será outro. Depois remova o `breakpoint()` — ou teste `PYTHONBREAKPOINT=0 python bagunca.py`, que ignora todos eles sem editar o arquivo.

9. O notebook e o mesmo interpretador.

```
python -m ipykernel install --user --name lab05 --display-name "Python (lab05)"
jupyter lab
```

No navegador, crie um notebook, escolha o kernel `Python (lab05)` e execute numa célula:

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

```
import sys
sys.executable
```

O resultado tem de ser o mesmo caminho do passo 1. Se for `/usr/bin/python3`, o kernel está errado — troque em **Kernel** → **Change Kernel**.

10. Reproduza a armadilha da ordem. Crie três células com o conteúdo A, B e C do capítulo. Execute na ordem **A, C, B, C** clicando célula por célula. Confirme que:

- a célula C falhou com **NameError** na primeira vez e funcionou na segunda;
- lendo a tela de cima para baixo, os contadores são [1], [3], [4] — a célula do meio rodou por último entre as duas primeiras, e a de baixo guarda apenas a execução mais recente. O [2], que foi a falha, **desapareceu da tela**: rodar C de novo sobrescreveu o resultado anterior.

Esse último detalhe é o que torna a armadilha perigosa. O erro aconteceu, você corrigiu por acidente, e o notebook não guarda memória de que houve problema.

11. Prove que está reprodutível. Use **Kernel** → **Restart Kernel and Run All Cells**. Agora os contadores são 1, 2, 3 de cima para baixo e nada falha, porque a ordem do arquivo passou a ser a ordem de execução. Faça isso sempre antes de mostrar um notebook para alguém.

12. Quebre de propósito para fixar. Numa célula nova, escreva `taxa = 0.25` e execute. Depois **apague a célula**. Rode a célula C: ela usa 0.25, um valor que não existe em nenhum lugar do arquivo. Reinicie o kernel e rode tudo: volta para 0.10. Esse é o fantasma que faz notebook não reproduzir.

13. Feche com deactivate (e Ctrl+C duas vezes no terminal do Jupyter).

5.9. Resumo

- Editor, terminal e kernel de notebook são três processos. Quando algo funciona num e não no outro, a pergunta é qual interpretador cada um usa: `import sys; print(sys.executable)` responde em dez segundos.
- Um ambiente virtual por projeto, e as três ferramentas apontando para ele. No VS Code isso se registra em `.vscode/settings.json`, que vai para o Git.
- Formatador arruma aparência; linter procura problema. O **ruff** faz as duas coisas e substitui a pilha antiga de ferramentas. A configuração vive no `pyproject.toml`.
- A automação tem um limite honesto: **ruff** remove import inútil e conserta espaçamento, mas não troca um laço por índice por `sum`, nem renomeia função. Arquivo aprovado pelo linter pode ser programa ruim.
- `breakpoint()` está na linguagem desde o 3.7 e substitui a depuração por `print`. `PYTHONBREAKPOINT=0` desliga todos.
- Notebook é para explorar. O contador **In [n]** conta execuções, não células: se não estiver em sequência, o notebook não é reproduzível. **Restart Kernel and Run All Cells** antes de dar por pronto; código com valor duradouro migra para `.py`.
- Versione a configuração (`pyproject.toml`, `.vscode/`, `.gitignore`), nunca o ambiente (`.venv/`, `__pycache__/`, checkpoints).

6. Sintaxe essencial: indentação, linhas lógicas e comentários

Você copiou um trecho de um site, colou no seu arquivo e o programa parou de rodar. A mensagem fala de indentação numa linha que parece idêntica às outras. Você apaga os espaços e digita de novo, um por um, e aí funciona — sem que nada tenha mudado na tela.

Mudou. O que estava lá era um caractere de tabulação e o que você digitou foram espaços; os dois desenham a mesma coisa e são bytes diferentes. Em quase toda linguagem isso seria irrelevante, porque o espaço em branco não significa nada. Em Python, o espaço em branco **é** a estrutura do programa, e este capítulo explica exatamente até onde essa afirmação vai.

6.1. O bloco é a indentação

Em C, Java, JavaScript e Rust, um bloco é o que está entre chaves; a indentação é enfeite para humanos. Em Python não existem chaves — e o `from __future__ import braces` do Capítulo 1 responde **not a chance** a quem pede. O bloco é definido pela quantidade de espaço à esquerda.

Isso não é convenção: é gramática, resolvida na análise léxica do Capítulo 2. O módulo `tokenize` mostra o mecanismo. Para este arquivo de três linhas:

```
if x:
    y = 1
z = 2
```

Os tokens que o Python produz são:

```
import tokenize

with open("t1.py", "rb") as f:
    for tok in tokenize.tokenize(f.readline):
        print(f"{tokenize.tok_name[tok.type]:<12} {tok.string!r}")
```

```
ENCODING      'utf-8'
NAME          'if'
NAME          'x'
OP            ':'
NEWLINE       '\n'
INDENT        '    '
NAME          'y'
OP            '='
NUMBER        '1'
```

6. Sintaxe essencial: indentação, linhas lógicas e comentários

```
NEWLINE      '\n'
DEDENT       ''
NAME         'z'
OP           '='
NUMBER       '2'
NEWLINE      '\n'
ENDMARKER    ''
```

Ali estão os tokens `INDENT` e `DEDENT`. Eles são o equivalente exato da chave que abre e da chave que fecha em outras linguagens — só que o Python os gera a partir do espaço em branco em vez de exigir que você os digite. O `DEDENT` sai com string vazia porque não corresponde a nenhum caractere: é uma marca posicional.

Figura 6.1 põe as duas notações lado a lado.

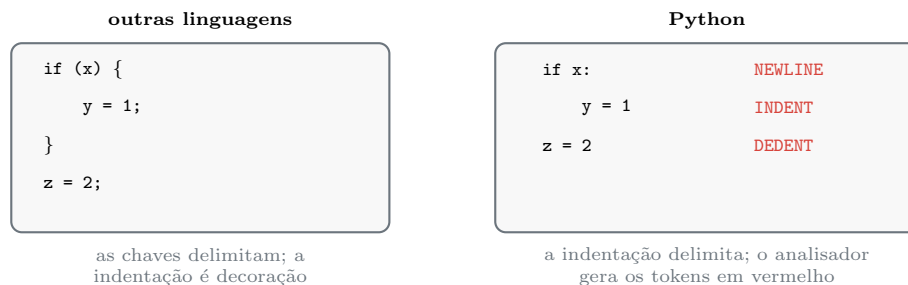


Figura 6.1.: `INDENT` e `DEDENT` são as chaves que você não digita.

A consequência é que **errar o espaço em branco é errar a sintaxe**, com mensagem de erro e tudo. Vale conhecer as quatro que aparecem.

Indentação onde não cabia bloco nenhum:

```
preco = 10  
    total = preco * 2
```

```
File "/tmp/lab06/e1.py", line 2  
    total = preco * 2  
IndentationError: unexpected indent
```

Bloco prometido e não entregue. Todo `:` no fim de uma linha abre um bloco, e o bloco tem de existir:

```
if preco > 5:  
print("caro")
```

```
File "/tmp/lab06/e2.py", line 2  
    print("caro")  
~~~~~  
IndentationError: expected an indented block after 'if' statement on line 1
```


Repare na precisão da mensagem: ela diz qual comando abriu o bloco e em que linha. Esse nível de detalhe é recente e economiza tempo.

Volta para um nível que não existe:

```
def f():
    a = 1
    b = 2
```

```
File "/tmp/lab06/e5.py", line 3
    b = 2
    ^
IndentationError: unindent does not match any outer indentation level
```

O `b` está a quatro espaços, mas o único nível aberto era o de oito. Não há para onde fechar.

Tabulação misturada com espaço — o erro da abertura do capítulo:

```
File "/tmp/lab06/e4.py", line 3
    b = 2
    ^
TabError: inconsistent use of tabs and spaces in indentation
```

`TabError` é uma subclasse de `IndentationError`, que por sua vez é subclasse de `SyntaxError`. Ou seja: os três são detectados na compilação, antes de qualquer linha rodar, exatamente como descrito no Capítulo 2.

Quanto ao tamanho da indentação, a linguagem só exige **consistência dentro do bloco**. Dois espaços funcionam:

```
def f():
    return 1

print(f())
```

```
1
```

Funciona e ninguém escreve assim. A PEP 8 pede **quatro espaços, nunca tabulação**, e essa é uma das poucas regras de estilo que o ecossistema trata como obrigatória: todo editor, todo formatador e todo projeto usam quatro espaços. Configure o editor para inserir espaços ao apertar Tab e o problema desaparece de vez.

6.2. Linha física e linha lógica

A segunda ideia estrutural do capítulo é que **uma instrução não precisa caber numa linha do arquivo**. O Python distingue a *linha física* (o que termina com Enter) da *linha lógica* (uma instrução completa).

Normalmente as duas coincidem. Dentro de parênteses, colchetes ou chaves, não:

6. Sintaxe essencial: indentação, linhas lógicas e comentários

```
clientes = [  
    "sitio-mangueira",  
    "fazenda-boavista",  
]  
  
total = (  
    10  
    + 20  
    + 30  
)
```

São duas linhas lógicas escritas em nove linhas físicas. Isso se chama **continuação implícita**, e é o mecanismo que os tokens revelam. Para o `total` acima:

ENCODING	'utf-8'
NAME	'total'
OP	'='
OP	'('
NL	'\n'
NUMBER	'1'
NL	'\n'
OP	'+'
NUMBER	'2'
NL	'\n'
OP	')'
NEWLINE	'\n'
ENDMARKER	''

Aqui está a distinção inteira, num lugar onde ela é visível: as quebras **dentro** dos parênteses viraram NL, e só a de depois do parêntese que fecha virou NEWLINE. NEWLINE termina uma linha lógica; NL é uma quebra de linha física que não termina nada. Uma linha lógica, quatro linhas físicas.

Figura 6.2 mostra o efeito.

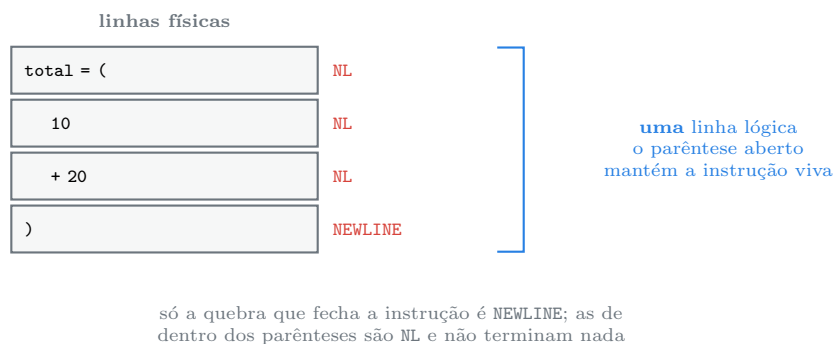


Figura 6.2.: Dentro de parênteses, a quebra de linha não termina a instrução.

Existe também a **continuação explícita**, com barra invertida no fim da linha:

```
soma = 1 + \
      2
```

Ela funciona, e a comunidade evita. O motivo é prático: um único espaço depois da barra invertida quebra o arquivo, e o espaço é invisível. Quando você precisa dividir uma linha, o caminho é acrescentar parênteses e usar a continuação implícita, que é robusta.

Isso resolve o problema real de manter linhas curtas. A PEP 8 pede no máximo 79 caracteres (as ferramentas modernas usam 88, como vimos no Capítulo 5), e a continuação implícita é como se obedece a esse limite sem ficar com código ilegível:

```
# em vez de uma linha de 120 caracteres
mensagem = f"cliente {cliente} teve {quedas} quedas entre {inicio} e {fim}"

# quebre dentro dos parênteses que já existem
mensagem = (
    f"cliente {cliente} teve {quedas} quedas "
    f"entre {inicio} e {fim}"
)
```

Duas strings literais adjacentes são concatenadas pelo compilador — outro caso de trabalho feito antes de o programa rodar, como o dobramento de constantes do Capítulo 2.

E o inverso, colocar várias instruções numa linha com ponto e vírgula, também é legal:

```
python3 -c "a = 1; b = 2; print(a + b)"
```

3

Legal e desaprovado. Em `python3 -c`, onde não há arquivo, é a única saída e está tudo bem. Em arquivo, é uma linha para cada instrução.

6.3. Comando e expressão

Uma distinção que evita confusão desde já: **expressão** é algo que tem valor; **comando** é algo que acontece. `2 + 3` é expressão. `x = 5` é comando. É por isso que o REPL do Capítulo 4 responde ao primeiro e fica calado com o segundo.

Onde a linguagem espera uma expressão, um comando é erro de sintaxe:

```
python3 -c "x = (z = 5)"
```

```

x = (z = 5)
~~~~~

```

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

A mensagem já sugere as duas coisas que provavelmente você queria. `==` compara e é expressão. E `:=`, o operador morsa do Capítulo 1, atribui e produz valor, o que o torna uma expressão:

```
python3 -c "x = (y := 5); print(x, y)"
```

```
5 5
```

Esse é justamente o motivo de o morsa ter sido criado: permitir atribuição em lugares que exigem expressão, como a condição de um `if` ou de um `while`. O Volume 3 volta ao assunto.

6.4. Nomes

Um identificador em Python começa com letra ou underscore e continua com letras, dígitos ou underscore. Não pode começar com dígito:

```
python3 -c "2gatos = 1"
```

```
  2gatos = 1
  ^
SyntaxError: invalid decimal literal
```

O que surpreende quem vem de outras linguagens é que “letra” quer dizer letra Unicode, não apenas ASCII. Isto é Python válido:

```
café = 3
preço_final = café * 2
print(preço_final)
```

```
6
```

Funciona, e este manual **não** recomenda. Nome de identificador em inglês ou em português sem acento, sempre — porque acento em nome de variável cria problemas de codificação em ferramentas antigas, dificulta digitar em teclado alheio e, pior, permite dois nomes visualmente idênticos e diferentes para o interpretador. Texto de mensagem, comentário e documentação, esses sim vão em português correto, com acento.

As palavras reservadas não podem ser usadas como nome. São 35:

```
import keyword

print(len(keyword.kwlist))
print(keyword.kwlist)
```

```
35
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
↪  'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally',
↪  'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal',
↪  'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

E existe uma segunda lista, curiosa e recente — as **palavras reservadas suaves**:

```
print(keyword.softkwlist)
```

```
['_', 'case', 'match', 'type']
```

Essas quatro só são especiais no contexto em que a gramática as espera. Fora dele, continuam sendo nomes comuns: `match = re.match(...)` é código perfeitamente válido, e é exatamente por isso que a lista suave existe. Quando o `match` foi acrescentado no Python 3.10, torná-lo palavra reservada de verdade quebraria uma quantidade enorme de código que já usava `match` como variável.

Sobre convenção de nomes, a PEP 8 é curta e o ecossistema segue à risca:

Tipo	Convenção	Exemplo
função, variável, módulo	snake_case	media_niveis, taxa_juros
classe	CamelCase	ConfiguracaoInvalida
constante	MAIUSCULA_COM_UNDERSCORE	TAXA_PADRAO
interno ao módulo	um underscore à frente	_cache

O **ruff** do Capítulo 5 verifica isso com as regras da família N, e foi o N802 que reclamou de `Media`.

6.5. Comentários e documentação

Comentário começa com o sinal de cerquilha e vale até o fim da linha física. Não existe comentário de bloco em Python.

O que existe, e é diferente, é a **docstring**: um literal de texto como primeira instrução de um módulo, função, classe ou método. A diferença não é estilística — comentário é descartado pelo compilador, docstring é um objeto guardado no atributo `__doc__`:

```
"""Documentacao do modulo."""
# comentario comum
```

```
def f():
    """Doc da funcao."""
    pass
```

```
python3 -c "import d; print(repr(d.__doc__)); print(repr(d.f.__doc__))"
```

```
'Documentacao do modulo.'
'Doc da funcao.'
```

6. Sintaxe essencial: indentação, linhas lógicas e comentários

O comentário não aparece em lugar nenhum; as duas docstrings estão acessíveis em tempo de execução. É delas que o **help** do Capítulo 4 tira o texto, e é nelas que geradores de documentação se apoiam. Um texto entre três aspas no meio de uma função, por outro lado, não é docstring: é uma expressão de string cujo valor é jogado fora — funciona como comentário multilinha por acidente, e não é para isso que serve.

Sobre o conteúdo dos comentários, a regra que economiza mais tempo de leitura: **comente o porquê, não o quê**.

```
# antipadrão: repete o código em português
# soma os níveis e divide pelo total
media = sum(níveis) / len(níveis)

# pythônico: explica o que o código não pode dizer
# o equipamento reporta -99 quando a porta esta apagada;
# incluir esses valores derrubaria a media do enlace
níveis = [n for n in níveis if n != -99]
```

O primeiro comentário vira mentira na primeira vez que alguém mudar a linha. O segundo carrega informação que não está em lugar nenhum do código.

E aquele **pass** do exemplo: ele existe porque a gramática exige um bloco depois do **:**, e às vezes você não tem nada para colocar ali ainda. **pass** é o comando que não faz nada, e serve exatamente para satisfazer a gramática.

6.6. Uma nota sobre codificação

Arquivo **.py** é **UTF-8 por padrão** desde o Python 3, e é por isso que **café** e os acentos deste manual funcionam sem cerimônia. Você ainda vai encontrar, em código antigo, uma linha como esta no topo:

```
# -*- coding: utf-8 -*-
```

É a declaração de codificação da PEP 263, obrigatória no Python 2, onde o padrão era ASCII. Hoje ela é redundante e pode ser apagada. Só faz diferença se o arquivo estiver **em outra** codificação, o que é um problema a resolver e não a documentar.

6.7. Jeito Pythônico

A PEP 8 é o guia de estilo da linguagem, e a maior parte dela se resume a coisas que uma ferramenta aplica sozinha. O resumo do que importa neste capítulo:

```
# pythônico
def media_níveis(níveis: list[float]) -> float:
    """Media dos níveis validos, ignorando portas apagadas."""
    validos = [n for n in níveis if n != -99]
    if not validos:
        raise ValueError("nenhum nível valido")
    return sum(validos) / len(validos)
```

Quatro espaços por nível, **snake_case**, docstring em vez de comentário no topo, duas linhas em branco entre funções de nível superior, uma instrução por linha, e nada de barra invertida.

Os antipadrões correspondentes:

```
# antipadrão: tabulação (ou mistura com espaço)
# antipadrão: continuação com barra invertida
mensagem = "cliente " + cliente + \
    " teve problemas"

# antipadrão: várias instruções numa linha
if not validos: raise ValueError("nenhum nível valido")

# antipadrão: comentário de bloco com três aspas no meio da função
"""
esta parte calcula a media
"""

# antipadrão: acento em identificador
média = sum(níveis) / len(níveis)
```

Sobre versões, o que mudou na sintaxe e a partir de quando:

- **Python 3.8** — o operador morsa `:=` (PEP 572), que faz da atribuição uma expressão.
- **Python 3.10** — `match` e `case` como palavras reservadas **suaves** (PEP 634), e mensagens de erro de sintaxe muito melhores, com o comando que abriu o bloco identificado por nome e linha.
- **Python 3.11** — marcação de coluna no traceback, com `~` e `^` apontando o trecho exato (PEP 657).
- **Python 3.12** — `type` entrou na lista de palavras suaves, junto com a sintaxe nova de genéricos (PEP 695), e f-strings deixaram de ter restrição sobre aspas e barras invertidas no interior (PEP 701).

Uma reflexão para fechar. A indentação obrigatória é a decisão de projeto mais criticada do Python por quem chega de fora, e a que a comunidade defende com mais convicção depois de um tempo. O argumento é simples: em toda linguagem, programador competente indenta o código de qualquer maneira. A diferença é que nas outras a indentação pode **mentir** — o bloco pode ser diferente do que os olhos veem, porque quem manda são as chaves. Em Python, o que você vê é o que executa. É o “legibilidade conta” da PEP 20 aplicado ao nível do caractere.

6.8. Laboratório

Objetivo: provocar cada erro de indentação de propósito, observar os tokens que o analisador produz e distinguir na prática linha física de linha lógica.

1. Ambiente.

```
mkdir -p ~/lab06 && cd ~/lab06
python3 -m venv .venv && source .venv/bin/activate
```

6. Sintaxe essencial: indentação, linhas lógicas e comentários

2. Provoque os quatro erros. Crie um arquivo para cada caso, rode e leia a mensagem inteira antes de corrigir. Este é o passo em que se aprende a reconhecer o erro pelo texto.

e1.py — indentação inesperada:

```
preco = 10
    total = preco * 2
```

e2.py — bloco que faltou:

```
preco = 10
if preco > 5:
    print("caro")
```

e5.py — volta para nível inexistente:

```
def f():
    a = 1
    b = 2
```

Confirme que os três dão `IndentationError` e que o texto de cada um é diferente.

3. Produza um `TabError`. Este exige cuidado, porque o editor pode converter tabulação em espaço automaticamente. Gere o arquivo pelo terminal, onde você controla os bytes:

```
printf 'def f():\n    a = 1\n\tb = 2\n' > e4.py
cat -A e4.py
python e4.py
```

O `cat -A` mostra os caracteres invisíveis: `^I` é tabulação e `$` é fim de linha. Você deve ver a terceira linha começando com `^I` e as outras com espaços. E o erro:

```
TabError: inconsistent use of tabs and spaces in indentation
```

4. Confirme o parentesco das exceções.

```
python -c "print(TabError.__mro__)"
```

```
(<class 'TabError'>, <class 'IndentationError'>, <class 'SyntaxError'>, <class
↳ 'Exception'>, <class 'BaseException'>, <class 'object'>)
```

`TabError` é um `IndentationError`, que é um `SyntaxError`. Por isso os três param o programa antes de a primeira linha rodar. A hierarquia de exceções é assunto do Capítulo 7 e do Volume 6.

5. Veja os tokens de indentação. Crie `t1.py` com o `if/y/z` do capítulo e rode o trecho do `tokenize`. Localize na saída o `INDENT` com quatro espaços e o `DEDENT` com string vazia.

6. Veja `NL` contra `NEWLINE`. Crie `t2.py`:


```
a = 1

# comentario
b = 2
```

Rode o mesmo `tokenize`. Você deve ver `NEWLINE` depois de cada instrução e `NL` na linha em branco e depois do comentário — porque nenhuma das duas termina instrução alguma. Agora `t3.py`:

```
total = (
    1
    + 2
)
```

Aqui há **um** `NEWLINE` só, no fim, e `NL` nas quebras internas. Uma linha lógica em quatro físicas.

7. Compare as duas continuações. Crie `cont.py` com o conteúdo do capítulo (a lista, o `total` entre parênteses e a soma com barra invertida) e rode:

```
python cont.py
```

```
['sitio-mangueira', 'fazenda-boa-vista'] 60 3
```

Agora **acrescente um espaço depois da barra invertida** da soma e rode de novo. O programa quebra, e a mensagem não ajuda muito, porque o caractere culpado é invisível. Desfaça. Foi por isso que a comunidade abandonou a barra invertida.

8. Explore as palavras reservadas.

```
python -c "import keyword; print(len(keyword.kwlist))"
python -c "import keyword; print(keyword.softkwlist)"
python -c "import keyword; print(keyword.iskeyword('match'),
↪ keyword.issoftkeyword('match'))"
```

A última linha deve responder `False True`: `match` não é palavra reservada, é suave. Confirme que dá para usá-la como variável:

```
python -c "match = 42; print(match)"
```

9. Teste um nome inválido e um exótico.

```
python -c "2gatos = 1"
python -c "café = 3; print(café * 2)"
```

O primeiro falha, o segundo imprime 6. Sabendo que funciona, continue não usando.

10. Prove que docstring é dado e comentário não. Crie `d.py` com o conteúdo do capítulo e rode:

```
python -c "import d; print(repr(d.__doc__)); print(repr(d.f.__doc__))"
python -c "import d; help(d.f)"
```

Não há forma de recuperar o comentário — ele não existe depois da compilação.

11. Deixe a ferramenta arrumar o que é mecânico.

```
python -m pip install ruff
printf 'x=1\ny  =  2\nif x>0:\n          print( x+y )\n' > torto.py
ruff format torto.py
cat torto.py
```

```
1 file reformatted
x = 1
y = 2
if x > 0:
    print(x + y)
```

O formatador normalizou os espaços e trouxe a indentação de oito para quatro. O que ele **não** faz é adivinhar a estrutura de um bloco errado:

```
printf 'x=1\nif x>0:\nprint(x)\n' > quebrado.py
ruff format quebrado.py; echo "exit=$?"
```

```
error: Failed to parse quebrado.py:3:1: Expected an indented block after `if`
↳ statement
exit=2
```

Arquivo que não compila não pode ser formatado — o formatador precisa da árvore sintática do Capítulo 2, e não há árvore aqui.

12. Feche com deactivate.

6.9. Resumo

- Em Python o bloco é a indentação, e isso é gramática: o analisador emite os tokens `INDENT` e `DEDENT`, que fazem o papel das chaves de outras linguagens. `tokenize` mostra isso.
- Quatro erros de espaço em branco valem reconhecer de cara: `unexpected indent`, `expected an indented block`, `unindent does not match any outer indentation level` e `TabError`. Todos são `SyntaxError` por herança e param o programa antes da primeira linha.
- A linguagem só exige consistência; a PEP 8 exige quatro espaços e nenhuma tabulação, e o ecossistema inteiro obedece.
- Linha física é o que termina com `Enter`; linha lógica é a instrução completa. Dentro de parênteses, colchetes ou chaves, a quebra vira `NL` e não termina nada — é a continuação implícita, e é como se quebra linha longa. Barra invertida funciona e se evita.
- Expressão tem valor, comando acontece. `=` é comando e não cabe onde se espera expressão; `==` compara e `:=` atribui produzindo valor.

- São 35 palavras reservadas e 4 suaves (`_`, `case`, `match`, `type`), que só valem no contexto da gramática e continuam servindo como nome comum.
- Comentário é descartado na compilação; docstring vira `__doc__` e alimenta o `help`. Comente o porquê, não o quê. Arquivo `.py` é UTF-8 por padrão, e `# -*- coding: utf-8 -*-` é herança do Python 2.

7. Erros e tracebacks: como ler o que o Python está dizendo

Um script que rodou a semana toda quebrou hoje de manhã. A tela cospe onze linhas de texto com caminhos de arquivo, setas e nomes que você não reconhece. A reação natural é rolar para o topo, procurar a primeira linha e tentar entender o que ela quer.

É o inverso do que funciona. Um traceback é organizado do mais antigo para o mais recente, e a informação que responde “o que aconteceu” está na **última** linha. Ler traceback é a habilidade que mais economiza tempo em Python, e a que menos se ensina.

7.1. O traceback do começo ao fim

Vamos quebrar um programa de verdade. `relatorio.py` calcula a média dos níveis de sinal óptico de um arquivo de log:

```
"""Gera o relatorio de quedas por cliente."""

from pathlib import Path

def ler_linhas(caminho):
    return Path(caminho).read_text(encoding="utf-8").splitlines()

def extrair_nivel(linha):
    campos = linha.split()
    return float(campos[3])

def media_niveis(caminho):
    linhas = ler_linhas(caminho)
    niveis = [extrair_nivel(l) for l in linhas]
    return sum(niveis) / len(niveis)

if __name__ == "__main__":
    print(media_niveis("sinais.log"))
```

E `sinais.log` tem um campo estragado na terceira linha — o equipamento não conseguiu ler e escreveu texto no lugar do número:

7. Erros e tracebacks: como ler o que o Python está dizendo

```
2026-03-11 03:14:02 sitio-mangueira -18.4
2026-03-11 05:02:19 fazenda-boavista -21.7
2026-03-11 06:47:55 chacara-do-ze sem-leitura
```

```
python3 relatorio.py
```

```
Traceback (most recent call last):
  File "/tmp/lab07/relatorio.py", line 22, in <module>
    print(media_niveis("sinais.log"))
    ~~~~~~^~~~~~
  File "/tmp/lab07/relatorio.py", line 17, in media_niveis
    niveis = [extrair_nivel(l) for l in linhas]
    ~~~~~~^~~
  File "/tmp/lab07/relatorio.py", line 12, in extrair_nivel
    return float(campos[3])
ValueError: could not convert string to float: 'sem-leitura'
```

Esse texto tem uma estrutura fixa, e vale nomear cada parte.

A primeira linha é um aviso de ordenação, não informação sobre o erro: `most recent call last` significa “a chamada mais recente está por último”. É o Python dizendo onde olhar, e quase todo mundo ignora.

No meio vêm os quadros (*frames*), um por chamada de função que estava em andamento. Cada quadro tem três linhas: o arquivo e a linha, o nome da função (`in media_niveis`), o código daquela linha, e — desde o Python 3.11 — a marcação de coluna com til e circunflexo apontando o trecho exato da expressão que estava sendo avaliado.

A última linha é a resposta: o tipo da exceção (`ValueError`), dois pontos, e a mensagem. `could not convert string to float: 'sem-leitura'` diz o que deu errado e qual dado causou o problema.

Figura 7.1 organiza a leitura.

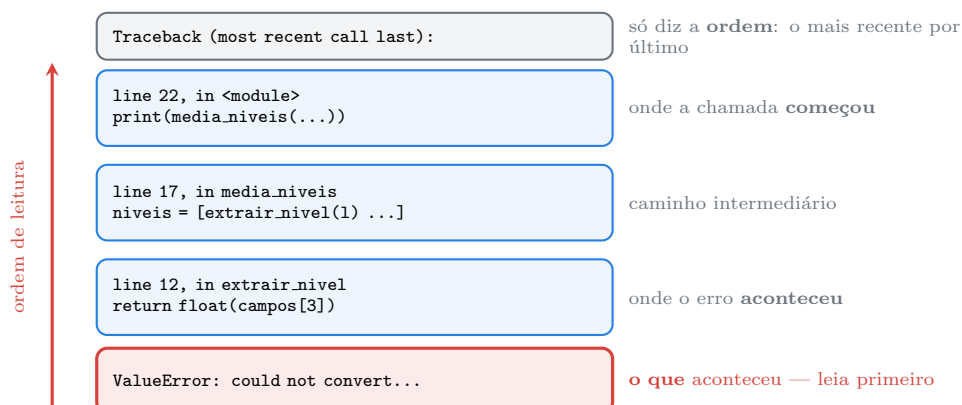


Figura 7.1.: O traceback imprime do mais antigo ao mais recente; leia de baixo para cima.

A ordem prática de leitura é esta:

1. **A última linha**, sempre. Tipo e mensagem respondem o que aconteceu.

2. **O quadro de baixo**, que é onde o erro estourou.
3. **Subindo**, até encontrar o primeiro quadro que seja **código seu**. É ali que você tem poder de agir.

O passo 3 importa mais do que parece, porque tracebacks reais atravessam bibliotecas. Se o arquivo de log não existir:

```
python3 -c "from relatorio import media_niveis; media_niveis('naoexiste.log')"
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    media_niveis('naoexiste.log')
    ~~~~~~^~~~~~
  File "/tmp/lab07/relatorio.py", line 16, in media_niveis
    linhas = ler_linhas(caminho)
  File "/tmp/lab07/relatorio.py", line 7, in ler_linhas
    return Path(caminho).read_text(encoding="utf-8").splitlines()
           ~~~~~~^~~~~~
  File "/usr/lib/python3.13/pathlib/_local.py", line 548, in read_text
    return PathBase.read_text(self, encoding, errors, newline)
           ~~~~~~^~~~~~
  File "/usr/lib/python3.13/pathlib/_abc.py", line 632, in read_text
    with self.open(mode='r', encoding=encoding, errors=errors, newline=newline)
    ↪ as f:
       ~~~~~~^~~~~~
  File "/usr/lib/python3.13/pathlib/_local.py", line 539, in open
    return io.open(self, mode, buffering, encoding, errors, newline)
           ~~~~~~^~~~~~
FileNotFoundError: [Errno 2] No such file or directory: 'naoexiste.log'
```

São sete quadros, e **quatro deles são da biblioteca padrão**. Nada em `/usr/lib/python3.13/pathlib/` está com defeito — aqueles quadros são só o caminho que a chamada percorreu. O quadro que interessa é o `ler_linhas`, na linha 7 do seu arquivo, e a informação decisiva continua sendo a última linha.

Daí a heurística que resolve a maioria dos casos: **procure o quadro mais baixo cujo caminho seja do seu projeto**. Erro dentro de biblioteca madura quase sempre significa que você passou para ela um valor que ela não esperava.

7.2. As exceções que você vai ver mais

Cada tipo de exceção é uma hipótese sobre a causa. Reconhecê-las de imediato é metade do trabalho.

NameError — o nome não existe. Desde o Python 3.10, com sugestão:

```
comprimento = 10
print(compriemnto)
```

7. Erros e tracebacks: como ler o que o Python está dizendo

```
NameError: name 'compriemnto' is not defined. Did you mean: 'comprimento'?
```

AttributeError — o objeto não tem aquele atributo ou método. Também sugere, quando o nome é parecido o bastante:

```
AttributeError: 'str' object has no attribute 'upperr'. Did you mean: 'upper'?
AttributeError: 'str' object has no attribute 'startswith'. Did you mean:
↳ 'startswith'?
```

E não sugere, quando o palpite é distante demais:

```
AttributeError: 'str' object has no attribute 'uppercase'
```

A sugestão vem de uma medida de semelhança entre textos: `upperr` e `uppe` chegam perto de `upper`, mas `uppercase` não. Ausência de sugestão não significa que o método não exista com outro nome — significa apenas que a heurística não achou nada próximo. Aí é `dir` e `help`, como no Capítulo 4.

TypeError — a operação não faz sentido para aqueles tipos:

```
File "<string>", line 1, in <module>
    print('3' + 4)
        ~~~~^~~~
TypeError: can only concatenate str (not "int") to str
```

Aqui a marcação de coluna se mostra útil: o `^` está exatamente sobre o `+`, e os `~` cobrem os operandos. Esse é o “explícito é melhor que implícito” do Capítulo 1 em ação — a linguagem se recusa a adivinhar se você queria `"34"` ou `7`.

ValueError — o tipo está certo e o valor não serve. `float("sem-leitura")` recebeu um texto, que é o tipo esperado, com conteúdo impossível de converter. A dupla `TypeError` e `ValueError` confunde no começo, e a distinção é essa: tipo errado contra valor errado.

KeyError e **IndexError** — a chave ou a posição não existe:

```
print(d['quedas'])
      ~~~~~
KeyError: 'quedas'
```

```
print(campos[5])
      ~~~~~
IndexError: list index out of range
```

IndexError é o que você recebe quando o arquivo de log tem uma linha com menos campos do que o esperado — e é por isso que `campos[3]` em `extrair_nivel` é uma bomba armada, assunto do laboratório.

ZeroDivisionError — divisão por zero. No nosso programa, ele aparece se o arquivo estiver vazio:


```
print(sum(niveis)/len(niveis))
~~~~~^~~~~~
ZeroDivisionError: division by zero
```

FileNotFoundError — arquivo ou diretório inexistente, com o [Errno 2] que vem do sistema operacional.

ModuleNotFoundError — o import falhou:

```
import requestss
ModuleNotFoundError: No module named 'requestss'
```

Nove vezes em dez a causa não é o nome: é o ambiente virtual errado. Antes de investigar qualquer coisa, rode o `import sys; print(sys.executable)` do Capítulo 5.

7.3. Exceções encadeadas

Quando uma exceção acontece **enquanto** outra estava sendo tratada, o Python mostra as duas. É a parte do traceback que mais assusta e a mais informativa.

7.3.1. Encadeamento implícito

```
dados = {"cliente": "sitio"}

try:
    quedas = dados["quedas"]
except KeyError:
    print("nao achei a chave, vou tentar converter")
    quedas = int(dados["cliente"])
```

```
nao achei a chave, vou tentar converter
Traceback (most recent call last):
  File "/tmp/lab07/enc1.py", line 4, in <module>
    quedas = dados["quedas"]
    ~~~~~^~~~~~
KeyError: 'quedas'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/tmp/lab07/enc1.py", line 7, in <module>
    quedas = int(dados["cliente"])
ValueError: invalid literal for int() with base 10: 'sitio'
```

A frase-chave é “**During handling of the above exception**”: durante o tratamento da exceção acima, outra ocorreu. Isso quase sempre indica que o seu `except` tem um bug — o plano B falhou. O erro que você precisa corrigir é o de baixo; o de cima explica por que o programa chegou lá.

7.3.2. Encadeamento explícito

O caso oposto é intencional. Você captura um erro técnico e o traduz para um erro que fala a língua da sua aplicação, preservando o original com `raise ... from`:

```
class ConfiguracaoInvalida(Exception):
    """Erro de dominio da aplicacao."""

def ler_taxa(dados):
    try:
        return float(dados["taxa"])
    except KeyError as erro:
        raise ConfiguracaoInvalida("falta a chave 'taxa'") from erro

ler_taxa({"cliente": "sitio"})
```

```
Traceback (most recent call last):
  File "/tmp/lab07/enc2.py", line 7, in ler_taxa
    return float(dados["taxa"])
          ~~~~~~^~~~~~
KeyError: 'taxa'
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "/tmp/lab07/enc2.py", line 12, in <module>
    ler_taxa({"cliente": "sitio"})
    ~~~~~~^~~~~~
  File "/tmp/lab07/enc2.py", line 9, in ler_taxa
    raise ConfiguracaoInvalida("falta a chave 'taxa'") from erro
ConfiguracaoInvalida: falta a chave 'taxa'
```

Agora a frase é “**The above exception was the direct cause**” — causa direta, não acidente. Quem lê recebe as duas informações: a mensagem de alto nível (“falta a chave taxa”) e o detalhe técnico (“KeyError: 'taxa'”).

A terceira forma é suprimir a origem com `from None`, quando o erro interno é ruído:

```
def ler_taxa(dados):
    try:
        return float(dados["taxa"])
    except KeyError:
        raise ValueError("falta a chave 'taxa'") from None
```

```
Traceback (most recent call last):
  File "/tmp/lab07/enc3.py", line 8, in <module>
    ler_taxa({})
    ~~~~~~^~~~~~
```

```
File "/tmp/lab07/enc3.py", line 5, in ler_taxa
    raise ValueError("falta a chave 'taxa'") from None
ValueError: falta a chave 'taxa'
```

Limpo — e você jogou fora informação. Use com parcimônia. Figura 7.2 resume as três.



Figura 7.2.: Três encadeamentos: acidente, tradução proposital e supressão.

7.4. Quando o traceback é gigante

Recursão infinita produz o traceback mais intimidador que existe — e o CPython tem a gentileza de resumir-lo:

```
def profundidade(n):
    return profundidade(n + 1)

profundidade(0)
```

```
Traceback (most recent call last):
  File "/tmp/lab07/rec.py", line 5, in <module>
    profundidade(0)
    ~~~~~^~~~
  File "/tmp/lab07/rec.py", line 2, in profundidade
    return profundidade(n + 1)
  File "/tmp/lab07/rec.py", line 2, in profundidade
    return profundidade(n + 1)
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Doze linhas, não mil. O [Previous line repeated 996 more times] substitui os quadros repetidos. `RecursionError` quase sempre significa uma de duas coisas: falta a condição de parada, ou existe um ciclo entre funções que chamam umas às outras.

7.5. SyntaxError é diferente de todos os outros

Vale reforçar a distinção do Capítulo 2 e do Capítulo 6, porque ela muda como se procura o problema:

7. Erros e tracebacks: como ler o que o Python está dizendo

```
File "/tmp/lab06/e2.py", line 2
    print("caro")
    ~~~~~
IndentationError: expected an indented block after 'if' statement on line 1
```

Não há `Traceback` (most recent call last) e não há quadros. Não pode haver: nenhuma função foi chamada, porque o programa nunca começou. `SyntaxError` e suas subclasses são detectadas na compilação; todo o resto acontece em execução. Se você vê quadros, o programa rodou; se não vê, nem chegou a rodar.

7.6. Jeito Pythônico

A postura pythônica diante de erro tem um nome: **EAFP**, de *Easier to Ask Forgiveness than Permission* — é mais fácil pedir perdão do que permissão. O oposto é o **LBYL**, *Look Before You Leap*, olhe antes de saltar, comum em outras linguagens.

```
# LBYL: verifica tudo antes
import os

if os.path.exists(caminho):
    if os.access(caminho, os.R_OK):
        with open(caminho) as f:
            dados = f.read()
    else:
        dados = ""
else:
    dados = ""

# EAFP: tenta, e trata o que der errado
try:
    dados = Path(caminho).read_text(encoding="utf-8")
except (FileNotFoundError, PermissionError):
    dados = ""
```

A segunda não é só mais curta: é **correta**, e a primeira não é. Entre o `os.path.exists` e o `open` existe um intervalo em que o arquivo pode ser apagado por outro processo — uma condição de corrida que a verificação prévia não elimina, só torna mais rara. A exceção é a única forma de saber que a operação falhou de fato.

Três antipadrões que valem nome:

```
# antipadrão: engolir tudo em silêncio
try:
    nivel = float(campos[3])
except:
    pass
```

Esse é o pior pedaço de código deste capítulo. O `except` sem tipo captura **tudo**, incluindo `KeyboardInterrupt` — o seu `Ctrl+C` — e `SystemExit`. Com `pass` no corpo, o erro desaparece sem rastro, e é o “erros nunca devem passar em silêncio” da PEP 20 sendo violado da forma mais direta possível. O programa continua com dados errados e você descobre semanas depois.

```
# antipadrão: capturar Exception por precaução
try:
    nivel = float(campos[3])
except Exception as erro:
    print("deu erro:", erro)
```

Melhor, porque não engole o `Ctrl+C` e ao menos avisa. Ainda ruim, porque trata igual coisas diferentes: campo com texto inválido, arquivo corrompido e um bug de digitação no nome de uma variável.

```
# pythônico: capture o que você sabe tratar, o mais específico possível
try:
    nivel = float(campos[3])
except (IndexError, ValueError) as erro:
    logging.warning("linha ignorada (%s): %r", erro, linha)
    continue
```

`IndexError` para a linha com poucos campos, `ValueError` para o campo não numérico, e um registro do que foi descartado. Qualquer outra exceção continua subindo — porque não é caso previsto, e programa que quebra alto é melhor que programa que mente.

Duas regras finais:

Nunca use `assert` para validar entrada. Como visto no Capítulo 2, `python -O` remove as asserções. Validação é `if` com `raise`.

Prefira exceção específica à mensagem genérica. Quando você levanta um erro, escolha o tipo mais preciso da hierarquia (`ValueError` para valor ruim, `TypeError` para tipo errado, `KeyError` para chave ausente) ou crie a sua, herdando de `Exception`. O Volume 6 trata da hierarquia completa e de como desenhar exceções próprias.

Sobre versões — a qualidade das mensagens de erro é uma das áreas em que o Python mais melhorou:

- **Python 3.10** — `NameError` e `AttributeError` passaram a sugerir nomes parecidos; mensagens de `SyntaxError` identificam o comando e a linha que abriram o bloco.
- **Python 3.11** — marcação de coluna com `~` e `^` (PEP 657) e `ExceptionGroup` com `except*` (PEP 654), para quando várias exceções acontecem em paralelo. `add_note` permite anexar contexto a uma exceção já levantada.
- **Python 3.12** — `sys.last_exc` guarda a última exceção no modo interativo.
- **Python 3.13** — `tracebacks` **coloridos** por padrão no terminal, com o tipo e a mensagem destacados. `PYTHON_COLORS=0` desliga, o que é necessário ao redirecionar para arquivo em ferramenta que não entende as sequências de cor.

7.7. Laboratório

Objetivo: provocar cada exceção comum de propósito, praticar a leitura de baixo para cima e transformar um programa frágil num programa que sobrevive a dados ruins.

1. Ambiente e programa.

```
mkdir -p ~/lab07 && cd ~/lab07
python3 -m venv .venv && source .venv/bin/activate
```

Crie `relatorio.py` e `sinais.log` com o conteúdo do capítulo.

2. Leia o traceback na ordem certa.

```
python relatorio.py
```

Antes de olhar o resto, responda três perguntas usando **só a última linha**: qual é o tipo do erro, o que ele diz e qual dado causou. Depois vá ao quadro de baixo e confirme em que função e em que linha aconteceu.

3. Conte os quadros de um erro que atravessa a biblioteca.

```
python -c "from relatorio import media_niveis; media_niveis('naoexiste.log')"
```

Conte quantos quadros são seus e quantos são de `/usr/lib`. Encontre o quadro mais baixo do seu arquivo — é ali que a correção entra.

4. Provoque o `IndexError` escondido. Uma linha com menos campos que o esperado é outra bomba armada em `campos[3]`. Para vê-la, use um arquivo **sem** a linha **sem-leitura**, senão o `ValueError` do passo anterior estoura primeiro e esconde este:

```
cat > curto.log <<'EOF'
2026-03-11 03:14:02 sitio-mangueira -18.4
2026-03-11 22:00:00 vila-sao-joao
EOF
python -c "from relatorio import media_niveis; media_niveis('curto.log')"
```

```
File "/tmp/lab07/relatorio.py", line 12, in extrair_nivel
    return float(campos[3])
           ~~~~~~^^~
IndexError: list index out of range
```

Duas coisas para notar. O traceback é quase idêntico ao do passo 2, com um tipo diferente na última linha — mais uma razão para lê-la primeiro. E a marcação de coluna mudou de lugar: agora aponta `campos[3]`, e não a chamada de `float`, porque o erro aconteceu na indexação, antes de `float` ser chamada.

O fato de o primeiro erro esconder o segundo é a razão de o passo 12 tratar os dois de uma vez.

5. Provoque o `ZeroDivisionError`.

```
: > vazio.log
python -c "from relatorio import media_niveis; media_niveis('vazio.log')"
```

O : > cria um arquivo vazio. Sem linhas, `len(niveis)` é zero.

6. Colecione as exceções comuns. Rode cada uma e leia a mensagem inteira:

```
python -c "comprimento = 10; print(compriemnto)"
python -c "'manual'.upperr()"
python -c "'manual'.uppercase()"
python -c "print('3' + 4)"
python -c "print({'cliente': 'sitio'}['quedas'])"
python -c "print(['a','b'][5])"
python -c "import requestss"
```

Compare a segunda com a terceira: uma sugere `upper`, a outra não. A semelhança entre os nomes é o critério.

7. Veja o encadeamento implícito. Crie `enc1.py` com o conteúdo do capítulo e rode. Localize a frase `During handling of the above exception`. Pergunte-se qual dos dois erros você corrigiria — e a resposta é o de baixo, porque o `except` é que está errado.

8. Veja o encadeamento explícito. Crie `enc2.py` e rode. Localize `The above exception was the direct cause`. Depois crie `enc3.py`, com `from None`, e compare: a segunda saída é mais limpa e esconde que a causa raiz era um `KeyError`.

9. Provoque o `RecursionError`.

```
python -c "
def profundidade(n):
    return profundidade(n + 1)

profundidade(0)
" 2>&1 | tail -4
```

Confirme o `[Previous line repeated ... more times]`. Depois conte o traceback inteiro:

```
python -c "
def profundidade(n):
    return profundidade(n + 1)

profundidade(0)
" 2>&1 | wc -l
```

Uma dúzia de linhas para mil chamadas.

10. Confirme que `SyntaxError` não tem quadros.

```
printf 'if True:\nprint(1)\n' > sint.py
python sint.py
```

7. Erros e tracebacks: como ler o que o Python está dizendo

Nenhum Traceback (most recent call last), nenhum quadro. O programa não rodou.

11. Entenda as cores. No 3.13 o traceback sai colorido — mas só quando a saída é um terminal de verdade. Confirme que redirecionar já desliga:

```
python relatorio.py 2> err.txt
cat -v err.txt | tail -2
```

```
    return float(campos[3])
ValueError: could not convert string to float: 'sem-leitura'
```

Texto puro, sem sequência de escape nenhuma: o Python detectou que não havia terminal e não coloriu. Para **ver** a cor que o terminal recebe, force um pseudoterminal com o utilitário `script`:

```
script -q -c "python relatorio.py" /dev/null 2>&1 | cat -v | tail -1
```

```
^[[1;35mValueError^[[0m: ^[[35mcould not convert string to float:
↪  'sem-leitura'^[[0m^M
```

Ali estão os códigos de cor em volta do tipo e da mensagem. E é assim que se desliga:

```
PYTHON_COLORS=0 script -q -c "python relatorio.py" /dev/null 2>&1 | cat -v |
↪  tail -1
```

```
ValueError: could not convert string to float: 'sem-leitura'^M
```

Como o desligamento automático já cobre redirecionamento, `PYTHON_COLORS=0` serve para o caso menos comum: uma ferramenta de CI que aloca um pseudoterminal e depois grava o resultado num log que ninguém colore de volta.

12. Conserte o programa. Reescreva `extrair_nivel` e `media_niveis` para sobreviver a dados ruins, aplicando o EAFP com exceções específicas:

```
import logging

def extrair_nivel(linha):
    campos = linha.split()
    try:
        return float(campos[3])
    except (IndexError, ValueError) as erro:
        logging.warning("linha ignorada (%s): %r", erro, linha)
        return None

def media_niveis(caminho):
    linhas = ler_linhas(caminho)
```



```
niveis = [n for l in linhas if (n := extrair_nivel(l)) is not None]
if not niveis:
    raise ValueError(f"nenhum nivel valido em {caminho}")
return sum(niveis) / len(niveis)
```

Salve como `relatorio2.py` (com `import logging` no topo) e junte os dois problemas num arquivo só:

```
echo "2026-03-11 22:00:00 vila-sao-joao" >> sinais.log
python relatorio2.py
```

```
WARNING:root:linha ignorada (could not convert string to float: 'sem-leitura'):
↳ '2026-03-11 06:47:55 chacara-do-ze sem-leitura'
WARNING:root:linha ignorada (list index out of range): '2026-03-11 22:00:00
↳ vila-sao-joao'
-20.049999999999997
```

As duas linhas ruins foram avisadas e descartadas — cada uma com o erro que a derrubou e o conteúdo exato da linha — e a média saiu das duas boas. Compare com o comportamento anterior, em que a primeira linha ruim matava o programa e a segunda nem chegava a ser vista.

E o arquivo vazio:

```
python -c "from relatorio2 import media_niveis; media_niveis('vazio.log')"
```

```
File "/tmp/lab07/relatorio2.py", line 24, in media_niveis
    raise ValueError(f"nenhum nivel valido em {caminho}")
ValueError: nenhum nivel valido em vazio.log
```

Agora o erro é um `ValueError` com mensagem sua, que diz **qual** arquivo, em vez de um `ZeroDivisionError` cru vindo de uma divisão que o leitor do log não fez ideia de onde veio.

E o `-20.049999999999997` no lugar de `-20.05`? É o ponto flutuante binário outra vez, como no notebook do Capítulo 5. O Volume 2 explica.

Repare no `:=` do Capítulo 6 dentro da compreensão: ele chama `extrair_nivel` uma vez, guarda o resultado e o testa. Sem ele, seria preciso chamar duas vezes ou escrever um laço.

13. Feche com `deactivate`.

7.8. Resumo

- Traceback imprime do mais antigo ao mais recente. Leia de baixo para cima: primeiro a última linha (tipo e mensagem), depois o quadro mais baixo, depois subindo até o primeiro quadro do seu projeto.
- Quadro em `/usr/lib` quase nunca é bug da biblioteca: é o caminho da chamada. O quadro que interessa é o mais baixo que seja código seu.

7. Erros e tracebacks: como ler o que o Python está dizendo

- Desde o 3.11, a marcação com `~` e `^` aponta a coluna exata da expressão. Desde o 3.10, `NameError` e `AttributeError` sugerem nomes parecidos — e a ausência de sugestão só quer dizer que nada ficou próximo o bastante.
- `TypeError` é tipo errado, `ValueError` é valor errado; `KeyError` e `IndexError` são chave e posição inexistentes; `ModuleNotFoundError` é quase sempre ambiente virtual errado.
- “During handling of the above exception” indica que o seu `except` falhou — corrija o erro de baixo. “The above exception was the direct cause” é `raise ... from`, tradução proposital. `from None` limpa a saída e joga informação fora.
- `SyntaxError` e subclasses não têm quadros, porque o programa nunca começou. Ver quadros significa que ele rodou.
- EAFP vence LBYL: tentar e tratar a exceção é mais curto e mais correto que verificar antes, que sofre de condição de corrida. Capture o tipo mais específico que você sabe tratar; `except:` pelado e `except Exception:` `pass` são os piores hábitos possíveis.

Parte II.

**Volume 2 — Tipos, Operadores e
Expressões**

8. Objetos, variáveis e o modelo de referências

O script de provisionamento roda há meses. Ele pega uma configuração padrão, troca a descrição pelo nome do cliente e devolve a configuração pronta. Hoje o segundo cliente da fila nasceu com uma porta que ninguém pediu.

```
PADRAO = {"vlan": 100, "portas": [1, 2], "descricao": "cliente novo"}

def provisionar(nome):
    config = PADRAO.copy()
    config["descricao"] = nome
    return config

sitio = provisionar("sitio-mangueira")
sitio["portas"].append(3)          # este cliente pediu a porta 3

fazenda = provisionar("fazenda-boa-vista")
print(fazenda)
```

```
{'vlan': 100, 'portas': [1, 2, 3], 'descricao': 'fazenda-boa-vista'}
```

A `fazenda-boa-vista` recebeu a porta 3 que era do `sitio-mangueira`. E tem mais: o `PADRAO`, que ninguém tocou, também foi alterado. Há um `.copy()` bem ali na segunda linha da função, e ele não impediu nada.

Nada disso é bug do Python. É o modelo de objetos da linguagem funcionando exatamente como documentado — e ele é diferente do que a intuição de “variável é uma caixa que guarda um valor” sugere. Este capítulo troca essa intuição pela correta. Sem ela, o Volume 4 inteiro (listas, dicionários, conjuntos) vira uma sequência de surpresas.

8.1. Tudo é objeto, e todo objeto tem três coisas

Em Python, **todo** valor é um objeto: o número 3, a string `"kali"`, a lista `[1, 2]`, a função `len`, o módulo `math`, a própria classe `int`. Não existe a divisão entre “tipos primitivos” e “objetos” que C++, Java ou C# fazem.

```
print(isinstance(1, object), isinstance(len, object), isinstance(int, object))
print(type(int))
```

```
True True True
<class 'type'>
```

8. Objetos, variáveis e o modelo de referências

Até `int` é um objeto — uma instância de `type`. Isso não é curiosidade: é o que permite guardar uma classe numa variável, passar uma função como argumento e escrever decoradores, temas dos volumes 3 e 8.

Todo objeto carrega três atributos, e a distinção entre eles é o assunto deste capítulo:

- **Identidade** — *qual* objeto é. Fixa da criação à destruição, obtida por `id()`.
- **Tipo** — o que ele sabe fazer. Também fixo, obtido por `type()`.
- **Valor** — o conteúdo. Este pode ou não mudar, e é o que separa mutável de imutável.

A `id()` devolve um inteiro que identifica o objeto de forma única enquanto ele existe. No CPython, esse número é o endereço de memória, mas isso é detalhe de implementação: o contrato é apenas “único e constante durante a vida do objeto”. O valor muda a cada execução do programa, então **nunca** o use como chave, nem o cole num teste.

8.2. O nome é uma etiqueta, não uma caixa

A afirmação central: em Python, atribuição **não copia valor**. Ela liga um nome a um objeto que já existe.

Comparar com C ajuda. Em C, `int a = 5;` reserva um espaço na memória chamado `a` e escreve 5 lá dentro; `int b = a;` reserva outro espaço e copia o 5. São dois valores independentes desde o primeiro instante.

Em Python, `a = 5` cria (ou reaproveita) um objeto inteiro com valor 5 e pendura nele a etiqueta `a`. `b = a` pendura no **mesmo objeto** uma segunda etiqueta. Não há cópia, não há segundo espaço.

Figura 8.1 mostra os dois estados que confundem: duas etiquetas no mesmo objeto e o que acontece quando uma delas é religada.

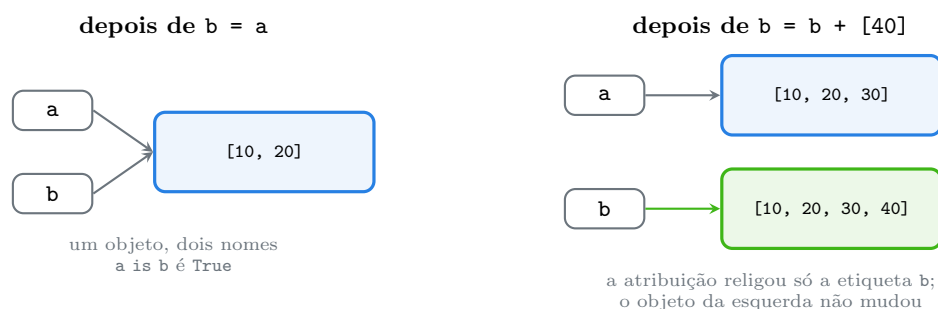


Figura 8.1.: Atribuição liga nomes a objetos. Religar um nome não altera o objeto antigo.

O REPL confirma cada passo:

```
>>> a = [10, 20]
>>> b = a
>>> id(a) == id(b)
True
>>> b.append(30)
>>> a
[10, 20, 30]
>>> b = b + [40]
```

```
>>> id(a) == id(b)
False
>>> a
[10, 20, 30]
>>> b
[10, 20, 30, 40]
```

Duas operações, dois efeitos opostos. `b.append(30)` é uma **mutação**: mexe no objeto, e quem estiver olhando para ele — inclusive `a` — vê a mudança. `b = b + [40]` é uma **relição**: cria um objeto novo e aponta `b` para ele, deixando o antigo intacto.

Essa distinção entre mutar e religar é a chave do capítulo inteiro. Guarde-a: `.append()`, `.sort()`, `d[k] = v`, `lst[0] = x` mutam; `nome = ...` religa.

8.3. Contagem de referências

Se ninguém apontar mais para um objeto, ele não serve para nada. O CPython aproveita isso: cada objeto guarda quantas referências apontam para ele, e quando esse número chega a zero a memória é liberada na hora.

```
import sys

lista = ["a"]
print(sys.getrefcount(lista))
outra = lista
print(sys.getrefcount(lista))
del outra
print(sys.getrefcount(lista))
```

```
2
3
2
```

Por que 2 e não 1? Porque passar `lista` como argumento para `getrefcount` cria mais uma referência temporária — a do próprio parâmetro. `sys.getrefcount` sempre devolve um a mais do que você espera, e a documentação diz isso explicitamente. O que interessa é a **variação**: subiu com `outra = lista`, voltou com `del outra`.

E note o que `del` faz: ele apaga o **nome**, não o objeto. O objeto morre como consequência, se aquele era o último nome.

A contagem de referências tem um ponto cego: objetos que apontam uns para os outros num ciclo nunca chegam a zero, mesmo sem ninguém de fora olhando. Para esse caso existe o coletor de lixo geracional, no módulo `gc`:

```
import gc

class No:
```

```
def __init__(self, nome):
    self.nome = nome
    self.par = None

def __del__(self):
    print("coletado:", self.nome)

x = No("x")
y = No("y")
x.par = y          # x aponta para y
y.par = x          # e y aponta de volta para x
del x, y
print("nomes apagados, objetos ainda vivos")
print("gc.collect() liberou:", gc.collect())
```

```
nomes apagados, objetos ainda vivos
coletado: x
coletado: y
gc.collect() liberou: 9
```

O `print` saiu **antes** dos “coletado” — prova de que apagar os dois nomes não bastou. Cada objeto continuava com uma referência: a do outro. Só o `gc.collect()` detectou o ciclo e desmontou. (O número 9 conta outros objetos internos que também estavam no lixo e varia entre execuções; o que importa é ser maior que zero.)

Sem ciclo, o comportamento é o esperado:

```
z = No("z")
del z
print("depois do del")
```

```
coletado: z
depois do del
```

Contagem de referências para o caso comum, coletor cíclico para o resto. O Volume 15 volta a esse assunto com `weakref` e ajuste de gerações.

8.4. Mutável e imutável

O terceiro atributo — o valor — pode ou não mudar depois da criação. Essa é a divisão mais importante entre os tipos nativos:

Imutáveis	Mutáveis
<code>int, float, complex, bool</code>	<code>list</code>
<code>str, bytes</code>	<code>dict</code>
<code>tuple, frozenset</code>	<code>set, bytearray</code>

Imutáveis	Mutáveis
range, None	instâncias de classes suas (por padrão)

Imutável significa que **não existe operação que altere o objeto**. Toda operação que parece alterar na verdade devolve um objeto novo:

```
nome = "kali"
antes = id(nome)
nome += "-linux"
print(antes == id(nome), nome)
```

```
False kali-linux
```

O += numa string não muda a string: cria outra e religa o nome. Numa lista, o mesmo += muda no lugar — é `list.__iadd__`, equivalente a `extend`. O mesmo operador, dois comportamentos, decididos pelo tipo do objeto da esquerda.

Imutabilidade é rasa. Uma tupla garante que **suas próprias posições** não mudam; nada garante sobre os objetos que estão nelas:

```
config = (["a"], 1)
config[0].append("b")
print(config)
config[1] = 2
```

```
(['a', 'b'], 1)
TypeError: 'tuple' object does not support item assignment
```

A tupla continua apontando para a mesma lista — só que a lista mudou. É por isso que uma tupla com lista dentro não pode ser chave de dicionário:

```
print(hash((1, 2)))
hash([1, 2])
```

```
-3550055125485641917
TypeError: unhashable type: 'list'
```

A regra é direta: hashável exige que o valor não mude, porque o dicionário calcula a posição a partir dele. Muda o valor, o objeto some da estrutura. O Volume 4 detalha.

8.5. *is e == não são a mesma pergunta*

Com o modelo na cabeça, os dois operadores ficam óbvios:

- `==` pergunta **valem o mesmo?** — compara valor, e o tipo decide como.

8. Objetos, variáveis e o modelo de referências

- `is` pergunta **são o mesmo objeto?** — compara identidade, e é o mesmo que `id(a) == id(b)`.

```
a = [1, 2]
b = [1, 2]
print(a == b, a is b)
print(1 == 1.0)
print(True == 1, False == 0)
```

```
True False
True
True True
```

Duas listas com o mesmo conteúdo são iguais e não são a mesma. `1 == 1.0` é verdadeiro apesar dos tipos diferentes, porque `int` e `float` sabem se comparar. E `True == 1` é verdadeiro porque `bool` é subclasse de `int` — assunto do Capítulo 11.

Agora a parte que engana. Digite isto no REPL:

```
>>> r = 257
>>> s = 257
>>> r is s
False
>>> r == s
True
>>> p = 256
>>> q = 256
>>> p is q
True
```

O CPython mantém um cache de inteiros pequenos, de `-5` a `256`, criados uma vez na inicialização. Todo `256` do seu programa é o mesmo objeto; cada `257` tende a ser um objeto novo. Comportamento idêntico com strings: nomes curtos parecidos com identificadores são *internados* automaticamente e compartilhados.

Só que o mesmo teste **dentro de um script** responde outra coisa:

```
def f():
    a = 257
    b = 257
    return a is b

print(f())
print(f.__code__.co_consts)
```

```
True
(None, 257)
```

`True`, e não `False`. O compilador guarda as constantes de cada bloco de código numa tabela — `co_consts` — e o 257 aparece ali **uma vez só**, reaproveitado nas duas atribuições. No REPL cada linha é uma unidade de compilação separada, então não há tabela comum e os objetos saem diferentes.

Isso é mais do que trivialidade. Significa que **o resultado de `is` entre valores iguais depende de otimização do interpretador**, e otimização não é contrato. Construa os valores em tempo de execução e a mágica some:

```
r = 257
s = int("257")
print(r == s, r is s)

w = "manual de python"
z = " ".join(["manual", "de", "python"])
print(z == w, z is w)
```

```
True False
True False
```

Daí a regra, sem exceção prática: **`is` só para singletons**. `None`, `True`, `False`, e sentinelas que você mesmo criou. Para qualquer outra coisa, `==`.

O Python leva isso tão a sério que avisa quando você erra:

```
SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
```

Esse aviso aparece na compilação, antes de o programa rodar. Ele não impede nada — e o código que ele denuncia costuma funcionar em teste e falhar em produção, quando o número passa de 256.

8.6. Cópia rasa e cópia profunda

Voltemos ao bug da abertura. `PADRAO.copy()` fez uma cópia **rasa**: criou um dicionário novo com as mesmas três chaves apontando para os mesmos três objetos. A string e o inteiro não importam, porque são imutáveis. A lista importa muito:

```
import copy

original = {"cliente": "sitio", "portas": [1, 2]}
raso = copy.copy(original)
raso["cliente"] = "fazenda"
raso["portas"].append(3)
print(original)
print(raso)
```

```
{'cliente': 'sitio', 'portas': [1, 2, 3]}
{'cliente': 'fazenda', 'portas': [1, 2, 3]}
```

8. Objetos, variáveis e o modelo de referências

Trocar `raso["cliente"]` foi religação numa chave do dicionário novo — o original não sentiu. Já `raso["portas"].append(3)` foi mutação no objeto compartilhado, e os dois viram.

`copy.deepcopy` percorre a estrutura inteira e duplica tudo que encontra:

```
original = {"cliente": "sitio", "portas": [1, 2]}
fundo = copy.deepcopy(original)
fundo["portas"].append(3)
print(original)
print(fundo)
```

```
{'cliente': 'sitio', 'portas': [1, 2]}
{'cliente': 'sitio', 'portas': [1, 2, 3]}
```

Figura 8.2 põe os dois lado a lado.

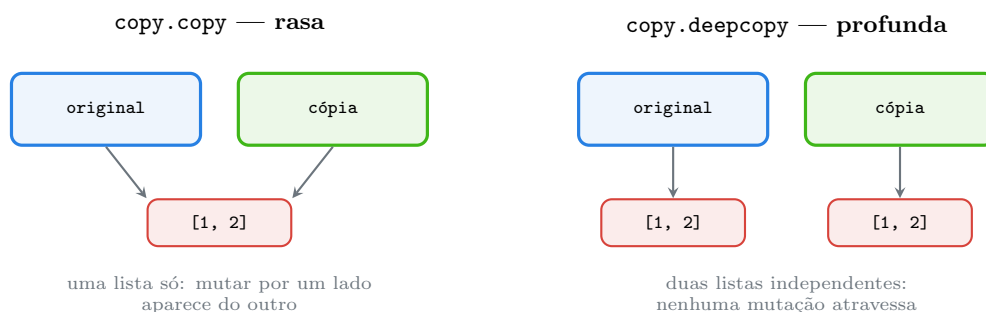


Figura 8.2.: A cópia rasa duplica o recipiente; a profunda duplica também o conteúdo.

Para listas há três formas equivalentes de cópia rasa, e todas produzem objeto novo:

```
a = [1, 2, 3]
b = a[:]
c = list(a)
```

O truque de multiplicação é a versão mais cruel dessa armadilha:

```
grade = [[0] * 3] * 2
grade[0][0] = 9
print(grade)

grade2 = [[0] * 3 for _ in range(2)]
grade2[0][0] = 9
print(grade2)
```

```
[[9, 0, 0], [9, 0, 0]]
[[9, 0, 0], [0, 0, 0]]
```

`[x] * 2` repete a **referência** duas vezes, não o objeto. Para matriz, sempre compreensão de lista — tema do Capítulo 26.

8.7. Argumentos de função

A pergunta “Python passa por valor ou por referência?” não tem resposta boa, porque a linguagem não faz nem uma coisa nem outra no sentido clássico. O que ela faz é **ligar o nome do parâmetro ao mesmo objeto do argumento**. Sabendo isso, o comportamento é previsível:

```
def acrescenta(destino, valor):
    destino.append(valor)          # muta o objeto

def substitui(destino, valor):
    destino = destino + [valor]    # religa o nome local
    return destino

alvo = [1]
acrescenta(alvo, 2)
print(alvo)
retorno = substitui(alvo, 3)
print(alvo, retorno)
```

```
[1, 2]
[1, 2] [1, 2, 3]
```

`acrescenta` mudou o objeto de quem chamou. `substitui` não: a atribuição dentro da função religou apenas o nome local `destino`, e o objeto de fora ficou como estava. Nenhuma regra especial sobre funções — é a mesma diferença entre mutar e religar, dentro de outro escopo.

E daí vem a armadilha mais famosa da linguagem:

```
def registrar(evento, log=[]):
    log.append(evento)
    return log

print(registrar("queda"))
print(registrar("religou"))
print(registrar.__defaults__)
```

```
['queda']
['queda', 'religou']
(['queda', 'religou'],)
```

O valor padrão é avaliado **uma vez**, quando o `def` executa, e fica guardado em `registrar.__defaults__` pelo resto da vida do programa. Duas chamadas independentes compartilham a mesma lista. A correção é o padrão de sentinela:

```
def registrar_ok(evento, log=None):
    if log is None:
        log = []
    log.append(evento)
    return log

print(registrar_ok("queda"))
print(registrar_ok("religou"))
```

```
['queda']
['religou']
```

Repare no `is None` — o único `is` que deve aparecer no código do dia a dia.

8.8. Jeito Pythônico

is para singletons, == para valor. `if x is None:` é a forma canônica, recomendada pela PEP 8, e não é preferência estética: `==` chama `__eq__`, que uma classe pode redefinir para dizer que é igual a `None` — enquanto `is` compara identidade e não pode mentir. `if not x:` **não** é substituto, porque `0`, `""`, `[]` e `False` também são falsos (Capítulo 11).

Não copie o que não muda. Copiar `str`, `int` ou `tuple` (de imutáveis) é gasto sem função — o objeto não pode mudar, compartilhá-lo é seguro. `copy.copy` de uma tupla imutável devolve a própria tupla.

Prefira o valor novo à mutação, quando for barato. Código que não muda o argumento de quem chamou é mais fácil de testar e não produz o bug da abertura:

```
# antipadrão: efeito colateral escondido no argumento
def normalizar(config):
    config["descricao"] = config["descricao"].strip().lower()
```

```
# pythônico: devolve novo, não mexe no de quem chamou
def normalizar(config):
    return config | {"descricao": config["descricao"].strip().lower()}
```

O operador `|` entre dicionários existe desde o **Python 3.9** (PEP 584) e produz um dicionário novo. Quando a mutação for mesmo o objetivo, deixe isso claro no nome (`atualizar_config`) e não devolva nada — é o contrato de `list.sort()`, que muda e devolve `None`, contra `sorted()`, que devolve lista nova. Essa simetria se repete por toda a biblioteca padrão e é um bom guia para desenhar as suas funções.

Use tupla quando a coleção não deve mudar. É documentação executável: quem lê sabe que o dado é fixo, e uma mutação acidental vira `TypeError` em vez de bug silencioso.

deepcopy é o último recurso. Ele é lento, percorre grafos inteiros, precisa lidar com ciclos e trava em objetos não copiáveis (soquete, arquivo aberto). Muitas vezes a resposta melhor é construir o objeto do zero — trocar a constante `PADRAO` por uma função `padrao()` que devolve

um dicionário novo a cada chamada, ou usar `dataclasses.field(default_factory=list)`, do Capítulo 46, que é o mesmo padrão de sentinela com outro nome.

Três antipadrões, para fechar:

```
if type(x) is list:           # frágil: recusa subclasses de list
if x is 257:                  # SyntaxWarning; funciona por acidente
def f(itens=[]):              # padrão mutável compartilhado entre chamadas
```

O primeiro se escreve `isinstance(x, list)`; o segundo, `x == 257`; o terceiro, `itens=None` com sentinela.

8.9. Laboratório

Objetivo: reproduzir o bug da abertura, provar cada afirmação do capítulo com `id`, `is` e `sys.getrefcount`, e consertar o script de provisionamento.

1. Ambiente.

```
mkdir -p ~/lab08 && cd ~/lab08
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. Reproduza o bug. Crie `provisionar.py` com o código da abertura e rode. Confirme que os dois clientes e o PADRAO compartilham a mesma lista:

```
python -c "
from provisionar import PADRAO, provisionar
a = provisionar('sitio'); a['portas'].append(3)
b = provisionar('fazenda')
print(b)
print(a['portas'] is b['portas'] is PADRAO['portas'])
"
```

```
{'vlan': 100, 'portas': [1, 2, 3], 'descricao': 'fazenda'}
True
```

O `True` é o diagnóstico: uma lista, três donos.

3. Etiquetas, não caixas. No REPL, refaça a sequência da Figura 8.1 e confirme cada `id`. Depois responda sem rodar, e só então confirme: depois de `x = [1]`; `y = x`; `y = [2]`, quanto vale `x`?

4. Mutar contra religar. Rode:

8. Objetos, variáveis e o modelo de referências

```
python -c "  
n = 'kali'; antes = id(n); n += '-linux'; print(antes == id(n))  
l = [1]; antes = id(l); l += [2]; print(antes == id(l))  
"
```

```
False  
True
```

O mesmo +=: religou a string, mudou a lista. Explique a diferença em uma frase antes de seguir.

5. Conte referências.

```
python -c "  
import sys  
lista = ['a']  
print(sys.getrefcount(lista))  
outra = lista  
print(sys.getrefcount(lista))  
del outra  
print(sys.getrefcount(lista))  
"
```

```
2  
3  
2
```

Justifique o 2 inicial (a referência do parâmetro de `getrefcount`).

6. Veja um ciclo sobreviver ao del. Crie `ciclo.py` com a classe `No` do capítulo e rode. Confirme que os coletado: saem **depois** do `print`, e só após o `gc.collect()`.

7. Imutabilidade é rasa.

```
python -c "  
t = (['a'], 1)  
t[0].append('b')  
print(t)  
t[1] = 2  
"
```

```
(['a', 'b'], 1)  
Traceback (most recent call last):  
  File "<string>", line 5, in <module>  
    t[1] = 2  
    ~~~~  
TypeError: 'tuple' object does not support item assignment
```

A lista dentro da tupla aceitou o `append`; a posição da tupla recusou a atribuição. Duas garantias diferentes, no mesmo objeto.

8. Provoque a divergência do is. Primeiro no REPL, uma linha por vez:


```
>>> r = 257
>>> s = 257
>>> r is s
False
```

Agora o mesmo em arquivo:

```
python -c "r = 257; s = 257; print(r is s)"
```

```
True
```

Mesmo código, respostas opostas. A causa é a tabela de constantes do bloco compilado:

```
python -c "
def f():
    a = 257
    b = 257
    return a is b
print(f.__code__.co_consts)
"
```

```
(None, 257)
```

Um 257 só na tabela, usado nas duas atribuições. É por isso que `is` vale apenas para singletons.

9. Ouça o aviso.

```
python -c "print(1 is 1.0)"
```

```
<string>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
False
```

10. Compare as cópias. Crie `copias.py` com os blocos de `copy.copy` e `copy.deepcopy` do capítulo, rode e confirme as quatro linhas de saída. Depois teste a multiplicação:

```
python -c "
g = [[0]*3]*2; g[0][0] = 9; print(g)
h = [[0]*3 for _ in range(2)]; h[0][0] = 9; print(h)
"
```

```
[[9, 0, 0], [9, 0, 0]]
[[9, 0, 0], [0, 0, 0]]
```

11. Argumentos. Crie `args.py` com `acrescenta` e `substitui`, rode e explique por que só a primeira alterou alvo.

12. Padrão mutável.

```
python -c "  
def registrar(evento, log=[]):  
    log.append(evento); return log  
print(registrar('queda')); print(registrar('religou'))  
print(registrar.__defaults__)  
"
```

```
['queda']  
['queda', 'religou']  
(['queda', 'religou'],)
```

O `__defaults__` mostra a lista guardada na função. Reescreva com `log=None` e confirme que as duas chamadas voltam a ser independentes.

13. Conserte o provisionamento. Duas soluções, e a segunda é melhor:

```
import copy  
  
def provisionar_a(nome):  
    config = copy.deepcopy(PADRAO)  
    config["descricao"] = nome  
    return config  
  
def padrao():  
    return {"vlan": 100, "portas": [1, 2], "descricao": "cliente novo"}  
  
def provisionar_b(nome):  
    return padrao() | {"descricao": nome}
```

Rode o teste do passo 2 contra as duas:

```
{'vlan': 100, 'portas': [1, 2], 'descricao': 'fazenda'}  
False
```

O `False` é o conserto: cada cliente com a sua lista. A versão `b` dispensa `deepcopy`, não depende de uma constante global mutável e é a que você quer manter.

14. Feche com `deactivate`.

8.10. Resumo

- Todo valor em Python é objeto, e todo objeto tem identidade (`id`), tipo (`type`) e valor. Identidade e tipo são fixos; valor muda só nos mutáveis.
- Atribuição não copia: liga um nome a um objeto existente. `b = a` cria uma segunda etiqueta no mesmo objeto.

- Mutar (`append`, `d[k] = v`) altera o objeto e é visto por todos os nomes que apontam para ele; religar (`nome = ...`) troca só a etiqueta e deixa o objeto antigo intacto.
- O CPython libera memória por contagem de referências; ciclos exigem o coletor do módulo `gc`. `del` apaga o nome, não o objeto.
- `==` compara valor, `is` compara identidade. Use `is` apenas com `None`, `True`, `False` e sentinelas — o resultado de `is` entre iguais depende de cache de inteiros e da tabela de constantes do bloco compilado, e muda entre REPL e script.
- Cópia rasa (`copy.copy`, `d.copy()`, `lst[:]`) duplica o recipiente e compartilha o conteúdo; `copy.deepcopy` duplica tudo, com custo. `[[0]*3]*2` repete referência, não objeto.
- Argumentos são ligados ao mesmo objeto do chamador: a função pode mutá-lo, mas religar o parâmetro não afeta quem chamou. Valor padrão é avaliado uma vez no `def` — para mutáveis, use `None` como sentinela.

9. Números inteiros, de ponto flutuante e complexos

O fabricante do equipamento entrega uma ferramenta em C que classifica as leituras de potência óptica em faixas de 2 dBm. Você reescreveu a mesma classificação em Python para integrar ao relatório, conferiu com uma dúzia de leituras e bateu tudo. Aí chegou a primeira leitura negativa.

```
printf("%d %d\n", -7 / 2, -7 % 2);
```

```
-3 -1
```

```
print(-7 // 2, -7 % 2)
```

```
-4 1
```

Nenhuma das duas está errada. C trunca em direção ao zero; Python arredonda para baixo. São duas convenções legítimas para dividir inteiros negativos, e a linguagem escolheu a que ninguém que vem de C espera.

Este capítulo é sobre os três tipos numéricos nativos — `int`, `float` e `complex` — e sobre as decisões de projeto que os tornam diferentes dos equivalentes de outras linguagens. A precisão do `float`, que é um assunto por si só, fica para o Capítulo 10.

9.1. `int` não tem teto

Em C, Java ou Rust, um inteiro tem tamanho fixo e estoura. Em Python, não:

```
import math

print(2 ** 100)
print(math.factorial(30))
```

```
1267650600228229401496703205376
265252859812191058636308480000000
```

Não há `long`, não há `int64`, não há `OverflowError` em aritmética inteira. O `int` do Python é de **precisão arbitrária**: cresce enquanto houver memória. O CPython guarda o número como uma sequência de dígitos de 30 bits, e o objeto engorda conforme necessário:

```
import sys

print(sys.getsizeof(0), sys.getsizeof(2 ** 30), sys.getsizeof(2 ** 100))
print((2 ** 100).bit_length(), (255).bit_length())
```

```
28 32 40
101 8
```

Vinte e oito bytes para o zero — o cabeçalho de qualquer objeto Python — e quatro bytes a mais por dígito interno. `bit_length()` diz quantos bits são necessários para representar o valor, e é a forma correta de perguntar “de que tamanho é este número”.

O preço da comodidade é desempenho: aritmética com inteiro grande é feita em software, não pela ALU do processador. Para a maioria dos programas isso não aparece; quando aparece, a resposta é NumPy, do Volume 13, que usa inteiros de tamanho fixo.

Há **um** limite, e ele não é de aritmética, é de conversão para texto:

```
print(sys.get_int_max_str_digits())
str(10 ** 5000)
```

```
4300
ValueError: Exceeds the limit (4300 digits) for integer string conversion; use
↪ sys.set_int_max_str_digits() to increase the limit
```

Esse limite entrou no **Python 3.11** (e foi retroportado até o 3.9) para fechar uma negação de serviço: converter um inteiro gigante para decimal é quadrático, e um servidor que aceitasse `int(entrada_do_usuario)` podia ser travado com uma string de alguns megabytes. A aritmética continua ilimitada; só a conversão decimal tem freio, e ele se ajusta com `sys.set_int_max_str_digits()`. Conversão para hexadecimal, octal ou binário não é afetada, porque nessas bases o algoritmo é linear.

9.2. Literais e bases

```
print(1_000_000, 0xFF, 0o755, 0b1010)
print(hex(255), oct(493), bin(10))
print(int("ff", 16), int("1010", 2), int(" 42  "))
```

```
1000000 255 493 10
0xff 0o755 0b1010
255 10 42
```

O sublinhado como separador de milhar vem da **PEP 515** (Python 3.6) e é ignorado pelo interpretador: serve só para o leitor. Use em constantes grandes — `1_500_000` e `0xFF_FF` custam nada e evitam a conferência de zeros no olho.

`int(texto, base)` aceita a base como segundo argumento e ignora espaço em volta. O que ele **não** aceita é ponto decimal: `int("3.5")` levanta `ValueError`. Para isso, `int(float("3.5"))`, e aí a truncagem é explícita.

9.3. As duas divisões

Python separa o que outras linguagens juntam. `/` é **sempre** divisão real e **sempre** devolve `float`, mesmo quando dá exato:

```
print(7 / 2, type(7 / 2))
print(4 / 2, type(4 / 2))
```

```
3.5 <class 'float'>
2.0 <class 'float'>
```

Essa foi uma das mudanças que quebraram a compatibilidade no Python 3, e por bom motivo: no Python 2, `1/2` valia 0, e o número de bugs que isso causou é folclore da comunidade.

Para divisão inteira existe `//`, o operador de piso. E aqui está a raiz do problema da abertura:

```
for a, b in [(7, 3), (-7, 3), (7, -3), (-7, -3)]:
    print(a, b, a // b, a % b, a == b * (a // b) + a % b)
```

```
7 3 2 1 True
-7 3 -3 2 True
7 -3 -3 -2 True
-7 -3 2 -1 True
```

Dois regras explicam a tabela inteira:

1. `a // b` arredonda para **baixo** (em direção a $-\infty$), não em direção ao zero.
2. `a % b` tem sempre o **sinal do divisor**.

E a última coluna mostra por que: a identidade `a == b * (a // b) + a % b` vale nos quatro casos. Python escolheu preservar essa invariante; C escolheu preservar `(-a)/b == -(a/b)`. Não dá para ter as duas.

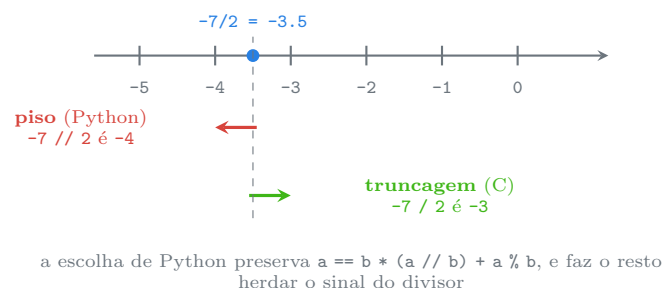


Figura 9.1.: Duas convenções para dividir negativos. Python arredonda para baixo; C trunca.

Na prática, a convenção de Python é a que você quer quase sempre, porque é a aritmética de relógio:

```
print(-1 % 24)
```

23

Uma hora antes da meia-noite é 23h. Em C, `-1 % 24` é `-1`, e quem usa isso como índice de um vetor de 24 posições ganha uma falha de segmentação. A convenção “estranha” de Python é a que não quebra.

`divmod` devolve os dois de uma vez, com uma divisão só:

```
print(divmod(-7, 2))
```

```
(-4, 1)
```

E cuidado com o tipo: `//` entre floats continua sendo piso, mas devolve `float`.

```
print(7.0 // 2, type(7.0 // 2))
```

```
3.0 <class 'float'>
```

Potenciação tem operador próprio, `**`, e a versão de três argumentos de `pow` calcula `(base ** expo) % mod` sem construir o número intermediário — é o que torna criptografia de chave pública viável:

```
print(2 ** 10, 2 ** -1, pow(3, 100, 7))
print(2 ** 0.5)
```

```
1024 0.5 4
1.4142135623730951
```

Repare: `2 ** -1` devolve `0.5`, um `float`. Expoente negativo sobre inteiro sai da aritmética inteira, como tem de ser.

9.4. Operadores bit a bit

Sobre `int`, os seis operadores bit a bit funcionam como em C, com a diferença de que o número não tem largura fixa:

```
ip = 0xC0A80105          # 192.168.1.5
mascara = 0xFFFFFF00     # /24

print(hex(ip & mascara))
print(hex(ip | 0xFF))
print(hex(ip >> 8), hex(ip & 0xFF))
print(~5, ~5 == -(5 + 1))
```



```
0xc0a80100
0xc0a801ff
0xc0a801 0x5
-6 True
```

& para máscara, | para ligar bits, ^ para inverter, >> e << para deslocar. O ~ é o que confunde: como o `int` é de precisão arbitrária e com sinal, o complemento de um número é sempre $-(n + 1)$. Não existe “inverter os 32 bits” sem dizer quais 32 — para isso, `n ^ 0xFFFFFFFF`.

Para endereçamento IP de verdade, use `ipaddress` da biblioteca padrão em vez de fazer a conta à mão; o exemplo acima é para ver o operador funcionando.

9.5. float é IEEE 754 de 64 bits

O `float` do Python é o `double` do C: 64 bits, no formato IEEE 754 de precisão dupla. Não há `float32`, não há decimal — há exatamente um tipo de ponto flutuante nativo, e ele tem uma estrutura fixa.

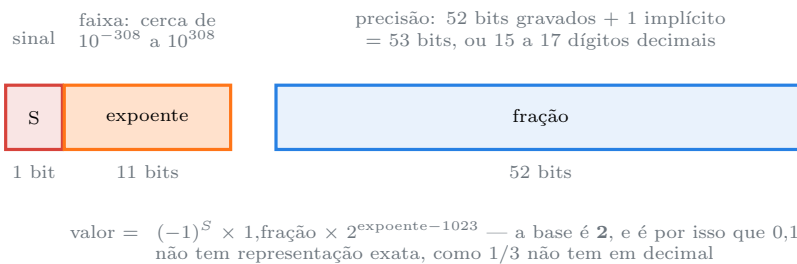


Figura 9.2.: Os 64 bits de um `float`: um de sinal, onze de expoente, cinquenta e dois de fração.

Os limites do formato estão em `sys.float_info`:

```
print(sys.float_info.max)
print(sys.float_info.min)
print(sys.float_info.epsilon)
print(sys.float_info.dig, sys.float_info.mant_dig)
```

```
1.7976931348623157e+308
2.2250738585072014e-308
2.220446049250313e-16
15 53
```

`dig` é o número de dígitos decimais que sempre sobrevivem a uma ida e volta; `mant_dig`, os 53 bits de precisão da figura; `epsilon`, a menor diferença perceptível em torno de 1,0.

Diferente do `int`, o `float` **estoura** — e não com exceção:

```
print(1e308 * 10, -1e308 * 10)
print(float("inf"), float("-inf"), float("nan"))
```

```
inf -inf
inf -inf nan
```

Ultrapassar `float_info.max` produz `inf`, silenciosamente. `inf` se comporta como o infinito matemático (`inf > 10**100` é verdadeiro), mas `inf - inf` é `nan` — *not a number*, o resultado de uma operação indefinida.

E `nan` tem uma propriedade que quebra a intuição:

```
n = float("nan")
print(n == n, n is n, math.isnan(n))
```

```
False True True
```

nan não é igual a si mesmo. É o que a norma IEEE 754 manda: comparar com um valor indefinido não pode dar verdadeiro. Consequência prática: `if x == x` é um teste válido de “não é nan”, e `lista.count(float('nan'))` pode devolver 1 mesmo com a igualdade falsa, porque as coleções testam identidade antes de igualdade. Para detectar, sempre `math.isnan`.

Divisão por zero, no entanto, é exceção em Python, e não `inf` como em C:

```
1.0 / 0.0
```

```
ZeroDivisionError: float division by zero
```

9.6. round e a regra do banqueiro

```
print(round(2.5), round(3.5), round(-2.5), round(0.5))
```

```
2 4 -2 0
```

`round(2.5)` é 2, não 3. Isso não é bug e não é o `float`: é o **arredondamento para o par mais próximo** (*banker's rounding*), especificado na norma IEEE 754 e adotado pelo Python 3. Quando o valor cai exatamente no meio, vai para o par. A razão é estatística: arredondar sempre para cima enviesaria a soma de uma série de valores para cima, e em contabilidade esse viés acumula dinheiro do nada.

Já este caso é o `float`:

```
print(round(2.675, 2))
```

```
2.67
```

Nada a ver com regra de arredondamento: 2.675 não é exatamente 2,675 em binário — é um tiquinho menos —, então o vizinho mais próximo com duas casas é mesmo 2,67. Quando o arredondamento decimal precisa ser exato, o tipo certo é `Decimal`, e esse é o assunto do próximo capítulo.

`round` com o segundo argumento negativo arredonda para a esquerda da vírgula, o que é útil e pouco conhecido:

```
print(round(1234, -2))
```

```
1200
```

E note a diferença entre truncar e arredondar para baixo:

```
print(int(-3.99), math.floor(-3.99), math.ceil(-3.99), math.trunc(-3.99))
```

```
-3 -4 -3 -3
```

`int()` e `math.trunc()` truncam em direção ao zero; `math.floor()` vai para baixo. Com números positivos os dois coincidem, e é por isso que o erro só aparece quando chega o primeiro negativo — exatamente como na abertura.

9.7. A torre numérica

Misturar tipos numéricos funciona, e o resultado sobe para o tipo mais geral: `int` → `float` → `complex`.

```
print(3 + 0.5, type(3 + 0.5))
print(1 + True, int(True))
```

```
3.5 <class 'float'>
2 1
```

`bool` é subclasse de `int`, com `True` valendo 1 — assunto do Capítulo 11. A promoção é automática **entre** esses tipos e não existe fora deles: `"3" + 4` continua sendo `TypeError`, como no Capítulo 7.

Um efeito colateral da promoção merece atenção: `int` grande somado a `float` perde precisão, porque o resultado é `float`.

```
print(10 ** 16 + 1.0)
print(float(2 ** 53) == float(2 ** 53 + 1))
```

```
1e+16
True
```

Acima de 2^3 o `float` não distingue mais inteiros consecutivos. Se o seu domínio tem identificadores grandes — carimbo de tempo em nanossegundos, chave primária, valor em centavos —, mantenha tudo em `int` e não deixe um `float` entrar por acidente no meio da conta.

9.8. Complexos

Python tem números complexos nativos, sem importar nada. O sufixo é `j`, não `i`, por convenção da engenharia elétrica:

```
z = 3 + 4j
print(z, z.real, z.imag, abs(z), z.conjugate())
print((1 + 2j) * (3 - 1j))
```

```
(3+4j) 3.0 4.0 5.0 (3-4j)
(5+5j)
```

`abs()` de um complexo é o módulo — por isso 5,0 para `3+4j`. As partes `real` e `imag` são sempre `float`.

Para funções sobre complexos existe `cmath`, o par de `math`:

```
import cmath

print(cmath.sqrt(-1))
print(cmath.polar(1j))
```

```
1j
(1.0, 1.5707963267948966)
```

`math.sqrt(-1)` levanta `ValueError`; `cmath.sqrt(-1)` devolve `1j`. É a diferença entre “domínio real” e “domínio complexo”, e não uma inconsistência.

Complexos aparecem pouco em código de aplicação e muito em processamento de sinais, FFT e cálculo numérico — território do Volume 13.

9.9. Jeito Pythonico

Escolha o operador pelo resultado que você quer, não pelo tipo da entrada. / quando o resultado é uma medida (média, taxa, proporção); // quando é uma contagem (quantos blocos cabem, quantas páginas). Escrever `int(a / b)` para dividir inteiros é o antipadrão clássico: passa pelo `float`, perde precisão acima de 2^3 e trunca em vez de arredondar para baixo.

```
# antipadrão: passeia pelo float sem precisar
paginas = int(total / por_pagina)
```

```
# pythonico: aritmética inteira do começo ao fim
paginas = -(-total // por_pagina)      # divisão inteira arredondando para cima
```

O truque `-(-a // b)` é o idioma consagrado para “divisão para cima” sem `math.ceil` e sem `float`. Quando a legibilidade importar mais que a economia, `math.ceil(Fraction(total,`

`por_pagina))` diz a mesma coisa em voz alta e continua exato — `Fraction` é assunto do próximo capítulo.

Nunca compare float com ==. Use `math.isclose`, que existe desde o Python 3.5 (PEP 485):

```
print(0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3))
```

```
False
True
```

`math.isclose` usa tolerância relativa por padrão, o que é o certo para grandezas de escala desconhecida; para valores próximos de zero, passe `abs_tol` também, porque a tolerância relativa a zero é zero.

Não use float para dinheiro. É a regra que mais aparece em revisão de código e a do capítulo seguinte: centavos em `int` ou valores em `Decimal`, nunca reais em `float`.

Prefira comparação encadeada. `1 < x < 10` é Python legítimo, avalia `x` uma vez só e lê como matemática:

```
if 0 <= indice < len(itens):
```

Use _ nos literais longos e as bases nativas nas máscaras. `0xFF_FF_FF_00` diz “máscara de rede” para quem lê; `4294967040` não diz nada.

Um antipadrão frequente:

```
# antipadrão: testar nan com igualdade
if resultado == float("nan"):      # nunca é verdadeiro
```

```
# pythônico
if math.isnan(resultado):
```

Sobre versões, o que mudou de fato:

- **Python 3.0** — `/` virou divisão real; `//` passou a ser a divisão inteira. É a diferença de comportamento mais comum em código antigo portado.
- **Python 3.6** — sublinhado em literais numéricos (PEP 515).
- **Python 3.8** — `math.prod`, o produto que faltava ao lado de `sum`.
- **Python 3.9** — `math.lcm`, ao lado de `math.gcd`.
- **Python 3.11** — limite de dígitos na conversão de `int` para `str`; `math.exp2` e `math.cbrt`.
- **Python 3.12** — `math.sumprod`, para produto escalar sem NumPy.

9.10. Laboratório

Objetivo: medir o `int` de precisão arbitrária, reproduzir a divergência C/Python da abertura, ver os limites do `float` e classificar leituras negativas sem errar a faixa.

1. Ambiente.

```
mkdir -p ~/lab09 && cd ~/lab09
python3 -m venv .venv && source .venv/bin/activate
python --version
```

```
Python 3.13.12
```

2. Meça o inteiro crescendo.

```
python -c "
import sys
for e in (0, 30, 100, 1000):
    n = 2 ** e
    print(e, n.bit_length(), sys.getsizeof(n))
"
```

```
0 1 28
30 31 32
100 101 40
1000 1001 160
```

Cada 30 bits de valor custam 4 bytes de objeto. Nenhum estouro em vista.

3. Encoste no limite que existe.

```
python -c "
import sys
print(sys.get_int_max_str_digits())
print(len(hex(10 ** 5000)))
print(len(str(10 ** 5000)))
"
```

```
4300
4155
Traceback (most recent call last):
  File "<string>", line 5, in <module>
    print(len(str(10 ** 5000)))
    ~~~~
ValueError: Exceeds the limit (4300 digits) for integer string conversion; use
↪ sys.set_int_max_str_digits() to increase the limit
```

Hexadecimal passa com 4155 caracteres, decimal não passa com 5001. Agora destrave:

```
python -c "
import sys
sys.set_int_max_str_digits(6000)
print(len(str(10 ** 5000)))
"
```

```
5001
```

Pense em por que o padrão é conservador num servidor que aceita número vindo do usuário.

4. Reproduza a divergência da abertura. Se houver gcc:

```
cat > div.c <<'EOF'
#include <stdio.h>
int main(void) {
    printf("%d %d\n", -7 / 2, -7 % 2);
    printf("%d %d\n", -1 / 24, -1 % 24);
    return 0;
}
EOF
gcc -O2 -o div div.c && ./div
python -c "print(-7 // 2, -7 % 2); print(-1 // 24, -1 % 24)"
```

```
-3 -1
0 -1
```

```
-4 1
-1 23
```

A segunda linha é a que importa: a hora anterior à meia-noite é 23 em Python e -1 em C.

5. Confira a invariante.

```
python -c "
for a in (-7, 7):
    for b in (-3, 3):
        print(a, b, a // b, a % b, a == b * (a // b) + a % b)
"
```

```
-7 -3 2 -1 True
-7 3 -3 2 True
7 -3 -3 -2 True
7 3 2 1 True
```

Quatro True. Anote a regra do sinal do resto — sempre o do divisor.

6. Truncar não é arredondar para baixo.

9. Números inteiros, de ponto flutuante e complexos

```
python -c "  
import math  
for x in (3.99, -3.99):  
    print(x, int(x), math.floor(x), math.ceil(x), math.trunc(x))  
"
```

```
3.99 3 3 4 3  
-3.99 -3 -4 -3 -3
```

Na linha positiva, `int` e `floor` concordam. Na negativa, não. É o motivo de o bug só aparecer com dado negativo.

7. Máscara de rede na mão.

```
python -c "  
ip, m = 0xC0A80105, 0xFFFFF00  
print(hex(ip & m), hex(ip | ~m & 0xFFFFFFFF))  
print('.'.join(str((ip >> d) & 0xFF) for d in (24, 16, 8, 0)))  
"
```

```
0xc0a80100 0xc0a801ff  
192.168.1.5
```

Rede e broadcast do /24. Repare no `& 0xFFFFFFFF` depois do `~`: sem ele, o complemento de um `int` sem largura fixa vira negativo.

8. Os limites do float.

```
python -c "  
import sys  
fi = sys.float_info  
print(fi.max, fi.epsilon, fi.dig, fi.mant_dig)  
print(fi.max * 1.0000001)  
print(2.0 ** 53, 2.0 ** 53 == 2.0 ** 53 + 1)  
"
```

```
1.7976931348623157e+308 2.220446049250313e-16 15 53  
inf  
9007199254740992.0 True
```

O estouro virou `inf` sem exceção nenhuma. E acima de 2^3 o `float` não conta mais de um em um.

9. nan contra si mesmo.

```
python -c "  
import math  
n = float('nan')  
print(n == n, math.isnan(n))
```



```
print([n] == [n])
print(math.inf - math.inf, math.inf > 10**100)
"
```

```
False True
True
nan True
```

A terceira linha é a pegadinha: a lista compara identidade antes de igualdade, então `[n] == [n]` é `True` mesmo com `n == n` falso.

10. Arredondamento do banqueiro.

```
python -c "
print([round(x) for x in (0.5, 1.5, 2.5, 3.5, 4.5)])
print(round(2.675, 2), round(1234, -2))
"
```

```
[0, 2, 2, 4, 4]
2.67 1200
```

Todos os resultados são pares. Some `[round(x) for x in ...]` e compare com a soma de arredondar sempre para cima — a diferença é o viés que a regra elimina.

11. Complexos.

```
python -c "
import cmath
z = 3 + 4j
print(z.real, z.imag, abs(z))
print(cmath.sqrt(-1), cmath.polar(1j))
"
```

```
3.0 4.0 5.0
1j (1.0, 1.5707963267948966)
```

12. Classifique as leituras sem errar a faixa. O problema da abertura, resolvido. Crie `faixas.py`:

```
"""Classifica leituras de potencia optica em faixas de 2 dBm."""

LEITURAS = [-18.4, -21.7, -7.0, -6.9, -3.0, 0.0]
LARGURA = 2

def faixa(valor):
    """Indice da faixa, arredondando para baixo, como manda a reta numerica."""
    return int(valor // LARGURA)
```

```
for leitura in LEITURAS:
    inicio = faixa(leitura) * LARGURA
    print(f"{leitura:>6} -> faixa {faixa(leitura):>3} [{inicio}, {inicio +
↵ LARGURA})")
```

```
python faixas.py
```

```
-18.4 -> faixa -10 [-20, -18)
-21.7 -> faixa -11 [-22, -20)
-7.0 -> faixa -4 [-8, -6)
-6.9 -> faixa -4 [-8, -6)
-3.0 -> faixa -2 [-4, -2)
0.0 -> faixa 0 [0, 2)
```

Cada leitura caiu no intervalo que a contém, inclusive a de borda exata (-7.0). Agora troque `//` por `int(valor / LARGURA)` e rode de novo:

```
-7.0 -3 [-6, -4)
-6.9 -3 [-6, -4)
```

As duas leituras foram para a faixa -3, cujo intervalo é [-6, -4) — e que **não contém** nenhuma das duas. Esse é o bug da ferramenta em C, e a linha que o corrige é uma barra a mais.

13. Feche com deactivate.

9.11. Resumo

- `int` tem precisão arbitrária: não estoura, cresce com a memória, e `bit_length()` diz o tamanho. O único limite é a conversão para decimal, freada em 4300 dígitos desde o 3.11 por segurança.
- `/` é sempre divisão real e devolve `float`; `//` é divisão de **piso**, arredondando para $-\infty$, e `%` herda o sinal do divisor. A escolha preserva `a == b * (a // b) + a % b` e é o oposto da truncagem de C.
- `int()` e `math.trunc()` truncam em direção ao zero; `math.floor()` vai para baixo. Com valores positivos coincidem — o bug só nasce com o primeiro negativo.
- `float` é IEEE 754 de 64 bits: 53 bits de precisão, 15 a 17 dígitos decimais, faixa até $\sim 10^3$. Estouro vira `inf` em silêncio; `nan` não é igual a si mesmo e só `math.isnan` detecta.
- `round` usa arredondamento para o par mais próximo, por norma e para evitar viés; `round(x, -n)` arredonda à esquerda da vírgula.
- Tipos numéricos se promovem automaticamente (`int` $\rightarrow$ `float` $\rightarrow$ `complex`), mas acima de 2^3 o `float` deixa de distinguir inteiros consecutivos — mantenha identificadores e dinheiro fora dele.
- Compare `float` com `math.isclose`, nunca com `==`; use `-(-a // b)` para divisão inteira arredondando para cima; use `_` e bases nativas nos literais que o leitor precisa reconhecer.

10. Precisão numérica: decimal, fractions e as armadilhas do float

A fatura tem três linhas. Link de 100 Mbps por R\$ 39,90, IP fixo por R\$ 19,90, suporte por R\$ 5,10. Qualquer pessoa soma isso de cabeça: R\$ 64,90.

```
ITENS = [("Link 100M", 39.90), ("IP fixo", 19.90), ("Suporte", 5.10)]

total = 0.0
for nome, valor in ITENS:
    total += valor

print(repr(total))
print(total == 64.90)
```

```
64.89999999999999
False
```

O relatório imprime 64.90, porque a formatação esconde o problema. O teste automatizado, que compara com 64.90, falha. E a conciliação com o sistema do banco, que trabalha em centavos, acusa uma diferença de um centavo num lote de trinta mil faturas.

Nada disso é bug do Python. O mesmo cálculo, em C, Java, JavaScript, Go ou numa planilha, dá o mesmo resultado — porque todos usam o mesmo `float` IEEE 754 de 64 bits do Capítulo 9. O que muda é que o Python te dá três saídas, e este capítulo é sobre escolher a certa.

10.1. Por que 0,1 não é 0,1

O `float` guarda o número em **base 2**. Um número tem representação finita em base 2 se, e somente se, seu denominador for uma potência de 2. Meio (1/2), um quarto (1/4), 0,75 (3/4): exatos. Um décimo (1/10) tem o fator 5 no denominador, e vira dízima periódica em binário — exatamente como 1/3 vira dízima em decimal.

Como só há 53 bits de precisão, a dízima é cortada. O que fica guardado não é 0,1:

```
print(f"{0.1:.55f}")
print((0.1).hex())
print((0.1).as_integer_ratio())
```

```
0.1000000000000000055511151231257827021181583404541015625
0x1.9999999999999ap-4
(3602879701896397, 36028797018963968)
```

10. Precisão numérica: decimal, fractions e as armadilhas do float

Três formas de ver a mesma coisa. A primeira mostra o valor decimal exato do objeto que o Python chama de 0.1. A segunda mostra a mantissa em hexadecimal — repare no 999999999999a, que é o 0,1999... periódico cortado e arredondado no último dígito. A terceira dá a fração exata: 3602879701896397 dividido por 2⁵⁵.

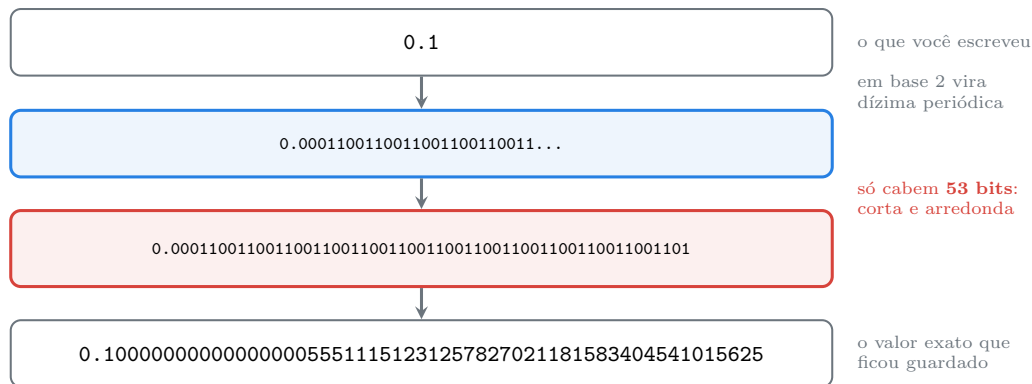


Figura 10.1.: O caminho de 0.1 até o número que o computador realmente guarda.

Todos os “erros” de ponto flutuante saem daí. O `float` não erra a conta: ele acerta a conta sobre números que não são os que você digitou.

E o Python é honesto sobre isso — só que discretamente. Desde a versão 3.1, `repr()` de um `float` mostra a **menor** cadeia de dígitos que volta ao mesmo objeto, e é por isso que 0.1 aparece como 0.1 e não como a monstruosidade acima. A informação não some; ela fica escondida até o momento em que se acumula o suficiente para vazar.

10.2. O erro se acumula (e a soma de Python não deixa mais)

Somar cem mil décimos deveria dar dez mil:

```
total = 0.0
for _ in range(1000):
    total += 0.1

print(repr(total))
print(total == 100.0, abs(total - 100.0))
```

```
99.9999999999986
False 1.4068746168049984e-12
```

Cada soma arredonda um pouquinho, e mil arredondamentos deixam rastro na décima segunda casa. Agora o mesmo cálculo com a função embutida:

```
print(repr(sum([0.1] * 1000)))
```

```
100.0
```

Exato. E não por sorte: desde o **Python 3.12**, `sum()` usa **somatório compensado** (algoritmo de Neumaier) quando os valores são `float`, mantendo um termo de correção que recupera os bits perdidos a cada passo. O contraste mais brutal:

```
print(repr(sum([1e16, 1.0, -1e16])))

acumulado = 0.0
for v in [1e16, 1.0, -1e16]:
    acumulado += v
print(repr(acumulado))
```

```
1.0
0.0
```

No laço, o 1.0 é engolido: somado a 10^1 , ele cai fora dos 53 bits e desaparece. `sum()` guarda a perda e a devolve no fim. Em Python 3.11 as duas linhas davam 0.0.

Isso é motivo suficiente para preferir `sum(lista)` a acumular num laço — e não só por elegância. Quando a compensação de `sum` não basta, existe `math.fsum`, que é exato por construção (soma em precisão estendida e arredonda uma vez só) e mais lento.

10.3. Comparar float com tolerância

Se a igualdade exata não vale, o que vale é a proximidade:

```
import math

print(0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3))
```

```
False
True
```

`math.isclose(a, b)` existe desde o Python 3.5 (PEP 485) e usa **tolerância relativa** de 10 por padrão: os valores podem diferir em uma parte por bilhão. Isso é o certo para grandezas de escala desconhecida — mas tem um ponto cego:

```
print(math.isclose(1e-18, 0.0))
print(math.isclose(1e-18, 0.0, abs_tol=1e-12))
```

```
False
True
```

Tolerância relativa a zero é zero, então **nada** é “próximo de zero” por padrão. Sempre que um dos lados puder ser zero, passe `abs_tol` com a menor diferença que ainda importa no seu domínio.

10.4. decimal: aritmética como a de um contador

O módulo `decimal` implementa a norma IEEE 854 / IBM para aritmética decimal. Ele guarda os números em **base 10**, com precisão configurável, e as contas saem como saem no papel:

```
from decimal import Decimal

print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
print(Decimal("1.10") + Decimal("2.20"))
```

```
True
0.3
3.30
```

Repare no 3.30, com o zero à direita preservado. `Decimal` guarda a **escala** — o número de casas decimais — como parte do valor, e isso é exatamente o que uma fatura precisa: R\$ 3,30 e R\$ 3,3 são o mesmo número e não são a mesma apresentação.

A regra número um do módulo: **construa sempre a partir de string**, nunca de `float`.

```
print(Decimal("0.1"))
print(Decimal(0.1))
```

```
0.1
0.10000000000000000055511151231257827021181583404541015625
```

`Decimal(0.1)` está correto — ele representa fielmente o `float` que recebeu, com todo o lixo binário. O problema é que o lixo já estava lá antes de o `Decimal` entrar em cena. Passar pelo `float` é o erro; `Decimal` só o torna visível.

O contexto controla precisão e arredondamento:

```
from decimal import getcontext, localcontext

print(getcontext().prec, getcontext().rounding)
print(Decimal(1) / Decimal(7))

with localcontext() as ctx:
    ctx.prec = 50
    print(Decimal(1) / Decimal(7))
```

```
28 ROUND_HALF_EVEN
0.1428571428571428571428571429
0.14285714285714285714285714285714285714
```

Vinte e oito dígitos significativos por padrão, arredondamento para o par mais próximo — o mesmo do `round` embutido, pelo mesmo motivo estatístico do Capítulo 9. `localcontext()` altera só dentro do bloco `with` e restaura na saída, o que é a forma segura de mexer nisso.

Para dinheiro, o método essencial é **quantize**, que fixa a escala e arredonda **de forma exata**:

```
from decimal import ROUND_HALF_UP

v = Decimal("2.675")
print(v.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))
print(round(2.675, 2))
```

```
2.68
2.67
```

Aqui está a diferença que fecha o assunto. `round(2.675, 2)` devolve 2.67 porque o `float` 2.675 é um tiquinho **menor** que 2,675 — o vizinho mais próximo com duas casas é mesmo 2,67. `Decimal("2.675")` é exatamente 2,675, e o arredondamento comercial manda subir. A legislação brasileira e a maioria dos contratos pedem `ROUND_HALF_UP`; a norma IEEE pede `ROUND_HALF_EVEN`. Escolha explicitamente, e escreva no código qual você escolheu.

Duas armadilhas do `decimal` que pegam quem vem do `float`:

```
Decimal("0.1") + 0.2
```

```
TypeError: unsupported operand type(s) for +: 'decimal.Decimal' and 'float'
```

Aritmética entre `Decimal` e `float` é **proibida**, de propósito, para você não misturar sem perceber. Comparação, porém, funciona:

```
print(Decimal("0.1") > 0.1)
```

```
False
```

E o resultado é `False` porque o `float` 0.1 é ligeiramente maior que um décimo exato — a comparação é feita com precisão total, e o `Decimal` perde por $5,55 \times 10^{-16}$.

A segunda armadilha é a divisão inteira, que **não** segue a regra do Capítulo 9:

```
print(Decimal("-7") // Decimal("2"), Decimal("-7") % Decimal("2"))
print(-7 // 2, -7 % 2)
```

```
-3 -1
-4 1
```

`Decimal` trunca em direção ao zero, como C; `int` arredonda para baixo. É o único ponto do Python em que os dois convivem, e a norma decimal é a razão — ela especifica truncagem. Se você escrever uma função que aceita os dois tipos, `//` vai devolver respostas diferentes conforme o que entrar.

10.5. fractions: exatidão sem arredondamento nenhum

Onde `Decimal` é exato em base 10, `Fraction` é exato e ponto. Ele guarda numerador e denominador inteiros, já reduzidos:

```
from fractions import Fraction

f = Fraction(1, 3)
print(f, f + Fraction(1, 6), f * 3, f * 3 == 1)
print(Fraction(3, 6))
```

```
1/3 1/2 1 True
1/2
```

`1/3 * 3 == 1` é verdadeiro — coisa que nem `float` nem `Decimal` conseguem, porque um terço não tem representação finita nem em base 2 nem em base 10. Toda aritmética racional é exata, sem contexto, sem precisão a configurar.

O mesmo cuidado com a construção vale aqui:

```
print(Fraction("0.1"))
print(Fraction(0.1))
print(Fraction(0.1).limit_denominator(1000))
```

```
1/10
3602879701896397/36028797018963968
1/10
```

A segunda linha é o `float` 0,1 mostrado como fração exata — o mesmo par de inteiros de `as_integer_ratio`. E `limit_denominator` é a ferramenta que recupera a fração “que a pessoa quis dizer” a partir de um `float`, útil para converter medidas e para achar aproximações racionais.

O preço é o desempenho. Uma soma, 200 mil vezes, no mesmo equipamento:

<code>int</code>	30 ns	1.3x
<code>float</code>	23 ns	1.0x
<code>Decimal</code>	68 ns	3.0x
<code>Fraction</code>	503 ns	21.9x

`Decimal` custa cerca de três vezes um `float`; `Fraction` custa vinte e duas. Para uma fatura de trinta linhas, irrelevante. Para um laço de milhões de iterações, decisivo — e o número exato varia com a máquina e com o tamanho dos operandos, então meça o seu caso antes de decidir.

10.6. Centavos em *int*: a resposta que quase sempre serve

Antes de alcançar o *decimal*, considere a alternativa mais simples: **guarde dinheiro em centavos, como inteiro**. Não há arredondamento, não há contexto, não há dependência, e a aritmética é a mais rápida da linguagem.

```
CENTAVOS = [3990, 1990, 510]

total = sum(CENTAVOS)
print(total, f"R$ {total // 100},{total % 100:02d}")
```

```
6490 R$ 64,90
```

Exato, e é assim que a maioria dos sistemas financeiros trabalha por dentro. As duas fronteiras exigem cuidado: na entrada, converter para centavos **sem passar pelo float**; na saída, formatar.

E é na conversão que mora o erro mais comum de todos:

```
print(4.35 * 100)
print(int(4.35 * 100), round(4.35 * 100))
```

```
434.99999999999994
434 435
```

`int(4.35 * 100)` dá **434 centavos**. Um centavo a menos, silenciosamente, e `int()` trunca justamente na direção que esconde o problema. A forma correta nunca multiplica um *float* por 100:

```
from decimal import Decimal

print(int(Decimal("4.35") * 100))
```

```
435
```

Nem *Decimal* é obrigatório se o dado nasce como texto — "4.35" vira 435 com um **replace** do separador e um **int**, sem tocar em ponto flutuante em momento algum.

Figura 10.2 resume a decisão.

O último ramo merece defesa: para uma medida física, *float* é a escolha **certa**, não uma concessão. A potência óptica lida pelo equipamento tem incerteza na primeira casa decimal; discutir o erro na décima sexta é ruído. Exatidão só importa quando o número é uma contagem ou um contrato, não quando é uma medição.

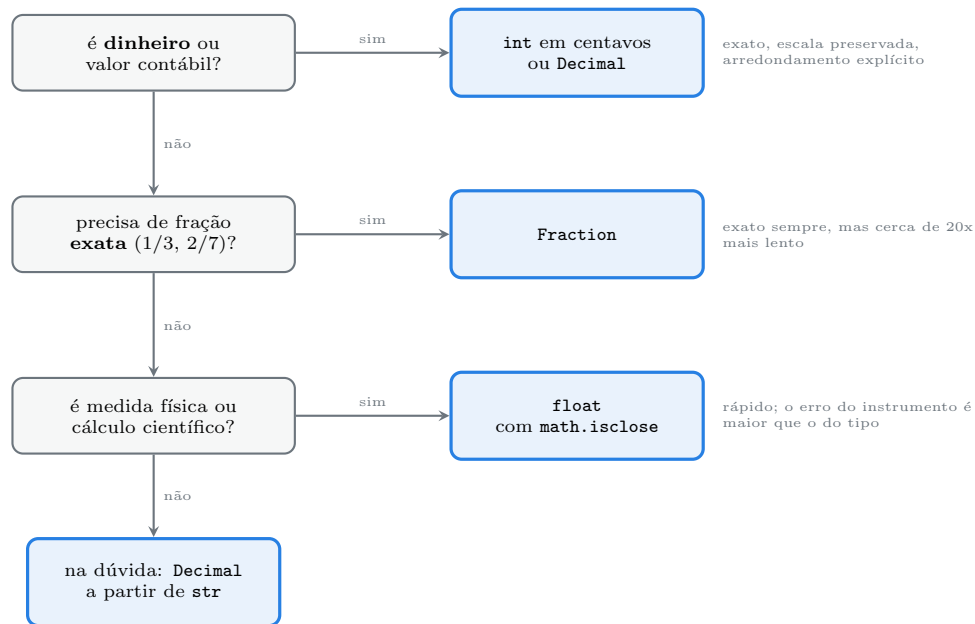


Figura 10.2.: Qual tipo numérico usar, e por quê.

10.7. Jeito Pythônico

Decimal nasce de str, nunca de float. Se o valor vem de JSON, de CSV ou de formulário, ele já é texto — construa direto e o float nunca entra na história. Se vier de um float alheio, use `Decimal(str(x))` e assuma conscientemente o arredondamento do repr.

Arredondamento é decisão de negócio, e vai explícito. `quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)` diz o que faz. Confiar no padrão do contexto funciona até alguém mudar o contexto.

Prefira sum() a acumular em laço. Desde o 3.12 ele compensa o erro de graça, e mesmo antes era mais rápido e mais legível:

```
# antipadrão
total = 0.0
for v in valores:
    total += v
```

```
# pythônico
total = sum(valores)
# quando a exatidão da soma for crítica:
total = math.fsum(valores)
```

Nunca == entre float. E nunca `math.isclose(x, 0.0)` sem `abs_tol` — o padrão relativo nunca dá verdadeiro contra zero.

Não misture os tipos numa mesma cadeia de cálculo. A conversão vai nas bordas: converte na entrada, calcula num tipo só, formata na saída. Um Decimal que encosta num float no meio do caminho perde tudo o que estava garantindo.

```
# antipadrão: exatidão jogada fora no meio
subtotal = Decimal("39.90") + Decimal("19.90")
com_desconto = float(subtotal) * 0.9
```

```
# pythônico: fica em Decimal do começo ao fim
subtotal = Decimal("39.90") + Decimal("19.90")
com_desconto = (subtotal * Decimal("0.9")).quantize(Decimal("0.01"),
                                                    rounding=ROUND_HALF_UP)
```

Formate no fim, calcule antes. `f"{total:.2f}"` é apresentação e mente sobre o valor guardado — foi o que escondeu o erro na fatura da abertura. Se o número precisa ser 64,90 de verdade, ele tem de ser 64,90 no tipo, não na string.

Um antipadrão que aparece muito em código de importação de dados:

```
centavos = int(float(linha["valor"]) * 100)    # perde um centavo em 4.35
```

```
centavos = int(Decimal(linha["valor"]) * 100)  # exato
```

Sobre versões:

- **Python 3.1** — `repr()` de `float` passou a mostrar a menor cadeia que faz a ida e volta. Antes saía `0.100000000000000001`.
- **Python 3.2** — comparação entre `Decimal` e `float` passou a ser exata (antes dava resultado arbitrário).
- **Python 3.3** — o `decimal` ganhou implementação em C (`_decimal`), entre 30 e 100 vezes mais rápida que a versão anterior em Python puro.
- **Python 3.5** — `math.isclose` (PEP 485).
- **Python 3.12** — `sum()` com somatório compensado para `float`; `Fraction` ganhou suporte a formatação com `format()`.

10.8. Laboratório

Objetivo: enxergar o valor real de um `float`, medir o erro acumulado, reproduzir o centavo perdido e reescrever a fatura da abertura de três formas exatas.

1. Ambiente.

```
mkdir -p ~/lab10 && cd ~/lab10
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. Reproduza a fatura. Crie `fatura.py` com o código da abertura e rode:

10. Precisão numérica: decimal, fractions e as armadilhas do float

```
python fatura.py
```

```
64.89999999999999
False
```

Depois imprima `f"{total:.2f}"` e observe: o relatório sai bonito e errado.

3. Veja o valor real.

```
python -c "
for x in (0.1, 0.5, 0.25, 39.90):
    print(x, '->', f'{x:.25f}', x.hex())
"
```

```
0.1 -> 0.1000000000000000055511151 0x1.999999999999ap-4
0.5 -> 0.500000000000000000000000 0x1.0000000000000p-1
0.25 -> 0.250000000000000000000000 0x1.0000000000000p-2
39.9 -> 39.899999999999985789145285 0x1.3f3333333333p+5
```

Meio e um quarto são exatos (denominador é potência de 2); um décimo e 39,90 não são. Explique a diferença antes de continuar.

4. Meça o erro acumulado.

```
python -c "
acc = 0.0
for _ in range(1000):
    acc += 0.1
print(repr(acc), acc == 100.0, abs(acc - 100.0))
"
```

```
99.9999999999986 False 1.4068746168049984e-12
```

5. Veja a compensação de sum.

```
python -c "
import math
print(repr(sum([0.1] * 1000)))
print(repr(sum([1e16, 1.0, -1e16])))
acc = 0.0
for v in [1e16, 1.0, -1e16]:
    acc += v
print(repr(acc))
print(repr(math.fsum([1e16, 1.0, -1e16])))
"
```

```
100.0
1.0
0.0
1.0
```

O 1.0 sumiu no laço e sobreviveu em `sum`. Se tiver um Python 3.11 à mão (`uv run --python 3.11`), rode a segunda linha lá e veja 0.0 — a compensação entrou no 3.12.

6. O ponto cego do `isclose`.

```
python -c "
import math
print(math.isclose(0.1 + 0.2, 0.3))
print(math.isclose(1e-18, 0.0))
print(math.isclose(1e-18, 0.0, abs_tol=1e-12))
"
```

```
True
False
True
```

7. Perca um centavo.

```
python -c "
print(4.35 * 100)
print(int(4.35 * 100), round(4.35 * 100))
from decimal import Decimal
print(int(Decimal('4.35') * 100))
"
```

```
434.999999999999994
434 435
435
```

Multiplique mentalmente por trinta mil faturas e escreva quanto some.

8. Decimal a partir de `str` contra a partir de `float`.

```
python -c "
from decimal import Decimal
print(Decimal('0.1'))
print(Decimal(0.1))
print(Decimal('0.1') == Decimal(0.1))
print(Decimal('0.1') > 0.1)
"
```

```
0.1
0.1000000000000000055511151231257827021181583404541015625
False
False
```

10. Precisão numérica: decimal, fractions e as armadilhas do float

A última linha é a mais sutil: o `float 0.1` é maior que um décimo, então o `Decimal` exato perde a comparação.

9. Contexto e arredondamento.

```
python -c "  
from decimal import Decimal, getcontext, localcontext, ROUND_HALF_UP,  
    ↪ ROUND_HALF_EVEN  
print(getcontext().prec, getcontext().rounding)  
print(Decimal(1) / Decimal(7))  
with localcontext() as ctx:  
    ctx.prec = 50  
    print(Decimal(1) / Decimal(7))  
q = Decimal('0.01')  
for v in ('2.675', '2.665', '2.345', '-2.675'):  
    d = Decimal(v)  
    print(v, d.quantize(q, rounding=ROUND_HALF_UP), d.quantize(q,  
    ↪ rounding=ROUND_HALF_EVEN))  
"
```

```
28 ROUND_HALF_EVEN  
0.1428571428571428571428571429  
0.14285714285714285714285714285714285714285714  
2.675 2.68 2.68  
2.665 2.67 2.66  
2.345 2.35 2.34  
-2.675 -2.68 -2.68
```

Compare as duas colunas em 2.665 e 2.345: o arredondamento comercial sobe, o do banqueiro vai para o par. Confira qual regra o seu contrato exige.

10. A divisão inteira que muda de regra.

```
python -c "  
from decimal import Decimal  
print(Decimal('-7') // Decimal('2'), Decimal('-7') % Decimal('2'))  
print(-7 // 2, -7 % 2)  
"
```

```
-3 -1  
-4 1
```

`Decimal` trunca, `int` arredonda para baixo. Isso é uma armadilha real numa função que aceita os dois tipos.

11. Exatidão total com Fraction.

```
python -c "  
from fractions import Fraction  
print(Fraction(1, 3) * 3 == 1)
```

```

print(sum(Fraction(1, 10) for _ in range(10)) == 1)
print(Fraction(0.1))
print(Fraction(0.1).limit_denominator(1000))
print(float(Fraction(22, 7)))
"

```

```

True
True
3602879701896397/36028797018963968
1/10
3.142857142857143

```

12. Meça o custo.

```

python -c "
import timeit
n = 200000
f = timeit.timeit('a + b', setup='a, b = 1.1, 2.2', number=n)
d = timeit.timeit('a + b', setup='from decimal import Decimal; a, b =
↳ Decimal('1.1'), Decimal('2.2')\n', number=n)
r = timeit.timeit('a + b', setup='from fractions import Fraction; a, b =
↳ Fraction(11,10), Fraction(22,10)', number=n)
i = timeit.timeit('a + b', setup='a, b = 110, 220', number=n)
for nome, t in (('int', i), ('float', f), ('Decimal', d), ('Fraction', r)):
    print(f'{nome:<9} {t/n*1e9:7.0f} ns {t/f:5.1f}x')
"

```

int	30 ns	1.3x
float	23 ns	1.0x
Decimal	68 ns	3.0x
Fraction	503 ns	21.9x

Os seus números vão diferir; as **proporções** não devem diferir muito.

13. Reescreva a fatura, três vezes. Crie fatura3.py:

```

"""A mesma fatura, exata de tres formas."""

from decimal import Decimal, ROUND_HALF_UP

ITENS = [("Link 100M", "39.90"), ("IP fixo", "19.90"), ("Suporte", "5.10")]
CENTAVO = Decimal("0.01")

# a) Decimal a partir de string
total_d = sum(Decimal(v) for _, v in ITENS)

# b) centavos em int, sem tocar em float
total_c = sum(int(Decimal(v) * 100) for _, v in ITENS)

```

10. Precisão numérica: decimal, fractions e as armadilhas do float

```
# c) float, comparado com tolerancia
total_f = sum(float(v) for _, v in ITENS)

print("Decimal ", total_d, total_d == Decimal("64.90"))
print("centavos", total_c, total_c == 6490)
print("float   ", repr(total_f), total_f == 64.90)

desconto = (total_d * Decimal("0.9")).quantize(CENTAVO, rounding=ROUND_HALF_UP)
print("com 10% ", desconto)
```

```
python fatura3.py
```

```
Decimal  64.90 True
centavos 6490 True
float    64.899999999999999 False
com 10%  58.41
```

As duas primeiras passam no teste que a original reprovava, e a terceira mostra por que ela reprovava. Note que `total_d` saiu 64.90, com o zero à direita — a escala veio junto, sem formatação nenhuma.

14. Feche com `deactivate`.

10.9. Resumo

- `float` guarda em base 2: só frações com denominador potência de 2 são exatas. 0.1 fica guardado como 0,1000000000000000055511151231257827021181583404541015625, e todo “erro de ponto flutuante” nasce daí.
- Desde o 3.1, `repr()` mostra a menor cadeia que faz a ida e volta — o que esconde o problema até ele acumular. `f"{x:.55f}"`, `x.hex()` e `x.as_integer_ratio()` mostram o valor real.
- Desde o 3.12, `sum()` compensa o erro de arredondamento em `float`; acumular num laço não compensa. `math.fsum` é exato quando isso não basta.
- Compare `float` com `math.isclose`, e passe `abs_tol` sempre que um dos lados puder ser zero.
- `Decimal` é exato em base 10, preserva a escala e tem arredondamento explícito via `quantize`. Construa **sempre** de `str`; aritmética com `float` é `TypeError`, e `//` trunca em direção ao zero, ao contrário do `int`.
- `Fraction` é exato para qualquer racional (`1/3 * 3 == 1`), ao custo de ser cerca de vinte vezes mais lento que `float`.
- Dinheiro: centavos em `int` ou `Decimal`, decidido nas bordas do sistema. `int(4.35 * 100)` devolve 434, e esse é o bug de um centavo que aparece só na conciliação.

11. Booleanos, valor-verdade e os operadores lógicos

O relatório de potência óptica descarta as leituras que não vieram. A função tem uma linha, é legível, passou na revisão e roda em produção há meses.

```
LEITURAS = [
    {"cliente": "sitio-mangueira", "nivel": -18.4},
    {"cliente": "fazenda-boa-vista", "nivel": -21.7},
    {"cliente": "chacara-do-ze", "nivel": 0.0},
    {"cliente": "vila-sao-joao", "nivel": None},
    {"cliente": "recanto-azul"},
]

def validas(leituras):
    return [l for l in leituras if l.get("nivel")]

niveis = [l["nivel"] for l in validas(LEITURAS)]
print([l["cliente"] for l in validas(LEITURAS)])
print(len(niveis), sum(niveis) / len(niveis))
```

```
['sitio-mangueira', 'fazenda-boa-vista']
2 -20.049999999999997
```

O `chacara-do-ze` sumiu. A leitura dele é `0.0` — potência de exatamente zero dBm, que é um valor perfeitamente válido e, aliás, excelente. O filtro o tratou como ausente, e a média do relatório mudou de `-13,37` para `-20,05`, uma diferença que não parece erro nenhum para quem lê.

`if l.get("nivel")` não pergunta “veio a leitura?”. Pergunta “o valor da leitura é verdadeiro?”. São perguntas diferentes, e `0.0` responde `False` à segunda. Este capítulo é sobre a diferença.

11.1. `bool` é uma subclasse de `int`

Antes de tudo, um fato que explica muita coisa:

```
print(isinstance(True, int), issubclass(bool, int))
print(True + True, True * 3, sum([True, False, True]))
print([10, 20, 30][True])
```

```
True True
2 3 2
20
```

`bool` é `int`. `True` vale 1, `False` vale 0, e nada impede de usá-los como número. Isso é herança histórica: os booleanos só chegaram no Python 2.3, e antes disso a verdade era representada por 1 e 0 direto. Para não quebrar o código existente, o tipo novo foi feito subclasse do antigo.

A consequência mais útil é `sum()` sobre uma sequência de comparações, que conta quantas são verdadeiras:

```
print(sum(1 for l in LEITURAS if l.get("nivel") is not None))
print(sum(l.get("nivel") is not None for l in LEITURAS))
```

```
3
3
```

A consequência mais perigosa é em dicionários e conjuntos, onde 1 e `True` são a mesma chave:

```
print({1: "a", True: "b"})
print({0: "zero", False: "falso"})
print(len({1, True, 1.0}))
```

```
{1: 'b'}
{0: 'falso'}
1
```

O dicionário guardou a **primeira** chave e o **último** valor, porque `hash(True) == hash(1)` e `True == 1`: para o dicionário, é a mesma chave sendo sobrescrita. E `{1, True, 1.0}` tem um elemento só, porque os três são iguais. O Volume 4 explora o mecanismo; aqui basta saber que misturar 0/`False` ou 1/`True` como chaves é receita de dado desaparecido.

11.2. Todo objeto tem valor-verdade

Em Python, qualquer objeto pode aparecer num `if`. Não existe “só booleanos são condições” — existe uma regra que converte qualquer coisa em verdadeiro ou falso.

A lista dos que são **falsos** é curta e vale decorar:

```
valores = [0, 0.0, "", [], {}, set(), (), None, range(0), b""]
for v in valores:
    print(f"{v!r:<12} {bool(v)}")
```

```

0           False
0.0         False
''          False
[]          False
{}          False
set()       False
()          False
None        False
range(0, 0) False
b''         False

```

Zero de qualquer tipo numérico, coleção vazia de qualquer tipo, e `None`. Todo o resto é verdadeiro — inclusive coisas que enganam:

```

'0'         True
'False'     True
[0]         True
(0,)        True
{0}         True
-1          True
b'\x00'     True

```

A string `"False"` é verdadeira porque é uma string não vazia; o conteúdo dela não é lido. Uma lista contendo um zero é verdadeira porque tem um elemento. É a diferença entre o recipiente e o conteúdo, e ela derruba muito código de leitura de configuração.

Por baixo, existe um protocolo com três degraus:

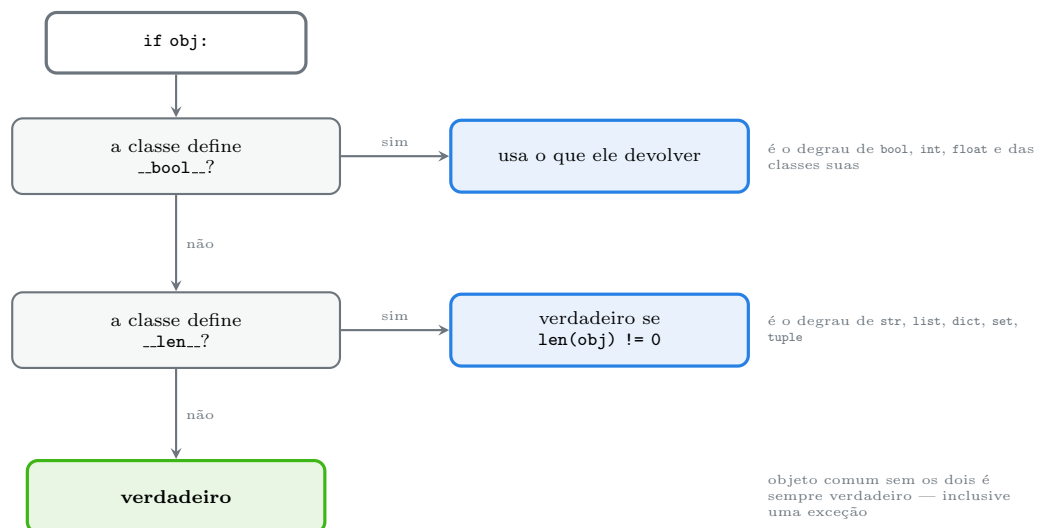


Figura 11.1.: Como o Python decide se um objeto é verdadeiro.

Dá para conferir os três degraus com quatro classes:

```

class Vazio:
    pass

```

```
class ComLen:
    def __len__(self):
        return 0

class ComBool:
    def __bool__(self):
        return False

class Ambos:
    def __len__(self):
        return 0

    def __bool__(self):
        return True

print(bool(Vazio()), bool(ComLen()), bool(ComBool()), bool(Ambos()))
```

```
True False False True
```

Vazio não define nenhum dos dois e é verdadeiro. Ambos define os dois, e `__bool__` ganha — a ordem da figura vale mesmo. É por isso que um objeto de negócio recém-criado é verdadeiro por padrão, e por que dar `__len__` a uma classe a torna falsa quando estiver vazia, muitas vezes sem que o autor perceba.

11.3. and e or não devolvem booleanos

Esta é a segunda surpresa do capítulo:

```
print(1 and 2, 0 and 2)
print(1 or 2, 0 or 2)
print(type(1 and 2))
```

```
2 0
1 2
<class 'int'>
```

`1 and 2` devolve 2, não `True`. Os operadores lógicos de Python devolvem **um dos operandos**, não um booleano. A regra:

- `a and b` — avalia `a`. Se `a` for falso, devolve `a` sem olhar `b`. Senão, devolve `b`.
- `a or b` — avalia `a`. Se `a` for verdadeiro, devolve `a` sem olhar `b`. Senão, devolve `b`.

Só `not` devolve booleano de verdade:

```
print(not 1, not 0, type(not 1))
```

```
False True <class 'bool'>
```

O “sem olhar b” é o **curto-circuito**, e é comportamento garantido, não otimização:

```
def efeito(nome, retorno):
    print("avaliou", nome)
    return retorno

print(efeito("A", False) and efeito("B", True))
```

```
avaliou A
False
```

B nunca rodou. Isso permite escrever guardas em uma linha — `if dados and dados[0] > 0` não estoura em lista vazia, porque a segunda parte só é avaliada quando a primeira é verdadeira.

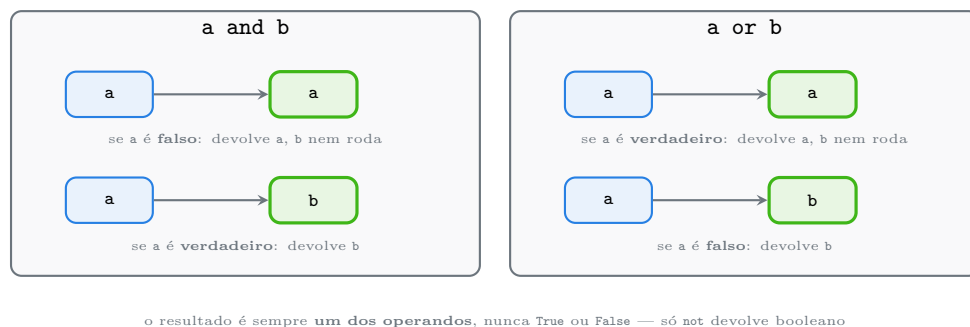


Figura 11.2.: `and` e `or` devolvem um operando e podem não avaliar o segundo.

Daí nasce o idioma `valor or padrao`, que é conveniente e é uma armadilha:

```
def conectar(porta=None):
    porta = porta or 8080
    return porta

print(conectar(), conectar(9090), conectar(0))
```

```
8080 9090 8080
```

`conectar(0)` devolveu 8080. A porta 0 é válida em soquete — significa “escolha uma porta livre” —, e o `or` a comeu, pela mesma razão que o filtro da abertura comeu a leitura zero. A versão correta pergunta o que realmente interessa:

11. Booleanos, valor-verdade e os operadores lógicos

```
def conectar(porta=None):
    if porta is None:
        porta = 8080
    return porta

print(conectar(), conectar(9090), conectar(0))
```

```
8080 9090 0
```

O mesmo vale para dicionários: `cfg.get("timeout")` or 30 devolve 30 quando o timeout configurado é 0; `cfg.get("timeout", 30)` devolve 0, que é o valor que a pessoa escreveu.

11.4. Precedência

De mais forte para mais fraco: aritmética, comparações, `not`, `and`, `or`.

```
print(True or False and False)
print((True or False) and False)
```

```
True
False
```

Sem parênteses, `and` agrupa primeiro: `True or (False and False)`. E `not` é mais forte que `and/or`, mas **mais fraco que as comparações**:

```
print(not 1 == 2)
print((not 1) == 2)
```

```
True
False
```

`not 1 == 2` é `not (1 == 2)`, que é o que quase todo mundo quer. Ainda assim, em expressão com três ou mais operadores lógicos, ponha parênteses: eles não custam nada e o leitor não precisa consultar tabela.

11.5. Comparação encadeada

Python permite escrever a desigualdade como na matemática:

```
x = 5
print(1 < x < 10)
print(3 > 2 > 1, 3 > 2 > 5)
```

```
True
True False
```

`a < b < c` é açúcar para `a < b and b < c`, com uma diferença importante: **b é avaliado uma vez só**. Isso importa quando b é uma chamada cara ou tem efeito colateral — `1 < proxima_leitura() < 10` lê o sensor uma vez, enquanto a versão com `and` leria duas.

O encadeamento vale para qualquer operador de comparação, incluindo `==`, `is` e `in` — e é aí que ele pode surpreender:

```
print(False == False in [False])
```

```
True
```

Isso é `(False == False) and (False in [False])`, que é `True and True`. Não é `False == (False in [False])`, que seria `False == True`, ou seja, `False`. Encadear `==` com `in` é legal e ilegível; não faça.

11.6. *all* e *any*

Duas embutidas que resumem uma sequência inteira, com curto-circuito:

```
print(all([1, 2, 0]), any([0, 0, 3]))
print(all(nivel < 0 for nivel in [-18.4, -21.7, 0.0]))
```

```
False True
False
```

O caso das sequências vazias merece atenção, porque não é intuitivo e é definido:

```
print(all([]), any([]))
```

```
True False
```

`all([])` é verdadeiro — “todos os elementos de um conjunto vazio satisfazem qualquer coisa” é verdade vacuamente, a mesma convenção da lógica matemática. `any([])` é falso, porque não há nenhum que satisfaça. Na prática: `if all(validacoes)` com a lista vazia deixa passar, e isso já causou incidente em mais de um sistema de checagem.

Com um gerador em vez de lista, `all` e `any` param no primeiro que decide — e isso é a diferença entre percorrer três itens e percorrer três milhões.

11.7. Jeito Pythônico

Pergunte o que você quer saber. Este é o capítulo inteiro numa frase:

11. Booleanos, valor-verdade e os operadores lógicos

```
# antipadrão: pergunta "é verdadeiro?" quando quer saber "existe?"
if not leitura.get("nivel"):
    continue
```

```
# pythônico: pergunta exatamente o que interessa
if leitura.get("nivel") is None:
    continue
```

`is None` para ausência, `== 0` para zero, `if not lista` para coleção vazia. Cada um responde a uma pergunta diferente, e usar o teste de verdade genérico para todas é o que produz o bug da abertura. A regra prática: **valor-verdade é seguro para coleções, perigoso para números e para valores que podem ser None.**

Para coleção vazia, use o valor-verdade. É o idioma da linguagem e a PEP 8 é explícita:

```
# antipadrão
if len(itens) > 0:
if itens != []:
```

```
# pythônico
if itens:
if not itens:
```

Funciona com qualquer coleção, é mais rápido (não constrói lista nem chama `len` desnecessariamente) e não assume o tipo.

Nunca compare com `True` ou `False`. A PEP 8 chama isso de “pior ainda”:

```
# antipadrão
if pronto == True:
if pronto is True:
```

```
# pythônico
if pronto:
```

`== True` falha para qualquer valor verdadeiro que não seja exatamente 1 — a lista `[1, 2]` é verdadeira e `[1, 2] == True` é falso. E `is True` é pior, porque exige o objeto singleton.

Prefira `dict.get(chave, padrao)` a `dict.get(chave) or padrao`. A primeira substitui só a chave ausente; a segunda substitui também 0, "" e [].

Não converta para bool sem necessidade. `bool(x)` num `if` é redundante — o `if` já faz isso. O lugar onde `bool()` é útil é quando você precisa do valor booleano como **dado**: numa chave de dicionário, num campo de JSON, num retorno de função que promete `bool`.

Evite bool como parâmetro posicional. `criar(True, False)` não diz nada na chamada. Use argumento nomeado, e o Volume 3 mostra como forçar isso com `*`:


```
criar(usuario, ativo=True, notificar=False)
```

Sobre versões e detalhes que valem lembrar:

- **Python 2.3** — `bool`, `True` e `False` passaram a existir como tipo. Antes eram 1 e 0.
- **Python 3.0** — `True` e `False` viraram palavras reservadas; antes eram nomes que se podia reatribuir (e havia código que fazia isso).
- `bool` **não pode** ser herdado (`class Meu(bool)` levanta `TypeError`), justamente para que `True` e `False` continuem sendo os dois únicos objetos do tipo.
- `^` entre booleanos é o “ou exclusivo” (`True ^ False` é `True`), mas **não tem curto-circuito** — é o operador bit a bit do Capítulo 9 agindo sobre 0 e 1. Não existe `xor` lógico na linguagem; quando precisar, `bool(a) != bool(b)` é mais claro.

11.8. Laboratório

Objetivo: mapear os valores falsos, ver o protocolo de verdade nos três degraus, provar que `and` e `or` devolvem operandos, e consertar o filtro que engoliu a leitura zero.

1. Ambiente.

```
mkdir -p ~/lab11 && cd ~/lab11
python3 -m venv .venv && source .venv/bin/activate
python --version
```

```
Python 3.13.12
```

2. Reproduza o bug. Crie `relatorio.py` com o código da abertura e rode:

```
python relatorio.py
```

```
['sitio-mangueira', 'fazenda-boa-vista']
2 -20.049999999999997
```

Confirme na lista que `chacara-do-ze` tem `"nivel": 0.0` e sumiu mesmo assim.

3. Monte a tabela de verdade dos nativos.

```
python -c "
valores = [0, 1, -1, 0.0, 0.1, '', '0', 'False', [], [0], {}, {'a': 1},
            set(), {0}, (), (0,), None, range(0), range(1), b'', b'\x00']
for v in valores:
    print(f'{v!r:<12} {bool(v)}')
"
```

11. Booleanos, valor-verdade e os operadores lógicos

```
0          False
1          True
-1         True
0.0        False
0.1        True
''         False
'0'        True
'False'    True
[]         False
[0]        True
{}         False
{'a': 1}   True
set()      False
{0}        True
()         False
(0,)       True
None       False
range(0, 0) False
range(0, 1) True
b''        False
b'\x00'    True
```

Marque as sete que enganam: -1, '0', 'False', [0], {0}, (0,) e b'\x00'.

4. Confirme os três degraus. Crie `degraus.py` com as quatro classes do capítulo e rode:

```
True False False True
```

Depois acrescente um `print` dentro de `__len__` de `Ambos` e rode de novo: ele **não** é chamado, porque `__bool__` respondeu primeiro.

5. bool é int.

```
python -c "
print(issubclass(bool, int), True + True, sum([True, False, True]))
print({1: 'a', True: 'b'})
print(len({1, True, 1.0}))
"
```

```
True 2 2
{1: 'b'}
1
```

A segunda linha é a que assusta: a chave `True` sobrescreveu o valor de `1` e o dicionário ficou com um par só.

6. and e or devolvem operandos.

```
python -c "
print(1 and 2, 0 and 2, 1 or 2, 0 or 2)
print('' or 'padrao', 'valor' or 'padrao')
print(type(1 and 2), type(not 1))
"
```

```
2 0 1 2
padrao valor
<class 'int'> <class 'bool'>
```

7. Veja o curto-circuito. Crie `curto.py` com a função `efeito` do capítulo, e rode as duas expressões. Em `A and B` com `A` falso, e em `A or B` com `A` verdadeiro, o `B` não aparece na saída:

```
avaliou A
False
---
avaliou A
True
```

8. Precedência.

```
python -c "
print(True or False and False, (True or False) and False)
print(not 1 == 2, (not 1) == 2)
"
```

```
True False
True False
```

9. Encadeamento avalia o meio uma vez.

```
python -c "
def leitura():
    print('leu o sensor')
    return 5
print(1 < leitura() < 10)
print('---')
print(1 < leitura() and leitura() < 10)
"
```

```
leu o sensor
True
---
leu o sensor
leu o sensor
True
```

11. Booleanos, valor-verdade e os operadores lógicos

Duas leituras na versão com **and**, uma na encadeada.

10. Sequência vazia em **all** e **any**.

```
python -c "  
print(all([]), any([]))  
print(all([1, 2, 0]), any([0, 0, 3]))  
print(all(n < 0 for n in [-18.4, -21.7, 0.0]))  
"
```

```
True False  
False True  
False
```

Pense no que `if all(validacoes):` faz quando `validacoes` chega vazia por engano.

11. O **or** que come o zero.

```
python -c "  
cfg = {'timeout': 0, 'retries': 3}  
print(cfg.get('timeout') or 30)  
print(cfg.get('timeout', 30))  
print(cfg.get('proxy') or 'nenhum')  
print(cfg.get('proxy', 'nenhum'))  
"
```

```
30  
0  
nenhum  
nenhum
```

As duas primeiras linhas divergem porque a chave **existe** com valor falso; as duas últimas coincidem porque a chave não existe. É exatamente a distinção que **or** não sabe fazer.

12. Conserte o relatório. Reescreva `validas` e confira a média:

```
def validas(leituras):  
    return [l for l in leituras if l.get("nivel") is not None]
```

```
python relatorio.py
```

```
['sitio-mangueira', 'fazenda-boa-vista', 'chacara-do-ze']  
3 -13.366666666666665
```

Três clientes e a média certa. Agora escreva uma terceira versão que também recuse leituras fora da faixa física plausível, usando encadeamento:

```
def validas(leituras):
    return [
        1
        for l in leituras
        if (n := l.get("nivel")) is not None and -40 <= n <= 10
    ]
```

O `:=` do Capítulo 6 evita chamar `get` duas vezes, e o encadeamento `-40 <= n <= 10` avalia `n` uma vez. Confirme que o resultado continua com os três clientes.

13. Feche com `deactivate`.

11.9. Resumo

- `bool` é subclasse de `int`: `True` vale 1 e `False` vale 0. Dá para somar comparações para contar; e 1/`True` (ou 0/`False`) colidem como chave de dicionário e como elemento de conjunto.
- Todo objeto tem valor-verdade. São falsos: zero de qualquer tipo numérico, coleção vazia de qualquer tipo e `None`. Todo o resto é verdadeiro, inclusive `"0"`, `"False"`, `[0]` e `b"\x00"`.
- O protocolo tem três degraus: `__bool__`, depois `__len__`, e na ausência dos dois o objeto é verdadeiro.
- `and` e `or` devolvem **um dos operandos**, não booleano, e fazem curto-circuito garantido. Só `not` devolve `bool`.
- `valor or padrao` e `dict.get(k) or padrao` substituem também 0, `" "` e `[]` — use `is None` ou `dict.get(k, padrao)` quando o zero for legítimo.
- Precedência: comparações, depois `not`, depois `and`, depois `or`. Comparação encadeada (`a < b < c`) avalia o operando do meio uma vez só.
- `all([])` é `True` e `any([])` é `False`, por convenção lógica. Ambos fazem curto-circuito, o que só rende com geradores.
- Pergunte o que você quer saber: `is None` para ausência, `== 0` para zero, `if colecao` para vazio. Nunca `== True`.

12. Strings: criação, indexação e fatiamento

O equipamento antigo não exporta CSV. Ele cospe registros de largura fixa, uma linha por leitura, com os campos em colunas combinadas há quinze anos. O parser tem cinco linhas e roda desde então:

```
LINHA = "2026-03-11 06:47:55 CHACARA-DO-ZE -18.4 OK"

data = LINHA[0:10]
hora = LINHA[11:18]

print(repr(data), repr(hora))
```

```
'2026-03-11' '06:47:5'
```

A hora perdeu o último dígito. 06:47:5 — que continua parecendo uma hora, entra no banco sem reclamação, e só aparece quando alguém tenta ordenar os eventos de um minuto e a ordem sai errada.

O erro é de um caractere: [11:18] em vez de [11:19]. E ele acontece porque quase todo mundo lê [11:18] como “do 11 ao 18” — quando a fatia de Python é **meio aberta**: inclui o começo, exclui o fim.

Este capítulo é sobre a **str** como sequência: como se escreve, como se indexa, como se fatia e por que a imutabilidade dela muda a forma de construir texto. Os métodos e a formatação ficam para o Capítulo 13; a codificação de caracteres, para o Capítulo 14.

12.1. Escrevendo uma string

Aspas simples e duplas são **idênticas** para o Python. A escolha é sobre o que está dentro:

```
print("ele disse 'oi'")
print('caminho: "C:\\temp"')
```

```
ele disse 'oi'
caminho: "C:\\temp"
```

Aspas triplas atravessam linhas e preservam a quebra:

```
texto = """Relatorio
de leituras"""
print(len(texto))
```

21

Vinte e um: nove de `Relatorio`, um do `\n` e onze de `de leituras`.

E dois literais adjacentes são **concatenados na compilação**, sem custo nenhum em execução — o idioma para quebrar uma string longa no fonte:

```
consulta = (
    "SELECT cliente, nivel "
    "FROM leituras "
    "WHERE nivel < -25"
)
print(consulta)
```

```
SELECT cliente, nivel FROM leituras WHERE nivel < -25
```

Repare que a vírgula ausente entre os pedaços é justamente o que faz isso funcionar — e é também a causa de um bug clássico: numa lista de strings, esquecer uma vírgula não dá erro, junta os dois itens em um.

```
PALAVRAS = [
    "erro",
    "aviso"          # vírgula esquecida
    "critico",
]
print(PALAVRAS)
```

```
['erro', 'avisocritico']
```

Escapes seguem a tradição de C: `\n` para nova linha, `\t` para tabulação, `\\` para a própria barra, `\x41` para um byte em hexadecimal, `\u00e9` e `\N{...}` para caracteres Unicode por código ou por nome.

```
print(repr("\x41\u00e9\N{SNAKE}"))
```

```
'Aé '
```

Quando as barras são conteúdo e não comando — caminhos do Windows, expressões regulares —, use uma string **crua**, com o prefixo `r`:

```
print(r"C:\novo\teste")
print(len(r"\n"), len("\n"))
```

```
C:\novo\teste
2 1
```

`r"\n"` são dois caracteres: uma barra e um `n`. É o formato certo para todo padrão do módulo `re` do Volume 10 — e a razão de o Python avisar `SyntaxWarning: invalid escape sequence` quando você escreve `"\d"` sem o `r`.

12.2. Imutável, como todo `str`

Já vimos no Capítulo 8 que `str` é imutável. A consequência prática aparece na primeira tentativa de “corrigir” um caractere:

```
nome = "kali"
nome[0] = "K"
```

```
TypeError: 'str' object does not support item assignment
```

Toda operação que parece alterar devolve uma string nova:

```
nome = "kali"
print(nome.replace("k", "K"), nome)
```

```
Kali kali
```

A original continua lá, intocada. Guarde o retorno ou você perde o resultado — é o erro número um de quem está começando.

12.3. Índices e fronteiras

Uma `str` é uma **sequência**, com índices a partir de zero e índices negativos contando do fim:

```
p = "python"
print(p[0], p[5], p[-1], p[-6])
print(type(p[0]), len(p[0]))
```

```
p n n p
<class 'str'> 1
```

Note a segunda linha: **não existe tipo caractere em Python**. `p[0]` é uma `str` de comprimento 1. Isso é diferente de C, Java e Rust, e simplifica a vida — não há conversão entre `char` e `String` para fazer.

Índice fora da faixa levanta erro:

```
p[6]
```

```
IndexError: string index out of range
```

E aqui está o ponto que resolve o bug da abertura: **fatia não usa índices, usa fronteiras**. Pense nos números como posições *entre* os caracteres, não sobre eles.

Com a figura na cabeça, tudo o mais decorre:

12. Strings: criação, indexação e fatiamento

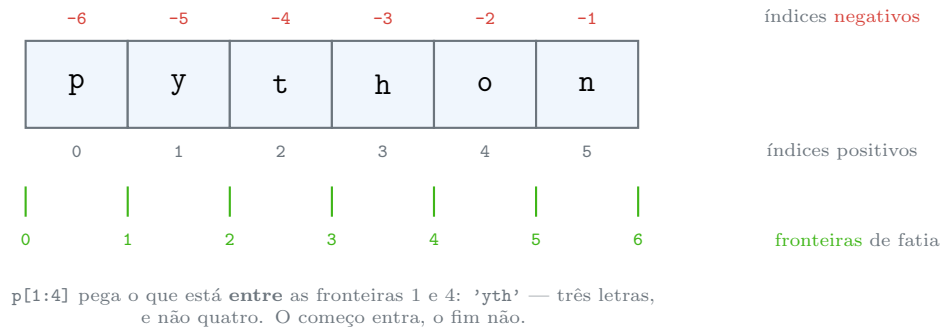


Figura 12.1.: Os números de uma fatia marcam fronteiras entre caracteres, não os caracteres.

```
p = "python"
print(p[0:3], p[:3], p[3:], p[:])
print(p[-3:], p[:-3], p[-4:-1])
```

```
pyt pyt hon python
hon pyt tho
```

Omitir o começo significa “do início”; omitir o fim, “até o final”. `p[:n]` e `p[n:]` sempre se encaixam de volta na string inteira, porque a fronteira `n` é a mesma dos dois lados. E o comprimento de `p[a:b]` é `b - a`, o que dá a conta da abertura: `[11:19]` tem oito caracteres, que é o tamanho de 06:47:55.

Fatia, ao contrário de índice, **não levanta erro** fora da faixa:

```
print(repr(p[10:20]), repr(p[3:100]), repr(p[-100:2]))
```

```
' 'hon' 'py'
```

Isso é conveniente e é uma faca de dois gumes: o parser da abertura teria falhado alto se a fatia levantasse `IndexError`. Como ela não levanta, campo fora da posição vira string vazia e segue viagem.

12.4. O terceiro número: o passo

A forma completa é `[início:fim:passo]`:

```
print(p[::-2], p[1::-2])
print(p[::-1])
```

```
pto yhn
nohtyp
```

`[::-1]` é o idioma para inverter uma sequência — e vale para lista e tupla também. Com passo negativo, a fatia caminha para trás, e é aí que a intuição falha:

```
print(repr(p[5:1:-1]), repr(p[1:5:-1]))
```

```
'noht' ''
```

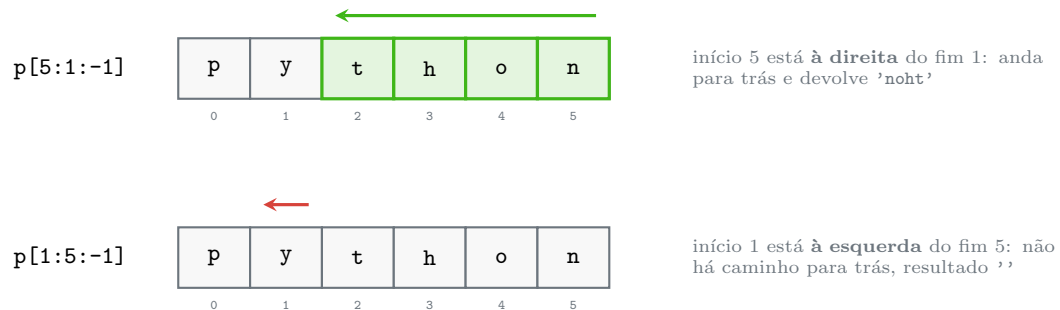


Figura 12.2.: Com passo negativo, o início precisa estar à direita do fim.

Passo zero é erro:

```
p[:,0]
```

```
ValueError: slice step cannot be zero
```

E uma sutileza que vale saber: `p[:]` de uma string devolve o **próprio objeto**, não uma cópia, porque `str` é imutável e copiar não teria função:

```
print(p[:] is p)
print([1, 2, 3][:] is [1, 2, 3])
```

```
True
False
```

Com lista, o `[:]` copia de verdade — é o idioma do Capítulo 8.

12.5. slice é um objeto

Os dois-pontos são açúcar para um objeto `slice`, que pode ser guardado numa constante e reusado. Para largura fixa, isso transforma um parser ilegível num que se lê:

```
CAMPOS = {
    "data": slice(0, 10),
    "hora": slice(11, 19),
    "cliente": slice(20, 36),
    "nivel": slice(36, 42),
    "estado": slice(43, None),
```

12. Strings: criação, indexação e fatiamento

```
}  
  
LINHA = "2026-03-11 06:47:55 CHACARA-DO-ZE    -18.4  OK"  
  
for nome, corte in CAMPOS.items():  
    print(f"{nome:<8} {LINHA[corte]!r}")
```

```
data      '2026-03-11'  
hora      '06:47:55'  
cliente   'CHACARA-DO-ZE    '  
nivel     '-18.4  '  
estado    'OK'
```

O layout do arquivo virou dado, num lugar só, com nome. Quando o fabricante mudar uma coluna, muda uma linha do dicionário — e não sete fatias espalhadas pelo código. É a diferença entre um parser que dá para revisar e um que não dá.

12.6. Concatenar, repetir, comparar

```
print("a" + "b", "ab" * 3, "-" * 20)
```

```
ab ababab -----
```

Somar `str` com número é `TypeError`, como no Capítulo 7 — a linguagem se recusa a adivinhar.

Comparação é **lexicográfica por ponto de código**, o que não é ordem alfabética:

```
print("Z" < "a", ord("Z"), ord("a"))  
print(sorted(["Zebra", "abacaxi", "Banana"]))  
print(sorted(["Zebra", "abacaxi", "Banana"], key=str.lower))
```

```
True 90 97  
['Banana', 'Zebra', 'abacaxi']  
['abacaxi', 'Banana', 'Zebra']
```

Toda maiúscula vem antes de toda minúscula, porque A é 65 e a é 97. E acentuadas vêm depois de todas as latinas sem acento (ç é 231), então `sorted` sozinho **não** ordena nomes em português como um humano espera. Para ordem alfabética de verdade existe `locale.strxfrm` ou a biblioteca PyICU; para o caso comum, `key=str.lower` já resolve a maiúscula.

Pertinência e busca:

```
p = "python"  
print("th" in p, "z" in p, p.index("th"), p.count("h"))
```

```
True False 2 1
```

`in` funciona com **subcadeias**, não só com um caractere — diferente de listas, onde `in` testa elementos. É o teste de substring mais legível que existe, e mais rápido que qualquer alternativa manual.

12.7. Construir texto: a armadilha do +=

Como `str` é imutável, `s += t` não pode alterar `s`: ele constrói uma string nova com o conteúdo das duas. Todo livro daí conclui que construir texto num laço é quadrático. **Em CPython, isso é só metade da verdade** — e a metade que falta importa.

```
def com_mais_igual(n):
    s = ""
    for _ in range(n):
        s += "x"
    return s

def com_alias(n):
    s = ""
    guarda = s          # segunda referência ao objeto
    for _ in range(n):
        s += "x"
        guarda = s
    return s

def com_join(n):
    return "".join("x" for _ in range(n))
```

n	+=	+= c/	alias	join
20000	1.7ms		8.0ms	1.7ms
40000	2.8ms		31.3ms	2.8ms
80000	5.3ms		132.1ms	7.5ms
160000	13.5ms		515.3ms	12.6ms

A coluna do meio quadruplica a cada vez que o `n` dobra — comportamento quadrático de livro. As outras duas apenas dobram.

A explicação é uma otimização do CPython: quando a string da esquerda tem **exatamente uma referência**, o interpretador sabe que ninguém mais a está olhando e redimensiona o buffer no lugar, em vez de copiar. Basta existir um segundo nome apontando para ela — `guarda = s` — e a otimização não pode agir, porque copiar sobre o buffer estragaria o outro nome.

Duas conclusões práticas:

1. `+=` num laço apertado é frágil. Funciona por um detalhe de implementação que some sem aviso quando o código muda, e que **não existe** em outras implementações do Python.
2. `"".join(...)` sempre foi linear, sem depender de nada. Escreva assim e não pense mais no assunto.

12.8. Jeito Pythônico

join para montar, nunca += em laço.

```
# antipadrão
linhas = ""
for l in registros:
    linhas += formatar(l) + "\n"
```

```
# pythônico
linhas = "\n".join(formatar(l) for l in registros)
```

O separador vem primeiro porque `join` é método da string separadora, o que assusta na primeira vez e é consistente: `"", ".join(nomes), "\n".join(linhas), "".join(pedacos)`. E o gerador dentro do `join` evita construir a lista intermediária.

Nomeie as fatias. Índice mágico é o pior tipo de constante:

```
# antipadrão
hora = linha[11:19]
```

```
# pythônico
HORA = slice(11, 19)
hora = linha[HORA]
```

Use `[::-1]` para inverter, não um laço. É uma linha em vez de quatro e, no teste do laboratório, algumas dezenas de vezes mais rápido.

Aspas: escolha uma e seja consistente. A PEP 8 não decide por você; o Black e o Ruff, sim — o padrão dos dois é aspas duplas, e trocar só quando a string contém aspas duplas. Se o projeto usa formatador, deixe ele decidir e pare de pensar nisso.

String crua para expressão regular e caminho do Windows. Sempre. `r"\d+"` e `r"C:\Users"`.

Não construa strings com dados de fora sem cuidado. Concatenar entrada do usuário em SQL, em comando de shell ou em HTML é a origem das três famílias mais comuns de vulnerabilidade. Cada uma tem a ferramenta certa — parâmetros de consulta, lista de argumentos em `subprocess`, escape de template — e o Volume 12 e o Volume 14 tratam disso.

Um antipadrão discreto:

```
# antipadrão: comparar tamanho para ver se está vazia
if len(texto) == 0:
```

```
# pythônico (@sec-cap011)
if not texto:
```

Sobre versões:

- **Python 3.0** — `str` passou a ser texto Unicode; `bytes` virou tipo separado (assunto do Capítulo 14).
- **Python 3.6** — f-strings (PEP 498), tema do próximo capítulo.
- **Python 3.7** — `str.isascii()`.
- **Python 3.9** — `str.removeprefix()` e `str.removesuffix()` (PEP 616), que substituem o uso errado de `strip` para remover prefixo.
- **Python 3.12** — f-strings passaram a aceitar qualquer expressão, inclusive com as mesmas aspas por dentro e por fora (PEP 701).
- **Python 3.13** — o aviso de sequência de escape inválida (`"\d"`) virou `SyntaxWarning` por padrão, a caminho de virar erro.

12.9. Laboratório

Objetivo: enxergar as fronteiras da fatia, medir a diferença entre `+=` e `join`, e reescrever o parser de largura fixa de forma que o próximo erro de coluna seja visível.

1. Ambiente.

```
mkdir -p ~/lab12 && cd ~/lab12
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. Reproduza o bug.

```
python -c "
LINHA = '2026-03-11 06:47:55 CHACARA-DO-ZE -18.4 OK'
print(repr(LINHA[0:10]), repr(LINHA[11:18]), repr(LINHA[11:19]))
print(len(LINHA))
"
```

```
'2026-03-11' '06:47:5' '06:47:55'
45
```

Confirme na régua da Figura 12.1 que o comprimento de `[a:b]` é `b - a`.

3. Percorra a régua.

```
python -c "
p = 'python'
for f in ('p[0]', 'p[-1]', 'p[0:3]', 'p[:3]', 'p[3:]', 'p[-3:]', 'p[: -3]',
↪ 'p[-4:-1]', 'p[:]'):
    print(f'{f:<10} {eval(f)!r}')
"
```

12. Strings: criação, indexação e fatiamento

```
p[0]      'p'
p[-1]     'n'
p[0:3]    'pyt'
p[:3]     'pyt'
p[3:]     'hon'
p[-3:]    'hon'
p[:-3]    'pyt'
p[-4:-1]  'tho'
p[:]      'python'
```

Confira que `p[:3] + p[3:]` reconstrói a string — a fronteira 3 é a mesma dos dois lados.

4. Índice estoura, fatia não.

```
python -c "
p = 'python'
print(repr(p[10:20]), repr(p[3:100]), repr(p[-100:2]))
print(p[10])
"
```

```
' ' 'hon' 'py'
Traceback (most recent call last):
  File "<string>", line 5, in <module>
    print(p[10])
    ~~~~~
IndexError: string index out of range
```

Pense em quais bugs a tolerância da fatia esconde — o da abertura é um deles.

5. Passo, inclusive negativo.

```
python -c "
p = 'python'
for f in ('p[::2]', 'p[1::2]', 'p[::-1]', 'p[::-2]', 'p[5:1:-1]', 'p[1:5:-1]',
↪      'p[-1:-4:-1]'):
    print(f'{f:<14} {eval(f)!r}')
print(p[::0])
"
```

```
p[::2]      'pto'
p[1::2]     'yhn'
p[::-1]     'nohtyp'
p[::-2]     'nhy'
p[5:1:-1]   'noht'
p[1:5:-1]   ''
p[-1:-4:-1] 'noh'
Traceback (most recent call last):
  File "<string>", line 6, in <module>
    print(p[::0])
    ~~~~~
ValueError: slice step cannot be zero
```


Compare `p[5:1:-1]` com `p[1:5:-1]` na Figura 12.2.

6. Meça a inversão.

```
python -c "
import timeit
def laco(s):
    saida = ''
    for c in s:
        saida = c + saida
    return saida
alvo = 'abcdefghij' * 100
print(laco(alvo) == alvo[::-1])
t1 = timeit.timeit('f(alvo)', globals={'f': laco, 'alvo': alvo}, number=2000)
tf = timeit.timeit('alvo[::-1]', globals={'alvo': alvo}, number=2000)
print(f'{t1/2000*1e6:.1f} us contra {tf/2000*1e6:.1f} us ({t1/tf:.0f}x)')
"
```

```
True
129.4 us contra 2.1 us (62x)
```

O fator exato varia bastante entre execuções (medi de 33 a 75 no mesmo equipamento); a ordem de grandeza, não.

7. Descubra quando o += fica quadrático. Crie `construir.py` com as três funções do capítulo e a medição, e rode:

```
python construir.py
```

n	+=	+= c/ alias	join
20000	1.7ms	8.0ms	1.7ms
40000	2.8ms	31.3ms	2.8ms
80000	5.3ms	132.1ms	7.5ms
160000	13.5ms	515.3ms	12.6ms

Os seus tempos vão diferir; o **formato** das colunas não deve. Confirme que a do meio quadruplica quando `n` dobra e as outras duas apenas dobram, e explique por que a única diferença entre as duas primeiras funções é a linha `guarda = s`.

8. Imutabilidade.

```
python -c "
nome = 'kali'
print(nome.replace('k', 'K'), nome)
nome[0] = 'K'
"
```

12. Strings: criação, indexação e fatiamento

```
Kali kali
Traceback (most recent call last):
  File "<string>", line 4, in <module>
    nome[0] = 'K'
    ~~~~^^^
TypeError: 'str' object does not support item assignment
```

A primeira linha imprimiu antes do erro: `replace` devolveu uma string nova (Kali) e a original (kali) não mudou.

9. Ordenação não é alfabética.

```
python -c "
nomes = ['Zebra', 'abacaxi', 'Banana', 'ácido', 'agua']
print(sorted(nomes))
print(sorted(nomes, key=str.lower))
print([ord(c) for c in 'ZaáčA'])
"
```

```
['Banana', 'Zebra', 'abacaxi', 'agua', 'ácido']
['abacaxi', 'agua', 'Banana', 'Zebra', 'ácido']
[90, 97, 225, 231, 65]
```

`key=str.lower` arrumou as maiúsculas, e nem assim `ácido` foi parar entre `abacaxi` e `agua`, que é onde um dicionário de português o colocaria: `á` é 225, maior que qualquer letra sem acento. Ordenação cultural é assunto de `locale`, retomado no Capítulo 14.

10. Literais adjacentes e a vírgula esquecida.

```
python -c "
PALAVRAS = [
    'erro',
    'aviso'
    'critico',
]
print(PALAVRAS, len(PALAVRAS))
"
```

```
['erro', 'avisocritico'] 2
```

Dois itens onde deveriam ser três, sem erro nenhum. Configure o seu editor para avisar (o Ruff tem a regra ISC para isso).

11. Reescreva o parser. Crie `parser.py`:

```
"""Le registros de largura fixa da OLT antiga."""

CAMPOS = {
    "data": slice(0, 10),
```

```

    "hora": slice(11, 19),
    "cliente": slice(20, 36),
    "nivel": slice(36, 42),
    "estado": slice(43, None),
}

LARGURA_MINIMA = 45

def analisar(linha):
    if len(linha) < LARGURA_MINIMA:
        raise ValueError(f"linha curta ({len(linha)} caracteres): {linha!r}")
    return {nome: linha[corte].strip() for nome, corte in CAMPOS.items()}

if __name__ == "__main__":
    LINHAS = [
        "2026-03-11 06:47:55 CHACARA-DO-ZE -18.4 OK",
        "2026-03-11 07:01:12 SITIO-MANGUEIRA -21.7 OK",
        "2026-03-11 07:15:00 CURTA",
    ]
    for linha in LINHAS:
        try:
            print(analisar(linha))
        except ValueError as erro:
            print("ignorada:", erro)

```

```
python parser.py
```

```

{'data': '2026-03-11', 'hora': '06:47:55', 'cliente': 'CHACARA-DO-ZE', 'nivel':
↪ '-18.4', 'estado': 'OK'}
{'data': '2026-03-11', 'hora': '07:01:12', 'cliente': 'SITIO-MANGUEIRA',
↪ 'nivel': '-21.7', 'estado': 'OK'}
ignorada: linha curta (25 caracteres): '2026-03-11 07:15:00 CURTA'

```

Três ganhos sobre a versão da abertura: o layout está num lugar só e com nome, a linha curta **falha alto** em vez de virar campo vazio, e trocar uma coluna é editar um número. O `strip` que limpa o preenchimento é assunto do próximo capítulo.

12. Feche com `deactivate`.

12.10. Resumo

- Aspas simples e duplas são equivalentes; aspas triplas atravessam linhas; literais adjacentes são concatenados na compilação — e a vírgula esquecida numa lista junta dois itens sem erro.
- String crua (`r"..."`) desliga os escapes: obrigatória em expressão regular e em caminho do Windows.

12. Strings: criação, indexação e fatiamento

- `str` é imutável: toda “alteração” devolve um objeto novo, e `s[0] = "x"` é `TypeError`. Não existe tipo caractere — `s[0]` é uma `str` de tamanho 1.
- Os números de uma fatia marcam **fronteiras entre** caracteres: `[a:b]` inclui `a`, exclui `b`, e tem `b - a` caracteres. Índice fora da faixa levanta `IndexError`; fatia fora da faixa devolve o que couber, inclusive `""`.
- Com passo negativo a fatia anda para trás, então o início precisa estar à direita do fim. `[::-1]` inverte; passo zero é `ValueError`.
- `slice(a, b)` é um objeto: guarde o layout de um formato de largura fixa em constantes com nome, não em números espalhados.
- Comparação é por ponto de código, não alfabética: toda maiúscula vem antes de toda minúscula, e acentuadas vêm depois de todas.
- `"".join(...)` é linear e sempre foi. `+=` num laço só é linear em CPython e só enquanto a string tiver uma única referência — basta um segundo nome para o tempo virar quadrático.

13. Métodos de string, formatação e f-strings

O importador de clientes recebe URLs de portal com e sem esquema, e normaliza tudo removendo o `https://`. A linha existe há dois anos:

```
url = "https://portal.exemplo.com"
print(repr(url.lstrip("https://")))
```

```
'ortal.exemplo.com'
```

Sumiu o `p` de `portal`. E o caso seguinte é pior:

```
print(repr("stackoverflow.com".lstrip("stack")))
print(repr("relatorio.log".rstrip(".log")))
```

```
'overflow.com'
'relatori'
```

`stackoverflow.com` virou `overflow.com`, e `relatorio.log` perdeu o `o` do nome além da extensão.

O motivo é que **`strip`, `lstrip` e `rstrip` não recebem um prefixo: recebem um conjunto de caracteres**. `lstrip("https://")` significa “remova do começo, um a um, todo caractere que esteja no conjunto `{h, t, p, s, :, /}`, até encontrar um que não esteja”. O `p` de `portal` está no conjunto, então foi. O `o` de `ortal` não está, e a remoção parou ali.

Este capítulo cobre os métodos de `str` e as três formas de formatar texto. É o capítulo de consulta do volume — e o que tem a maior densidade de armadilhas por linha.

13.1. `strip` é um conjunto, não um prefixo

A correção existe desde o **Python 3.9** (PEP 616):

```
print(repr(url.removeprefix("https://")))
print(repr("relatorio.log".removesuffix(".log")))
```

```
'portal.exemplo.com'
'relatorio'
```

13. Métodos de string, formatação e f-strings

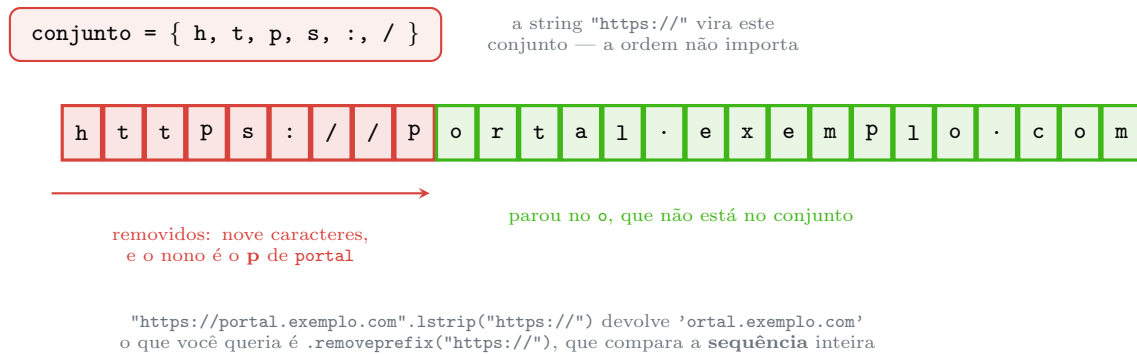


Figura 13.1.: `rstrip` consome caracteres do conjunto até achar um que não pertence.

`removeprefix` compara a sequência inteira e, se ela não estiver lá, devolve a string intacta — sem exceção e sem estrago. Antes do 3.9, o idioma correto era `s[len(p):]` `if s.startswith(p)` `else s`.

Onde `strip` é a ferramenta certa: **limpar espaço em branco**, que é o uso para o qual ele existe. Sem argumento, remove qualquer espaço, tabulação e quebra de linha das pontas:

```
t = " \t leitura \n "
print(repr(t.strip()), repr(t.lstrip()), repr(t.rstrip()))
```

```
'leitura' 'leitura \n ' ' \t leitura'
```

13.2. Os métodos, por família

13.2.1. Caixa

```
s = "Relatório de Leituras"
print(s.upper())
print(s.lower())
print(s.title())
print(s.capitalize())
```

```
RELATÓRIO DE LEITURAS
relatório de leituras
Relatório De Leituras
Relatório de leituras
```

`title()` põe maiúscula em toda palavra, inclusive em `De` — quase nunca é o que se quer em português. Para comparação sem diferenciar caixa, o método certo **não** é `lower()`:

```
print("straße".lower() == "STRASSE".lower())
print("straße".casefold() == "STRASSE".casefold())
```

```
False
True
```

`casefold()` é uma normalização mais agressiva, feita para comparação: ele sabe que o ß alemão equivale a ss. Regra: `lower()` para exibir, `casefold()` para comparar.

13.2.2. Busca e substituição

```
linha = "2026-03-11 06:47:55 CHACARA-DO-ZE -18.4 OK"

print(linha.find("CHACARA"), linha.find("ZZZ"))
print(linha.startswith("2026"), linha.endswith("OK"))
print(linha.startswith(("2025", "2026")))
print(linha.count("-"))
print(linha.replace("-", "/", 2))
```

```
20 -1
True True
True
5
2026/03/11 06:47:55 CHACARA-DO-ZE -18.4 OK
```

Três coisas para guardar. `find` devolve `-1` quando não acha, enquanto `index` levanta `ValueError` — prefira `index` quando a ausência for erro, para não deixar um `-1` virar índice válido. `startswith` e `endswith` aceitam uma **tupla** de opções, o que evita encadear `or`. E `replace` aceita um limite de substituições.

13.2.3. Divisão e junção

```
print("a,b,,c".split(","))
print("a b c".split())
print("a b c".split(" "))
```

```
['a', 'b', '', 'c']
['a', 'b', 'c']
['a', 'b', '', 'c']
```

`split()` **sem argumento** é um método diferente de `split(" ")`: ele quebra em qualquer sequência de espaço em branco, descarta os vazios e ignora as pontas. É o que você quer para texto humano. Com separador explícito, cada ocorrência gera um campo — inclusive vazios, que é o que você quer para CSV.

```
print("chave=valor=x".partition("="))
print("chave=valor=x".rpartition("="))
print("a,b,c".split(",", 1))
```

```
('chave', '=', 'valor=x')
('chave=valor', '=', 'x')
['a', 'b,c']
```

`partition` divide na **primeira** ocorrência e devolve sempre uma tupla de três elementos, mesmo quando não acha — o que dispensa checar tamanho antes de desempacotar. Para `chave=valor` com `=` no valor, é a ferramenta certa.

E `splitlines` entende as quebras de linha de todos os sistemas e não deixa vazio no fim:

```
print("l1\nl2\n".splitlines())
print("l1\nl2\n".split("\n"))
```

```
['l1', 'l2']
['l1', 'l2', '']
```

`join` já apareceu no Capítulo 12 e exige que todos os itens sejam `str`:

```
"-".join(["a", 1])
```

```
TypeError: sequence item 1: expected str instance, int found
```

A conversão vai dentro: `"-".join(str(x) for x in itens)`.

13.2.4. Testes de conteúdo

Os métodos `is*` são o campo minado do módulo:

```
for v in ["123", "12.3", " ", "²", "abc1", "", "café"]:
    print(f"{v!r: <8} digit={v.isdigit()} decimal={v.isdecimal()} "
          f"alpha={v.isalpha()} ascii={v.isascii()}")
```

```
'123'    digit=True decimal=True alpha=False ascii=True
'12.3'    digit=False decimal=False alpha=False ascii=True
' '       digit=True decimal=True alpha=False ascii=True
'²'       digit=True decimal=False alpha=False ascii=False
'abc1'    digit=False decimal=False alpha=False ascii=True
''        digit=False decimal=False alpha=False ascii=True
'café'    digit=False decimal=False alpha=True  ascii=False
```

Duas armadilhas de segurança aqui. `"12.3".isdigit()` é **falso** — o ponto não é dígito —, então `isdigit` não valida número decimal. E `" ".isdigit()` é **verdadeiro**: são os algarismos de largura total do japonês, que passam no teste e explodem em `int()`... ou pior, não explodem, porque `int()` os aceita. Para validar entrada numérica, o teste honesto é tentar converter:


```
try:
    valor = float(entrada)
except ValueError:
    ...
```

É o EAFP do Capítulo 7 outra vez. E note que `"".isalpha()` é falso — string vazia falha em todos os `is*`, o que é a resposta certa e às vezes esquecida.

13.2.5. Preenchimento e tradução

```
print(repr("42".zfill(6)), repr("-42".zfill(6)))
print(repr("ab".ljust(6, "."), repr("ab".center(6, ".")))
```

```
'000042' '-00042'
'ab....' '..ab..'
```

`zfill` é mais esperto que `rjust(6, "0")`: ele põe os zeros **depois** do sinal. Fora esse caso, prefira a mini-linguagem de formatação da próxima seção, que faz o mesmo dentro do molde.

`translate` com `str.maketrans` troca vários caracteres numa passada só, e é a forma rápida de remover acentos quando a tabela é fixa:

```
ACENTOS = str.maketrans("áâãäåêëíîóôõüçÀÃÄÅÊËÍÎÓÔÕÜÇ",
                        "aaaaeeioooouucAAAAEEIIOOOUUC")
print("Configuração Ótima da Estação".translate(ACENTOS))
```

```
Configuracao Otima da Estacao
```

Funciona e é explícito. A alternativa geral, que não depende de tabela escrita à mão, usa `unicodedata` e é assunto do Capítulo 14.

13.3. Três formas de formatar

Python tem três mecanismos de formatação, todos vivos e nenhum obsoleto por decreto:

```
cliente, nivel = "chacara-do-ze", -18.4

print("%s: %.1f dBm" % (cliente, nivel))
print("{}: {:.1f} dBm".format(cliente, nivel))
print(f"{cliente}: {nivel:.1f} dBm")
```

```
chacara-do-ze: -18.4 dBm
chacara-do-ze: -18.4 dBm
chacara-do-ze: -18.4 dBm
```

13. Métodos de string, formatação e f-strings

`%` é herança de C, do Python 1. Continua funcionando e ainda é o formato usado pelo módulo `logging` (por um bom motivo, adiante). Não use em código novo fora disso: ele confunde tupla com argumento único (`"%s" % (1, 2)` é `TypeError`) e não tem a mini-linguagem completa.

`.format()` chegou no Python 2.6 e é o mecanismo completo. Ainda é a escolha certa quando o molde é **dado**, e não código — vindo de um arquivo de configuração, do banco ou de tradução.

f-string chegou no Python 3.6 (PEP 498) e é o padrão para tudo o mais. É a mais legível, porque a expressão está onde ela aparece, e a mais rápida:

```
caso A: interpolacao simples, sem spec
%          104 ns  1.25x
.format()  227 ns  2.71x
f-string   84 ns  1.00x
concatenacao 128 ns  1.54x

caso B: com especificacao de formato
%          408 ns  1.10x
.format()  555 ns  1.50x
f-string   371 ns  1.00x
```

A f-string ganha porque não é chamada de função: o compilador a transforma em instruções de construção de string direto no bytecode.

13.4. A mini-linguagem de formatação

`.format()` e f-strings compartilham a mesma especificação, que é a parte depois dos dois-pontos. Vale aprender uma vez.

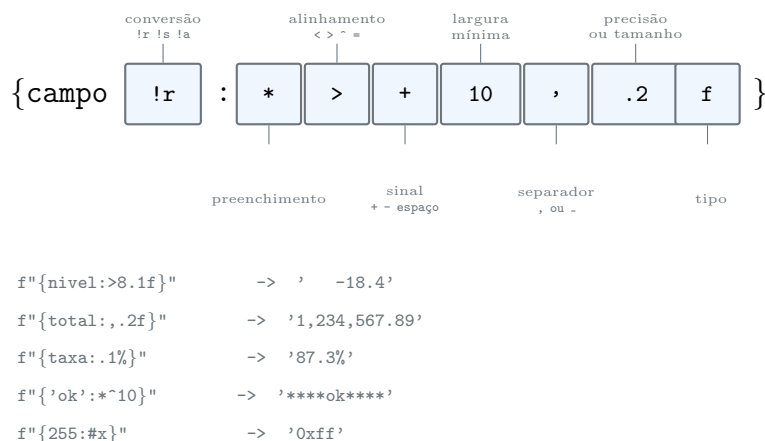


Figura 13.2.: A especificação de formato, campo a campo. Tudo é opcional.

Na prática:

```
print(f"{42:5d}|", f"{42:<5d}|", f"{42:^5d}|", f"{42:05d}|")
print(f"{1234567.891:,.2f}", f"{1234567.891:_.2f}")
print(f"{0.1234:.1%}", f"{255:#x}", f"{255:#b}")
```

```
print(f"{42:+d}", f"{-42:+d}")
print(f"{'ok':*^10}")
```

```
    42| 42    |  42  | 00042|
1,234,567.89 1_234_567.89
12.3% 0xff 0b11111111
+42 -42
****ok****
```

Números alinham à direita por padrão, texto à esquerda — o que é quase sempre o que se quer numa tabela. A largura pode vir de uma variável, com chaves aninhadas:

```
larg = 8
print(f"{'x':>{larg}}|")
```

```
x|
```

E o objeto pode ter a sua própria linguagem de formato. `datetime` é o exemplo mais útil:

```
import datetime

agora = datetime.datetime(2026, 3, 11, 6, 47, 55)
print(f"{agora:%d/%m/%Y %H:%M}")
```

```
11/03/2026 06:47
```

Isso funciona porque a especificação inteira é repassada ao `__format__` do objeto, tema do Volume 8.

13.5. Detalhes de f-string que valem saber

As três conversões. `!s` chama `str()`, `!r` chama `repr()`, `!a` chama `ascii()`:

```
n = "café"
print(f"{n!r}", f"{n!s}", f"{n!a}")
```

```
'café'  café  'caf\xe9'
```

Use `!r` em mensagem de erro e em log: ele mostra as aspas e revela espaço em branco invisível, que é exatamente o que você quer ver quando está depurando.

O = de depuração, do Python 3.8, imprime a expressão junto com o valor:

13. Métodos de string, formatação e f-strings

```
x = 5
print(f"x = {x}", f"x*2 = {x*2}")
```

```
x = 5 x*2 = 10
```

É a forma mais rápida de instrumentar um trecho sem escrever o nome duas vezes.

Chave literal se escreve dobrada:

```
print(f"{{literal}} e {x}")
```

```
{{literal}} e 5
```

Desde o **Python 3.12** (PEP 701), a f-string aceita qualquer expressão dentro, incluindo as mesmas aspas e a barra invertida:

```
d = {"cliente": "ze"}
print(f"{d['cliente']}")
print(f"{'\n'.join(['a', 'b'])}")
```

```
ze
a
b
```

Nas versões anteriores as duas linhas eram `SyntaxError`. Se o seu código precisa rodar em 3.11 ou antes, continue usando aspas diferentes por dentro e por fora.

13.6. Quando o molde não é código

f-string é **código compilado**, avaliado no lugar. Ela não serve para molde que vem de fora, e tentar fazer isso é uma vulnerabilidade: avaliar texto de terceiro como expressão dá acesso ao interior do programa.

Para molde vindo de arquivo, banco ou tradução, as opções seguras são duas. `str.format` com um molde de dados:

```
molde = "{cliente:<20}{nivel:>8.1f}" # veio da configuração
print(molde.format(cliente="ze", nivel=-18.4))
```

```
ze                -18.4
```

Ou `string.Template`, que é deliberadamente burro — só substitui `$nome`, sem chamar método nem acessar atributo:

```
from string import Template

t = Template("Cliente $cliente teve $n quedas")
print(t.substitute(cliente="ze", n=3))
print(t.safe_substitute(cliente="ze"))
```

```
Cliente ze teve 3 quedas
Cliente ze teve $n quedas
```

`substitute` levanta `KeyError` se faltar chave; `safe_substitute` deixa o marcador como está. Quando o molde é escrito por um usuário, `Template` é a escolha, porque `str.format` ainda permite alcançar atributos (`{obj.__class__}`) e vaziar informação.

13.7. Jeito Pythônico

f-string por padrão; `.format()` quando o molde é dado; `%` só no logging.

O caso do logging merece explicação, porque parece inconsistência:

```
# antipadrão
logging.debug(f"leitura de {cliente}: {nivel:.1f}")
```

```
# pythônico
logging.debug("leitura de %s: %.1f", cliente, nivel)
```

Com a f-string, a string é montada **sempre**, mesmo quando o nível `DEBUG` está desligado e a mensagem vai direto para o lixo. Com os `%` e os argumentos separados, o `logging` só monta se for realmente registrar. Num laço quente com log de depuração desligado, a diferença é medível. O Volume 9 volta a isso.

`removeprefix/removesuffix`, nunca `strip`, para tirar prefixo ou sufixo. É o assunto da abertura e provavelmente o erro mais comum deste capítulo em código real.

`split()` sem argumento para texto de humano, com argumento para formato de máquina. E `partition` quando o separador pode aparecer no valor.

Valide convertendo, não com `is*`. `isdigit` aceita algarismo de largura total e recusa ponto decimal; `try: float(x) except ValueError:` é honesto.

`casefold()` para comparar, `lower()` para exibir. E, se a comparação for de dado do usuário, o Capítulo 14 acrescenta a normalização Unicode que falta.

Não construa comando, consulta ou HTML por concatenação. Vale repetir do capítulo anterior porque aqui a tentação é maior: `f"SELECT * FROM x WHERE id = {entrada}"` é injeção de SQL, e a f-string a torna curta e bonita. Parâmetros de consulta no Volume 10, `subprocess` com lista no Volume 12, `template` com escapamento no Volume 14.

Um antipadrão discreto de alinhamento:

13. Métodos de string, formatação e f-strings

```
# antipadrão: conta o preenchimento na mão
print(cliente + " " * (20 - len(cliente)) + str(nivel))
```

```
# pythônico
print(f"{cliente:<20}{nivel:>8.1f}")
```

Sobre versões:

- **Python 2.6** — `str.format` e a mini-linguagem.
- **Python 3.6** — f-strings (PEP 498) e separador `_` em números formatados.
- **Python 3.8** — `o =` de depuração dentro da f-string.
- **Python 3.9** — `str.removeprefix` e `str.removesuffix` (PEP 616).
- **Python 3.12** — f-strings sem restrição de aspas e barra invertida (PEP 701).
- **Python 3.14** — t-strings (PEP 750), moldes que devolvem um objeto em vez de uma string pronta, para que a biblioteca faça o escapamento certo. É a resposta estrutural para injeção de SQL e HTML, e ainda estava chegando ao ecossistema quando este livro foi escrito.

13.8. Laboratório

Objetivo: reproduzir o estrago do `strip`, percorrer as famílias de métodos, dominar a mini-linguagem de formatação e produzir um relatório alinhado de verdade.

1. Ambiente.

```
mkdir -p ~/lab13 && cd ~/lab13
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. Reproduza a abertura.

```
python -c "
url = 'https://portal.exemplo.com'
print(repr(url.lstrip('https://')))
print(repr(url.removeprefix('https://')))
print(repr('stackoverflow.com'.lstrip('stack')))
print(repr('relatorio.log'.rstrip('.log')))
print(repr('relatorio.log'.removesuffix('.log')))
"
```

```
'ortal.exemplo.com'
'portal.exemplo.com'
'overflow.com'
'relatori'
'relatorio'
```

Escreva o conjunto de caracteres de "stack" e confirme, letra a letra, por que `stackoverflow` perdeu justamente cinco caracteres.

3. `strip` no uso para o qual serve.

```
python -c "
t = ' \t leitura \n '
print(repr(t.strip()), repr(t.lstrip()), repr(t.rstrip()))
print(repr(' -18.4 '.strip()))
"
```

```
'leitura' 'leitura \n ' ' \t leitura'
'-18.4'
```

4. Caixa e comparação.

```
python -c "
print('straße'.lower() == 'STRASSE'.lower())
print('straße'.casefold() == 'STRASSE'.casefold())
print('relatório de leituras'.title())
"
```

```
False
True
Relatório De Leituras
```

O De maiúsculo mostra por que `title()` quase nunca serve para português.

5. `split` tem dois comportamentos.

```
python -c "
print('a b c'.split())
print('a b c'.split(' '))
print('a,b,c'.split(','))
print('chave=valor=x'.partition('='))
print('l1\nl2\n'.splitlines(), 'l1\nl2\n'.split('\n'))
"
```

```
['a', 'b', 'c']
['a', 'b', '', 'c']
['a', 'b', '', 'c']
('chave', '=', 'valor=x')
['l1', 'l2'] ['l1', 'l2', '']
```

6. O campo minado dos `is*`.

13. Métodos de string, formatação e f-strings

```
python -c "  
for v in ['123', '12.3', ' ', '²', '', 'café']:  
    print(f'{v!r:<8} digit={v.isdigit()} decimal={v.isdecimal()}  
    ↪ numeric={v.isnumeric()} ascii={v.isascii()}')  
print(int(' '), float('12.3'))  
"
```

```
'123'    digit=True decimal=True numeric=True ascii=True  
'12.3'    digit=False decimal=False numeric=False ascii=True  
' '      digit=True decimal=True numeric=True ascii=False  
'²'      digit=True decimal=False numeric=True ascii=False  
' '      digit=False decimal=False numeric=False ascii=True  
'café'    digit=False decimal=False numeric=False ascii=False
```

```
123 12.3
```

A última linha é o que assusta: `int()` **aceita** os algarismos japoneses de largura total. Um formulário que valida com `isdigit` e converte com `int` aceita entrada que ninguém previu.

7. Percorra a mini-linguagem.

```
python -c "  
print(f'{42:5d}|', f'{42:<5d}|', f'{42:~5d}|', f'{42:05d}|')  
print(f'{1234567.891:,.2f}', f'{1234567.891:_.2f}')  
print(f'{0.1234:.1%}', f'{255:#x}', f'{255:#b}', f'{255:o}')  
print(f'{3.14159:.3f}', f'{3.14159:10.3f}|', f'{1e6:e}', f'{1e6:g}')  
print(f'{42:+d}', f'{42: d}', f'{ -42:+d}')  
print(f'\{"'ok':*~10}\")  
"
```

```
 42| 42  | 42  | 00042|  
1,234,567.89 1_234_567.89  
12.3% 0xff 0b11111111 377  
3.142      3.142| 1.000000e+06 1e+06  
+42  42 -42  
****ok****
```

8. Depuração com `=` e `!r`.

```
python -c "  
n = 'café '  
print(f'{n = }')  
print(f'{n!r}', f'{n!s}', f'{n!a}')  
"
```

```
n = 'café '  
'café ' café 'caf\xe9 '
```


Repare como o `!r` denuncia os dois espaços do fim, que o `!s` esconde. É a razão de usar `!r` em `log`.

9. f-string do 3.12.

```
python -c "
d = {'cliente': 'ze'}
print(f'{d[\"cliente\"]}')
"
```

```
ze
```

Em Python 3.11 essa linha seria `SyntaxError`.

10. Meça as três formas. Crie `bench.py` com os dois casos do capítulo e rode:

```
caso A: interpolacao simples, sem spec
%          104 ns  1.25x
.format()  227 ns  2.71x
f-string   84 ns  1.00x
concatenacao 128 ns  1.54x

caso B: com especificacao de formato
%          408 ns  1.10x
.format()  555 ns  1.50x
f-string   371 ns  1.00x
```

Os seus números vão diferir. Confirme apenas a ordem: f-string na frente, `.format()` atrás.

11. Template para molde de fora.

```
python -c "
from string import Template
t = Template('Cliente \${cliente} teve \${n} quedas')
print(t.substitute(cliente='ze', n=3))
print(t.safe_substitute(cliente='ze'))
"
```

```
Cliente ze teve 3 quedas
Cliente ze teve ${n} quedas
```

12. Monte o relatório. Crie `relatorio.py`:

```
"""Relatorio alinhado de niveis opticos."""

LEITURAS = [
    ("chacara-do-ze", -18.4, "OK"),
    ("sitio-mangueira", -21.75, "OK"),
    ("fazenda-boa-vista", -7.0, "ALERTA"),
]
```

```

LARG = 20

def emitir(leituras):
    linhas = [
        f"{'CLIENTE':<{LARG}}{'NIVEL':>8} {'ESTADO':<8}",
        "-" * (LARG + 18),
    ]
    linhas += [
        f"{'cliente':<{LARG}}{'nivel':>8.1f} {'estado':<8}"
        for cliente, nivel, estado in leituras
    ]
    media = sum(n for _, n, _ in leituras) / len(leituras)
    linhas += ["-" * (LARG + 18), f"{'MEDIA':<{LARG}}{'media':>8.1f}"]
    return "\n".join(linhas)

if __name__ == "__main__":
    print(emitir(LEITURAS))

```

```
python relatorio.py
```

CLIENTE	NIVEL	ESTADO
chacara-do-ze	-18.4	OK
sitio-mangueira	-21.8	OK
fazenda-boavista	-7.0	ALERTA
MEDIA	-15.7	

Três coisas aconteceram aqui. A largura veio de uma constante aninhada nas chaves, então mudar **LARG** realinha a tabela inteira. Os números alinham à direita e o texto à esquerda, sem uma única conta de **len**. E a montagem foi por **join** sobre uma lista, como manda o Capítulo 12.

Agora troque **LARG** para 25 e confirme que só o alinhamento muda. Depois substitua o nome de um cliente por um mais longo que **LARG** e observe: a largura é **mínima**, não máxima — o campo cresce e a coluna desalinha. Para truncar, a precisão em texto resolve: `f"{'cliente':<{LARG}}.{LARG}"`.

13. Feche com deactivate.

13.9. Resumo

- **strip**, **lstrip** e **rstrip** recebem um **conjunto de caracteres**, não um prefixo: `"https://portal".lstrip("https://")` come o p de portal. Para prefixo e sufixo, **removeprefix** e **removesuffix**, do Python 3.9.
- **lower()** para exibir, **casefold()** para comparar. **title()** capitaliza toda palavra e quase nunca serve para português.
- **find** devolve -1, **index** levanta **ValueError**. **startswith** e **endswith** aceitam tupla de opções.

- `split()` sem argumento quebra em qualquer espaço e descarta vazios; com separador, cada ocorrência gera campo. `partition` divide na primeira ocorrência e sempre devolve três elementos.
- Os métodos `is*` não validam número: `"12.3".isdigit()` é falso e `" ".isdigit()` é verdadeiro. Valide tentando converter.
- As três formas de formatar compartilham a mesma mini-linguagem depois dos dois-pontos: preenchimento, alinhamento, sinal, largura, separador de milhar, precisão e tipo — tudo opcional, e a largura pode vir de variável.
- f-string por padrão (mais legível e mais rápida); `.format()` quando o molde é dado; `%` no `logging`, para adiar a montagem da mensagem; `string.Template` quando o molde vem do usuário.
- `!r` em mensagem de erro e log — ele revela espaço em branco invisível. `f"{x = }"` instrumenta um trecho sem repetir o nome.

14. Unicode, bytes e codificação de caracteres

O sistema de cadastro exporta os clientes num CSV. O script de importação abre o arquivo, lê e quebra:

```
from pathlib import Path

print(Path("clientes.csv").read_text(encoding="utf-8"))
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc1 in position 16:
  ↳ invalid start byte
```

Alguém procura o erro na internet, encontra a resposta de sempre e aplica:

```
print(repr(Path("clientes.csv").read_text(encoding="utf-8", errors="ignore")))
```

```
'cliente;nivel\nCHCARA DO Z;-18.4\nSTIO MANGUEIRA;-21.7\n'
```

Agora não quebra mais. E CHÁCARA DO ZÉ virou CHCARA DO Z, SÍTIO virou STIO, e trinta mil cadastros entraram no banco sem os acentos — silenciosamente, para sempre, porque o arquivo original foi arquivado e ninguém percebeu por três meses.

O `errors="ignore"` não consertou nada: ele mandou o Python jogar fora todo byte que não entendeu. O arquivo estava em **latin-1**, não em UTF-8, e a única correção certa era dizer isso:

```
print(Path("clientes.csv").read_text(encoding="latin-1"))
```

```
cliente;nivel
CHÁCARA DO ZÉ;-18.4
SÍTIO MANGUEIRA;-21.7
```

Este capítulo fecha o Volume 2 com o que separa `str` de `bytes`, por que a fronteira entre os dois é onde os bugs de texto nascem, e o que `len()` realmente conta.

14.1. Duas coisas diferentes: `str` e `bytes`

No Python 3, o modelo é explícito e não há conversão automática entre os dois:

- **`str`** é uma sequência de **pontos de código** Unicode. É texto. Não tem codificação — a codificação é interna e não é assunto seu.

14. Unicode, bytes e codificação de caracteres

- **bytes** é uma sequência de **números de 0 a 255**. É dado bruto: o que trafega em soquete, o que está gravado em arquivo, o que sai de uma chamada de sistema.

```
s = "Zé"
b = s.encode("utf-8")

print(s, len(s), type(s))
print(b, len(b), type(b))
```

```
Zé 2 <class 'str'>
b'Z\xc3\xa9' 3 <class 'bytes'>
```

Dois caracteres viraram três bytes. **encode** transforma texto em bytes; **decode** faz o caminho de volta. Guardar a direção é metade da batalha, e o macete que funciona: **texto codifica para bytes; bytes decodificam para texto**.

Um detalhe que confunde: indexar **bytes** devolve **int**, não **bytes**.

```
print(b[0], type(b[0]))
print(b[:1], type(b[:1]))
print(b.hex(" "))
```

```
90 <class 'int'>
b'Z' <class 'bytes'>
5a c3 a9
```

Indexar dá o número; fatiar dá **bytes**. Isso é lembrete permanente de que **bytes** não é “string de bytes” — é uma sequência de inteiros que às vezes se parece com texto.

Literal **bytes** leva o prefixo **b** e só aceita ASCII:

```
b"Zé"
```

```
SyntaxError: bytes can only contain ASCII literal characters
```

E **bytearray** é a versão mutável, útil para montar um buffer:

```
ba = bytearray(b"abc")
ba[0] = 65
ba.append(33)
print(ba, bytes(ba))
```

```
bytearray(b'Abc!') b'Abc!'
```

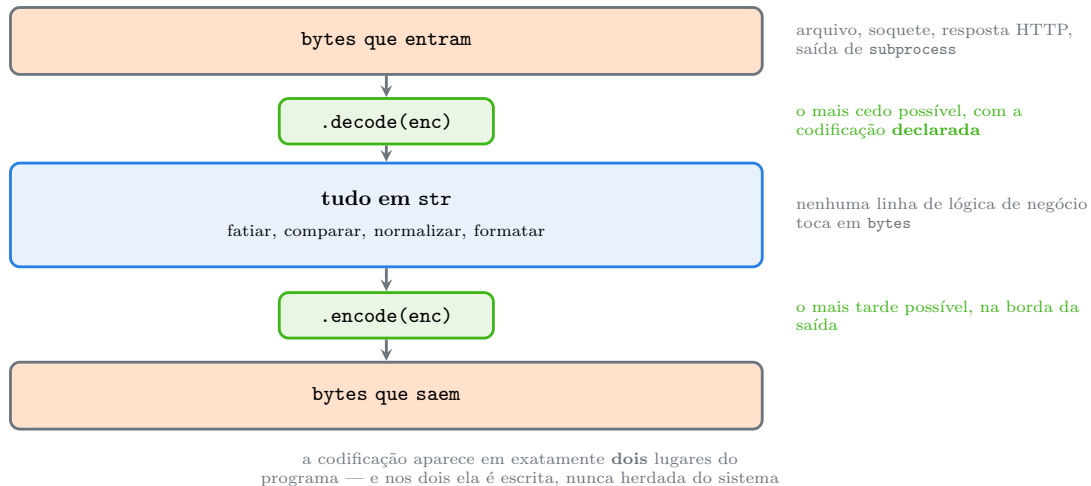


Figura 14.1.: Decodifique na entrada, trabalhe em `str`, codifique na saída.

14.2. O sanduíche Unicode

A disciplina que evita a classe inteira de bugs deste capítulo cabe em três palavras: **decodifique cedo, processe em `str`, codifique tarde**.

O erro da abertura foi de fatia do sanduíche: a codificação da entrada foi **adivinhada** em vez de declarada. E o `errors="ignore"` foi pior — trocou um erro alto por perda silenciosa de dados.

14.3. UTF-8 e as outras

Uma codificação é uma tabela: como transformar ponto de código em bytes e vice-versa.

UTF-8 representa qualquer ponto de código Unicode usando de um a quatro bytes:

```
import unicodedata

for c in "A", "é", "€", " ":
    e = c.encode("utf-8")
    print(f"{c}  U+{ord(c):04X}  {len(e)} bytes  {e.hex(' ')}")
    ↪ {unicodedata.name(c)}")
```

```
A  U+0041  1 bytes  41  LATIN CAPITAL LETTER A
é  U+00E9  2 bytes  c3 a9  LATIN SMALL LETTER E WITH ACUTE
€  U+20AC  3 bytes  e2 82 ac  EURO SIGN
   U+1F40D  4 bytes  f0 9f 90 8d  SNAKE
```

Três propriedades tornam o UTF-8 o padrão de fato:

1. **Compatível com ASCII.** Todo arquivo ASCII já é UTF-8 válido, byte por byte.
2. **Autossincronizante.** Byte de continuação sempre começa com 10, então nunca se confunde com um byte inicial. Perdeu o começo do fluxo? Dá para reencontrar a fronteira do próximo caractere.

14. Unicode, bytes e codificação de caracteres

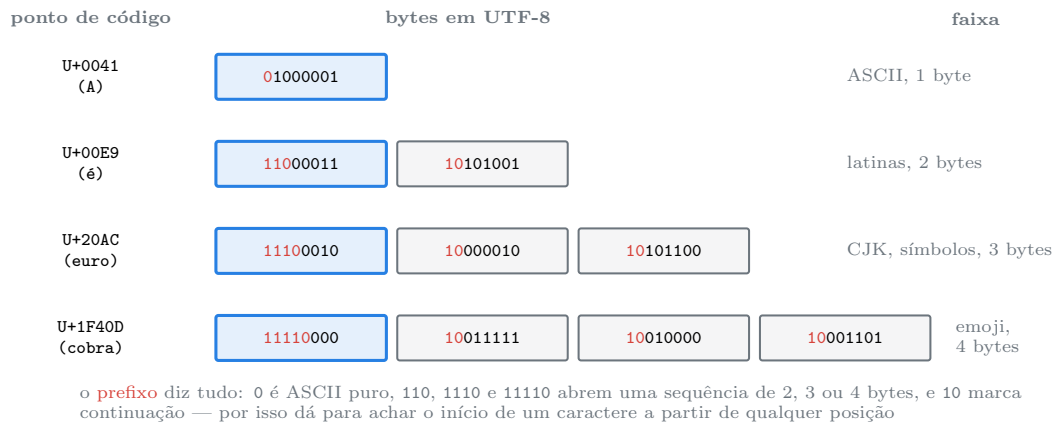


Figura 14.2.: UTF-8: o padrão de bits do primeiro byte diz quantos vêm atrás.

3. **Sem ambiguidade de ordem.** Não existe “UTF-8 big endian”, logo não há necessidade de marca de ordem de bytes.

latin-1 (ISO-8859-1) usa exatamente um byte por caractere e cobre 256 deles — o suficiente para o português, o que explica a longevidade dela em sistemas antigos. **cp1252** é a variante da Microsoft, quase igual, com símbolos nas posições 0x80–0x9F que a latin-1 deixa vazias.

E é da mistura entre elas que nasce o *mojibake*:

```
utf = "chácara do zé".encode("utf-8")
print(utf)
print(repr(utf.decode("latin-1")))
```

```
b'ch\xc3\xa1cara do z\xc3\xa9'
'chÃ;cara do zÃ©'
```

Ã; no lugar de á é a assinatura visual de “texto UTF-8 lido como latin-1”. Repare que **não houve erro**: latin-1 tem um caractere definido para cada um dos 256 bytes, então ela decodifica qualquer coisa e nunca falha. É o pior dos dois mundos — dado corrompido sem exceção nenhuma. UTF-8 no lugar de latin-1 falha alto, como na abertura; latin-1 no lugar de UTF-8 falha em silêncio.

14.4. Os tratadores de erro

O parâmetro `errors` decide o que fazer quando um byte (ou um caractere) não cabe:

```
latin = "CHÁCARA DO ZÉ".encode("latin-1")

print(repr(latin.decode("utf-8", errors="ignore")))
print(repr(latin.decode("utf-8", errors="replace")))
print(repr(latin.decode("utf-8", errors="backslashreplace")))
```



```
'CHCARA DO Z'
'CH CARA DO Z '
'CH\\xc1CARA DO Z\\xc9'
```

- `strict` (padrão) — levanta a exceção. **É quase sempre o certo**: um erro alto é melhor que dado errado.
- `ignore` — descarta em silêncio. É o vilão da abertura.
- `replace` — troca pelo caractere de substituição U+FFFD (o ? dentro de um losango). Perde a informação, mas deixa rastro visível.
- `backslashreplace` — preserva os bytes originais como escape, então é **reversível**. É a melhor escolha quando você precisa continuar e não quer perder nada.

Na direção da codificação, há mais um:

```
print(" ".encode("ascii", errors="xmlcharrefreplace"))
```

```
b'&#128013;'
```

E `surrogateescape`, que existe para um problema específico: nomes de arquivo no Linux são bytes, não texto, e podem conter sequências que não são UTF-8 válido.

```
ruim = b"arquivo-\xff.log"
nome = ruim.decode("utf-8", errors="surrogateescape")
print(repr(nome))
print(nome.encode("utf-8", errors="surrogateescape") == ruim)
```

```
'arquivo-\udcff.log'
True
```

Os bytes inválidos viram pontos de código reservados e voltam intactos na codificação. É assim que o Python trata nomes de arquivo, e é a razão de você conseguir abrir um arquivo cujo nome nenhum editor mostra direito.

14.5. `len()` conta pontos de código, não o que você vê

Esta é a parte que surpreende mesmo quem já sabe o resto:

```
casos = ["á", "ã", " ", " ", " "]
for c in casos:
    print(f"{c}  len={len(c)}  utf8={len(c.encode('utf-8'))}")
```

```
á  len=1  utf8=2
ã  len=2  utf8=3
   len=1  utf8=4
   len=5  utf8=18
   len=2  utf8=8
```

Os dois primeiros são **visualmente idênticos** e têm comprimentos diferentes. O primeiro é U+00E1, o caractere pré-composto. O segundo é a seguido de U+0301, o acento agudo combinante:

```
import unicodedata

nfd = "chácara"          # com acento combinante
print([unicodedata.name(ch) for ch in "á"])
```

```
['LATIN SMALL LETTER A', 'COMBINING ACUTE ACCENT']
```

A família de emoji tem cinco pontos de código: três pessoas e dois ZERO WIDTH JOINER no meio. A bandeira tem dois, que são as letras regionais B e R.

O que o usuário chama de “caractere” é um **grafema**, e o Python não conta grafemas — conta pontos de código. Para contar grafemas é preciso a biblioteca externa **regex** (com `\X`) ou **grapheme**. Na prática isso importa em três lugares: contar caracteres para um limite de campo, truncar texto sem partir um emoji ao meio, e alinhar tabela em terminal.

14.6. Normalização

Se "á" e "á" são strings diferentes, comparar texto do usuário fica perigoso:

```
nfc = "chácara"          # á pré-composto
nfd = "chácara"          # a + acento combinante

print(nfc == nfd, len(nfc), len(nfd))
print(nfc.casefold() == nfd.casefold())
print("chá" in nfd)
```

```
False 7 8
False
False
```

Nem a igualdade, nem o `casefold` do Capítulo 13, nem a busca por subcadeia funcionam. O `in` falhou porque "chá" (com á pré-composto) não aparece literalmente na versão decomposta.

A solução é normalizar antes de comparar. O `unicodedata` oferece quatro formas:

- **NFC** — compõe o máximo possível. É o formato canônico da web e o que você quer no armazenamento.
- **NFD** — decompõe o máximo possível. Útil para processar acentos separadamente (é o macOS que grava assim).
- **NFKC** e **NFKD** — as versões de *compatibilidade*, que além de compor ou decompor também unificam formas equivalentes: vira fi, vira 1, ² vira 2.

```
print(unicodedata.normalize("NFC", nfd) == nfc)
print(unicodedata.normalize("NFKC", " ² "))
```

```
True
12fi
```

E daí sai a receita de remover acentos sem tabela escrita à mão — melhor do que o `str.maketrans` do Capítulo 13, porque cobre o alfabeto inteiro:

```
def sem_acento(texto):
    decomposto = unicodedata.normalize("NFD", texto)
    return "".join(c for c in decomposto if unicodedata.category(c) != "Mn")

print(sem_acento("Configuração Ótima da Estação"))
```

```
Configuracao Otima da Estacao
```

Decompõe (separando a letra do acento) e descarta tudo que for da categoria `Mn`, *Mark*, *nonspacing* — os acentos combinantes. Funciona para qualquer idioma com acentuação decomponível.

E com isso a ordenação do Capítulo 12 finalmente fica em ordem de dicionário:

```
nomes = ["Zebra", "abacaxi", "ácido", "agua"]
print(sorted(nomes))
print(sorted(nomes, key=lambda s: (sem_acento(s).casefold(), s)))
```

```
['Zebra', 'abacaxi', 'agua', 'ácido']
['abacaxi', 'ácido', 'agua', 'Zebra']
```

A segunda é a ordem que um leitor de português espera. Para ordenação cultural completa — com as regras de cada idioma, incluindo os casos em que uma letra acentuada é uma letra distinta —, a resposta é `locale.strxfrm` ou PyICU.

14.7. A codificação padrão, e por que não confiar nela

```
import locale
import sys

print(sys.getdefaultencoding())
print(sys.getfilesystemencoding())
print(locale.getpreferredencoding(False))
```

```
utf-8
utf-8
UTF-8
```

14. Unicode, bytes e codificação de caracteres

Neste Kali, tudo é UTF-8. Num Windows com português do Brasil, `locale.getpreferredencoding()` costuma responder `cp1252` — e é exatamente essa diferença que faz um script funcionar na máquina de quem escreveu e quebrar na de quem usa.

`open()` sem `encoding=` usa a codificação preferida do sistema. Por isso a regra é curta e não tem exceção: **sempre passe `encoding=`**.

```
# antipadrão: depende do sistema operacional e do idioma dele
texto = Path("clientes.csv").read_text()
```

```
# pythônico: reproduzível em qualquer máquina
texto = Path("clientes.csv").read_text(encoding="utf-8")
```

Duas notas de versão importantes aqui. Desde o **Python 3.10**, `open()` sem `encoding=` emite `EncodingWarning` se o modo de aviso estiver ligado (`python -X warn_default_encoding`), justamente para achar essas chamadas. E a **PEP 686** define UTF-8 como padrão do interpretador; a mudança está programada para o Python 3.15, e até lá `PYTHONUTF8=1` ou `python -X utf8` liga o comportamento novo em qualquer versão a partir da 3.7.

14.8. O BOM

Alguns editores do Windows gravam três bytes no começo do arquivo (`EF BB BF`) para marcar que ele é UTF-8:

```
com_bom = "Zé".encode("utf-8-sig")
print(com_bom)
print(repr(com_bom.decode("utf-8")))
print(repr(com_bom.decode("utf-8-sig")))
```

```
b'\xef\xbb\xbfZ\xc3\xa9'
'Zé'
'Zé'
```

Lido como `utf-8`, o BOM vira um caractere invisível no começo da string — e o nome da primeira coluna do seu CSV passa a ser `cliente`, que não casa com `"cliente"` em lugar nenhum. É a causa do “a primeira coluna sumiu” em importação de planilha. O codec `utf-8-sig` consome o BOM na leitura; na escrita, prefira `utf-8` puro, porque BOM em UTF-8 não serve para nada em sistema que não seja Windows.

14.9. Jeito Pythônico

Sempre `encoding=`. Em `open`, em `read_text`, em `write_text`, em `subprocess.run(text=True, encoding=...)`, em `json.loads` de bytes. Se você não escreveu, o sistema escolheu por você.

`errors="strict"` é o padrão e deve continuar sendo. Quando precisar de tolerância, prefira `backslashreplace` (reversível) ou `replace` (visível) a `ignore` (silencioso). `ignore` só se justifica quando você já sabe exatamente o que está descartando e por quê.

Não adivinhe a codificação — descubra e documente. Quando o arquivo vem de fora sem contrato, `charset-normalizer` ou `chardet` dão um palpite; use o palpite para **decidir uma vez** e escreva a decisão no código, em vez de adivinhar a cada execução.

Normalize na fronteira, junto com a decodificação. Se o texto vai ser comparado, buscado ou usado como chave, `unicodedata.normalize("NFC", ...)` na entrada resolve a classe inteira de bugs de `"â" != "ã"`.

Para comparar identificador do usuário, normalize e faça casefold. Nessa ordem, e com NFKC quando o campo for nome de conta:

```
def chave(nome):
    return unicodedata.normalize("NFKC", nome).casefold()
```

Sem isso, `ADMIN`, `admin` e (largura total) são três contas diferentes num sistema e a mesma conta em outro — que é como se fabrica um ataque de personificação.

Não fatie bytes de texto. `b[:10]` pode cortar um caractere no meio e produzir uma sequência inválida. Decodifique primeiro, fatie a `str` depois.

Guarde e transmita em UTF-8. Não há decisão a tomar em 2026: UTF-8 na saída, sempre, sem BOM.

Um antipadrão que vale nome:

```
# antipadrão: o "conserto" que apaga dados
texto = dados.decode("utf-8", errors="ignore")
```

```
# pythônico: falhe alto, ou preserve o que não entendeu
texto = dados.decode("utf-8") # deixa estourar
texto = dados.decode("utf-8", errors="backslashreplace") # reversível
```

Sobre versões:

- **Python 3.0** — `str` virou texto Unicode e `bytes` virou tipo separado, sem conversão implícita. É a mudança mais importante do Python 3 e a que mais quebrou código do Python 2.
- **Python 3.1** — `surrogateescape` para nomes de arquivo (PEP 383).
- **Python 3.7** — `PYTHONUTF8` e `-X utf8` (PEP 540); `PYTHONCOERCECLOCALE` para o *locale C*.
- **Python 3.10** — `EncodingWarning` (PEP 597) para encontrar chamadas sem `encoding=`.
- **Python 3.11** — a base de dados Unicode subiu para a versão 14; cada versão do Python traz uma atualização, então `unicodedata.unidata_version` diz qual você tem.
- **Python 3.15** — UTF-8 passa a ser o padrão do interpretador (PEP 686), tornando `encoding="utf-8"` redundante. Escrever mesmo assim continua sendo boa prática enquanto houver 3.13 em produção.

14.10. Laboratório

Objetivo: reproduzir o estrago da abertura, ver os bytes de cada codificação, medir a diferença entre ponto de código e grafema, e escrever um importador que não perde acento.

1. Ambiente.

14. Unicode, bytes e codificação de caracteres

```
mkdir -p ~/lab14 && cd ~/lab14
python3 -m venv .venv && source .venv/bin/activate
python --version
```

```
Python 3.13.12
```

2. Fabrique o arquivo do problema.

```
python -c "
from pathlib import Path
conteudo = 'cliente;nivel\nCHÁCARA DO ZÉ;-18.4\nSÍTIO MANGUEIRA;-21.7\n'
Path('clientes.csv').write_bytes(conteudo.encode('latin-1'))
print(Path('clientes.csv').read_bytes()[:30])
"
```

```
b'cliente;nivel\nCH\xc1CARA DO Z\xc9;-1'
```

O `\xc1` de um byte só é a assinatura da latin-1: em UTF-8, `Á` seriam dois bytes.

3. Reproduza os três desfechos.

```
python -c "
from pathlib import Path
d = Path('clientes.csv').read_bytes()
try:
    d.decode('utf-8')
except UnicodeDecodeError as e:
    print('ERRO:', e)
print(repr(d.decode('utf-8', errors='ignore'))))
print(repr(d.decode('utf-8', errors='replace'))))
print(repr(d.decode('utf-8', errors='backslashreplace'))))
print(repr(d.decode('latin-1'))))
"
```

```
ERRO: 'utf-8' codec can't decode byte 0xc1 in position 16: invalid start byte
'cliente;nivel\nCHCARA DO Z;-18.4\nSTIO MANGUEIRA;-21.7\n'
'cliente;nivel\nCH CARA DO Z ; -18.4\nS TIO MANGUEIRA;-21.7\n'
'cliente;nivel\nCH\\xc1CARA DO Z\\xc9;-18.4\nS\\xcdTIO MANGUEIRA;-21.7\n'
'cliente;nivel\nCHÁCARA DO ZÉ;-18.4\nSÍTIO MANGUEIRA;-21.7\n'
```

Compare linha a linha: `ignore` apagou, `replace` marcou, `backslashreplace` preservou de forma reversível, e só a última está certa.

4. Descubra a codificação por tentativa.

```
python -c "
from pathlib import Path
d = Path('clientes.csv').read_bytes()
for enc in ('utf-8', 'latin-1', 'cp1252'):
    try:
        print(f'{enc:<10} OK      {d.decode(enc).splitlines()[1]!r}')
    except UnicodeDecodeError as e:
        print(f'{enc:<10} FALHA {e.reason} em {e.start}')
```

```
utf-8      FALHA invalid start byte em 16
latin-1    OK      'CHÁCARA DO ZÉ;-18.4'
cp1252     OK      'CHÁCARA DO ZÉ;-18.4'
```

Latin-1 e cp1252 concordam aqui — e sempre concordam quando não há bytes na faixa 0x80–0x9F. Como latin-1 **nunca** falha, ela dá certo em qualquer arquivo: “não deu erro” não é prova de que a codificação é essa.

5. Veja o mojibake ao contrário.

```
python -c "
utf = 'chácara do zé'.encode('utf-8')
print(utf)
print(repr(utf.decode('latin-1')))
```

```
b'ch\xc3\xa1cara do z\xc3\xa9'
'chÃ¡cara do zÃ©'
```

Sem exceção nenhuma. Grave o padrão ã, ã©, ã§: sempre que ele aparecer numa tela, é UTF-8 lido como latin-1.

6. Conte os bytes de cada caractere.

```
python -c "
import unicodedata
for c in 'A', 'é', '€', ' ':
    e = c.encode('utf-8')
    print(f'{c} U+{ord(c):04X} {len(e)}B {e.hex()}')
    ↪ {unicodedata.name(c)}')
print([f'{x:08b}' for x in 'é'.encode('utf-8')])
print([f'{x:08b}' for x in ' '.encode('utf-8')])
```

```
A U+0041 1B 41 LATIN CAPITAL LETTER A
é U+00E9 2B c3 a9 LATIN SMALL LETTER E WITH ACUTE
€ U+20AC 3B e2 82 ac EURO SIGN
U+1F40D 4B f0 9f 90 8d SNAKE
['11000011', '10101001']
['11110000', '10011111', '10010000', '10001101']
```

14. Unicode, bytes e codificação de caracteres

Confira os prefixos na Figura 14.2: 110 abre dois bytes, 11110 abre quatro, e todo byte de continuação começa com 10.

7. len não é o que você vê.

```
python -c "  
casos = ['á', 'á', '\U0001F40D', '\U0001F468\U0001F469\U0001F467',  
↪ '\U0001F1E7\U0001F1F7']  
for c in casos:  
    print(f'{c} len={len(c)} utf8={len(c.encode(\"utf-8\"))} {[hex(ord(x))  
↪ for x in c]}')  
"
```

```
á len=1 utf8=2 ['0xe1']  
á len=2 utf8=3 ['0x61', '0x301']  
len=1 utf8=4 ['0x1f40d']  
len=5 utf8=18 ['0x1f468', '0x200d', '0x1f469', '0x200d', '0x1f467']  
len=2 utf8=8 ['0x1f1e7', '0x1f1f7']
```

As duas primeiras linhas são o mesmo á na tela e strings diferentes na memória.

8. Normalize.

```
python -c "  
import unicodedata  
nfc, nfd = 'chácara', 'chácara'  
print(nfc, nfd)  
print(nfc == nfd, nfc.casefold() == nfd.casefold(), 'chá' in nfd)  
print(unicodedata.normalize('NFC', nfd) == nfc)  
print('chá' in unicodedata.normalize('NFC', nfd))  
print(nfc.encode('utf-8').hex(' '))  
print(nfd.encode('utf-8').hex(' '))  
"
```

```
chácara chácara  
False False False  
True  
True  
63 68 c3 a1 63 61 72 61  
63 68 61 cc 81 63 61 72 61
```

Três False na segunda linha — igualdade, casefold e busca, todos falhando em strings que a tela mostra idênticas. E os bytes na quinta e sexta linha explicam por quê.

9. NFKC unifica formas de compatibilidade.

```
python -c "  
import unicodedata  
print(unicodedata.normalize('NFKC', '²'))  
print('²' == 'fi', unicodedata.normalize('NFKC', '²') == 'fi')  
print(unicodedata.normalize('NFKC', '² '))  
"
```



```
12fi
False True
admin
```

A última linha é o motivo de NFKC entrar na comparação de nome de conta.

10. Remova acentos direito.

```
python -c "
import unicodedata
def sem_acento(t):
    d = unicodedata.normalize('NFD', t)
    return ''.join(c for c in d if unicodedata.category(c) != 'Mn')
print(sem_acento('Configuração Ótima da Estação'))
nomes = ['Zebra', 'abacaxi', 'ácido', 'agua']
print(sorted(nomes))
print(sorted(nomes, key=lambda s: (sem_acento(s).casefold(), s)))
"
```

```
Configuracao Otima da Estacao
['Zebra', 'abacaxi', 'agua', 'ácido']
['abacaxi', 'ácido', 'agua', 'Zebra']
```

11. Encontre o BOM.

```
python -c "
com = 'cliente;nivel'.encode('utf-8-sig')
print(com[:6])
print(repr(com.decode('utf-8').split(';')[0]))
print(repr(com.decode('utf-8-sig').split(';')[0]))
print(com.decode('utf-8').split(';')[0] == 'cliente')
"
```

```
b'\xef\xbb\xbfcli'
'cliente'
'cliente'
False
```

O False da última linha é o “a primeira coluna sumiu” de toda importação de planilha exportada pelo Excel.

12. Escreva o importador que não perde acento. Crie importar.py:

```
"""Le o CSV da OLT antiga sem perder acento e normaliza o texto na entrada."""

import csv
import unicodedata
from pathlib import Path
```

14. Unicode, bytes e codificação de caracteres

```
CODIFICACAO_ORIGEM = "latin-1"          # decidido uma vez, documentado aqui

def chave(texto):
    """Forma canonica para comparar e indexar."""
    return unicodedata.normalize("NFKC", texto).casefold()

def carregar(caminho):
    bruto = Path(caminho).read_text(encoding=CODIFICACAO_ORIGEM)
    bruto = unicodedata.normalize("NFC", bruto)
    return list(csv.DictReader(bruto.splitlines(), delimiter=";"))

if __name__ == "__main__":
    registros = carregar("clientes.csv")
    for r in registros:
        print(f"{r['cliente']:<20} {chave(r['cliente']):<20} {r['nivel']:>7}")

    saida = Path("clientes-utf8.csv")
    saida.write_text(
        "\n".join(f"{r['cliente']};{r['nivel']}" for r in registros) + "\n",
        encoding="utf-8",
    )
    print("gravado:", saida.stat().st_size, "bytes em utf-8")
```

```
python importar.py
```

```
CHÁCARA DO ZÉ      chácara do zé      -18.4
SÍTIO MANGUEIRA    sítio mangueira    -21.7
gravado: 45 bytes em utf-8
```

Todas as camadas do sanduíche estão ali: a codificação de origem declarada numa constante com nome, a normalização NFC logo depois da decodificação, o texto em `str` durante o processamento, uma função de chave canônica para comparar, e UTF-8 explícito na saída.

13. Confirme que a saída está certa.

```
python -c "
from pathlib import Path
d = Path('clientes-utf8.csv').read_bytes()
print(d)
print(d.decode('utf-8'))
"
```

```
b'CH\xc3\x81CARA DO Z\xc3\x89;-18.4\nS\xc3\x8dTIO MANGUEIRA;-21.7\n'
CHÁCARA DO ZÉ;-18.4
SÍTIO MANGUEIRA;-21.7
```

Dois bytes por letra acentuada — agora é UTF-8 de verdade.

14. Feche com `deactivate`.

14.11. Resumo

- `str` é sequência de pontos de código Unicode; `bytes` é sequência de inteiros de 0 a 255. Não há conversão implícita: `encode` vai de texto para bytes, `decode` volta. Indexar `bytes` devolve `int`; fatiar devolve `bytes`.
- Sanduíche Unicode: decodifique na entrada, processe só em `str`, codifique na saída. A codificação aparece em dois lugares do programa, e nos dois é escrita.
- UTF-8 usa de 1 a 4 bytes, é compatível com ASCII e autossincronizante. latin-1 usa 1 byte e **nunca falha ao decodificar** — por isso corrompe em silêncio, enquanto UTF-8 falha alto. `Ãj` na tela é a assinatura do erro.
- `errors="ignore"` apaga dados sem avisar; prefira `strict` (padrão), `replace` (visível) ou `backslashreplace` (reversível). `surrogateescape` é o que permite tratar nome de arquivo inválido.
- `len()` conta pontos de código, não grafemas: "á" pode ter comprimento 1 ou 2, uma família de emoji tem 5 e uma bandeira tem 2.
- Normalize antes de comparar: NFC para armazenar, NFD para processar acento, NFKC para comparar identificador — junto com `casefold`. Sem isso, strings visualmente idênticas são diferentes para `==`, para `in` e para chave de dicionário.
- `open()` sem `encoding=` usa o padrão do sistema, que é cp1252 em Windows brasileiro e utf-8 em Linux. Passe sempre, até a PEP 686 chegar no 3.15.
- Remoção correta de acentos: NFD e descarte da categoria Mn. Ordenação de dicionário: essa chave mais `casefold`, ou `locale.strxfrm` para as regras completas.

Parte III.

**Volume 3 — Controle de Fluxo e
Funções**

15. Condicionais: if, elif, else e a expressão condicional

O sistema de monitoramento de uma rede de fibra recebe, a cada cinco minutos, o nível óptico de milhares de ONUs. A faixa saudável vai de -28 dBm a -8 dBm: abaixo disso o sinal está fraco demais para o equipamento entender, acima disso está forte demais e satura o receptor. Alguém escreveu o alerta:

```
def alerta(nivel):  
    if nivel < -28 or -8:  
        return "fora de faixa"  
    return "ok"
```

Lê-se em voz alta como português correto — “se o nível for menor que -28 ou -8 ” — e o resultado é este:

```
for n in (-20.0, -3.0, -30.0):  
    print(n, alerta(n))
```

```
-20.0 fora de faixa  
-3.0 fora de faixa  
-30.0 fora de faixa
```

Todas as ONUs da rede aparecem fora de faixa. A equipe de campo passou uma semana perseguindo alarme falso antes de alguém rodar isto:

```
print(bool(-8))  
print((-20.0 < -28) or -8)
```

```
True  
-8
```

O `or` não distribui a comparação. O Python avalia `nivel < -28` (falso), passa para o segundo operando, encontra `-8`, e `-8` é um objeto verdadeiro — como qualquer número diferente de zero, pela regra do Capítulo 11. A condição inteira é sempre verdadeira, e o `if` sempre entra. Não houve erro de sintaxe, não houve traceback, não houve aviso: o código está perfeitamente correto para o Python e perfeitamente errado para a rede.

A forma certa é uma cadeia de comparação, e ela também é mais curta:

15. Condicionais: *if*, *elif*, *else* e a expressão condicional

```
def alerta(nivel):
    if not -28 <= nivel <= -8:
        return "fora de faixa"
    return "ok"

for n in (-20.0, -3.0, -30.0, -8.0):
    print(n, alerta(n))
```

```
-20.0 ok
-3.0 fora de faixa
-30.0 fora de faixa
-8.0 ok
```

Este capítulo é sobre decidir: a sintaxe do `if`, o que exatamente o Python considera verdadeiro na hora de decidir, e as três ou quatro construções que substituem escadas de condicionais por código que se lê de uma vez.

15.1. `if`, `elif`, `else`

A estrutura é a que se espera, com a indentação do Capítulo 6 delimitando os blocos e os dois-pontos fechando cada cabeçalho:

```
nivel = -19.5

if nivel < -28:
    estado = "critico"
elif nivel < -25:
    estado = "atencao"
elif nivel <= -8:
    estado = "normal"
else:
    estado = "alto demais"

print(estado)
```

```
normal
```

Três detalhes que importam mais do que parecem:

- `elif` é uma palavra só. Não existe `else if` em Python, e escrever `else:` seguido de um `if` indentado funciona, mas empurra tudo um nível para a direita.
- `elif` e `else` são **opcionais**, e pode haver quantos `elif` você quiser. Um `if` sozinho é uma condicional completa.
- **A primeira condição verdadeira vence.** As de baixo nem chegam a ser avaliadas — e isso não é detalhe de desempenho, é semântica.

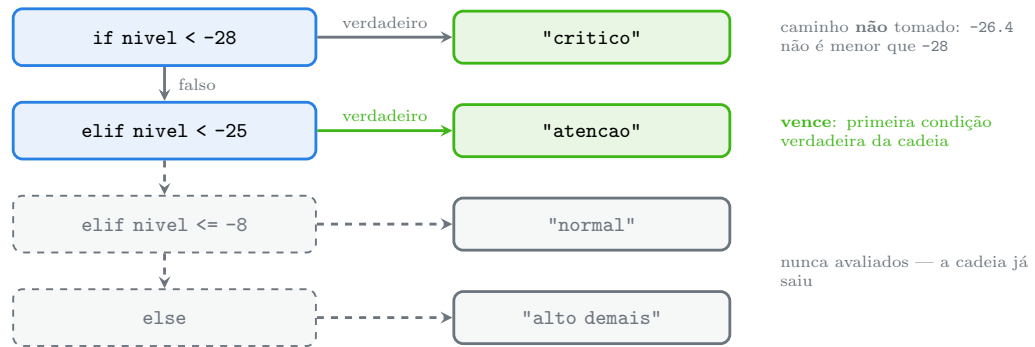


Figura 15.1.: A cadeia é avaliada de cima para baixo e para no primeiro teste verdadeiro.

Dá para ver a parada acontecendo com uma função que anuncia quando é chamada:

```
def testa(rotulo, valor):
    print(f"testando {rotulo}")
    return valor

if testa("A", False):
    print("A venceu")
elif testa("B", True):
    print("B venceu")
elif testa("C", True):
    print("C venceu")
else:
    print("nenhuma")
```

```
testando A
testando B
B venceu
```

C era verdadeiro e nunca foi testado. Se `testa("C", ...)` gravasse no banco, a gravação não teria acontecido. Condição com efeito colateral é uma armadilha justamente por isso.

Como consequência prática: **ordem importa e ordem esconde bug**. Numa escada de faixas numéricas, trocar duas linhas de lugar muda o resultado sem mudar nada visível. Se você inverter os dois primeiros `elif` do exemplo acima, -26.4 continua caindo em `atencao`, mas -31.0 também — porque `-31 < -25` é verdadeiro e a linha do `critico` deixou de ser alcançável para todo mundo.

15.2. O que o `if` considera verdadeiro

O `if` não exige um `bool`. Ele chama o protocolo de valor-verdade do Capítulo 11 em qualquer objeto:

15. Condicionais: if, elif, else e a expressão condicional

```
for v in (0, 0.0, 0j, "", b"", [], (), {}, set(), None, range(0), "0", [0],
↪ -1):
    print(f"{v!r:>10} {bool(v)}")
```

```
      0  False
     0.0 False
      0j False
       '' False
     b'' False
        [] False
        () False
        {} False
     set() False
      None False
range(0, 0) False
      '0'  True
      [0]  True
      -1  True
```

Zero de qualquer tipo numérico, contêiner vazio e `None` são falsos; o resto é verdadeiro. Repare em `"0"` e `[0]`: a string com o dígito zero e a lista que contém um zero são **verdadeiras**, porque não estão vazias.

Isso torna idiomático testar contêiner diretamente:

```
portas = []

if portas:
    print("tem porta")
else:
    print("lista vazia")
```

lista vazia

`if portas:` em vez de `if len(portas) > 0:`. É mais curto, funciona para qualquer sequência e não força o objeto a calcular um comprimento que pode ser caro.

E é aqui que mora o segundo bug clássico do capítulo. Quando 0 é um valor legítimo, `if not x:` confunde “não informado” com “informado como zero”:

```
def desconto_ruim(p=None):
    if not p:
        p = 10
    return p

def desconto(p=None):
    if p is None:
        p = 10
    return p
```

```
for arg in (0, 5, None):
    print(f"{str(arg):>4}  ruim={desconto_ruim(arg):>3}
      ↪ certo={desconto(arg):>3}")
```

```
    0  ruim= 10  certo=  0
    5  ruim=  5  certo=  5
None  ruim= 10  certo= 10
```

O cliente que tinha desconto de 0% recebeu 10%. A regra é curta: **para testar ausência, use `is None`**; para testar vazio, use o valor-verdade. Vale também para string ("" é um nome vazio informado, `None` é nome não informado) e para lista (`[]` é “consultei e não achou nada”, `None` é “não consultei”).

O `is` aqui é o mesmo do Capítulo 8: compara identidade, e `None` é um singleton, então `is None` é a comparação certa — e mais rápida que `== None`, que ainda por cima pode ser sequestrada por uma classe que defina `__eq__`.

15.3. Comparações encadeadas

O Python permite escrever a desigualdade dupla como se escreve em matemática:

```
x = 5
print(1 < x < 10)
print(1 < x and x < 10)
```

```
True
True
```

As duas formas são equivalentes em resultado, mas não em execução: na cadeia, o operando do meio é avaliado **uma vez só**.

```
def um():
    print("avaliou uma vez")
    return 5

print(0 < um() < 10)
```

```
avaliou uma vez
True
```

Escrito como `0 < um() and um() < 10`, a função seria chamada duas vezes. O bytecode mostra o truque — o Python duplica o valor na pilha em vez de recalculá-lo:

15. Condicionais: if, elif, else e a expressão condicional

```
import dis

dis.dis("a < b < c")
```

0	RESUME	0
1	LOAD_NAME	0 (a)
	LOAD_NAME	1 (b)
	SWAP	2
	COPY	2
	COMPARE_OP	2 (<)
	COPY	1
	TO_BOOL	
	POP_JUMP_IF_FALSE	5 (to L1)
	POP_TOP	
	LOAD_NAME	2 (c)
	COMPARE_OP	2 (<)
	RETURN_VALUE	
L1:	SWAP	2
	POP_TOP	
	RETURN_VALUE	

O POP_JUMP_IF_FALSE é o curto-circuito: se a primeira comparação falha, c nunca é carregado.

A cadeia funciona com qualquer operador de comparação e com mais de dois:

```
print(-28 <= -19.5 <= -8)
print("a" < "b" < "c")
print(1 == 1 != 2)
```

```
True
True
True
```

O último merece atenção: `1 == 1 != 2` significa `1 == 1 and 1 != 2`, não `(1 == 1) != 2`. Encadear operadores de igualdade com operadores de ordem é legal e quase sempre confuso — use quando for uma faixa (`a <= x <= b`) e evite o resto.

15.4. Cláusulas de guarda em vez de pirâmide

Condicional aninhada cresce para a direita e o código fica difícil de seguir depois do terceiro nível:

```
def registrar_aninhado(leitura):
    if leitura is not None:
        if leitura.get("nivel") is not None:
            if -28 <= leitura["nivel"] <= -8:
                return "ok"
```

```

        else:
            return "fora de faixa"
    else:
        return "sem nivel"
else:
    return "sem leitura"

```

O caso normal — o "ok" — é o mais escondido de todos, e cada `else` está a dezenas de colunas do `if` que ele fecha. A mesma função, escrita com **cláusulas de guarda**, trata cada caso excepcional na hora e devolve:

```

def registrar(leitura):
    if leitura is None:
        return "sem leitura"
    nivel = leitura.get("nivel")
    if nivel is None:
        return "sem nivel"
    if not -28 <= nivel <= -8:
        return "fora de faixa"
    return "ok"

```

```

casos = [None, {}, {"nivel": -19.5}, {"nivel": -31.0}]
for c in casos:
    print(f"{str(c):<18} {registrar_aninhado(c):<14} {registrar(c)}")

```

None	sem leitura	sem leitura
{}	sem nivel	sem nivel
{'nivel': -19.5}	ok	ok
{'nivel': -31.0}	fora de faixa	fora de faixa

Idênticas no comportamento, distantes na legibilidade. A versão com guardas tem um nível de indentação, lê-se de cima para baixo como uma lista de pré-condições e termina no caminho feliz.

Numa função sem `return` — dentro de um laço, por exemplo — a guarda usa `continue`, que é assunto do capítulo 17.

15.5. A expressão condicional

`if` é uma **instrução**: ele não produz valor, então não cabe dentro de uma expressão. Quando você precisa escolher um valor, o operador ternário do Python resolve:

```

nivel = -19.5
rotulo = "ok" if -28 <= nivel <= -8 else "fora"
print(rotulo)

```

15. Condicionais: if, elif, else e a expressão condicional

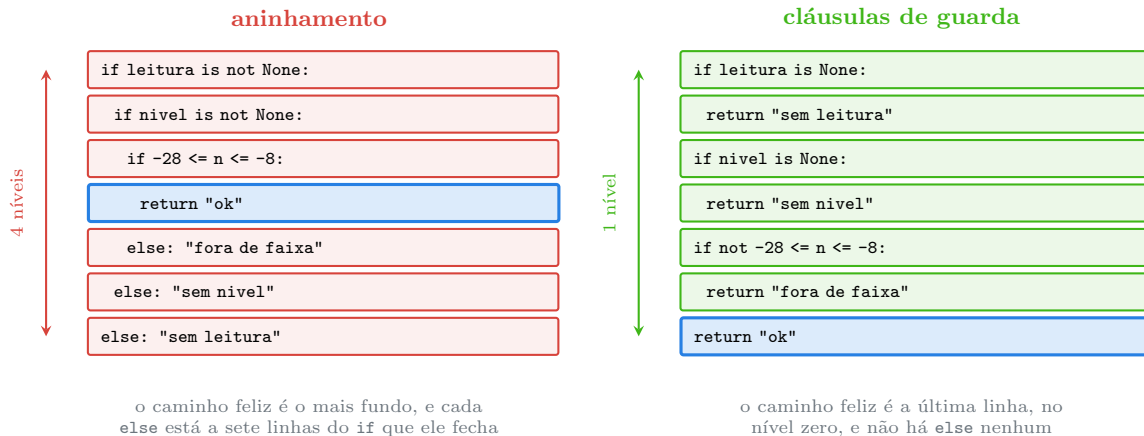


Figura 15.2.: A mesma lógica em pirâmide e em guardas: só a indentação muda, e ela muda tudo.

```
ok
```

A ordem é `valor_se_verdadeiro if condição else valor_se_falso` — a condição fica no meio, ao contrário do `? : de C e JavaScript`. É estranho no primeiro dia e natural no terceiro, porque se lê como português: “*ok*”, *se o nível estiver na faixa; senão, “fora”*.

O **else** é **obrigatório**. Não existe expressão condicional sem alternativa — o Python precisa produzir um valor nos dois caminhos.

Onde ele rende de verdade é dentro de outra expressão, em lugares onde um `if` de bloco não caberia:

```
print(["ok" if -28 <= n <= -8 else "fora") for n in (-20, -3, -30)])
```

```
['ok', 'fora', 'fora']
```

Dentro de uma f-string do Capítulo 13, cuidando de plural:

```
for falta in (0, 1, 3):
    print(f'{falta} porta{"s" if falta != 1 else ""} livre{"s" if falta != 1
    ↪ else ""}')

```

```
0 portas livres
1 porta livre
3 portas livres

```

E para tratar ausência sem quebrar a formatação:

```
valores = {"onu-01": -19.5, "onu-02": None, "onu-03": -31.0}
for nome, n in valores.items():
    print(f"{nome}: {'sem leitura' if n is None else f'{n:.1f} dBm'}")
```

```
onu-01: -19.5 dBm
onu-02: sem leitura
onu-03: -31.0 dBm
```

Ternários aninhados são legais e funcionam:

```
n = -31.0
print("critico" if n < -28 else "atencao" if n < -25 else "ok")
```

```
critico
```

Funciona, e é o limite. Com quatro faixas isso vira uma linha que ninguém revisa direito — a partir daí, uma função com cadeia de `if` e `return` é mais barata de ler e de testar.

15.6. O operador morsa em condições

Desde o **Python 3.8** (PEP 572), `:=` atribui **dentro** de uma expressão. O caso que justifica o operador é exatamente o do `if`: você precisa do resultado de uma chamada tanto para testar quanto para usar.

```
import re

registro = "onu-07 nivel=-24.3 status=up"
if (m := re.search(r"nivel=(-?\d+\.\d+)", registro)):
    print("nivel:", float(m.group(1)))
else:
    print("sem nivel")
```

```
nivel: -24.3
```

Sem ele, ou você chama `re.search` duas vezes, ou atribui antes do `if` e deixa a variável viva num escopo mais largo do que precisa. Com ele, `m` nasce e é testado na mesma linha.

Duas regras de higiene: os parênteses em torno do `:=` são quase sempre dispensáveis, mas deixá-los evita ambiguidade de precedência e sinaliza a atribuição para quem lê; e `:=` **não** substitui `=` na atribuição comum — usar o morsa fora de uma expressão é `SyntaxError`.

15.7. Escada de `elif` ou tabela?

Quando toda a cadeia testa igualdade contra a mesma variável, ela não é lógica: é uma tabela escrita como código.

15. Condicionais: *if*, *elif*, *else* e a expressão condicional

```
def taxa_elif(plano):
    if plano == "basico":
        return 79.90
    elif plano == "turbo":
        return 129.90
    elif plano == "gamer":
        return 199.90
    else:
        raise ValueError(f"plano desconhecido: {plano!r}")
```

Um dicionário diz o mesmo em três linhas, e o preço vira dado em vez de código:

```
TARIFAS = {"basico": 79.90, "turbo": 129.90, "gamer": 199.90}

print(TARIFAS["turbo"])
print(TARIFAS.get("empresarial", 0.0))
```

```
129.9
0.0
```

Um dicionário de funções faz o mesmo com comportamento no lugar do valor — é o *dispatch table*, e é o que substitui o **switch** que o Python não tem. Isso vale para **igualdade contra constantes**. Quando as condições são faixas, testes compostos ou desestruturação de dados, a escada de **if** continua sendo a resposta certa — ou o **match**, que estrutura o caso de desestruturação e é o assunto do capítulo 17.

15.8. Jeito Pythônico

Teste o objeto, não o comprimento dele. **if portas:** em vez de **if len(portas) > 0:**, **if not texto:** em vez de **if texto == "":**. A PEP 8 é explícita nesse ponto.

Para ausência, is None. Sempre que 0, "" ou [] forem valores legítimos, **if x is None:** é a única forma correta de perguntar “veio alguma coisa?”. **== None** funciona por acidente e **not x** funciona até o dia em que alguém passa zero.

Compare em cadeia. **-28 <= nivel <= -8** é mais curto, avalia o meio uma vez e não tem como virar o bug do **or -8** da abertura.

Prefira guardas a aninhamento. Trate os casos excepcionais no começo, saia cedo, e deixe o caminho normal sem indentação nenhuma. Três níveis de **if** dentro de **if** é o sinal de que a função quer virar duas.

Não escreva if x == True:. É redundante e é sutilmente diferente de **if x:** para objetos que não são **bool**. Pela mesma razão, **if x is True:** só se você realmente quer o singleton **True** e não “algo verdadeiro” — o que é raro.


```
# antipadrão
if usuario.ativo == True:
    ...
if len(erros) != 0:
    ...
if resultado != None:
    ...
```

```
# pythônico
if usuario.ativo:
    ...
if erros:
    ...
if resultado is not None:
    ...
```

Não faça a mesma atribuição nos dois ramos. Um `if/else` que só muda o valor de uma variável é uma expressão condicional disfarçada:

```
# antipadrão
if -28 <= nivel <= -8:
    rotulo = "ok"
else:
    rotulo = "fora"
```

```
# pythônico
rotulo = "ok" if -28 <= nivel <= -8 else "fora"
```

Ternário só quando cabe numa linha e numa leitura. A partir de dois `else` encadeados, escreva uma função.

Dicionário no lugar de escada de igualdades. Se todos os testes são `variavel == constante`, a estrutura de dados certa é um `dict`.

Sobre versões:

- **Python 3.8** — operador morsa `:=` (PEP 572), que tornou o `if (m := ...) idiomático`.
- **Python 3.10** — instrução `match/case` (PEP 634), para os casos em que a decisão é sobre a *forma* do dado, não sobre um valor escalar.
- **Python 3.12** — mensagens de erro de sintaxe mais precisas para `if` sem dois-pontos e para `=` onde se queria `==`, o que encurta a caçada descrita no Capítulo 7.

15.9. Laboratório

Objetivo: reproduzir o alarme falso da abertura, ver a tabela de valores falsos com os próprios olhos, medir a diferença entre `not x` e `is None`, e terminar com um monitor de rede que classifica leituras e diz o que fazer com cada uma.

1. Ambiente.

15. Condicionais: if, elif, else e a expressão condicional

```
mkdir -p ~/lab15 && cd ~/lab15
python3 -m venv .venv && source .venv/bin/activate
python --version
```

```
Python 3.13.12
```

2. Reproduza o bug do or.

```
python -c "
def alerta(nivel):
    if nivel < -28 or -8:
        return 'fora de faixa'
    return 'ok'
for n in (-20.0, -3.0, -30.0):
    print(n, alerta(n))
print('bool(-8) =', bool(-8))
"
```

```
-20.0 fora de faixa
-3.0 fora de faixa
-30.0 fora de faixa
bool(-8) = True
```

Três leituras completamente diferentes, um único veredito. A última linha é a causa.

3. Corrija com a cadeia de comparação.

```
python -c "
def alerta(nivel):
    if not -28 <= nivel <= -8:
        return 'fora de faixa'
    return 'ok'
for n in (-20.0, -3.0, -30.0, -8.0, -28.0):
    print(f'{n:>6} {alerta(n)}')
"
```

```
-20.0 ok
-3.0  fora de faixa
-30.0 fora de faixa
-8.0  ok
-28.0 ok
```

Repare nos dois extremos: -8.0 e -28.0 entram como ok porque <= inclui a borda. Faixa é sempre uma decisão sobre bordas, e é onde mora metade dos bugs de classificação.

4. Veja a tabela de valores falsos.

```
python -c "
for v in (0, 0.0, 0j, '', b'', [], (), {}, set(), None, range(0), '0', [0],
↪ -1):
    print(f'{v!r:>10} {bool(v)}')
```

```

0 False
0.0 False
0j False
'' False
b'' False
[] False
() False
{} False
set() False
None False
range(0, 0) False
'0' True
[0] True
-1 True
```

Guarde as três últimas linhas: "0" e [0] não são vazios, e -1 não é zero.

5. not x contra is None.

```
python -c "
def desconto_ruim(p=None):
    if not p:
        p = 10
    return p
def desconto(p=None):
    if p is None:
        p = 10
    return p
for arg in (0, 5, None):
    print(f'{str(arg):>4} ruim={desconto_ruim(arg):>3}
↪ certo={desconto(arg):>3}')
"
```

```

0 ruim= 10 certo= 0
5 ruim= 5 certo= 5
None ruim= 10 certo= 10
```

A primeira linha é o bug: desconto zero virou desconto de dez por cento.

6. Escreva o classificador. Crie classificar.py:

```
"""Classifica o nivel optico de uma ONU em faixas operacionais."""

CRITICO = -28.0
```

15. Condicionais: if, elif, else e a expressão condicional

```
ATENCAO = -25.0
SATURACAO = -8.0

def classificar(nivel):
    if nivel is None:
        return "sem leitura"
    if nivel < CRITICO:
        return "critico"
    if nivel < ATENCAO:
        return "atencao"
    if nivel <= SATURACAO:
        return "normal"
    return "saturado"

if __name__ == "__main__":
    for n in (None, -31.0, -26.4, -19.5, -3.0):
        print(f"{str(n):>7} {classificar(n)}")
```

```
python classificar.py
```

```
None    sem leitura
-31.0    critico
-26.4    atencao
-19.5    normal
-3.0     saturado
```

Cinco guardas, um nível de indentação, nenhum `else`. Cada `return` fecha um caso, e o último é o que sobra.

7. Ternário dentro de f-string.

```
python -c "
for falta in (0, 1, 3):
    print(f'{falta} porta{"s" if falta != 1 else ""} livre{"s" if falta
↵ != 1 else ""}')
"
```

```
0 portas livres
1 porta livre
3 portas livres
```

8. Morsa numa varredura de log.

```
cat > enlace.log <<'LOG'
2026-08-11 09:00:01 onu-01 nivel=-19.5 status=up
2026-08-11 09:00:02 onu-02 status=down
2026-08-11 09:00:03 onu-03 nivel=-31.0 status=up
```

```
2026-08-11 09:00:04 onu-04 nivel=-24.3 status=up
LOG

python -c "
import re
from pathlib import Path
padrao = re.compile(r'(onu-\d+) nivel=(-?\d+\.\d+)')
for linha in Path('enlace.log').read_text(encoding='utf-8').splitlines():
    if (m := padrao.search(linha)):
        print(f'{m.group(1)} {float(m.group(2))>7.1f} dBm')
    else:
        print(f'{linha.split()[2]} sem leitura')
"
```

```
onu-01    -19.5 dBm
onu-02    sem leitura
onu-03    -31.0 dBm
onu-04    -24.3 dBm
```

Uma busca por linha, e o resultado usado dentro do bloco. Sem `:=`, seriam duas buscas ou uma variável a mais.

9. Guardas contra pirâmide.

```
python -c "
def registrar(leitura):
    if leitura is None:
        return 'sem leitura'
    nivel = leitura.get('nivel')
    if nivel is None:
        return 'sem nivel'
    if not -28 <= nivel <= -8:
        return 'fora de faixa'
    return 'ok'
for c in (None, {}, {'nivel': -19.5}, {'nivel': -31.0}):
    print(f'{str(c):<18} {registrar(c)}')
"
```

```
None          sem leitura
{}            sem nivel
{'nivel': -19.5} ok
{'nivel': -31.0} fora de faixa
```

10. Tabela no lugar da escada.

```
python -c "
TARIFAS = {'basico': 79.90, 'turbo': 129.90, 'gamer': 199.90}
print(TARIFAS['turbo'])
print(TARIFAS.get('empresarial', 0.0))
"
```

15. Condicionais: if, elif, else e a expressão condicional

```
try:
    TARIFAS['empresarial']
except KeyError as e:
    print('KeyError:', e)
"
```

```
129.9
0.0
KeyError: 'empresarial'
```

[] para quando a chave ausente é erro; .get com padrão para quando não é. A escolha entre os dois é uma condicional que você não precisou escrever.

11. Monte o monitor. Crie leituras.csv:

```
cat > leituras.csv <<'CSV'
onu;nivel
onu-01;-19.5
onu-02;
onu-03;-31.0
onu-04;-26.4
onu-05;-3.0
onu-06;-8.0
CSV
```

E monitor.py:

```
"""Relatorio de niveis opticos: classifica cada ONU e resume por faixa."""

import csv
from collections import Counter
from pathlib import Path

from classificar import classificar

ACAO = {
    "critico": "abrir chamado de campo",
    "atencao": "agendar inspecao",
    "normal": "",
    "saturado": "instalar atenuador",
    "sem leitura": "verificar se a ONU responde",
}

def ler(caminho):
    texto = Path(caminho).read_text(encoding="utf-8")
    for linha in csv.DictReader(texto.splitlines(), delimiter=";"):
        bruto = linha["nivel"].strip()
        yield linha["onu"], (float(bruto) if bruto else None)
```

```

if __name__ == "__main__":
    contagem = Counter()
    for onu, nivel in ler("leituras.csv"):
        faixa = classificar(nivel)
        contagem[faixa] += 1
        medida = "   ---   " if nivel is None else f"{nivel:>7.1f}"
        print(f"{onu}   {medida}   {faixa:<12} {ACAO[faixa]}")

    print("-" * 52)
    problemas = sum(v for k, v in contagem.items() if k != "normal")
    print(f"{contagem.total()} ONUs, {problemas} exigem acao")
    for faixa, n in contagem.most_common():
        print(f"   {faixa:<12} {n}")

```

python monitor.py

```

onu-01   -19.5   normal
onu-02   ---     sem leitura   verificar se a ONU responde
onu-03   -31.0   critico      abrir chamado de campo
onu-04   -26.4   atencao      agendar inspecao
onu-05   -3.0    saturado     instalar atenuador
onu-06   -8.0    normal

-----
6 ONUs, 4 exigem acao
normal      2
sem leitura  1
critico     1
atencao     1
saturado    1

```

Todas as construções do capítulo estão nas trinta linhas: guardas em `classificar`, `is None` para distinguir ausência de zero, ternário em `float(bruto) if bruto else None` e em `medida`, e dois dicionários fazendo o papel de escadas de `elif` que ninguém precisou escrever.

12. Feche com `deactivate`.

15.10. Resumo

- `if/elif/else` é avaliado de cima para baixo e para no primeiro teste verdadeiro; as condições abaixo não são avaliadas, então ordem é semântica e condição com efeito colateral é armadilha.
- `if` aceita qualquer objeto e aplica o valor-verdade: zero de qualquer tipo, contêiner vazio e `None` são falsos. `"0"` e `[0]` são verdadeiros.
- Para ausência, `is None`; para vazio, o valor-verdade. Confundir os dois é o bug que transforma desconto zero em desconto padrão.
- `a <= x <= b` avalia `x` uma única vez e curto-circuita. `x < a or b` não é comparação com dois limites — é o bug da abertura.

15. Condicionais: *if*, *elif*, *else* e a expressão condicional

- Cláusulas de guarda com **return** antecipado substituem pirâmides de **if** aninhado: um nível de indentação e o caminho normal no fim, sem **else**.
- A expressão condicional **A if cond else B** produz valor e cabe onde um bloco não caberia (f-string, compreensão, argumento). O **else** é obrigatório e o aninhamento tem limite de um.
- **:=** (3.8) atribui dentro da condição, evitando a chamada dupla no par testar-e-usar.
- Escada de **elif** comparando a mesma variável com constantes é um dicionário disfarçado; faixas e formas de dados continuam sendo **if** ou **match**.

16. Laços: while, for e o protocolo de iteração

A rotina de limpeza do inventário tem uma linha de trabalho: tirar da lista as ONUs que estão fora do ar. O código tem quatro linhas e não parece ter onde errar.

```
onus = ["onu-01", "onu-02", "offline", "offline", "onu-05", "offline"]

for nome in onus:
    if nome == "offline":
        onus.remove(nome)

print(onus)
```

```
['onu-01', 'onu-02', 'onu-05', 'offline']
```

Sobrou uma. Nenhum erro, nenhum aviso, e a lista final está errada de um jeito que passa em qualquer revisão rápida — quem lê o código vê a intenção, não o resultado.

O motivo é que o **for** **não** percorre os elementos de uma lista: ele pede um item por vez a um iterador, e o iterador de lista trabalha com um índice interno que só sabe andar para frente. Cada **remove** encurta a lista sob os pés desse índice.

Dá para ver o índice andando se você desmontar o **for** na mão:

```
onus = ["onu-01", "onu-02", "offline", "offline", "onu-05", "offline"]
it = iter(onus)
i = 0
while True:
    try:
        nome = next(it)
    except StopIteration:
        break
    print(f"posicao {i} o for ve {nome!r:<10} tamanho={len(onus)}")
    if nome == "offline":
        onus.remove(nome)
    i += 1
```

```
posicao 0 o for ve 'onu-01' tamanho=6
posicao 1 o for ve 'onu-02' tamanho=6
posicao 2 o for ve 'offline' tamanho=6
posicao 3 o for ve 'onu-05' tamanho=5
posicao 4 o for ve 'offline' tamanho=5
```

16. Laços: *while*, *for* e o protocolo de iteração

Na posição 2 o laço encontra o primeiro "offline" e o remove. A lista encolhe, tudo desliza uma casa para a esquerda, o segundo "offline" passa a ocupar a posição 2 — e o índice interno já está indo para a 3. O elemento nunca é visto. O laço, que deveria dar seis voltas, deu cinco.

Este capítulo é sobre o mecanismo por trás disso: o que **while** e **for** realmente fazem, o que é um iterador, e por que quase todo laço escrito com índice em Python é um laço escrito errado.

16.1. **while**: repetir enquanto uma condição valer

O **while** é a repetição mais crua: testa uma condição e executa o bloco enquanto ela for verdadeira, com as mesmas regras de valor-verdade do Capítulo 15.

```
n = 0
saldo = 100.0
while saldo > 0:
    saldo -= 30.0
    n += 1
print(n, round(saldo, 2))
```

```
4 -20.0
```

Quatro voltas, e o saldo terminou negativo — a condição só é testada no **começo** de cada volta, nunca no meio. Se o bloco precisa parar no meio, isso é **break**, assunto do capítulo 17.

O uso natural do **while** é o caso em que você não sabe de antemão quantas iterações haverá: consumir uma fila, tentar de novo até dar certo, ler até o fim de um fluxo.

```
fila = ["job-1", "job-2", "job-3"]
while fila:
    tarefa = fila.pop(0)
    print("rodando", tarefa, "| restam", len(fila))
```

```
rodando job-1 | restam 2
rodando job-2 | restam 1
rodando job-3 | restam 0
```

Note o **while fila**: — a lista vazia é falsa, então a condição é o próprio esvaziamento. Escrever **while len(fila) > 0**: diria a mesma coisa com mais palavras.

O que **não** se faz em Python é usar **while** como contador:

```
# antipadrão: o for de C escrito em Python
i = 0
while i < len(linhas):
    processa(linhas[i])
    i += 1
```

Três oportunidades de errar (inicializar, comparar, incrementar) para fazer o que **for linha in linhas**: faz sem nenhuma. Todo laço que sabe quantas voltas vai dar é um **for**.

16.2. *for*: percorrer um iterável

O *for* do Python não é o *for* de C. Não existe contador, não existe condição de parada, não existe incremento. Existe um **iterável** e um alvo:

```
for nome, nivel in [("onu-01", -19.5), ("onu-03", -31.0)]:
    print(f"{nome}: {nivel:>7.1f}")
```

```
onu-01:   -19.5
onu-03:  -31.0
```

O alvo pode ser um desempacotamento, como acima. E o iterável pode ser qualquer coisa que saiba produzir itens: lista, tupla, string, dicionário, arquivo aberto, **range**, um objeto seu.

```
d = {"a": 1, "b": 2}
for chave in d:
    print(chave, end=" ")
print()
for chave, valor in d.items():
    print(chave, valor, end=" ")
print()
```

```
a b
a 1 b 2
```

Iterar um dicionário direto dá as **chaves**; `.items()` dá os pares e `.values()` dá os valores. Errar isso é o `TypeError: cannot unpack non-sequence` mais comum de quem vem de outra linguagem.

16.3. O protocolo de iteração

Por baixo, o *for* faz exatamente três coisas: chama `iter()` no objeto, chama `next()` repetidamente no resultado, e para quando aparece `StopIteration`.

```
it = iter(["a", "b", "c"])
print(it)
print(next(it))
print(next(it))
print(next(it))
try:
    next(it)
except StopIteration:
    print("StopIteration")
```

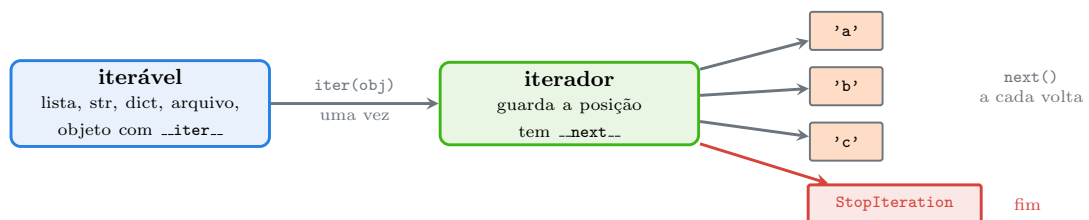
16. Laços: while, for e o protocolo de iteração

```
<list_iterator object at 0x7a01b33df0a0>
a
b
c
StopIteration
```

O endereço muda a cada execução; o resto não. E é assim que o `for` é implementado — o código abaixo é o `for` escrito à mão:

```
seq = ["a", "b"]
it = iter(seq)
while True:
    try:
        item = next(it)
    except StopIteration:
        break
    print("processando", item)
```

```
processando a
processando b
```



o que o `for` faz, em três passos: chama `iter()` no objeto **uma única vez**; chama `next()` no iterador a cada volta e liga o resultado ao alvo do laço; captura o `StopIteration` e encerra o bloco sem propagar a exceção. O iterador é **descartável** — depois do `StopIteration` ele não volta ao começo, e é por isso que um `iter()` consumido devolve lista vazia na segunda vez.

Figura 16.1.: `iter()` uma vez, `next()` a cada volta, `StopIteration` no fim.

A distinção entre **iterável** e **iterador** é a que resolve metade das confusões:

- Um **iterável** sabe produzir um iterador (`__iter__`). Lista, tupla, string, dicionário e `range` são iteráveis, e cada `for` sobre eles começa do zero.
- Um **iterador** sabe produzir o próximo item (`__next__`), guarda a posição e **se esgota**. Todo iterador também é iterável — `iter(it) is it` —, o que é o truque que permite passar um iterador direto para um `for`.

```
nums = [1, 2, 3]
it = iter(nums)
print(list(it))
```

```
print(list(it))
print(iter(it) is it)
```

```
[1, 2, 3]
[]
True
```

A segunda chamada devolve lista vazia. Isso morde de verdade quando o iterador vem de longe: `zip`, `enumerate`, `map`, `filter`, `reversed` e o objeto de arquivo aberto são **iteradores**, não sequências, e consumir duas vezes devolve nada na segunda. Quando você precisa passar mais de uma vez, materialize com `list()` antes.

Cada tipo tem o seu iterador:

```
print(type(iter("abc")), type(iter({"a": 1})), type(iter((1, 2))))
print(type(iter(range(3))))
```

```
<class 'str_ascii_iterator'> <class 'dict_keyiterator'> <class
↳ 'tuple_iterator'>
<class 'range_iterator'>
```

16.3.1. Um iterável seu

Implementar o protocolo é implementar dois métodos. O padrão correto separa o iterável (que pode ser percorrido muitas vezes) do iterador (que se esgota):

```
class Rampa:
    def __init__(self, inicio, fim):
        self.inicio = inicio
        self.fim = fim

    def __iter__(self):
        return RampaIterador(self.inicio, self.fim)

class RampaIterador:
    def __init__(self, atual, fim):
        self.atual = atual
        self.fim = fim

    def __iter__(self):
        return self

    def __next__(self):
        if self.atual > self.fim:
            raise StopIteration
        valor = self.atual
        self.atual += 1
        return valor
```

```
r = Rampa(1, 4)
print(list(r))
print(list(r))
print(sum(r), max(r))
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]
10 4
```

Duas listas iguais: cada `list(r)` pediu um iterador novo. E, de graça, o objeto passou a funcionar com `sum`, `max`, `sorted`, `in` e desempacotamento — tudo que consome iterável passa a aceitar `Rampa`. Se `__iter__` devolvesse `self`, a segunda linha sairia vazia e o `sum` veria uma rampa gasta.

Na prática você quase nunca escreve essas duas classes: um gerador com `yield` faz o mesmo em quatro linhas, e é assunto do Volume 8. Mas é este o protocolo que o gerador implementa por baixo.

16.4. `range` não é uma lista

```
r = range(0, 10, 3)
print(r, list(r), len(r), r[2], 9 in r, 4 in r)
```

```
range(0, 10, 3) [0, 3, 6, 9] 4 6 True False
```

`range` é uma **sequência preguiçosa**: guarda início, fim e passo, e calcula cada valor na hora. Tem comprimento, aceita índice e responde `in` — e o `in` é calculado por aritmética, não por varredura.

```
import sys

print(sys.getsizeof(range(1_000_000)), sys.getsizeof(list(range(1_000_000))))
```

```
48 8000056
```

Quarenta e oito bytes contra oito megabytes. `range` é o motivo de nunca precisar materializar uma sequência de índices.

O que não se faz com ele é isto:

```
# antipadrão: índice só para poder indexar
for i in range(len(linhas)):
    print(i, linhas[i])
```

```
# pythônico
for i, valor in enumerate(linhas, start=1):
    print(i, valor)
```

```
1 -19.5
2 -31.0
3 -8.0
```

`enumerate` produz pares (índice, valor) e aceita `start`, o que resolve de uma vez a numeração começando em 1 para relatório. `range(len(...))` só se justifica quando você realmente precisa do índice para outra coisa — escrever numa posição, por exemplo.

16.5. zip: percorrer em paralelo

```
nomes = ["onu-01", "onu-02", "onu-03"]
niveis = [-19.5, -31.0]
print(list(zip(nomes, niveis)))
```

```
[('onu-01', -19.5), ('onu-02', -31.0)]
```

`zip` para no **mais curto**, em silêncio. A terceira ONU sumiu do relatório e ninguém foi avisado. Desde o **Python 3.10** existe a proteção:

```
try:
    print(list(zip(nomes, niveis, strict=True)))
except ValueError as e:
    print("ValueError:", e)
```

```
ValueError: zip() argument 2 is shorter than argument 1
```

Sempre que as sequências **deveriam** ter o mesmo tamanho, `strict=True` transforma um bug silencioso em uma exceção. Quando o truncamento é intencional, deixe sem — e escreva no comentário que é intencional.

16.6. Mudar o que se está percorrendo

De volta ao problema da abertura. A regra geral é: **não modifique um contêiner enquanto itera sobre ele**. Em lista, o resultado é silencioso e errado; em dicionário e conjunto, o Python detecta e falha alto:

16. Laços: while, for e o protocolo de iteração

```
estoque = {"onu-01": 3, "onu-02": 0, "onu-03": 5}
try:
    for chave, qtd in estoque.items():
        if qtd == 0:
            del estoque[chave]
except RuntimeError as e:
    print("RuntimeError:", e)
```

RuntimeError: dictionary changed size during iteration

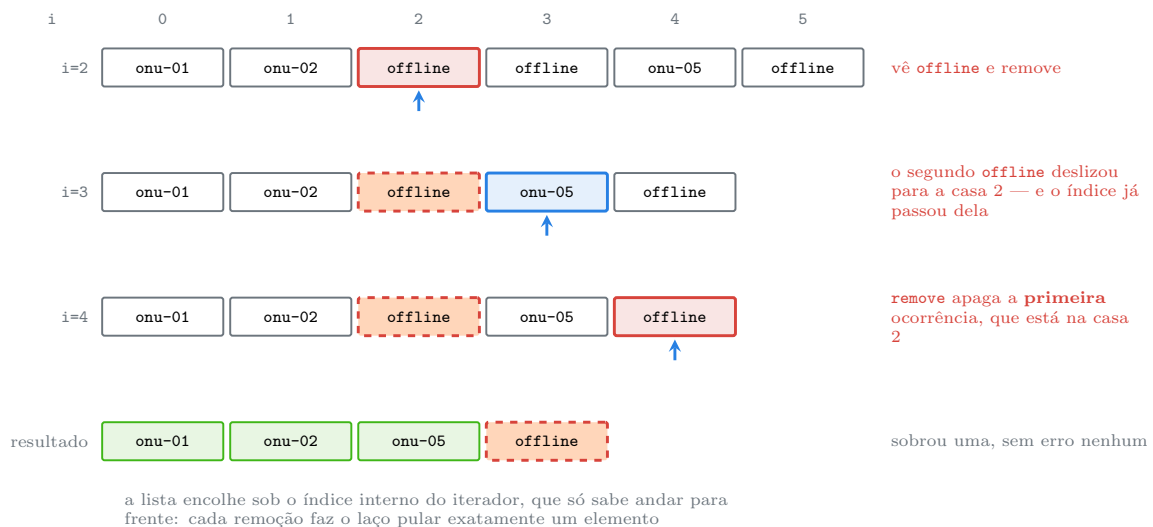


Figura 16.2.: Remover durante o laço desloca a lista e faz o índice interno pular elementos.

Há três saídas corretas, em ordem de preferência:

```
onus = ["onu-01", "onu-02", "offline", "offline", "onu-05", "offline"]

# 1. construir uma lista nova (o normal)
ativas = [n for n in onus if n != "offline"]

# 2. substituir o conteúdo no lugar, quando outras referências apontam para a
↪ lista
onus[:] = [n for n in onus if n != "offline"]

print(ativas)
print(onus)
```

```
['onu-01', 'onu-02', 'onu-05']
['onu-01', 'onu-02', 'onu-05']
```

A primeira é a resposta padrão: filtrar é produzir uma coleção nova, não mutilar a antiga. A segunda usa a atribuição a fatia do Capítulo 12 para trocar o conteúdo mantendo o mesmo objeto — importa quando outra parte do programa guarda uma referência a essa lista, pela mecânica do Capítulo 8. A terceira é iterar sobre uma **cópia**, que é o que salva o dicionário:


```
estoque = {"onu-01": 3, "onu-02": 0, "onu-03": 5}
for chave in list(estoque):
    if estoque[chave] == 0:
        del estoque[chave]
print(estoque)
```

```
{'onu-01': 3, 'onu-03': 5}
```

`list(estoque)` congela as chaves antes do laço começar; o que muda é o dicionário, não a lista que está sendo percorrida.

A sintaxe `[n for n in onus if ...]` é uma compreensão de lista, assunto do Volume 4. Ela aparece aqui porque é a resposta idiomática ao problema deste capítulo, e vale ler por enquanto como “um `for` que produz uma lista”.

16.7. Laços aninhados

```
matriz = [[1, 2, 3], [4, 5, 6]]
for linha in matriz:
    for valor in linha:
        print(valor, end=" ")
    print()
```

```
1 2 3
4 5 6
```

Funciona e é legível em dois níveis. A partir do terceiro, e sempre que os dois laços percorrem coleções independentes, `itertools.product` diz o mesmo em uma linha — e o custo continua sendo o produto dos tamanhos. Dois laços de mil itens são um milhão de voltas, o que é a origem da maioria dos “ficou lento quando os dados cresceram”.

16.8. Jeito Pythônico

Itere sobre o objeto, não sobre índices. `for item in colecao:` é o padrão; `for i, item in enumerate(colecao):` quando o índice também é necessário; `range(len(...))` praticamente nunca.

```
# antipadrão
for i in range(len(nomes)):
    print(nomes[i], niveis[i])
```

```
# pythônico
for nome, nivel in zip(nomes, niveis, strict=True):
    print(nome, nivel)
```

while para o que não tem contagem conhecida; **for** para todo o resto. **while** com contador manual é **for** escrito com três chances a mais de errar.

Nunca modifique a coleção que está sendo percorrida. Construa uma nova, ou itere sobre uma cópia (`list(d)`), ou atribua à fatia (`lst[:] = ...`).

zip(..., strict=True) sempre que os tamanhos deveriam bater (3.10+). O truncamento silencioso é a categoria de bug mais barata de evitar e mais cara de achar.

Cuidado com iteradores consumidos. `zip`, `enumerate`, `map`, `filter`, `reversed` e arquivos abertos são de uma passada só. Se vai usar duas vezes, `list()` antes.

for _ in range(n) quando o valor não importa. O sublinhado é a convenção para “variável que existe só porque a sintaxe exige”.

Não conte na mão o que a biblioteca já conta. `sum`, `min`, `max`, `any`, `all`, `sorted` e `collections.Counter` substituem laços de acumulação inteiros, e cada um deles roda em C.

```
# antipadrão
total = 0
for n in niveis:
    total += n
media = total / len(niveis)
```

```
# pythônico
media = sum(niveis) / len(niveis)
```

Iterável reutilizável, iterador descartável. Ao escrever uma classe iterável, `__iter__` devolve um objeto **novo**; só um iterador devolve `self`.

Sobre versões:

- **Python 3.0** — `range`, `zip` e `map` passaram a ser preguiçosos; no Python 2 devolviam listas.
- **Python 3.6** — o dicionário passou a preservar a ordem de inserção (detalhe de implementação no 3.6, garantia da linguagem no 3.7), então a ordem de iteração de um `dict` é previsível.
- **Python 3.10** — `zip(..., strict=True)` (PEP 618).
- **Python 3.12** — laços mais rápidos por conta da especialização adaptativa do interpretador (PEP 659), sem mudança de sintaxe.

16.9. Laboratório

Objetivo: reproduzir a remoção que pula elementos, desmontar o `for` no protocolo que ele usa, medir a diferença entre `range` e lista, e terminar escrevendo um iterável próprio que calcula média móvel.

1. Ambiente.

```
mkdir -p ~/lab16 && cd ~/lab16
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. Reproduza o bug.

```
python -c "
onus = ['onu-01', 'onu-02', 'offline', 'offline', 'onu-05', 'offline']
for nome in onus:
    if nome == 'offline':
        onus.remove(nome)
print(onus)
"
```

```
['onu-01', 'onu-02', 'onu-05', 'offline']
```

3. Instrumente o laço para ver o índice andar.

```
python -c "
onus = ['onu-01', 'onu-02', 'offline', 'offline', 'onu-05', 'offline']
it = iter(onus)
i = 0
while True:
    try:
        nome = next(it)
    except StopIteration:
        break
    print(f'posicao {i} o for ve {nome!r:<10} tamanho={len(onus)}')
    if nome == 'offline':
        onus.remove(nome)
    i += 1
print('sobrou:', onus)
"
```

```
posicao 0 o for ve 'onu-01' tamanho=6
posicao 1 o for ve 'onu-02' tamanho=6
posicao 2 o for ve 'offline' tamanho=6
posicao 3 o for ve 'onu-05' tamanho=5
posicao 4 o for ve 'offline' tamanho=5
sobrou: ['onu-01', 'onu-02', 'onu-05', 'offline']
```

Cinco voltas numa lista de seis, e a posição 3 já mostra `onu-05` — o segundo `offline` foi pulado. Compare com a Figura 16.2.

4. As duas correções.

```
python -c "
onus = ['onu-01', 'onu-02', 'offline', 'offline', 'onu-05', 'offline']
print([n for n in onus if n != 'offline'])
onus[:] = [n for n in onus if n != 'offline']
print(onus)
"
```

```
['onu-01', 'onu-02', 'onu-05']  
['onu-01', 'onu-02', 'onu-05']
```

5. O dicionário falha alto.

```
python -c "  
estoque = {'onu-01': 3, 'onu-02': 0, 'onu-03': 5}  
try:  
    for chave, qtd in estoque.items():  
        if qtd == 0:  
            del estoque[chave]  
except RuntimeError as e:  
    print('RuntimeError:', e)  
estoque = {'onu-01': 3, 'onu-02': 0, 'onu-03': 5}  
for chave in list(estoque):  
    if estoque[chave] == 0:  
        del estoque[chave]  
print(estoque)  
"
```

```
RuntimeError: dictionary changed size during iteration  
{'onu-01': 3, 'onu-03': 5}
```

Repare que a exceção veio **depois** de a remoção já ter acontecido. Falhar alto não desfaz nada: protege o resto, não o que já foi feito.

6. O protocolo na mão.

```
python -c "  
it = iter(['a', 'b'])  
print(type(it))  
print(next(it))  
print(next(it))  
try:  
    next(it)  
except StopIteration:  
    print('StopIteration')  
"
```

```
<class 'list_iterator'>  
a  
b  
StopIteration
```

7. Iterador se esgota; iterável não.

```
python -c "
nums = [1, 2, 3]
it = iter(nums)
print(list(it))
print(list(it))
print('iter(it) is it:', iter(it) is it)
print('iter(nums) is nums:', iter(nums) is nums)
"
```

```
[1, 2, 3]
[]
iter(it) is it: True
iter(nums) is nums: False
```

As duas últimas linhas são a definição operacional da diferença: o iterador devolve a si mesmo, o iterável devolve um objeto novo.

8. range contra lista.

```
python -c "
import sys
print(sys.getsizeof(range(1_000_000)), sys.getsizeof(list(range(1_000_000))))
r = range(0, 10, 3)
print(list(r), len(r), r[2], 9 in r, 4 in r)
"
```

```
48 8000056
[0, 3, 6, 9] 4 6 True False
```

9. enumerate e zip.

```
python -c "
nomes = ['onu-01', 'onu-02', 'onu-03']
niveis = [-19.5, -31.0]
print(list(zip(nomes, niveis)))
try:
    list(zip(nomes, niveis, strict=True))
except ValueError as e:
    print('ValueError:', e)
for i, (nome, nivel) in enumerate(zip(nomes, niveis), start=1):
    print(i, nome, nivel)
"
```

```
[('onu-01', -19.5), ('onu-02', -31.0)]
ValueError: zip() argument 2 is shorter than argument 1
1 onu-01 -19.5
2 onu-02 -31.0
```

16. Laços: while, for e o protocolo de iteração

O desempacotamento aninhado `i, (nome, nivel)` combina os dois sem índice manual em lugar nenhum.

10. while consumindo fila.

```
python -c "  
fila = ['job-1', 'job-2', 'job-3']  
while fila:  
    print('rodando', fila.pop(0), '| restam', len(fila))  
"
```

```
rodando job-1 | restam 2  
rodando job-2 | restam 1  
rodando job-3 | restam 0
```

Aqui a mutação é o **objetivo** do laço, e por isso é `while`, não `for`.

11. Escreva um iterável próprio. Crie janela.py:

```
"""Media movel sobre uma sequencia de leituras, como iteravel proprio."""  
  
class JanelaMovel:  
    """Iteravel (reutilizavel) que produz a media de cada janela de tamanho  
    ↪ n."""  
  
    def __init__(self, dados, tamanho):  
        if tamanho < 1:  
            raise ValueError("tamanho da janela precisa ser >= 1")  
        self.dados = list(dados)  
        self.tamanho = tamanho  
  
    def __iter__(self):  
        return _JanelaIterador(self.dados, self.tamanho)  
  
class _JanelaIterador:  
    def __init__(self, dados, tamanho):  
        self.dados = dados  
        self.tamanho = tamanho  
        self.pos = 0  
  
    def __iter__(self):  
        return self  
  
    def __next__(self):  
        fim = self.pos + self.tamanho  
        if fim > len(self.dados):  
            raise StopIteration  
        fatia = self.dados[self.pos:fim]  
        self.pos += 1  
        return sum(fatia) / self.tamanho
```

```

if __name__ == "__main__":
    leituras = [-19.5, -19.8, -21.2, -25.9, -27.1, -31.0]
    janela = JanelaMovel(leituras, 3)

    for i, media in enumerate(janela, start=1):
        print(f"janela {i}: {media:>7.2f} dBm")

    print("de novo (o iteravel e reutilizavel):", [round(m, 2) for m in
↵ janela])
    print("pior media:", round(min(janela), 2))

    it = iter(janela)
    print("primeiro next():", round(next(it), 2))
    print("resto:", [round(m, 2) for m in it])
    print("esgotado:", list(it))

```

```
python janela.py
```

```

janela 1: -20.17 dBm
janela 2: -22.30 dBm
janela 3: -24.73 dBm
janela 4: -28.00 dBm
de novo (o iteravel e reutilizavel): [-20.17, -22.3, -24.73, -28.0]
pior media: -28.0
primeiro next(): -20.17
resto: [-22.3, -24.73, -28.0]
esgotado: []

```

As três últimas linhas são o protocolo inteiro: o `for` já tinha consumido um iterador e o `min` outro, mas o **iterável** continuou inteiro; o iterador que você pegou na mão, não. E `min(janela)` funcionou sem nenhuma linha a mais, porque implementar `__iter__` já entrega a função inteira da biblioteca padrão que consome iteráveis.

12. Use o iterável em algo real. Crie `tendencia.py`:

```

"""Detecta ONUs em degradacao comparando cada leitura com a anterior."""

from janela import JanelaMovel

HISTORICO = {
    "onu-01": [-19.5, -19.6, -19.4, -19.5],
    "onu-03": [-24.0, -26.1, -28.7, -31.0],
    "onu-04": [-27.0, -26.2, -25.1, -24.4],
}

for onu, leituras in HISTORICO.items():
    quedas = 0
    for anterior, atual in zip(leituras, leituras[1:]):

```

```
    if atual < anterior:
        quedas += 1
media = sum(JanelaMovel(leituras, 2)) / (len(leituras) - 1)
veredito = "degradando" if quedas >= 3 else "estavel"
print(f"{onu} quedas={quedas}/{len(leituras) - 1} media={media:>7.2f}
↪ {veredito}")
```

python tendencia.py

```
onu-01 quedas=2/3 media= -19.50 estavel
onu-03 quedas=3/3 media= -27.43 degradando
onu-04 quedas=0/3 media= -25.67 estavel
```

O `zip(leituras, leituras[1:])` é o idioma para percorrer pares consecutivos: a mesma sequência contra ela mesma deslocada de um. Nenhum índice, nenhum `range(len(...))`, e o `zip` para sozinho no penúltimo elemento.

13. Feche com `deactivate`.

16.10. Resumo

- `while` repete enquanto a condição for verdadeira e testa só no início da volta; é a escolha certa quando o número de iterações não é conhecido. `while` com contador manual é um `for` mal escrito.
- `for` percorre um **iterável**: chama `iter()` uma vez, `next()` a cada volta, e encerra ao capturar `StopIteration`.
- Iterável produz um iterador novo a cada `iter()`; iterador guarda a posição, devolve `self` em `__iter__` e se esgota. `zip`, `enumerate`, `map`, `filter` e arquivos abertos são iteradores de uma passada só.
- Implementar `__iter__` e `__next__` faz o objeto funcionar com `for`, `sum`, `min`, `sorted`, `in` e desempacotamento de uma vez.
- `range` é sequência preguiçosa: 48 bytes para um milhão de valores, com índice, `len` e `in` por aritmética.
- `enumerate` no lugar de `range(len(...))`; `zip` para percorrer em paralelo, com `strict=True` (3.10+) sempre que os tamanhos deveriam bater.
- Modificar a coleção durante a iteração pula elementos em lista (silenciosamente) e levanta `RuntimeError` em dicionário e conjunto. Construa uma coleção nova, itere sobre `list(d)` ou atribua a `lst[:]`.
- `zip(seq, seq[1:])` percorre pares consecutivos; `sum`, `max`, `any` e `all` substituem laços de acumulação inteiros.

17. break, continue, else em laços e o casamento de padrões

O inventário da rede é um dicionário de OLTs, cada uma com as suas ONUs. A função procura a **primeira** ONU com nível crítico:

```
REDE = {
    "olt-norte": {"onu-01": -19.5, "onu-02": -31.2, "onu-03": -20.1},
    "olt-sul": {"onu-04": -18.0, "onu-05": -29.8},
    "olt-leste": {"onu-06": -21.4},
}

def primeira_critica(rede):
    achada = None
    for olt, onus in rede.items():
        for onu, nivel in onus.items():
            if nivel < -28:
                achada = (olt, onu, nivel)
                break
    return achada

print(primeira_critica(REDE))
```

```
('olt-sul', 'onu-05', -29.8)
```

A primeira crítica é a **onu-02**, com $-31,2$ dBm, na **olt-norte**. A função devolveu a **onu-05**, com $-29,8$ — que é a **última**, e a menos grave das duas. O chamado foi aberto para o cliente errado.

O **break** fez exatamente o que ele faz: saiu do laço **mais interno**. O laço externo continuou percorrendo as outras OLTs, encontrou outra crítica e sobrescreveu **achada**. Em Python não existe **break** com rótulo, então o **break** de dentro nunca sai de dois níveis.

A correção mais limpa não é um **break** melhor — é não precisar de **break**:

```
def primeira_critica(rede):
    for olt, onus in rede.items():
        for onu, nivel in onus.items():
            if nivel < -28:
                return olt, onu, nivel
    return None
```

```
print(primeira_critica(REDE))
```

```
('olt-norte', 'onu-02', -31.2)
```

`return` sai de **todos** os laços de uma vez, porque sai da função inteira. É por isso que “extraia a busca para uma função” é a resposta padrão para `break` aninhado.

Este capítulo trata das três instruções que alteram o curso de um laço — `break`, `continue` e a cláusula `else` que quase ninguém conhece — e do `match`, que desde o Python 3.10 substituiu escadas inteiras de condicionais quando a decisão é sobre a **forma** do dado.

17.1. `break`: abandonar o laço

`break` encerra o laço imediatamente, sem terminar a volta atual e sem avaliar a condição de novo. Vale para `for` e para `while`, e afeta **um** nível.

Quando o `break` aninhado não pode virar função, a saída é uma bandeira explícita:

```
achada = None
for olt, onus in REDE.items():
    for onu, nivel in onus.items():
        if nivel < -28:
            achada = (olt, onu, nivel)
            break
    if achada is not None:
        break
print(achada)
```

```
('olt-norte', 'onu-02', -31.2)
```

Funciona, e é feio: dois `break` e um teste extra para fazer o trabalho de um `return`. Prefira a função.

17.2. `continue`: pular para a próxima volta

`continue` abandona o resto do bloco e vai direto para a próxima iteração — no `while`, isso significa reavaliar a condição; no `for`, pedir o próximo item.

O uso que se paga é o de filtro na entrada, que é a cláusula de guarda do Capítulo 15 aplicada a laço:

```
soma, lidos, descartados = 0.0, 0, 0
for linha in linhas:
    if not linha.strip():
        descartados += 1
        continue
    if linha.startswith("#"):
```

```

        descartados += 1
        continue
    try:
        valor = float(linha)
    except ValueError:
        descartados += 1
        continue
    soma += valor
    lidos += 1

```

Cada caso ruim é tratado e descartado na hora, e o corpo útil do laço fica no fim, sem indentação nenhuma. A alternativa com `if` aninhado empurraria as duas últimas linhas para quatro níveis de profundidade.

Um cuidado no `while`: um `continue` que pula o incremento é laço infinito.

```

# antipadrão: se a linha for vazia, i nunca cresce
i = 0
while i < len(linhas):
    if not linhas[i]:
        continue
    processa(linhas[i])
    i += 1

```

É mais um argumento para não usar `while` com contador.

17.3. *else* em laço: “terminou sem *break*”

Esta é a construção mais mal-batizada do Python. O `else` de um laço executa quando o laço **termina naturalmente** — sem `break`. Se tivesse sido chamado `nobreak`, ninguém teria dificuldade.

```

for i in range(3):
    print(i, end=" ")
else:
    print("| else rodou")

for i in range(3):
    if i == 1:
        break
    print(i, end=" ")
else:
    print("| else rodou")
print("| else nao rodou")

```

```

0 1 2 | else rodou
0 | else nao rodou

```

17. *break, continue, else em laços e o casamento de padrões*

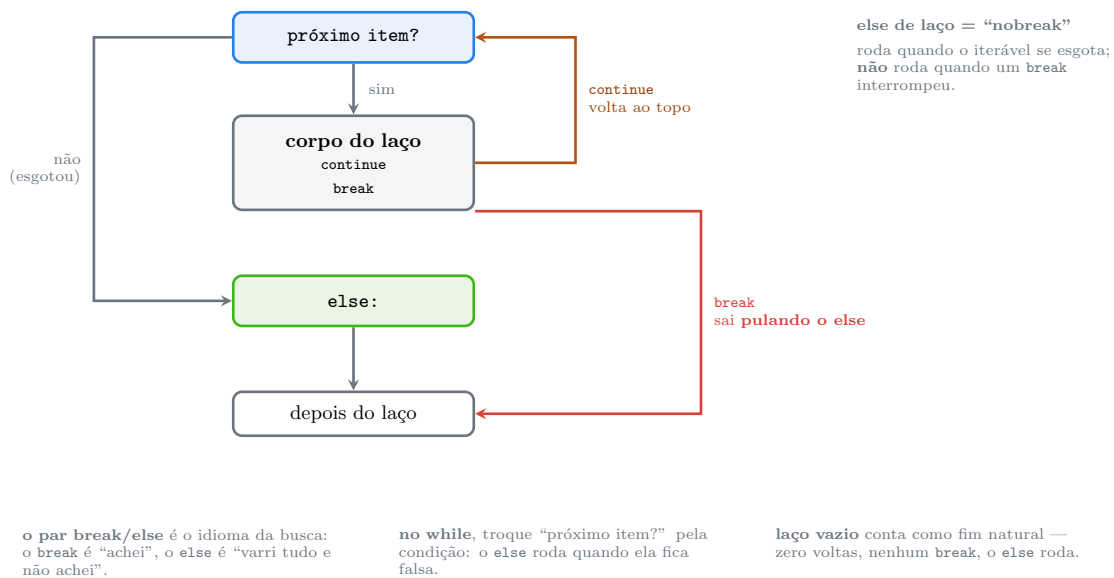


Figura 17.1.: `continue` volta ao topo, `break` sai pulando o `else`, e o `else` só roda no fim natural.

O par `break/else` existe para um caso específico: a busca que precisa distinguir “achei” de “varri tudo e não achei”, sem bandeira nenhuma.

```
alvo = "onu-99"
for onu in REDE["olt-norte"]:
    if onu == alvo:
        print("encontrada")
        break
else:
    print(f"{alvo} nao esta na olt-norte")
```

```
onu-99 nao esta na olt-norte
```

O exemplo canônico é o teste de primalidade, onde o `else` significa “nenhum divisor foi encontrado”:

```
for n in (91, 97, 2):
    for divisor in range(2, int(n**0.5) + 1):
        if n % divisor == 0:
            print(f"{n} = {divisor} x {n // divisor}")
            break
    else:
        print(f"{n} e primo")
```

```
91 = 7 x 13
97 e primo
2 e primo
```

O caso do 2 mostra o comportamento com laço que não dá volta nenhuma: `range(2, 2)` é vazio, o `for` termina sem `break`, e o `else` roda. **Laço vazio conta como “terminou sem `break`”** — é o detalhe que mais surpreende.

O `while` tem a mesma cláusula, com o mesmo significado:

```
tentativas = 0
while tentativas < 3:
    tentativas += 1
    if tentativas == 5:
        print("conectou")
        break
else:
    print("desistiu depois de", tentativas, "tentativas")
```

```
desistiu depois de 3 tentativas
```

Este é o idioma da retentativa: o `break` é o sucesso, o `else` é o esgotamento das tentativas. Sem ele, seria preciso uma variável `sucesso = False` e um `if` depois do laço.

Dito isso: o `else` de laço divide opiniões, e com razão — a palavra sugere a coisa errada. A recomendação honesta é usá-lo quando ele elimina uma bandeira, sempre com um comentário ou um nome de função que deixe claro o que “sem `break`” significa ali; e não usá-lo em código que passa por muita gente diferente.

17.4. *match*: casar com a forma do dado

Desde o **Python 3.10** (PEP 634 a 636), o `match` compara um valor contra **padrões estruturais**. Ele não é o `switch` de C: não é uma cadeia de igualdades, é desestruturação com teste embutido.

O caso que justifica a instrução é o de dados que chegam de fora — JSON de uma API, evento de fila, linha de comando:

```
def descrever(evento):
    match evento:
        case {"tipo": "alarme", "onu": onu, "nivel": nivel} if nivel < -28:
            return f"CRITICO {onu} em {nivel} dBm"
        case {"tipo": "alarme", "onu": onu}:
            return f"alarme comum em {onu}"
        case {"tipo": "reboot", "onu": onu, **resto}:
            return f"reboot de {onu} (extras: {resto})"
        case {"tipo": tipo}:
            return f"evento desconhecido: {tipo}"
        case _:
            return "sem tipo"

eventos = [
    {"tipo": "alarme", "onu": "onu-02", "nivel": -31.2},
    {"tipo": "alarme", "onu": "onu-01", "nivel": -19.5},
```

17. *break, continue, else* em laços e o casamento de padrões

```
    {"tipo": "reboot", "onu": "onu-05", "causa": "queda de energia"},
    {"tipo": "sync"},
    {"origem": "olt"},
]
for e in eventos:
    print(descrever(e))
```

```
CRITICO onu-02 em -31.2 dBm
alarme comum em onu-01
reboot de onu-05 (extras: {'causa': 'queda de energia'})
evento desconhecido: sync
sem tipo
```

Em cinco `case` estão: teste de valor literal ("`alarme`"), extração de campos para variáveis (`onu`, `nivel`), guarda (`if nivel < -28`), captura do resto do dicionário (`**resto`) e curinga (`_`). Escrever isso com `if` e `.get()` daria três vezes mais linhas, e cada acesso teria que ser defendido contra chave ausente.

Dois pontos importantes sobre padrão de mapeamento: ele casa por **subconjunto** — `{"tipo": "alarme"}` casa com um dicionário que tenha outras chaves além dessa —, e a ordem dos `case` importa como na cadeia de `elif`, então o padrão mais específico vem primeiro. Foi por isso que a guarda `if nivel < -28` ficou no topo.

17.4.1. Padrões de sequência

```
def roteia(comando):
    match comando.split():
        case ["listar"]:
            return "lista tudo"
        case ["listar", olt]:
            return f"lista a {olt}"
        case ["reiniciar", *onus] if onus:
            return f"reinicia {len(onus)}: " + ", ".join(onus)
        case ["sair" | "quit" | "exit"]:
            return "tchau"
        case []:
            return "comando vazio"
        case _:
            return "nao entendi"

for c in ["listar", "listar olt-sul", "reiniciar onu-01 onu-02", "quit", "",
↪ "voar"]:
    print(f"{c!r:<28} -> {roteia(c)}")
```

```
'listar'                -> lista tudo
'listar olt-sul'         -> lista a olt-sul
'reiniciar onu-01 onu-02' -> reinicia 2: onu-01, onu-02
```

```
'quit'          -> tchau
''              -> comando vazio
'voar'         -> nao entendi
```

O padrão de sequência casa por **tamanho e posição**: `["listar"]` só casa com lista de um elemento. O `*onus` absorve o resto, `|` é alternativa, e a guarda `if onus` rejeita reiniciar sem argumento.

Uma armadilha específica: `str` e `bytes` **não** casam como sequência, mesmo sendo sequências.

```
match "abc":
    case [a, b, c]:
        print("string casou como sequencia?", a, b, c)
    case str() as s:
        print("string nao e sequencia para o match:", s)
```

```
string nao e sequencia para o match: abc
```

A exceção é deliberada: sem ela, `case [x]` casaria com qualquer string de um caractere, e ninguém quer isso.

17.4.2. Padrões de classe

```
from dataclasses import dataclass

@dataclass
class Ponto:
    x: float
    y: float

def onde(p):
    match p:
        case Ponto(x=0, y=0):
            return "origem"
        case Ponto(x=0, y=y):
            return f"eixo vertical em y={y}"
        case Ponto(x=x, y=0):
            return f"eixo horizontal em x={x}"
        case Ponto():
            return "fora dos eixos"
        case _:
            return "nao e um ponto"

for p in [Ponto(0, 0), Ponto(0, 3), Ponto(2, 0), Ponto(1, 1), "abc"]:
    print(onde(p))
```

17. *break, continue, else em laços e o casamento de padrões*

```
origem
eixo vertical em y=3
eixo horizontal em x=2
fora dos eixos
nao e um ponto
```

Ponto(x=0, y=0) **não** é uma chamada: é um padrão que testa o tipo e os atributos. Ponto() testa só o tipo, e é o `isinstance` escrito como padrão. Também funciona com os tipos embutidos, o que dá validação de tipo de graça:

```
match (200, "ok"):
    case (int(codigo), str(texto)):
        print("codigo", codigo, "texto", texto)
```

```
codigo 200 texto ok
```

padrão	casa com
<code>case -28.0:</code>	literal — só o valor igual (==, mas is para None, True e False)
<code>case Limiar.CRITICO:</code>	valor com nome pontilhado — compara com a constante; a forma correta de usar constantes
<code>case CRITICO:</code>	captura! nome simples nunca compara: casa com tudo e rebatiza o valor
<code>case [_, segundo, *resto]:</code>	sequência — lista/tupla com 2 ou mais itens (str e bytes não entram)
<code>case {"tipo": t, **extra}:</code>	mapeamento — casa por subconjunto : outras chaves podem existir
<code>case Ponto(x=0, y=y):</code>	classe — testa o tipo e os atributos; Ponto() sozinho é um <code>isinstance</code>
<code>case "sair" "quit":</code>	alternativa — qualquer um dos lados
<code>case [x] if x > 0:</code>	guarda — o padrão casa e a condição é verdadeira
<code>case ..:</code>	curinga — casa com tudo sem ligar nome nenhum

Figura 17.2.: Os padrões do `match`, e o único que quase todo mundo escreve errado na primeira vez.

17.4.3. O erro que todo mundo comete uma vez

Um nome simples num padrão **nunca** é uma comparação: é uma captura.

```
CRITICO = -28.0

def faixa(nivel):
    match nivel:
        case CRITICO:
            return f"casou com o limite? CRITICO agora vale {CRITICO}"
```



```
print(faixa(-19.5))
print("CRITICO no modulo:", CRITICO)
```

```
casou com o limite? CRITICO agora vale -19.5
CRITICO no modulo: -28.0
```

O `case CRITICO`: casou com `-19,5` — casaria com qualquer coisa — e ligou o nome `CRITICO` ao valor recebido, como variável local. A constante do módulo continua intacta, o que torna o bug ainda mais difícil de ver: o teste “a constante mudou?” não acusa nada.

Quando há um `case` depois, o Python detecta e recusa o programa:

```
CRITICO = -28.0

def faixa(nivel):
    match nivel:
        case CRITICO:
            return "no limite"
        case _:
            return "outro"
```

```
File "/home/vitor/lab17/captura2.py", line 6
    case CRITICO:
        ~~~~~
SyntaxError: name capture 'CRITICO' makes remaining patterns unreachable
```

A mensagem é excelente e diz o nome do problema: *name capture*. A correção é usar um **nome pontilhado**, que o `match` trata como valor:

```
class Limiar:
    CRITICO = -28.0
    SATURACAO = -8.0

def faixa(nivel):
    match nivel:
        case Limiar.CRITICO:
            return "exatamente no limite critico"
        case Limiar.SATURACAO:
            return "exatamente no limite de saturacao"
        case _:
            return "entre os limites"

for n in (-28.0, -8.0, -19.5):
    print(f"{n:>7} {faixa(n)}")
```

```
-28.0  exatamente no limite critico
-8.0   exatamente no limite de saturacao
-19.5  entre os limites
```

Uma classe simples, um `Enum` ou um módulo importado — qualquer coisa com ponto no meio serve. É a razão de o `match` combinar tão bem com `enum.Enum`.

17.5. Jeito Pythônico

break aninhado quer virar função. Extraia a busca e use `return`; ele sai de todos os níveis e ainda dá um nome ao que o laço faz.

continue como guarda no topo do laço. Trate e descarte o caso ruim na primeira linha, deixe o trabalho útil sem indentação.

else de laço só para eliminar bandeira. É o par de `break` numa busca ou numa retentativa. Se você precisou explicar o `else` em comentário longo, uma função com `return` é mais clara.

Não use `match` como `switch`. Para comparar uma variável contra constantes, o dicionário do Capítulo 15 continua melhor. `match` se paga quando há **estrutura**: sequência de tamanho variável, dicionário aninhado, tipos diferentes no mesmo canal.

```
# antipadrão: match usado como escada de igualdades
match plano:
    case "basico":
        preco = 79.90
    case "turbo":
        preco = 129.90
```

```
# pythônico
TARIFAS = {"basico": 79.90, "turbo": 129.90}
preco = TARIFAS[plano]
```

Constante em case sempre com ponto. `case Limiar.CRITICO:`, `case Status.ATIVO:`. Nome simples é captura, e captura silenciosa.

Termine o match com `case _:` quando a entrada vem de fora. Sem curinga, um valor não previsto simplesmente não faz nada — e “não faz nada” é o pior desfecho possível para dado malformado.

Nada de `for ... else` combinado com `continue` para simular fluxo. O bloco `for/else/continue/break` da busca aninhada existe, funciona e é ilegível; a função com `return` faz o mesmo em metade das linhas.

Sobre versões:

- **Python 3.10** — `match/case` (PEP 634, 635 e 636). É a mudança de sintaxe mais visível da série 3.x recente.
- **Python 3.11** — mensagens de erro do `match` mais precisas, incluindo o `name capture` acima.
- **Python 3.12** — o compilador passou a otimizar melhor o despacho de `match` sobre literais.
- `break`, `continue` e o `else` de laço existem desde sempre e não mudaram.

17.6. Laboratório

Objetivo: reproduzir o `break` que sai só de um nível, usar o par `break/else` numa busca, ver o `case CONSTANTE` capturar em silêncio, e terminar com um console de operação que despacha comandos com `match`.

1. Ambiente e dados.

```
mkdir -p ~/lab17 && cd ~/lab17
python3 -m venv .venv && source .venv/bin/activate
python --version

cat > rede.py <<'PY'
REDE = {
    "olt-norte": {"onu-01": -19.5, "onu-02": -31.2, "onu-03": -20.1},
    "olt-sul": {"onu-04": -18.0, "onu-05": -29.8},
    "olt-leste": {"onu-06": -21.4},
}
PY
```

```
Python 3.13.12
```

2. Reproduza o `break` de um nível só.

```
python -c "
from rede import REDE
achada = None
for olt, onus in REDE.items():
    for onu, nivel in onus.items():
        if nivel < -28:
            achada = (olt, onu, nivel)
            break
print('achada:', achada)
"
```

```
achada: ('olt-sul', 'onu-05', -29.8)
```

A `onu-02`, com `-31,2`, foi encontrada primeiro e depois sobrescrita.

3. Corrija com `return`.

```
python -c "
from rede import REDE
def primeira_critica(rede):
    for olt, onus in rede.items():
        for onu, nivel in onus.items():
            if nivel < -28:
                return olt, onu, nivel
    return None
print(primeira_critica(REDE))
"
```

17. break, continue, else em laços e o casamento de padrões

```
('olt-norte', 'onu-02', -31.2)
```

4. for/else numa busca.

```
python -c "  
from rede import REDE  
alvo = 'onu-99'  
for onu in REDE['olt-norte']:  
    if onu == alvo:  
        print('encontrada')  
        break  
else:  
    print(f'{alvo} nao esta na olt-norte')  
"
```

```
onu-99 nao esta na olt-norte
```

5. for/else e o laço vazio.

```
python -c "  
for n in (91, 97, 2):  
    for divisor in range(2, int(n**0.5) + 1):  
        if n % divisor == 0:  
            print(f'{n} = {divisor} x {n // divisor}')  
            break  
    else:  
        print(f'{n} e primo')  
"
```

```
91 = 7 x 13  
97 e primo  
2 e primo
```

Para $n = 2$, o `range(2, 2)` é vazio: zero voltas, nenhum `break`, e o `else` roda mesmo assim.

6. continue como filtro.

```
cat > enlace.log <<'LOG'  
-19.5  
# leitura suspensa  
-31.0  
  
erro  
-8.0  
LOG  
  
python -c "  
from pathlib import Path
```

```
soma, lidos, descartados = 0.0, 0, 0
for linha in Path('enlace.log').read_text(encoding='utf-8').splitlines():
    if not linha.strip() or linha.startswith('#'):
        descartados += 1
        continue
    try:
        valor = float(linha)
    except ValueError:
        descartados += 1
        continue
    soma += valor
    lidos += 1
print(f'{lidos} lidos, {descartados} descartados, media {soma / lidos:.2f}')
"
```

```
3 lidos, 3 descartados, media -19.50
```

7. while/else como retentativa.

```
python -c "
tentativas = 0
while tentativas < 3:
    tentativas += 1
    if tentativas == 5:
        print('conectou')
        break
else:
    print('desistiu depois de', tentativas, 'tentativas')
"
```

```
desistiu depois de 3 tentativas
```

8. match sobre sequência.

```
python -c "
def roteia(comando):
    match comando.split():
        case ['listar']:
            return 'lista tudo'
        case ['listar', olt]:
            return f'lista a {olt}'
        case ['reiniciar', *onus] if onus:
            return f'reinicia {len(onus)}: ' + ', '.join(onus)
        case ['sair' | 'quit' | 'exit']:
            return 'tchau'
        case []:
            return 'comando vazio'
        case _:
            return 'nao entendi'
"
```

17. *break, continue, else em laços e o casamento de padrões*

```
for c in ['listar', 'listar olt-sul', 'reiniciar onu-01 onu-02', 'quit', '',
↪   'voar']:
    print(f'{c!r:<28} -> {roteia(c)}')
"
```

'listar'	-> lista tudo
'listar olt-sul'	-> lista a olt-sul
'reiniciar onu-01 onu-02'	-> reinicia 2: onu-01, onu-02
'quit'	-> tchau
''	-> comando vazio
'voar'	-> nao entendi

9. Veja a captura silenciosa. Crie captura.py:

```
CRITICO = -28.0

def faixa(nivel):
    match nivel:
        case CRITICO:
            return f"casou com o limite? CRITICO agora vale {CRITICO}"

print(faixa(-19.5))
print("CRITICO no modulo:", CRITICO)
```

```
python captura.py
```

```
casou com o limite? CRITICO agora vale -19.5
CRITICO no modulo: -28.0
```

10. Agora com um case depois. Crie captura2.py:

```
CRITICO = -28.0

def faixa(nivel):
    match nivel:
        case CRITICO:
            return "no limite"
        case _:
            return "outro"
```

```
python captura2.py
```

```
File "/home/vitor/lab17/captura2.py", line 6
    case CRITICO:
        ~~~~~
SyntaxError: name capture 'CRITICO' makes remaining patterns unreachable
```

O mesmo erro nos dois arquivos; só o segundo é detectado. Isso é o argumento inteiro a favor de sempre terminar o `match` com `case _:`.

11. A forma correta. Crie `limiar.py`:

```
class Limiar:
    CRITICO = -28.0
    SATURACAO = -8.0

def faixa(nivel):
    match nivel:
        case Limiar.CRITICO:
            return "exatamente no limite critico"
        case Limiar.SATURACAO:
            return "exatamente no limite de saturacao"
        case _:
            return "entre os limites"

for n in (-28.0, -8.0, -19.5):
    print(f"{n:>7} {faixa(n)}")
```

```
python limiar.py
```

```
-28.0  exatamente no limite critico
-8.0   exatamente no limite de saturacao
-19.5  entre os limites
```

12. Monte o console de operação. Crie `console.py`:

```
"""Console de operacao: le comandos de um arquivo e despacha com match."""

from pathlib import Path

from rede import REDE

def executar(comando):
    """Devolve (mensagem, encerrar)."""
    match comando.split():
        case [] | ["#", *_]:
            return None, False
        case ["listar"]:
```

17. break, continue, else em laços e o casamento de padrões

```
        return " ".join(REDE), False
    case ["listar", olt] if olt in REDE:
        onus = REDE[olt]
        return f"{olt}: " + ", ".join(f"{k}={v}" for k, v in onus.items()),
            ↪ False
    case ["listar", olt]:
        return f"OLT desconhecida: {olt}", False
    case ["nivel", onu]:
        for olt, onus in REDE.items():
            if onu in onus:
                return f"{onu} em {olt}: {onus[onu]} dBm", False
            return f"{onu} nao encontrada", False
    case ["criticas", *_]:
        achadas = [
            f"{olt}/{onu}"
            for olt, onus in REDE.items()
            for onu, nivel in onus.items()
            if nivel < -28
        ]
        return ("criticas: " + ", ".join(achadas)) if achadas else
            ↪ "nenhuma", False
    case ["sair" | "quit"]:
        return "encerrando", True
    case _:
        return f"comando invalido: {comando!r}", False

if __name__ == "__main__":
    linhas = Path("comandos.txt").read_text(encoding="utf-8").splitlines()
    for numero, linha in enumerate(linhas, start=1):
        mensagem, encerrar = executar(linha)
        if mensagem is None:
            continue
        print(f"{numero:>2}| {mensagem}")
        if encerrar:
            break
    else:
        print("fim do arquivo sem comando de saida")
```

E o arquivo de comandos:

```
cat > comandos.txt <<'TXT'
listar

# so um comentario
listar olt-sul
listar olt-oeste
nivel onu-05
nivel onu-99
criticas
voar
```



```
sair
listar
TXT

python console.py
```

```
1| olt-norte olt-sul olt-leste
4| olt-sul: onu-04=-18.0, onu-05=-29.8
5| OLT desconhecida: olt-oeste
6| onu-05 em olt-sul: -29.8 dBm
7| onu-99 nao encontrada
8| criticas: olt-norte/onu-02, olt-sul/onu-05
9| comando invalido: 'voar'
10| encerrando
```

As linhas 2 e 3 sumiram do relatório: são a linha em branco e o comentário, tratados pelo `case [] | ["#", *]` que devolve `None` e aciona o `continue`. A numeração continua sendo a do arquivo porque vem do `enumerate`, não de um contador de linhas processadas. E o `break` da linha 10 impediu que o `listar` da 11 rodasse.

13. Tire o `sair` e veja o `else` do `laço`.

```
sed -i '/^sair$/d' comandos.txt
python console.py
```

```
1| olt-norte olt-sul olt-leste
4| olt-sul: onu-04=-18.0, onu-05=-29.8
5| OLT desconhecida: olt-oeste
6| onu-05 em olt-sul: -29.8 dBm
7| onu-99 nao encontrada
8| criticas: olt-norte/onu-02, olt-sul/onu-05
9| comando invalido: 'voar'
10| olt-norte olt-sul olt-leste
fim do arquivo sem comando de saida
```

A última linha é o `else` do `for` fazendo o seu único trabalho: dizer que o arquivo acabou sem que ninguém tenha dado `break`.

14. Feche com `deactivate`.

17.7. Resumo

- `break` sai do `laço mais interno` — não existe `break` com rótulo. Para sair de `laços aninhados`, extraia uma função e use `return`.
- `continue` pula para a próxima volta e é a forma idiomática de filtrar entrada no topo do `laço`; no `while` com contador, é gerador de `laço infinito`.
- O `else` de `laço` deveria chamar-se `nobreak`: roda quando o `laço` termina naturalmente, inclusive quando ele não dá volta nenhuma, e não roda depois de um `break`.

17. *break, continue, else em laços e o casamento de padrões*

- O par **break/else** elimina a variável-bandeira em buscas e retentativas.
- **match** (3.10+) casa **estrutura**, não valor: literais, sequências por tamanho e posição, mapeamentos por subconjunto, classes por tipo e atributos, alternativas com **|**, guardas com **if** e o curinga **_**.
- **str** e **bytes** não casam como sequência, por decisão de projeto.
- Nome simples em **case** é **captura**, não comparação: casa com tudo e rebatiza o valor. Constante só com nome pontilhado (**Limiar.CRITICO**, **Status.ATIVO**).
- Terminar o **match** com **case _**: transforma “não fez nada” em erro visível, e faz o compilador detectar capturas acidentais.

18. Definindo funções: parâmetros, argumentos e retorno

O script de relatório cresceu, e alguém fez a coisa certa: extraiu o cálculo da média para uma função.

```
def media_dbm(leituras):
    if leituras:
        total = sum(leituras)
        media = total / len(leituras)
        print(f"media calculada: {media:.2f}")

resultado = media_dbm([-19.5, -21.3, -20.2])
print("resultado:", resultado)
print("resultado + 3 =", resultado + 3)
```

```
media calculada: -20.33
resultado: None
Traceback (most recent call last):
  File "/home/vitor/lab18/media_ruim.py", line 10, in <module>
    print("resultado + 3 =", resultado + 3)
                                ~~~~~~^~~
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

A média foi calculada, apareceu na tela, e mesmo assim **resultado** é **None**. A função **imprimiu** o valor em vez de **devolver** o valor — e uma função sem **return** devolve **None**, sempre, sem avisar.

O detalhe cruel é a primeira linha da saída: o número certo apareceu. Quem testou no terminal viu **-20.33** e deu por encerrado. O erro só aparece quando alguém tenta usar o resultado, possivelmente num arquivo diferente, dias depois — e o **NoneType** do traceback não diz nada sobre a função que faltou devolver.

```
def media(leituras):
    if not leituras:
        return None
    return sum(leituras) / len(leituras)

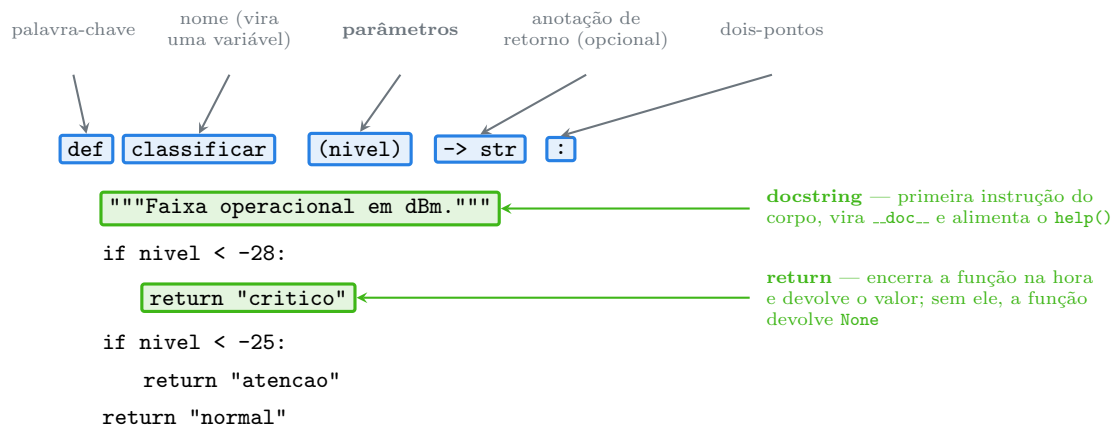
print(media([-19.5, -21.3, -20.2]))
print(media([]))
```

```
-20.333333333333332
None
```

Agora o `None` é uma decisão escrita, não um acidente. Este capítulo é sobre o que acontece quando você escreve `def`: o objeto que nasce, o quadro que a chamada empilha, como os argumentos chegam nos parâmetros e o que exatamente volta.

18.1. Anatomia de uma definição

```
def classificar(nivel):
    """Devolve a faixa operacional de um nivel optico em dBm."""
    if nivel < -28:
        return "critico"
    if nivel < -25:
        return "atencao"
    return "normal"
```



corpo — delimitado por indentação, não por chaves. Executa **só na chamada**: erro de digitação dentro do corpo passa despercebido até alguém chamar a função. O que o `def` faz na definição é criar um objeto do tipo `function` e ligá-lo ao nome `classificar` — nada mais.

Figura 18.1.: As partes de um `def`, e o que cada uma faz na definição e na chamada.

Duas coisas que essa figura resume e que valem repetir:

O `def` cria um objeto. Não é declaração, é instrução executável — ela roda, constrói um objeto do tipo `function` e o liga a um nome, do mesmo jeito que uma atribuição comum liga um nome a uma lista.

```
def classificar(nivel):
    """Devolve a faixa operacional de um nivel optico em dBm."""
    ...
```

```
print(type(classificar), classificar.__name__)
print(classificar.__doc__)
apelido = classificar
print(apelido(-31.0), apelido is classificar)
```

```
<class 'function'> classificar
Devolve a faixa operacional de um nível optico em dBm.
critico True
```

`apelido` e `classificar` são dois nomes para o mesmo objeto, exatamente como no Capítulo 8. O que isso destrava — passar função como argumento, guardar em dicionário, devolver de outra função — é o assunto do Capítulo 21.

O corpo só roda na chamada. Um `def` com erro de digitação lá dentro é aceito sem reclamar; o `NameError` aparece na primeira chamada, que pode ser em produção.

18.2. Parâmetro e argumento

A distinção é chata e útil:

- **Parâmetro** é o nome na definição (`nível`).
- **Argumento** é o valor na chamada (`-31.0`).

Argumentos podem ser passados por **posição** ou por **nome**:

```
def area(base, altura):
    return base * altura / 2

print(area(3, 4))
print(area(altura=4, base=3))
```

```
6.0
6.0
```

Passar por nome documenta a chamada e imuniza contra troca de ordem. As regras completas — valores padrão, `*args`, `**kwargs`, parâmetros só-posicionais — são o Capítulo 19.

18.3. return

`return` faz duas coisas ao mesmo tempo: **encerra** a função e **devolve** um valor. É por isso que a cláusula de guarda do Capítulo 15 funciona tão bem dentro de função.

Sem valor, `return` devolve `None`; sem `return` nenhum, também:

```
def registrar(nivel):
    if nivel < -28:
        return "critico"
    print("nao critico")
    return

print(registrar(-31.0))
print(registrar(-19.5))
```

```
critico
nao critico
None
```

Um `return` nu é uma saída antecipada legítima. O que não é legítimo é a função **às vezes** devolver e às vezes não — o caso da abertura. Se um caminho devolve `str`, todos devem devolver `str` ou `None` de propósito, e o `None` precisa estar escrito.

18.3.1. “Vários valores” é uma tupla

Python não devolve múltiplos valores; devolve **um** objeto, que costuma ser uma tupla:

```
def estatisticas(leituras):
    return min(leituras), max(leituras), sum(leituras) / len(leituras)

r = estatisticas([-19.5, -31.0, -20.2])
print(r, type(r))
pior, melhor, med = estatisticas([-19.5, -31.0, -20.2])
print(f"{pior=} {melhor=} {med=:.2f}")
```

```
(-31.0, -19.5, -23.566666666666666) <class 'tuple'>
pior=-31.0 melhor=-19.5 med=-23.57
```

As vírgulas do `return` montam a tupla; o desempacotamento do lado esquerdo a desmonta. A partir de três ou quatro valores, uma tupla posicional vira armadilha (quem lembra a ordem?), e a resposta é `NamedTuple` ou `dataclass`, do Volume 7.

E a sintaxe `f"{pior=}"`, do Capítulo 13, é o depurador de pobre que todo mundo usa: imprime o nome e o valor juntos.

18.4. O que a chamada faz

Chamar uma função empilha um **quadro** (*frame*): um espaço novo onde os parâmetros já nascem ligados aos argumentos e onde vivem as variáveis locais. Quando a função retorna, o quadro é desempilhado.

É a pilha que o traceback do Capítulo 7 imprime:

```
TABELA = {"onu-01": -19.5, "onu-02": -31.2}
```

```
def nivel_de(onu):
    return TABELA[onu]
```

```
def relatar(onu):
    return f"{onu}: {nivel_de(onu)} dBm"
```

```
print(relatar("onu-01"))
print(relatar("onu-99"))
```

```
onu-01: -19.5 dBm
```

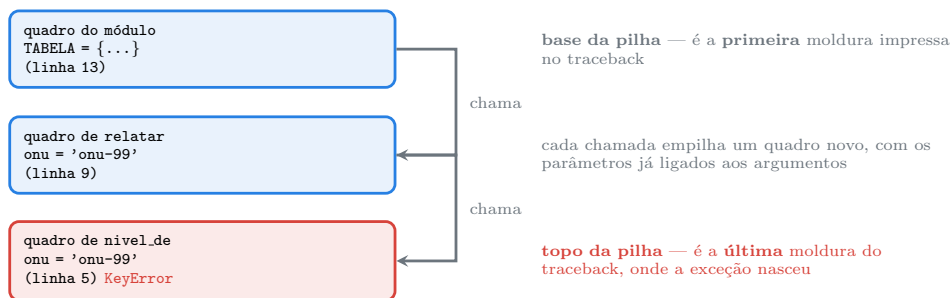
```
Traceback (most recent call last):
```

```
File "/home/vitor/lab18/consulta.py", line 13, in <module>
    print(relatar("onu-99"))
    ~~~~~~^~~~~~
```

```
File "/home/vitor/lab18/consulta.py", line 9, in relatar
    return f"{onu}: {nivel_de(onu)} dBm"
           ~~~~~~^~~~~~
```

```
File "/home/vitor/lab18/consulta.py", line 5, in nivel_de
    return TABELA[onu]
           ~~~~~~^~~~~~
```

```
KeyError: 'onu-99'
```



Ao retornar, o quadro é descartado e as variáveis locais dele somem. Por isso o traceback se lê de **baixo para cima** quando se quer a causa, e de **cima para baixo** quando se quer o caminho: a última moldura é onde estourou, a primeira é quem começou tudo.

Figura 18.2.: Três chamadas, três quadros — e as três molduras do traceback, na mesma ordem.

18.5. Como os argumentos chegam

Python não é “por valor” nem “por referência”: o que acontece é que o parâmetro vira **mais um nome** para o objeto que veio de fora, exatamente como uma atribuição. As consequências são duas, e é preciso as duas para não se enganar.

Mutar o objeto afeta quem chamou:

```
def anexar(destino, item):
    destino.append(item)

lista = ["a"]
anexar(lista, "b")
print("depois de anexar: ", lista)
```

```
depois de anexar:  ['a', 'b']
```

Reatribuir o parâmetro não afeta ninguém:

```
def rebatizar(destino):
    destino = ["novo"]
    return destino

volta = rebatizar(lista)
print("depois de rebatizar:", lista, "| devolvido:", volta)
```

```
depois de rebatizar: ['a', 'b'] | devolvido: ['novo']
```

A atribuição dentro da função só reapontou o nome **local**. Dá para ver com id:

```
def inspeciona(obj):
    print("dentro:          ", id(obj))
    obj = "outro"
    print("depois da atribuicao: ", id(obj))

texto = "original"
print("fora, antes:          ", id(texto))
inspeciona(texto)
print("fora, depois:         ", id(texto), texto)
```

```
fora, antes:          124197974277104
dentro:              124197974277104
depois da atribuicao: 124197974127408
fora, depois:        124197974277104 original
```

Os identificadores mudam a cada execução; o padrão não muda. Entrou com o mesmo id (é o mesmo objeto), a atribuição criou outro id só lá dentro, e o de fora ficou igual.

Daí a regra prática: **uma função ou devolve um valor novo, ou modifica o que recebeu — não as duas coisas**, e o nome deve dizer qual das duas. Modificar o argumento sem avisar é a origem de bugs que atravessam módulos.

18.6. Docstring

A docstring é a primeira instrução do corpo, entre aspas triplas, e é o que `help()` mostra:

```
help(classificar)
```

```
Help on function classificar in module __main__:
```

```
classificar(nivel)
    Devolve a faixa operacional de um nivel optico em dBm.
```

A PEP 257 pede uma linha imperativa curta (“Devolve...”, “Calcula...”, “Grava...”), e, se precisar de mais, uma linha em branco e o resto. Comentário `#` explica **como**; docstring explica **o que** e **para quem** chama.

18.7. Anotações de tipo

```
def normaliza(nome: str, casas: int) -> str:
    return nome.strip().lower()[:casas]
```

```
print(normaliza("  ONU-Norte-01  ", 8))
print(normaliza.__annotations__)
print(normaliza(123, 2))
```

```
onu-nort
{'nome': <class 'str'>, 'casas': <class 'int'>, 'return': <class 'str'>}
```

Traceback (most recent call last):

```
File "/home/vitor/lab18/anotacao.py", line 7, in <module>
    print(normaliza(123, 2))
          ~~~~~~^~~~~~
File "/home/vitor/lab18/anotacao.py", line 2, in normaliza
    return nome.strip().lower()[:casas]
           ~~~~~~^~~~~~
AttributeError: 'int' object has no attribute 'strip'
```

A anotação dizia `str` e o `123` passou sem reclamação: **o interpretador não verifica anotações em tempo de execução**. Elas ficam guardadas em `__annotations__` e servem para o verificador estático (`mypy`, `pyright`), para o editor e para quem lê. O erro veio depois, de dentro, quando o `int` não soube fazer `.strip()`. Tipagem é o Volume 11; aqui basta a regra: anotar é documentar de forma verificável, não é validar.

18.8. Funções que devolvem `None` de propósito

Parte da biblioteca padrão devolve `None` deliberadamente, para deixar claro que a operação foi **no lugar**:

```
onus = ["onu-03", "onu-01", "onu-02"]
devolvido = onus.sort()
print("sort devolveu:", devolvido)
print("lista agora: ", onus)

outras = ["onu-03", "onu-01", "onu-02"]
print("sorted devolveu:", sorted(outras), "| original:", outras)
```

```
sort devolveu: None
lista agora:  ['onu-01', 'onu-02', 'onu-03']
sorted devolveu: ['onu-01', 'onu-02', 'onu-03'] | original: ['onu-03',
↳ 'onu-01', 'onu-02']
```

`onus = onus.sort()` é o erro clássico: a lista ordenada é jogada fora e `onus` vira `None`. A convenção vale também para `list.append`, `dict.update` e `random.shuffle` — e vale para o seu código: se a função muda o argumento, não devolva ele também, porque isso convida a encadear chamadas que parecem funcionais e não são.

18.9. Recursão, em uma seção curta

Uma função pode chamar a si mesma. Precisa de um caso base, e o Python limita a profundidade:

```
import sys

def profundidade(n):
    return profundidade(n + 1)

def fatorial(n):
    """Fatorial de n, com caso base explicito."""
    if n <= 1:
        return 1
    return n * fatorial(n - 1)

print("limite:", sys.getrecursionlimit())
print("fatorial(20):", fatorial(20))
try:
    profundidade(0)
except RecursionError as e:
    print("RecursionError:", e)
```

```
limite: 1000
fatorial(20): 2432902008176640000
RecursionError: maximum recursion depth exceeded
```

Mil quadros e acabou. O Python **não** faz otimização de chamada de cauda, e a decisão é deliberada: Guido van Rossum argumentou que ela destruiria os `tracebacks`, que são mais valiosos. Em Python, recursão é ferramenta para estrutura recursiva (árvore, JSON aninhado, sistema de arquivos), não substituto de laço.

18.10. Jeito Pythônico

Uma função, uma responsabilidade, um nome de verbo. `carregar`, `classificar`, `formatar`, `resumir`. Se o nome precisa de “e” (`carrega_e_valida`), são duas funções.

Calcular e imprimir são trabalhos diferentes. A função devolve; quem chamou decide se imprime, grava ou soma. Foi essa mistura que produziu o bug da abertura, e ela também impede testar a função sem capturar a saída padrão.

Torne o `None` explícito. Se a função pode não ter resposta, escreva `return None` no caminho correspondente. `None` implícito é indistinguível de esquecimento.

Não misture cálculo com mutação. Ou devolve valor novo (`sorted`), ou muda no lugar e devolve `None` (`sort`).

Docstring em toda função pública, na forma imperativa, dizendo o que devolve.

Anote os tipos quando a assinatura não for óbvia. Custa pouco, o editor usa na hora, e o `mypy` transforma em verificação de verdade.

```
# antipadrão: imprime, não devolve, e o nome não diz o que faz
def dados(x):
    print(sum(x) / len(x))
```

```
# pythônico
def media(leituras: list[float]) -> float:
    """Devolve a media aritmetica das leituras."""
    return sum(leituras) / len(leituras)
```

Prefira `list[float]` a `typing.List[float]`. A forma nova vale desde o **Python 3.9** (PEP 585); a antiga só sobrevive em código legado.

Cuidado com o argumento mutável que você não pediu. Se a função guarda o que recebeu (numa lista de estado, num atributo), guarde uma cópia — senão quem chamou pode mudar por baixo depois.

Sobre versões:

- **Python 3.0** — anotações de função (PEP 3107), sem semântica definida.
- **Python 3.5** — módulo `typing` e o significado padrão das anotações (PEP 484).
- **Python 3.8** — parâmetros só-posicionais com `/` (PEP 570), assunto do Capítulo 19.
- **Python 3.9** — `list[float]` e `dict[str, int]` direto nos tipos embutidos (PEP 585).
- **Python 3.10** — `X | Y` no lugar de `Union[X, Y]` (PEP 604).
- **Python 3.12** — sintaxe nova de genéricos, `def f[T](x: T) -> T:` (PEP 695).
- **Python 3.14** — anotações passam a ser avaliadas de forma preguiçosa (PEP 649), o que muda o comportamento de `__annotations__` em referências adiante. Na 3.13, `from __future__ import annotations` é o jeito de antecipar isso.

18.11. Laboratório

Objetivo: ver o `None` implícito estourar longe da causa, ler a pilha de chamadas num traceback de três molduras, medir a diferença entre mutar e reatribuir um parâmetro, e terminar quebrando um script em funções pequenas e testáveis no REPL.

1. Ambiente.

```
mkdir -p ~/lab18 && cd ~/lab18
python3 -m venv .venv && source .venv/bin/activate
python --version
```

```
Python 3.13.12
```

2. O `None` implícito. Crie `media_ruim.py`:

```
def media_dbm(leituras):
    if leituras:
        total = sum(leituras)
        media = total / len(leituras)
        print(f"media calculada: {media:.2f}")

resultado = media_dbm([-19.5, -21.3, -20.2])
print("resultado:", resultado)
print("resultado + 3 =", resultado + 3)
```

```
python media_ruim.py
```

```
media calculada: -20.33
resultado: None
Traceback (most recent call last):
  File "/home/vitor/lab18/media_ruim.py", line 10, in <module>
    print("resultado + 3 =", resultado + 3)
                                ~~~~~~^~~
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

Repare que o traceback aponta a **linha 10**, e o erro está na **linha 5** — a que deveria ser um `return`.

3. A versão que devolve.

```
python -c "
def media(leituras):
    if not leituras:
        return None
    return sum(leituras) / len(leituras)
print(media([-19.5, -21.3, -20.2]))
print(media([]))
"
```

```
-20.333333333333332
None
```

4. A função é um objeto.

```
python -c "
def classificar(nivel):
    '''Devolve a faixa operacional de um nivel optico em dBm.'''
    if nivel < -28:
        return 'critico'
    if nivel < -25:
        return 'atencao'
    return 'normal'
print(type(classificar), classificar.__name__)
print(classificar.__doc__)
apelido = classificar
print(apelido(-31.0), apelido is classificar)
"
```

```
<class 'function'> classificar
Devolve a faixa operacional de um nivel optico em dBm.
critico True
```

5. Devolver uma tupla.

```
python -c "
def estatisticas(leituras):
    return min(leituras), max(leituras), sum(leituras) / len(leituras)
r = estatisticas([-19.5, -31.0, -20.2])
print(r, type(r))
pior, melhor, med = estatisticas([-19.5, -31.0, -20.2])
print(f'{pior=} {melhor=} {med=:.2f}')
"
```

```
(-31.0, -19.5, -23.566666666666666) <class 'tuple'>
pior=-31.0 melhor=-19.5 med=-23.57
```

6. Mutar contra reatribuir.

```
python -c "
def anexar(destino, item):
    destino.append(item)
def rebatizar(destino):
    destino = ['novo']
    return destino
lista = ['a']
anexar(lista, 'b')
print('depois de anexar: ', lista)
```

18. Definindo funções: parâmetros, argumentos e retorno

```
volta = rebatizar(lista)
print('depois de rebatizar:', lista, '| devolvido:', volta)
"
```

```
depois de anexar: ['a', 'b']
depois de rebatizar: ['a', 'b'] | devolvido: ['novo']
```

7. Confirme com id.

```
python -c "
def inspeciona(obj):
    print('dentro: ', id(obj))
    obj = 'outro'
    print('depois da atribuicao: ', id(obj))
texto = 'original'
print('fora, antes: ', id(texto))
inspeciona(texto)
print('fora, depois: ', id(texto), texto)
"
```

```
fora, antes:      124197974277104
dentro:           124197974277104
depois da atribuicao: 124197974127408
fora, depois:     124197974277104 original
```

Os números mudam a cada execução. O que importa é que a primeira e a segunda linha são iguais, a terceira é diferente, e a quarta voltou a ser a primeira.

8. Anotação não é validação. Crie anotacao.py:

```
def normaliza(nome: str, casas: int) -> str:
    return nome.strip().lower()[casas]
```

```
print(normaliza(" ONU-Norte-01 ", 8))
print(normaliza.__annotations__)
print(normaliza(123, 2))
```

```
python anotacao.py
```

```
onu-nort
{'nome': <class 'str'>, 'casas': <class 'int'>, 'return': <class 'str'>}
Traceback (most recent call last):
  File "/home/vitor/lab18/anotacao.py", line 7, in <module>
    print(normaliza(123, 2))
    ~~~~~~^~~~~~
  File "/home/vitor/lab18/anotacao.py", line 2, in normaliza
    return nome.strip().lower()[casas]
    ~~~~~~
AttributeError: 'int' object has no attribute 'strip'
```

9. sort contra sorted.

```
python -c "
onus = ['onu-03', 'onu-01', 'onu-02']
devolvido = onus.sort()
print('sort devolveu:', devolvido)
print('lista agora: ', onus)
outras = ['onu-03', 'onu-01', 'onu-02']
print('sorted devolveu:', sorted(outras), '| original:', outras)
"
```

```
sort devolveu: None
lista agora:  ['onu-01', 'onu-02', 'onu-03']
sorted devolveu: ['onu-01', 'onu-02', 'onu-03'] | original: ['onu-03',
↪  'onu-01', 'onu-02']
```

10. Recursão e o limite. Crie recursao.py:

```
import sys

def profundidade(n):
    return profundidade(n + 1)

def fatorial(n):
    """Fatorial de n, com caso base explicito."""
    if n <= 1:
        return 1
    return n * fatorial(n - 1)

print("limite:", sys.getrecursionlimit())
print("fatorial(20):", fatorial(20))
try:
    profundidade(0)
except RecursionError as e:
    print("RecursionError:", e)
```

```
python recursao.py
```

```
limite: 1000
fatorial(20): 2432902008176640000
RecursionError: maximum recursion depth exceeded
```

11. Leia a pilha num traceback. Crie consulta.py:

18. Definindo funções: parâmetros, argumentos e retorno

```
TABELA = {"onu-01": -19.5, "onu-02": -31.2}

def nivel_de(onu):
    return TABELA[onu]

def relatar(onu):
    return f"{onu}: {nivel_de(onu)} dBm"

print(relatar("onu-01"))
print(relatar("onu-99"))
```

```
python consulta.py
```

```
onu-01: -19.5 dBm
Traceback (most recent call last):
  File "/home/vitor/lab18/consulta.py", line 13, in <module>
    print(relatar("onu-99"))
    ~~~~~~^~~~~~
  File "/home/vitor/lab18/consulta.py", line 9, in relatar
    return f"{onu}: {nivel_de(onu)} dBm"
    ~~~~~~^~~~~~
  File "/home/vitor/lab18/consulta.py", line 5, in nivel_de
    return TABELA[onu]
    ~~~~~~^~~~~~
KeyError: 'onu-99'
```

Três molduras, três quadros — compare com a Figura 18.2. A de baixo é onde o erro nasceu; a de cima é quem começou.

12. Quebre um script em funções. Crie leituras.csv e relatorio.py:

```
cat > leituras.csv <<'CSV'
onu;nivel
onu-01;-19.5
onu-02;-31.2
onu-03;
onu-04;-26.4
onu-05;-8.0
CSV
```

```
"""Relatorio de niveis opticos, escrito como funcoes pequenas e testaveis."""

import csv
from pathlib import Path

CRITICO = -28.0
```



```

ATENCAO = -25.0
SATURACAO = -8.0

def carregar(caminho):
    """Le o CSV e devolve uma lista de pares (onu, nivel ou None)."""
    texto = Path(caminho).read_text(encoding="utf-8")
    linhas = csv.DictReader(texto.splitlines(), delimiter=";")
    return [
        (l["onu"], float(l["nivel"])) if l["nivel"].strip() else None) for l in
↪ linhas
    ]

def classificar(nivel):
    """Faixa operacional de um nivel em dBm; None quando nao ha leitura."""
    if nivel is None:
        return "sem leitura"
    if nivel < CRITICO:
        return "critico"
    if nivel < ATENCAO:
        return "atencao"
    if nivel <= SATURACAO:
        return "normal"
    return "saturado"

def resumir(registros):
    """Devolve (total, quantidade de problemas, pior leitura registrada)."""
    medidos = [n for _, n in registros if n is not None]
    problemas = sum(1 for _, n in registros if classificar(n) != "normal")
    return len(registros), problemas, min(medidos)

def formatar(onu, nivel):
    """Uma linha de relatorio, ja alinhada."""
    medida = " --- " if nivel is None else f"{nivel:>7.1f}"
    return f"{onu} {medida} {classificar(nivel)}"

def main(caminho="leituras.csv"):
    registros = carregar(caminho)
    for onu, nivel in registros:
        print(formatar(onu, nivel))
    total, problemas, pior = resumir(registros)
    print("-" * 34)
    print(f"{total} ONUs, {problemas} com problema, pior leitura {pior} dBm")

if __name__ == "__main__":
    main()

```

```
python relatorio.py
```

```
onu-01    -19.5  normal
onu-02    -31.2  critico
onu-03     ---   sem leitura
onu-04    -26.4  atencao
onu-05     -8.0  normal
-----
5 ONUs, 3 com problema, pior leitura -31.2 dBm
```

13. Agora use as funções sem rodar o script. É este o ganho: cada peça pode ser exercitada sozinha.

```
python -c "
import relatorio
print(relatorio.classificar(-31.0), relatorio.classificar(None),
      ↪ relatorio.classificar(-19.5))
print(relatorio.formatar('onu-09', -26.0))
print(relatorio.resumir([('a', -19.5), ('b', None), ('c', -31.0)]))
print(relatorio.carregar.__doc__)
"
```

```
critico sem leitura normal
onu-09    -26.0  atencao
(3, 2, -31.0)
Le o CSV e devolve uma lista de pares (onu, nivel ou None).
```

Nenhuma dessas quatro chamadas leu arquivo, imprimiu relatório ou dependeu do resto do programa. Uma função que só devolve é uma função que se testa em uma linha — e é por isso que “devolva, não imprima” é a regra que mais rende neste capítulo. O `if __name__ == "__main__":` que protege o `main()` é o que permite importar sem executar; a mecânica dele é o Capítulo 15 aplicada a uma variável do módulo, e o assunto completo é o Volume 5.

14. Feche com `deactivate`.

18.12. Resumo

- `def` é instrução executável: cria um objeto `function` e liga a um nome. O corpo só roda na chamada, então erro dentro dele aparece tarde.
- Função sem `return` devolve `None`, e função que **imprime** não é função que **devolve** — a confusão entre as duas é o bug de abertura deste capítulo.
- `return` encerra e devolve ao mesmo tempo; devolver “vários valores” é devolver uma tupla, desmontada por desempacotamento.
- Parâmetro é o nome na definição; argumento é o valor na chamada, passado por posição ou por nome.
- Cada chamada empilha um quadro com os parâmetros ligados aos argumentos; a pilha é exatamente o que o `traceback` imprime, da base para o topo.

- O parâmetro é mais um nome para o objeto recebido: `mutar` afeta quem chamou, reatribuir não afeta ninguém.
- Uma função ou devolve valor novo (`sorted`) ou muda no lugar e devolve `None` (`sort`) — nunca as duas.
- Docstring diz o que a função faz e alimenta o `help()`; anotações documentam tipos e **não** são verificadas em execução.
- Recursão precisa de caso base e esbarra em `sys.getrecursionlimit()` (1000 por padrão); o Python não otimiza chamada de cauda, por decisão de projeto.

19. Argumentos nomeados, valores padrão, `*args` e `**kwargs`

A função guarda o histórico de ONUs visitadas numa lista, e quem chama pode passar a sua própria lista ou deixar que a função comece uma:

```
def registrar(onu, historico=[]):
    historico.append(onu)
    return historico

print(registrar("onu-01"))
print(registrar("onu-02"))
print(registrar("onu-03"))
print("o padrao em si:", registrar.__defaults__)
```

Três chamadas independentes, três listas de um elemento — é o que a leitura do código promete. É o seguinte que acontece:

```
['onu-01']
['onu-01', 'onu-02']
['onu-01', 'onu-02', 'onu-03']
o padrao em si: (['onu-01', 'onu-02', 'onu-03'],)
```

Cada chamada devolveu uma lista **maior** que a anterior, e a última linha explica tudo: o valor padrão não é “uma lista vazia”, é **aquela** lista, criada uma única vez e guardada dentro do objeto função. Toda chamada que omite o argumento recebe a mesma lista de sempre, com tudo que as chamadas anteriores deixaram lá.

Num programa de verdade, isso vaza estado entre requisições, entre testes e entre clientes — e o vazamento é cumulativo, então o sintoma aparece depois de horas no ar.

Este capítulo é sobre as regras que governam a passagem de argumentos: quando o padrão é avaliado, como forçar nome em vez de posição, e o que `*` e `**` fazem dos dois lados da chamada.

19.1. Valores padrão são avaliados uma vez, na definição

Essa é a frase inteira. O `def` roda quando o módulo é importado; a expressão do padrão roda ali, naquele instante, e o objeto resultante fica preso ao objeto função.

19. Argumentos nomeados, valores padrão, **args* e ***kwargs*

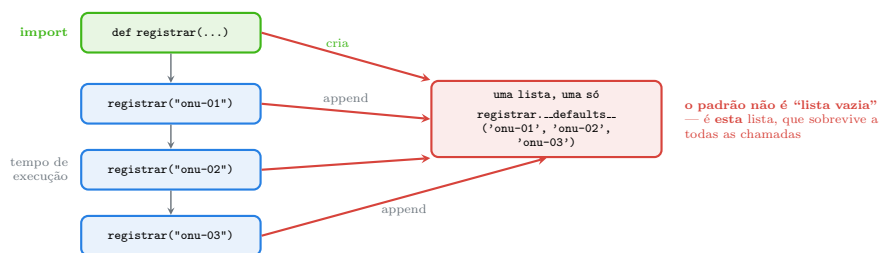
```
import time

def carimbo(t=time.time()):
    return t

print(f"{carimbo():.3f}")
time.sleep(1)
print(f"{carimbo():.3f}")
print(f"{carimbo(time.time()):.3f}")
```

```
1786439046.981
1786439046.981
1786439047.981
```

Um segundo separa a primeira da segunda chamada, e as duas devolveram o mesmo instante — o instante em que o `def` foi executado. Só a terceira, que passou o argumento, viu o relógio andar.



A regra que evita a classe inteira: valor padrão só imutável (`None`, número, string, tupla). Quando o padrão precisa ser mutável, o padrão é `None` e a função cria o objeto no corpo — aí ele nasce de novo a cada chamada.

Figura 19.1.: O padrão é criado uma vez, na definição, e todas as chamadas compartilham o mesmo objeto.

A correção é sempre a mesma — `None` como sentinela e a criação dentro do corpo:

```
def registrar(onu, historico=None):
    if historico is None:
        historico = []
    historico.append(onu)
    return historico

print(registrar("onu-01"))
print(registrar("onu-02"))
meu = ["onu-00"]
print(registrar("onu-03", meu), meu)
```

```
['onu-01']
['onu-02']
['onu-00', 'onu-03'] ['onu-00', 'onu-03']
```

As duas primeiras chamadas agora são independentes. E a terceira mostra o outro lado da moeda do Capítulo 18: quando a lista é passada, a função continua modificando a de quem chamou — o que aqui é o comportamento desejado, e por isso a última linha imprime a mesma lista duas vezes.

O padrão mutável é tão conhecido que o **ruff** tem uma regra própria para ele (B006), e vale ligar.

19.2. Argumentos por nome

Todo parâmetro comum pode ser preenchido por posição ou por nome:

```
def alerta(onu, nivel, unidade="dBm", urgente=False):
    marca = "!! " if urgente else "  "
    return f"{marca}{onu}: {nivel} {unidade}"

print(alerta("onu-01", -19.5))
print(alerta("onu-02", -31.2, urgente=True))
print(alerta(nivel=-8.0, onu="onu-05", unidade="dBm"))
print(alerta.__defaults__)
```

```
onu-01: -19.5 dBm
!! onu-02: -31.2 dBm
onu-05: -8.0 dBm
('dBm', False)
```

Três coisas se veem aí. O nomeado permite **pular** parâmetros com padrão (**urgente=True** sem mencionar **unidade**). Permite **reordenar** (**nivel** antes de **onu**). E os padrões ficam guardados em **__defaults__**, na ordem em que aparecem.

Duas regras de sintaxe: argumento posicional nunca vem depois de nomeado na chamada, e parâmetro com padrão nunca vem antes de parâmetro sem padrão na definição — as duas coisas são **SyntaxError**.

Os erros de chamada valem decorar, porque a mensagem diz exatamente o que falta:

```
def alerta(onu, nivel, unidade="dBm"):
    return f"{onu}: {nivel} {unidade}"
```

```
TypeError: alerta() missing 1 required positional argument: 'nivel'
TypeError: alerta() takes from 2 to 3 positional arguments but 4 were given
TypeError: alerta() got an unexpected keyword argument 'unit'
TypeError: alerta() got multiple values for argument 'onu'
```

19. Argumentos nomeados, valores padrão, **args* e ***kwargs*

O último é o mais curioso: `alerta("onu-01", -19.5, onu="onu-02")` deu o valor de `onu` duas vezes, uma por posição e outra por nome.

19.3. **args*: os posicionais que sobraram

Um parâmetro precedido de `*` recolhe todos os argumentos posicionais restantes numa **tupla**:

```
def media(*leituras):
    print(f" recebi {leituras!r} ({type(leituras).__name__})")
    return sum(leituras) / len(leituras) if leituras else 0.0

print(media(-19.5, -21.3))
print(media())
print(media(*[-19.5, -21.3, -20.2]))
```

```
recebi (-19.5, -21.3) (tuple)
-20.4
recebi () (tuple)
0.0
recebi (-19.5, -21.3, -20.2) (tuple)
-20.333333333333332
```

Sem argumento nenhum, `leituras` é uma tupla vazia — nunca `None`. E a terceira chamada mostra o `*` do **outro lado**: na definição ele empacota; na chamada ele desempacota.

O nome `args` é convenção, não sintaxe: `*leituras` é igual de válido e diz mais.

19.4. ***kwargs*: os nomeados que sobraram

O `**` faz o mesmo com argumentos nomeados, recolhendo-os num **dicionário**:

```
def conectar(host, **opcoes):
    print(f" {host} com {opcoes!r} ({type(opcoes).__name__})")
    porta = opcoes.get("porta", 22)
    return f"{host}:{porta}"

print(conectar("olt-norte"))
print(conectar("olt-sul", porta=830, timeout=5))
config = {"porta": 161, "comunidade": "publica"}
print(conectar("olt-leste", **config))
```

```
olt-norte com {} (dict)
olt-norte:22
olt-sul com {'porta': 830, 'timeout': 5} (dict)
olt-sul:830
olt-leste com {'porta': 161, 'comunidade': 'publica'} (dict)
olt-leste:161
```


O dicionário preserva a ordem em que os nomes foram passados, e as chaves são sempre strings.

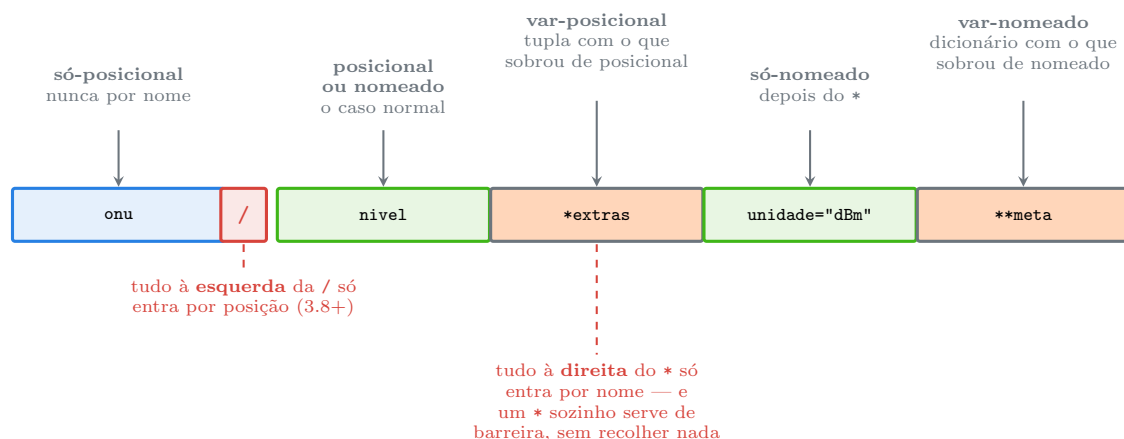
19.5. A ordem canônica dos parâmetros

Tudo junto, na única ordem possível:

```
def registrar(onu, /, nivel, *extras, unidade="dBm", **meta):
    return {
        "onu": onu,
        "nivel": nivel,
        "extras": extras,
        "unidade": unidade,
        "meta": meta,
    }

print(registrar("onu-01", -19.5))
print(registrar("onu-01", -19.5, "manha", "campo", unidade="mW",
    ↪ tecnico="ana"))
```

```
{'onu': 'onu-01', 'nivel': -19.5, 'extras': (), 'unidade': 'dBm', 'meta': {}}
{'onu': 'onu-01', 'nivel': -19.5, 'extras': ('manha', 'campo'), 'unidade':
    ↪ 'mW', 'meta': {'tecnico': 'ana'}}
```



A ordem é obrigatória e a lista pode pular qualquer região, mas nunca trocá-las de lugar. `inspect.signature(f).parameters` devolve o nome e a categoria de cada parâmetro — é como se confere uma assinatura alheia sem ler o código.

Figura 19.2.: As cinco regiões de uma lista de parâmetros, separadas por `/` e por `*`.

A barra `/` marca o fim dos parâmetros **só-posicionais** (PEP 570, **Python 3.8**). Passar `onu` por nome deixa de ser possível:

```
registrar(onu="onu-01", nivel=-19.5)
```

```
TypeError: registrar() missing 1 required positional argument: 'onu'
```

A mensagem parece estranha — `onu` foi passado! —, mas do ponto de vista da função o `onu=` foi para dentro de ***meta* e o parâmetro posicional continuou vazio. Isso é o que as funções embutidas sempre fizeram: `len(obj=[1,2])` também não funciona.

Um detalhe que morde: com **extras* no meio, o terceiro posicional **não** vai para `unidade`.

```
print(registrar("onu-01", -19.5, "dBm")["extras"])
```

```
('dBm',)
```

`unidade` está depois do **extras*, então é **só-nomeado**: `"dBm"` foi engolido pelos extras em silêncio. Quando um parâmetro fica depois de um ***, a única forma de preenchê-lo é pelo nome.

E é justamente para isso que serve o *** sozinho, sem nome:

```
def desligar(onu, *, confirmado=False):
    if not confirmado:
        return f"{onu}: recusado, falta confirmar"
    return f"{onu}: desligada"

print(desligar("onu-07"))
print(desligar("onu-07", confirmado=True))
print(desligar("onu-07", True))
```

```
onu-07: recusado, falta confirmar
onu-07: desligada
TypeError: desligar() takes 1 positional argument but 2 were given
```

`desligar("onu-07", True)` é recusado. É assim que se elimina a chamada ilegível com booleano solto — e a função fica documentada no ponto de uso.

Para inspecionar uma assinatura alheia:

```
import inspect

def registrar(onu, /, nivel, *extras, unidade="dBm", **meta):
    pass

sig = inspect.signature(registrar)
print(sig)
for nome, p in sig.parameters.items():
    print(f" {nome:<10} {p.kind}")
```

```
(onu, /, nivel, *extras, unidade='dBm', **meta)
onu          POSITIONAL_ONLY
nivel        POSITIONAL_OR_KEYWORD
extras       VAR_POSITIONAL
unidade      KEYWORD_ONLY
meta         VAR_KEYWORD
```

19.6. * e ** na chamada

Do lado da chamada, os mesmos símbolos fazem o inverso: **espalham** uma sequência em argumentos posicionais e um dicionário em argumentos nomeados.

```
def alerta(onu, nivel, unidade="dBm"):
    return f"{onu}: {nivel} {unidade}"

args = ("onu-01", -19.5)
kwargs = {"unidade": "mW"}
print(alerta(*args, **kwargs))

linhas = [("onu-01", -19.5), ("onu-02", -31.2)]
for linha in linhas:
    print(alerta(*linha))
```

```
onu-01: -19.5 mW
onu-01: -19.5 dBm
onu-02: -31.2 dBm
```

Combinar os dois lados é o que permite escrever um invólucro que repassa argumentos sem conhecer a assinatura de baixo:

```
import time

def com_tempo(funcao, *args, **kwargs):
    """Chama funcao(*args, **kwargs) e devolve (resultado, milissegundos)."""
    inicio = time.perf_counter()
    resultado = funcao(*args, **kwargs)
    return resultado, (time.perf_counter() - inicio) * 1000

def soma_lenta(a, b, atraso=0.05):
    time.sleep(atraso)
    return a + b

valor, ms = com_tempo(soma_lenta, 2, 3)
print(f"soma_lenta(2, 3) = {valor} em ~{ms:.0f} ms")
```

19. Argumentos nomeados, valores padrão, **args* e ***kwargs*

```
valor, ms = com_tempo(soma_lenta, 2, 3, atraso=0.2)
print(f"com atraso=0.2 = {valor} em ~{ms:.0f} ms")
valor, ms = com_tempo(sorted, [3, 1, 2], reverse=True)
print(f"sorted(...) = {valor} em ~{ms:.2f} ms")
```

```
soma_lenta(2, 3) = 5 em ~50 ms
com atraso=0.2 = 5 em ~200 ms
sorted(...) = [3, 2, 1] em ~0.01 ms
```

O mesmo `com_tempo` serviu para uma função sua e para uma função embutida, com assinaturas completamente diferentes. Esse par `*args`, `**kwargs` no meio é a base de todo decorador, que é o Volume 8.

19.7. Jeito Pythônico

Valor padrão só imutável. `None`, número, string, tupla, `frozenset`. Para lista, dicionário, conjunto ou qualquer objeto seu, o padrão é `None` e o corpo cria.

```
# antipadrão
def registrar(onu, historico=[], meta={}):
    ...
```

```
# pythônico
def registrar(onu, historico=None, meta=None):
    historico = [] if historico is None else historico
    meta = {} if meta is None else meta
```

Nada de expressão viva no padrão. `time.time()`, `datetime.now()`, `uuid4()`, leitura de arquivo, consulta ao ambiente — tudo isso congela no instante do `import`.

Booleano em chamada é só-nomeado. Um `*` antes dele custa um caractere e elimina a chamada `desligar("onu-07", True, False)`, que ninguém consegue ler.

`*args` e `**kwargs` só quando você realmente não sabe a lista. Repasse genérico (decoradores, invólucros, subclasses) é o caso legítimo. Usá-los como “assinatura preguiçosa” apaga a documentação, engana o editor e transforma erro de digitação em chave silenciosa dentro de `kwargs`.

```
# antipadrão: a assinatura não diz nada e "tiemout" passa batido
def conectar(**opcoes):
    ...
```

```
# pythônico: o que é conhecido, explícito; o resto, se houver, no kwargs
def conectar(host, *, porta=22, timeout=5.0, **extras):
    ...
```

Passe por nome tudo que não for óbvio no ponto de chamada. Uma chamada com três literais seguidos (`formatar(x, 2, True)`) só se entende abrindo a definição.

Não mute o argumento recebido sem que o nome diga isso. É a regra do Capítulo 18 aplicada aos padrões: se a função recebe uma lista e a modifica, o nome dela deve ser `anexar_em`, não `calcular`.

Sobre versões:

- **Python 3.0** — parâmetros só-nomeados com `*` (PEP 3102).
- **Python 3.7** — `**kwargs` passou a preservar a ordem de inserção, junto com a garantia de ordem dos dicionários.
- **Python 3.8** — parâmetros só-posicionais com `/` (PEP 570); antes, só as funções em C tinham isso.
- **Python 3.11** — mensagens de `TypeError` de chamada mais precisas, indicando o argumento problemático.
- **Python 3.12** — `**kwargs` tipado com `TypedDict` via `Unpack` (PEP 692), para quem usa verificador estático.

19.8. Laboratório

Objetivo: ver o padrão mutável acumular estado entre chamadas, cronometrar o padrão que congelou no `import`, percorrer as cinco regiões de uma assinatura e terminar com um formatador de relatório configurável por opções nomeadas.

1. Ambiente.

```
mkdir -p ~/lab19 && cd ~/lab19
python3 -m venv .venv && source .venv/bin/activate
python --version
```

```
Python 3.13.12
```

2. O padrão mutável. Crie `acumula.py`:

```
def registrar(onu, historico=[]):
    historico.append(onu)
    return historico

print(registrar("onu-01"))
print(registrar("onu-02"))
print(registrar("onu-03"))
print("o padrao em si:", registrar.__defaults__)
```

```
python acumula.py
```

19. Argumentos nomeados, valores padrão, **args* e ***kwargs*

```
['onu-01']
['onu-01', 'onu-02']
['onu-01', 'onu-02', 'onu-03']
o padrao em si: (['onu-01', 'onu-02', 'onu-03'],)
```

A última linha é a prova material: o padrão guardado no objeto função **cresceu**.

3. A correção com sentinela.

```
python -c "
def registrar(onu, historico=None):
    if historico is None:
        historico = []
    historico.append(onu)
    return historico
print(registrar('onu-01'))
print(registrar('onu-02'))
meu = ['onu-00']
print(registrar('onu-03', meu), meu)
"
```

```
['onu-01']
['onu-02']
['onu-00', 'onu-03'] ['onu-00', 'onu-03']
```

4. O padrão que parou o relógio. Crie quando.py:

```
import time

def carimbo(t=time.time()):
    return t

print(f"{carimbo():.3f}")
time.sleep(1)
print(f"{carimbo():.3f}")
print(f"{carimbo(time.time()):.3f}")
```

```
python quando.py
```

```
1786439046.981
1786439046.981
1786439047.981
```

Os números mudam a cada execução; o que não muda é as duas primeiras linhas serem **idênticas** e a terceira ser um segundo maior.

5. Nomeados, e o que eles destravam.

```
python -c "
def alerta(onu, nivel, unidade='dBm', urgente=False):
    marca = '!! ' if urgente else '  '
    return f'{marca}{onu}: {nivel} {unidade}'
print(alerta('onu-01', -19.5))
print(alerta('onu-02', -31.2, urgente=True))
print(alerta(nivel=-8.0, onu='onu-05', unidade='dBm'))
print(alerta.__defaults__)
"
```

```
onu-01: -19.5 dBm
!! onu-02: -31.2 dBm
onu-05: -8.0 dBm
('dBm', False)
```

6. Os quatro TypeError de chamada. Crie erros.py:

```
def alerta(onu, nivel, unidade="dBm"):
    return f"{onu}: {nivel} {unidade}"

chamadas = [
    lambda: alerta("onu-01"),
    lambda: alerta("onu-01", -19.5, "dBm", "extra"),
    lambda: alerta("onu-01", -19.5, unit="dBm"),
    lambda: alerta("onu-01", -19.5, onu="onu-02"),
]
for c in chamadas:
    try:
        print(c())
    except TypeError as e:
        print("TypeError:", e)
```

```
python erros.py
```

```
TypeError: alerta() missing 1 required positional argument: 'nivel'
TypeError: alerta() takes from 2 to 3 positional arguments but 4 were given
TypeError: alerta() got an unexpected keyword argument 'unit'
TypeError: alerta() got multiple values for argument 'onu'
```

Quatro mensagens diferentes para quatro erros diferentes. Vale ler cada uma com calma: elas aparecem muito, e cada uma diz precisamente o que aconteceu.

7. *args.

```
python -c "
def media(*leituras):
    print(f' recebi {leituras!r} ({type(leituras).__name__})')
    return sum(leituras) / len(leituras) if leituras else 0.0
"
```

19. Argumentos nomeados, valores padrão, **args* e ***kwargs*

```
print(media(-19.5, -21.3))
print(media())
print(media(*[-19.5, -21.3, -20.2]))
"
```

```
recebi (-19.5, -21.3) (tuple)
-20.4
recebi () (tuple)
0.0
recebi (-19.5, -21.3, -20.2) (tuple)
-20.333333333333332
```

8. ****kwargs**.

```
python -c "
def conectar(host, **opcoes):
    print(f' {host} com {opcoes!r} ({type(opcoes).__name__})')
    porta = opcoes.get('porta', 22)
    return f'{host}:{porta}'
print(conectar('olt-norte'))
print(conectar('olt-sul', porta=830, timeout=5))
config = {'porta': 161, 'comunidade': 'publica'}
print(conectar('olt-leste', **config))
"
```

```
olt-norte com {} (dict)
olt-norte:22
olt-sul com {'porta': 830, 'timeout': 5} (dict)
olt-sul:830
olt-leste com {'porta': 161, 'comunidade': 'publica'} (dict)
olt-leste:161
```

9. A assinatura completa. Crie assinatura.py:

```
def registrar(onu, /, nivel, *extras, unidade="dBm", **meta):
    return {
        "onu": onu,
        "nivel": nivel,
        "extras": extras,
        "unidade": unidade,
        "meta": meta,
    }

print(registrar("onu-01", -19.5))
print(registrar("onu-01", -19.5, "manha", "campo", unidade="mW",
    ↪ tecnico="ana"))
try:
    registrar(onu="onu-01", nivel=-19.5)
```



```
except TypeError as e:
    print("TypeError:", e)
print(registrar("onu-01", -19.5, "dBm")["extras"])
```

```
python assinatura.py
```

```
{'onu': 'onu-01', 'nivel': -19.5, 'extras': (), 'unidade': 'dBm', 'meta': {}}
{'onu': 'onu-01', 'nivel': -19.5, 'extras': ('manha', 'campo'), 'unidade':
↪ 'mW', 'meta': {'tecnico': 'ana'}}
TypeError: registrar() missing 1 required positional argument: 'onu'
('dBm',)
```

A última linha é a que pega gente experiente: "dBm" foi para extras, não para unidade.

10. A barreira *. Crie sokw.py:

```
def desligar(onu, *, confirmado=False):
    if not confirmado:
        return f"{onu}: recusado, falta confirmar"
    return f"{onu}: desligada"

print(desligar("onu-07"))
print(desligar("onu-07", confirmado=True))
try:
    print(desligar("onu-07", True))
except TypeError as e:
    print("TypeError:", e)
```

```
python sokw.py
```

```
onu-07: recusado, falta confirmar
onu-07: desligada
TypeError: desligar() takes 1 positional argument but 2 were given
```

11. Leia a assinatura pelo inspect.

```
python -c "
import inspect
def registrar(onu, /, nivel, *extras, unidade='dBm', **meta):
    pass
sig = inspect.signature(registrar)
print(sig)
for nome, p in sig.parameters.items():
    print(f' {nome:<10} {p.kind}')
"
```

19. Argumentos nomeados, valores padrão, **args* e ***kwargs*

```
(onu, /, nivel, *extras, unidade='dBm', **meta)
onu          POSITIONAL_ONLY
nivel        POSITIONAL_OR_KEYWORD
extras       VAR_POSITIONAL
unidade      KEYWORD_ONLY
meta         VAR_KEYWORD
```

As cinco categorias da Figura 19.2, nomeadas pelo próprio Python.

12. Espalhar na chamada.

```
python -c "
def alerta(onu, nivel, unidade='dBm'):
    return f'{onu}: {nivel} {unidade}'
args = ('onu-01', -19.5)
kwargs = {'unidade': 'mW'}
print(alerta(*args, **kwargs))
linhas = [('onu-01', -19.5), ('onu-02', -31.2)]
for linha in linhas:
    print(alerta(*linha))
"
```

```
onu-01: -19.5 mW
onu-01: -19.5 dBm
onu-02: -31.2 dBm
```

13. O invólucro que repassa tudo. Crie repassa.py:

```
"""Um invólucro que repassa argumentos sem conhecer a assinatura de baixo."""

import time

def com_tempo(funcao, *args, **kwargs):
    """Chama funcao(*args, **kwargs) e devolve (resultado, milissegundos)."""
    inicio = time.perf_counter()
    resultado = funcao(*args, **kwargs)
    return resultado, (time.perf_counter() - inicio) * 1000

def soma_lenta(a, b, atraso=0.05):
    time.sleep(atraso)
    return a + b

valor, ms = com_tempo(soma_lenta, 2, 3)
print(f"soma_lenta(2, 3) = {valor} em ~{ms:.0f} ms")
valor, ms = com_tempo(soma_lenta, 2, 3, atraso=0.2)
print(f"com atraso=0.2 = {valor} em ~{ms:.0f} ms")
valor, ms = com_tempo(sorted, [3, 1, 2], reverse=True)
print(f"sorted(...) = {valor} em ~{ms:.2f} ms")
```

```
python repassa.py
```

```
soma_lenta(2, 3) = 5 em ~50 ms
com atraso=0.2  = 5 em ~200 ms
sorted(...)    = [3, 2, 1] em ~0.01 ms
```

14. Junte tudo num relatório configurável. Crie relatorio.py:

```
"""Formatador de relatorio com padroes seguros e opcoes nomeadas."""

LARGURA = 46

def formatar(onu, nivel, *, unidade="dBm", casas=1, marca=None):
    """Uma linha de relatorio; marca destaca a linha quando fornecida."""
    prefixo = "" if marca is None else f"{marca} "
    medida = "sem leitura" if nivel is None else f"{nivel:.{casas}f} {unidade}"
    return f"{prefixo}{onu}: {medida}"

def relatorio(leituras, *, titulo="Niveis opticos", **estilo):
    """Monta o relatorio inteiro; estilo e repassado para formatar()."""
    linhas = [titulo.center(LARGURA, "="), ""]
    for onu, nivel in leituras:
        linhas.append(formatar(onu, nivel, **estilo))
    linhas.append("=" * LARGURA)
    return "\n".join(linhas)

LEITURAS = [("onu-01", -19.5), ("onu-02", -31.234), ("onu-03", None)]

print(relatorio(LEITURAS))
print()
print(relatorio(LEITURAS, titulo="Varredura noturna", casas=2, marca="*"))
```

```
python relatorio.py
```

```
=====Niveis opticos=====

onu-01: -19.5 dBm
onu-02: -31.2 dBm
onu-03: sem leitura
=====

=====Varredura noturna=====

* onu-01: -19.50 dBm
* onu-02: -31.23 dBm
* onu-03: sem leitura
=====
```

19. Argumentos nomeados, valores padrão, `*args` e `**kwargs`

Todos os padrões são imutáveis (`None`, `1`, `"dBm"`), toda opção é só-nomeada, e o `**estilo` repassa para `formatar` o que `relatorio` não precisa conhecer. Se alguém escrever `casa=2` em vez de `casas=2`, o erro aparece na hora, como `unexpected keyword argument` — porque `formatar` não tem `**kwargs`. Essa é a diferença entre repassar e engolir.

15. Feche com `deactivate`.

19.9. Resumo

- O valor padrão é avaliado **uma vez**, quando o `def` executa, e fica guardado em `__defaults__`. Padrão mutável é estado compartilhado entre todas as chamadas.
- A correção é `None` como sentinela e a criação do objeto no corpo da função.
- Expressão viva no padrão (`time.time()`, `datetime.now()`) congela no instante do `import`.
- Argumento nomeado permite pular parâmetros com padrão e reordenar; posicional depois de nomeado é `SyntaxError`.
- `*args` recolhe os posicionais restantes numa tupla (vazia, nunca `None`); `**kwargs` recolhe os nomeados restantes num dicionário.
- Na chamada, `*` e `**` fazem o inverso: espalham sequência e dicionário em argumentos.
- A ordem é fixa: só-posicionais, `/`, comuns, `*args`, só-nomeados, `**kwargs`. `/` é 3.8+; `*` sozinho serve de barreira para forçar nome.
- Parâmetro depois de um `*` é **só-nomeado** — o posicional extra vai para `*args` em silêncio.
- `inspect.signature` mostra assinatura e categoria de cada parâmetro sem abrir o código.

20. Escopo, closures e a regra LEGB

O contador de alarmes vive no módulo, e a função incrementa. É a coisa mais simples do capítulo, e não funciona:

```
alarmes = 0

def registrar(onu):
    alarmes += 1
    print(f"{onu} registrada, total {alarmes}")

registrar("onu-01")
```

```
Traceback (most recent call last):
  File "/home/vitor/lab20/contador.py", line 9, in <module>
    registrar("onu-01")
    ~~~~~~^~~~~~
  File "/home/vitor/lab20/contador.py", line 5, in registrar
    alarmes += 1
    ~~~~~~
UnboundLocalError: cannot access local variable 'alarmes' where it is not
↪ associated with a value
```

“Não consigo acessar a variável local `alarmes`” — mas `alarmes` não é local, está lá em cima, no módulo, valendo zero. E se a função só `ler`, funciona:

```
alarmes = 0

def ler():
    print("leitura funciona:", alarmes)

ler()
```

```
leitura funciona: 0
```

A diferença não é o valor, é a **atribuição**. Existe uma regra que decide tudo neste capítulo: se um nome recebe atribuição em qualquer ponto do corpo de uma função, ele é local **na função inteira** — inclusive nas linhas antes da atribuição. E `alarmes += 1` é uma atribuição.

Dá para ver a decisão já tomada, antes de qualquer execução, no código compilado:

```

alarmes = 0

def escreve():
    alarmes = alarmes + 1

def le():
    print(alarmes)

print("escreve co_varnames:", escreve.__code__.co_varnames)
print("le co_varnames:", le.__code__.co_varnames)
print("le co_names: ", le.__code__.co_names)

escreve co_varnames: ('alarmes',)
le co_varnames: ()
le co_names: ('print', 'alarmes')

```

Em `escreve`, `alarmes` está entre as **variáveis locais**; em `le`, está entre os **nomes globais**. O compilador classificou os dois na hora de compilar, olhando só para a presença da atribuição — não para o que existe em tempo de execução.

Este capítulo é sobre onde o Python procura um nome, o que muda essa busca, e o que acontece quando uma função guarda um pedaço do escopo em que nasceu.

20.1. LEGB: a ordem da busca

Quando o Python encontra um nome, ele procura em quatro lugares, nesta ordem:

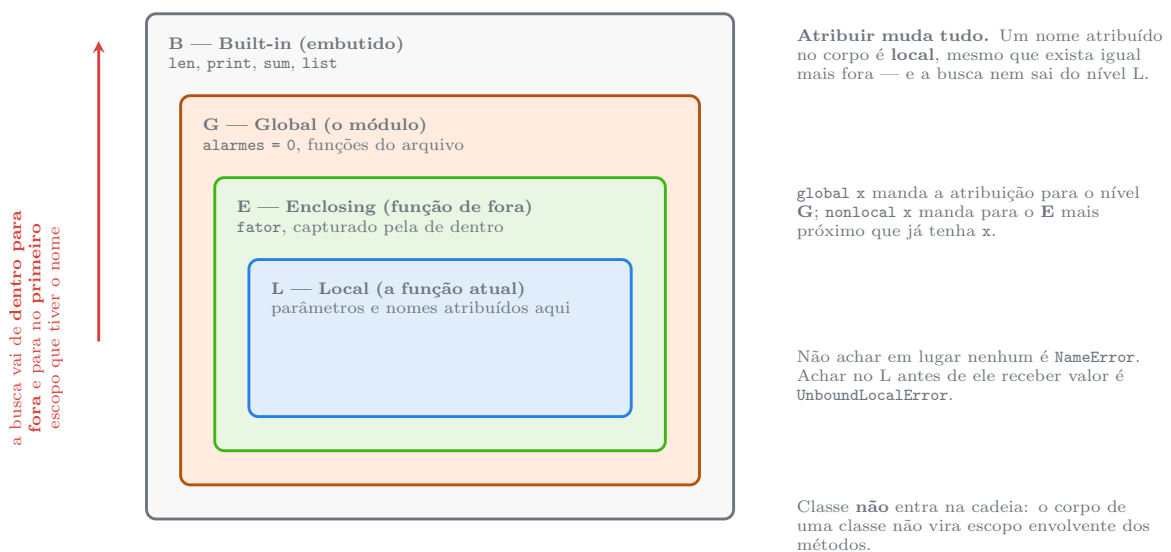


Figura 20.1.: A ordem LEGB, e as três palavras que a alteram.

Um exemplo com os quatro níveis de uma vez:

```
x = "global"

def externa():
    x = "da externa"

    def interna():
        print("interna ve:", x)

    interna()
    print("externa ve:", x)

externa()
print("modulo ve:", x)
print("embutida:", len("abc"))
```

```
interna ve: da externa
externa ve: da externa
modulo ve: global
embutida: 3
```

`interna` não tem `x` local, então subiu um nível e encontrou o `x` de `externa` (E) — o global (G) nem foi consultado. E `len` veio do nível B, que é o último da fila.

20.2. *global e nonlocal*

Para **atribuir** num escopo de fora, é preciso declarar. São as duas únicas palavras que fazem isso:

```
alarmes = 0

def registrar(onu):
    global alarmes
    alarmes += 1
    return f"{onu}: total {alarmes}"

print(registrar("onu-01"))
print(registrar("onu-02"))
print("no modulo:", alarmes)
```

```
onu-01: total 1
onu-02: total 2
no modulo: 2
```

`nonlocal` faz o mesmo para o escopo envolvente mais próximo — e **não** alcança o global:

```
def contador_local():
    total = 0

    def somar(n):
        nonlocal total
        total += n
        return total

    return somar

c = contador_local()
print(c(3), c(4), c(10))
outro = contador_local()
print("contador novo:", outro(1))
```

```
3 7 17
contador novo: 1
```

`c` acumulou até 17; `outro`, criado por outra chamada de `contador_local`, começou do zero. Cada chamada da função externa produz um `total` novo — e é exatamente isso que faz de `somar` um **closure**.

Sobre `global`: ele funciona e quase nunca é a resposta. Estado global mutável mexido de dentro de funções é a receita de bug que só aparece com a ordem certa de chamadas. A alternativa é devolver o valor novo, passar um objeto que carregue o estado, ou usar o fechamento da seção seguinte.

20.3. Sombrear um nome embutido

Como o nível B é o **último** da busca, qualquer nome definido no módulo ou na função tapa o embutido de mesmo nome:

```
niveis = [-19.5, -31.0, -8.0]
sum = 0
for n in niveis:
    sum = sum + n
print("soma manual:", sum)
print(sum(niveis))
```

```
soma manual: -58.5
Traceback (most recent call last):
  File "/home/vitor/lab20/sombra.py", line 6, in <module>
    print(sum(niveis))
    ~~~~^~~~~~
TypeError: 'float' object is not callable
```


O erro nem menciona `sum`: diz que um `float` não é chamável. Quando o sombreamento acontece dentro de uma função grande, a distância entre a causa e o sintoma é de dezenas de linhas. Os nomes que mais aparecem sombreados por acidente são `list`, `dict`, `set`, `str`, `id`, `type`, `input`, `next`, `filter`, `max`, `min` e `sum` — e o `ruff` sinaliza todos eles (regra A001). Quando o nome certo mesmo é `list`, a convenção é um sublinhado no fim: `list_`.

20.4. Closures

Um **closure** é uma função que continua enxergando variáveis do escopo onde nasceu, mesmo depois de esse escopo ter terminado:

```
def multiplicador(fator):
    def aplicar(valor):
        return valor * fator

    return aplicar

dobro = multiplicador(2)
triplo = multiplicador(3)
print(dobro(10), triplo(10))
print("fator ainda vive:", dobro.__closure__)
print("valor guardado: ", dobro.__closure__[0].cell_contents)
print("nomes capturados:", dobro.__code__.co_freevars)
```

```
20 30
fator ainda vive: (<cell at 0x77dfacce39a0: int object at 0xa56c48>,)
valor guardado: 2
nomes capturados: ('fator',)
```

Quando `multiplicador(2)` retornou, o quadro dele foi desempilhado — mas `fator` não morreu junto. O Python percebeu, **na compilação**, que `aplicar` usa um nome que não é dele, e promoveu esse nome a **célula**: uma caixinha que fica pendurada no objeto função e sobrevive ao quadro.

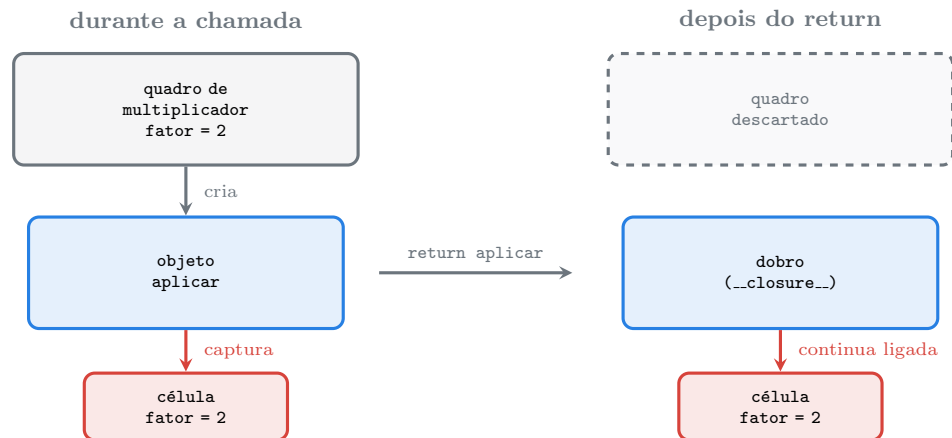
Closure é a base de decorador, de *callback* com contexto, de fábrica de funções e de boa parte do que o Capítulo 21 vai mostrar.

20.5. A armadilha da captura tardia

O closure captura a **variável**, não o valor dela no instante da criação. Num laço, isso produz o erro mais famoso da categoria:

```
funcoes = []
for fator in (2, 3, 4):
    funcoes.append(lambda v: v * fator)

print([f(10) for f in funcoes])
```



A célula é decidida na **compilação**: o Python vê que `aplicar` usa um nome que não é dele nem global, e promove esse nome a célula em vez de deixá-lo morrer com o quadro. Por isso `dobro` e `triplo`, vindos de duas chamadas diferentes, têm células **diferentes** — e é assim que cada um lembra do próprio `fator`.

Figura 20.2.: O quadro morre no `return`; a célula com o valor capturado, não.

```
[40, 40, 40]
```

As três funções compartilham a mesma variável `fator`, e quando alguém finalmente as chama, `fator` já vale 4 — o valor que sobrou depois do laço. Não é um bug de escopo: é o escopo funcionando exatamente como definido.

Há duas correções idiomáticas:

```
certas = []
for fator in (2, 3, 4):
    certas.append(lambda v, fator=fator: v * fator)

print([f(10) for f in certas])

from functools import partial

def escala(fator, valor):
    return valor * fator

com_partial = [partial(escala, f) for f in (2, 3, 4)]
print([f(10) for f in com_partial])
print("fator depois do laço:", fator)
```

```
[20, 30, 40]
```

```
[20, 30, 40]
```

```
4
```

A primeira usa o valor padrão do Capítulo 19 justamente porque ele é avaliado **na definição** — o mesmo comportamento que causa o bug do padrão mutável aqui resolve o problema. A segunda usa `functools.partial`, que fixa o argumento de verdade e é mais explícita.

E a última linha mostra outra coisa importante: **o for não cria escopo**. `fator` continua vivo, valendo 4, depois do laço terminar.

20.6. Compreensões têm escopo próprio; laços, não

```
quadrados = [n * n for n in range(3)]
print(quadrados)
print(n)
```

```
[0, 1, 4]
NameError: name 'n' is not defined
```

```
for m in range(3):
    pass
print("depois do for, m =", m)
```

```
depois do for, m = 2
```

A compreensão roda num escopo próprio (mudança do Python 3, para não poluir o ambiente); o `for` de bloco vaza a variável. Isso é útil quando você quer o último valor, e é fonte de confusão quando você não quer — outra razão para não reaproveitar nome de variável de laço.

20.7. `locals()` e `globals()`

```
def f(a, b=2):
    c = a + b
    print("locals:", locals())

f(1)
g = 10
print("g esta em globals:", "g" in globals())
print("len esta em builtins:", "len" in dir(__builtins__))
```

```
locals: {'a': 1, 'b': 2, 'c': 3}
g esta em globals: True
len esta em builtins: True
```

São ótimas para depurar e para entender, e ruins para programar: escrever em `locals()` dentro de uma função não tem efeito garantido, e criar nomes dinamicamente com `globals()[nome] = valor` é o caminho mais curto para código que nenhuma ferramenta consegue analisar. Quando a chave é dinâmica, o que você quer é um dicionário seu.

20.8. Jeito Pythônico

Prefira parâmetro e retorno a estado global. Uma função que recebe tudo que usa e devolve tudo que produz é testável em uma linha e não depende de ordem de execução.

```
# antipadrão
total = 0

def somar(n):
    global total
    total += n
```

```
# pythônico
def somar(total, n):
    return total + n
```

nonlocal é aceitável; global quase nunca. nonlocal mora dentro de um fechamento pequeno e explícito. global alcança o módulo inteiro.

Não sobreie nomes embutidos. list, dict, sum, id, type, input. Se o nome perfeito for um deles, use list_ ou um nome mais específico (itens, mapa, total).

Não conte com a variável vazada do for. Ela existe, mas depender disso é frágil e não sobrevive a quem transformar o laço numa compreensão.

Ao criar funções em laço, fixe o valor. lambda v, fator=fator: ... ou functools.partial. Sem isso, todas capturam a mesma variável.

Função pequena, escopo pequeno. A regra LEGB só machuca em função longa, onde uma atribuição na linha 60 muda a natureza de um nome usado na linha 10.

Sobre versões:

- **Python 3.0** — nonlocal (PEP 3104), e as compreensões passaram a ter escopo próprio.
- **Python 3.11** — o UnboundLocalError deixou de dizer apenas *local variable referenced before assignment* e passou a nomear a variável, como no traceback da abertura.
- **Python 3.12** — parâmetros de tipo genérico (PEP 695) introduzem um escopo lexical novo, o *annotation scope*, que é o primeiro escopo criado no Python desde as compreensões.
- **Python 3.13** — locals() teve a semântica finalmente especificada (PEP 667): em função, devolve um instantâneo independente, então escrever nele nunca afeta as variáveis reais.

20.9. Laboratório

Objetivo: provocar o UnboundLocalError e ver que a decisão é do compilador, medir o alcance de global e nonlocal, ver a célula de um fechamento com os próprios olhos, cair na captura tardia e sair dela.

1. Ambiente.

```
mkdir -p ~/lab20 && cd ~/lab20
python3 -m venv .venv && source .venv/bin/activate
python --version
```

```
Python 3.13.12
```

2. O erro da abertura. Crie contador.py:

```
alarmes = 0

def registrar(onu):
    alarmes += 1
    print(f"{onu} registrada, total {alarmes}")

registrar("onu-01")
```

```
python contador.py
```

```
Traceback (most recent call last):
  File "/home/vitor/lab20/contador.py", line 9, in <module>
    registrar("onu-01")
    ~~~~~~^~~~~~
  File "/home/vitor/lab20/contador.py", line 5, in registrar
    alarmes += 1
    ~~~~~~
UnboundLocalError: cannot access local variable 'alarmes' where it is not
↪ associated with a value
```

3. Só ler funciona.

```
python -c "
alarmes = 0
def ler():
    print('leitura funciona:', alarmes)
ler()
"
```

```
leitura funciona: 0
```

4. A decisão é do compilador.

```
python -c "
alarmes = 0
def escreve():
    alarmes = alarmes + 1
```

```
def le():
    print(alarmes)
print('escreve co_varnames:', escreve.__code__.co_varnames)
print('le      co_varnames:', le.__code__.co_varnames)
print('le      co_names:   ', le.__code__.co_names)
"
```

```
escreve co_varnames: ('alarmes',)
le      co_varnames: ()
le      co_names:   ('print', 'alarmes')
```

Nenhuma das duas foi chamada. A classificação já estava feita.

5. **global e nonlocal**. Crie `global_nonlocal.py`:

```
alarmes = 0

def registrar(onu):
    global alarmes
    alarmes += 1
    return f"{onu}: total {alarmes}"

print(registrar("onu-01"))
print(registrar("onu-02"))
print("no modulo:", alarmes)

def contador_local():
    total = 0

    def somar(n):
        nonlocal total
        total += n
        return total

    return somar

c = contador_local()
print(c(3), c(4), c(10))
outro = contador_local()
print("contador novo:", outro(1))
```

```
python global_nonlocal.py
```

```
onu-01: total 1
onu-02: total 2
```

```
no modulo: 2
3 7 17
contador novo: 1
```

6. Os quatro níveis.

```
python -c "
x = 'global'
def externa():
    x = 'da externa'
    def interna():
        print('interna ve:', x)
    interna()
    print('externa ve:', x)
externa()
print('modulo ve:', x)
print('embutida:', len('abc'))
"
```

```
interna ve: da externa
externa ve: da externa
modulo ve: global
embutida: 3
```

7. Sombreie um embutido. Crie sombra.py:

```
niveis = [-19.5, -31.0, -8.0]
sum = 0
for n in niveis:
    sum = sum + n
print("soma manual:", sum)
print(sum(niveis))
```

```
python sombra.py
```

```
soma manual: -58.5
Traceback (most recent call last):
  File "/home/vitor/lab20/sombra.py", line 6, in <module>
    print(sum(niveis))
    ~~~~^~~~~~
TypeError: 'float' object is not callable
```

8. Veja a célula. Crie fechamento.py:

```
def multiplicador(fator):
    def aplicar(valor):
        return valor * fator
```

```
    return aplicar

dobro = multiplicador(2)
triplo = multiplicador(3)
print(dobro(10), triplo(10))
print("fator ainda vive:", dobro.__closure__)
print("valor guardado: ", dobro.__closure__[0].cell_contents)
print("nomes capturados:", dobro.__code__.co_freevars)
```

```
python fechamento.py
```

```
20 30
fator ainda vive: (<cell at 0x77dfacce39a0: int object at 0xa56c48>,)
valor guardado: 2
nomes capturados: ('fator',)
```

O endereço muda a cada execução. O que importa é que `__closure__` não é `None` e que a célula guarda o 2.

9. Captura tardia, e as duas saídas. Crie `tardio.py`:

```
funcoes = []
for fator in (2, 3, 4):
    funcoes.append(lambda v: v * fator)

print([f(10) for f in funcoes])

certas = []
for fator in (2, 3, 4):
    certas.append(lambda v, fator=fator: v * fator)

print([f(10) for f in certas])

from functools import partial

def escala(fator, valor):
    return valor * fator

com_partial = [partial(escala, f) for f in (2, 3, 4)]
print([f(10) for f in com_partial])
print("fator depois do laco:", fator)
```

```
python tardio.py
```



```
[40, 40, 40]
[20, 30, 40]
[20, 30, 40]
fator depois do laço: 4
```

10. Escopo de compreensão e de laço.

```
python -c "
quadrados = [n * n for n in range(3)]
print(quadrados)
try:
    print(n)
except NameError as e:
    print('NameError:', e)
for m in range(3):
    pass
print('depois do for, m =', m)
"
```

```
[0, 1, 4]
NameError: name 'n' is not defined
depois do for, m = 2
```

11. Olhe os dicionários de escopo.

```
python -c "
def f(a, b=2):
    c = a + b
    print('locals:', locals())
f(1)
g = 10
print('g esta em globals:', 'g' in globals())
print('len esta em builtins:', 'len' in dir(__builtins__))
"
```

```
locals: {'a': 1, 'b': 2, 'c': 3}
g esta em globals: True
len esta em builtins: True
```

12. Estado privado sem variável global. Crie acumulador.py:

```
"""Acumuladores de alarme por OLT, construídos com fechamento."""

def criar_monitor(olt, limite=-28.0):
    """Devolve (registrar, resumo) com estado privado por OLT."""
    total = 0
    criticas = []
```

```

def registrar(onu, nivel):
    nonlocal total
    total += 1
    if nivel < limite:
        criticas.append((onu, nivel))
    return len(criticas)

def resumo():
    pior = min(criticas, key=lambda par: par[1], default=None)
    return {
        "olt": olt,
        "lidas": total,
        "criticas": len(criticas),
        "pior": pior,
    }

return registrar, resumo

norte, resumo_norte = criar_monitor("olt-norte")
sul, resumo_sul = criar_monitor("olt-sul", limite=-30.0)

for onu, nivel in [("onu-01", -19.5), ("onu-02", -31.2), ("onu-03", -28.4)]:
    norte(onu, nivel)
for onu, nivel in [("onu-04", -29.0), ("onu-05", -33.1)]:
    sul(onu, nivel)

print(resumo_norte())
print(resumo_sul())
print("estado e privado:", resumo_norte()["lidas"], "!= ",
    ↪ resumo_sul()["lidas"])
print("nomes livres de registrar:", norte.__code__.co_freevars)

```

```
python acumulador.py
```

```

{'olt': 'olt-norte', 'lidas': 3, 'criticas': 2, 'pior': ('onu-02', -31.2)}
{'olt': 'olt-sul', 'lidas': 2, 'criticas': 1, 'pior': ('onu-05', -33.1)}
estado e privado: 3 != 2
nomes livres de registrar: ('criticas', 'limite', 'total')

```

Duas OLTs, dois estados completamente separados, nenhuma variável global e nenhuma classe. As duas funções devolvidas por `criar_monitor` compartilham as **mesmas** células — é assim que `resumo` enxerga o que `registrar` acumulou — e as células de `norte` são outras que as de `sul`.

Repare também que `criticas.append(...)` **não** precisou de `nonlocal`: mutar o objeto não é reatribuir o nome. Só `total`, que recebe `total += 1`, precisou da declaração. É a mesma distinção do Capítulo 18 entre mutar e reatribuir, agora do lado do escopo.

13. Feche com `deactivate`.

20.10. Resumo

- Um nome atribuído em qualquer ponto do corpo de uma função é local na função **inteira**; ler antes da atribuição dá `UnboundLocalError`. A classificação é feita na compilação, e aparece em `co_varnames` e `co_names`.
- A busca de nomes segue LEGB: Local, Enclosing, Global, Built-in — de dentro para fora, parando no primeiro que tiver o nome.
- `global` manda a atribuição para o módulo; `nonlocal` para o escopo envolvente mais próximo. Mutar um objeto não exige nenhum dos dois: só reatribuir o nome exige.
- Sombrear um embutido (`sum = 0`) não dá erro na hora — dá um `TypeError` confuso quando alguém tenta chamar o nome mais adiante.
- Um closure é uma função que mantém vivas, em células, as variáveis do escopo em que nasceu. Cada chamada da função externa cria células novas.
- A captura é da **variável**, não do valor: funções criadas dentro de um laço veem o último valor. A saída é `lambda ..., x=x:` ou `functools.partial`.
- Compreensão tem escopo próprio; `for` de bloco não tem, e a variável do laço sobrevive a ele.
- `locals()` e `globals()` servem para inspecionar, não para programar.

21. Funções como objetos: lambda, map, filter e ordem superior

O console de operação precisa despachar comandos para funções. A ideia é a certa — uma tabela em vez de uma escada de `elif`, como no Capítulo 15 —, e a execução tem um caractere a mais:

```
def reiniciar(onu):
    return f"{onu}: reiniciada"

def desligar(onu):
    return f"{onu}: desligada"

ACOES = {
    "reiniciar": reiniciar("onu-01"),
    "desligar": desligar("onu-01"),
}

print("tabela montada:", ACOES)
print(ACOES["reiniciar"]("onu-07"))
```

```
tabela montada: {'reiniciar': 'onu-01: reiniciada', 'desligar': 'onu-01:
↪ desligada'}
Traceback (most recent call last):
  File "/home/vitor/lab21/despacho_ruim.py", line 15, in <module>
    print(ACOES["reiniciar"]("onu-07"))
    ~~~~~~
TypeError: 'str' object is not callable
```

A tabela não guardou funções: guardou **resultados**. Os parênteses em `reiniciar("onu-01")` chamaram a função ali mesmo, na linha do dicionário, e o que ficou na tabela foi a string devolvida. Pior: se `reiniciar` fizesse o que o nome diz, as duas ONUs teriam sido reiniciadas na hora do `import` — antes de qualquer comando.

A correção é tirar os parênteses:

```
ACOES = {
    "reiniciar": reiniciar,
    "desligar": desligar,
    "diagnosticar": diagnosticar,
}

print("tabela montada:", {k: v.__name__ for k, v in ACOES.items()})
```

```
for comando in ("reiniciar", "diagnosticar", "voar"):
    acao = ACOES.get(comando)
    if acao is None:
        print(f"{comando}: comando desconhecido")
        continue
    print(acao("onu-07"))
```

```
tabela montada: {'reiniciar': 'reiniciar', 'desligar': 'desligar',
↪  'diagnosticar': 'diagnosticar'}
onu-07: reiniciada
onu-07: diagnostico em andamento
voar: comando desconhecido
```

Este capítulo fecha o Volume 3 com o que essa diferença de dois caracteres destrava: função guardada em estrutura de dados, passada como argumento, devolvida por outra função — e as ferramentas que a biblioteca padrão oferece para trabalhar assim.

21.1. O nome refere, os parênteses chamam

```
def classificar(nivel):
    return "critico" if nivel < -28 else "normal"

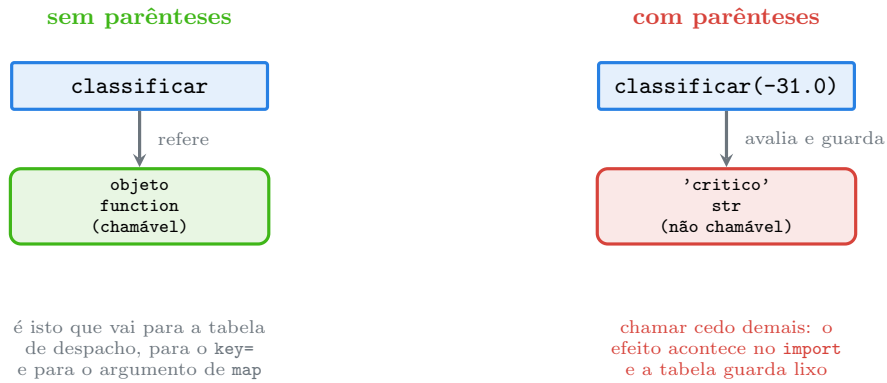
print(classificar)
print(classificar(-31.0))
print(callable(classificar), callable(classificar(-31.0)))
apelido = classificar
print(apelido(-31.0), apelido is classificar, apelido.__name__)
```

```
<function classificar at 0x730205c2fd80>
critico
True False
critico True classificar
```

O endereço muda a cada execução; o resto não. `classificar` é um objeto — chamável, atribuível, com atributos —, e `classificar(-31.0)` é o resultado de chamá-lo, que aqui é uma string comum.

21.2. Funções de ordem superior

Uma função de ordem superior é uma que **recebe** ou **devolve** função. As mais usadas estão embutidas, e o parâmetro se chama `key`:



callable(obj) responde qual dos dois você tem em mãos. E o erro que aparece quando se confunde — 'str' object is not callable — fala do **resultado**, nunca da função, que é o que torna esse bug demorado de achar.

Figura 21.1.: Sem parênteses o nome refere o objeto; com parênteses, o que sobra é o valor devolvido.

```
LEITURAS = [("onu-03", -31.2), ("onu-01", -19.5), ("onu-05", -8.0), ("onu-02",
↪ -26.4)]
```

```
print(sorted(LEITURAS))
print(sorted(LEITURAS, key=lambda par: par[1]))
print(sorted(LEITURAS, key=lambda par: par[1], reverse=True))
print(min(LEITURAS, key=lambda par: par[1]))
print(max(LEITURAS, key=lambda par: par[1]))
```

```
[('onu-01', -19.5), ('onu-02', -26.4), ('onu-03', -31.2), ('onu-05', -8.0)]
[('onu-03', -31.2), ('onu-02', -26.4), ('onu-01', -19.5), ('onu-05', -8.0)]
[('onu-05', -8.0), ('onu-01', -19.5), ('onu-02', -26.4), ('onu-03', -31.2)]
('onu-03', -31.2)
('onu-05', -8.0)
```

Sem **key**, a tupla é comparada elemento a elemento e a ordem é alfabética pelo nome. Com **key**, cada item é passado pela função e o **resultado** é que ordena. O item original é que volta na lista — **key** decide a ordem, não o conteúdo.

Chave composta é uma tupla, e resolve ordenação em vários critérios de uma vez:

```
LEITURAS = [
    ("onu-03", -31.2, "norte"),
    ("onu-01", -19.5, "sul"),
    ("onu-05", -8.0, "norte"),
    ("onu-02", -26.4, "sul"),
]
print(sorted(LEITURAS, key=lambda r: (r[2], r[1])))
```

```
[('onu-03', -31.2, 'norte'), ('onu-05', -8.0, 'norte'), ('onu-02', -26.4,  
↪ 'sul'), ('onu-01', -19.5, 'sul')]
```

Por OLT e, dentro da OLT, por nível. E o **sorted** é **estável**: itens com a mesma chave mantêm a ordem original, o que permite ordenar em duas passagens quando os critérios têm direções diferentes.

21.3. `lambda`

`lambda` cria uma função sem nome, a partir de **uma única expressão**:

```
key=lambda par: par[1]
```

Não há **return** (o valor da expressão é o retorno), não cabe instrução, não cabe **if** de bloco — só a expressão condicional do Capítulo 15. Essa limitação é deliberada: `lambda` existe para o caso em que a função é tão pequena que dar um nome a ela atrapalharia mais do que ajudaria.

O que **não** se faz é dar nome a um `lambda`:

```
# antipadrão: lambda com nome
classificar = lambda n: "critico" if n < -28 else "normal"

def classificar_def(n):
    return "critico" if n < -28 else "normal"

print(classificar(-31.0), classificar_def(-31.0))
print("nome do lambda:", classificar.__name__)
print("nome do def:   ", classificar_def.__name__)
classificar("texto")
```

```
critico critico
nome do lambda: <lambda>
nome do def:   classificar_def
TypeError: '<' not supported between instances of 'str' and 'int'
```

Os dois fazem a mesma coisa, e o **def** é melhor em tudo: aceita docstring, aparece pelo nome no traceback e no *profiler*, e não custa uma linha a mais. A PEP 8 é explícita nesse ponto, e o **ruff** sinaliza com E731.

Regra prática: `lambda` só como **argumento** de outra função, e só quando couber confortavelmente na mesma linha.

21.4. map e filter

`map` aplica uma função a cada item; `filter` mantém os itens para os quais a função é verdadeira. Os dois devolvem **iteradores** preguiçosos, com tudo que o Capítulo 16 disse sobre isso:

```
niveis = [-19.5, -31.2, -8.0, -26.4]
dobrados = map(lambda n: n * 2, niveis)
print(dobrados)
print(list(dobrados))
print(list(dobrados))
print(list(filter(lambda n: n < -20, niveis)))
```

```
<map object at 0x782d3463bcd0>
[-39.0, -62.4, -16.0, -52.8]
[]
[-31.2, -26.4]
```

O segundo `list(dobrados)` veio vazio: o iterador já tinha sido consumido.

E aqui vem a parte que separa Python de linguagens funcionais: **a resposta idiomática para os dois casos acima é uma compreensão**.

```
print([n * 2 for n in niveis])
print([n for n in niveis if n < -20])
```

```
[-39.0, -62.4, -16.0, -52.8]
[-31.2, -26.4]
```

A compreensão lê melhor, não exige `list()` em volta e, com `lambda`, ainda é mais rápida:

```
import timeit

setup = "niveis = [-19.5, -31.2, -8.0, -26.4] * 250"
a = timeit.timeit("[n * 2 for n in niveis]", setup=setup, number=2000)
b = timeit.timeit("list(map(lambda n: n * 2, niveis))", setup=setup,
    ↪ number=2000)
c = timeit.timeit("list(map(float.__neg__, niveis))", setup=setup, number=2000)
print(f"compreensao      {a:.3f}s")
print(f"map com lambda    {b:.3f}s")
print(f"map com builtin    {c:.3f}s")
```

```
compreensao      0.043s
map com lambda    0.084s
map com builtin   0.051s
```

Os números variam de máquina para máquina, e a ordem não: `map` com `lambda` é o mais lento dos três, porque cada item paga uma chamada de função Python. `map` só ganha da compreensão quando a função já existe e é implementada em C — `map(str.strip, linhas)`, `map(int, campos)`. Essa é a regra: **map quando a função já tem nome; compreensão quando você criaria um `lambda` para isso**.

21.5. operator: as funções que você ia escrever

O módulo `operator` traz prontas as chaves mais comuns, em C:

```
from operator import itemgetter

LEITURAS = [("onu-03", -31.2), ("onu-01", -19.5), ("onu-05", -8.0)]
print(sorted(LEITURAS, key=itemgetter(1)))
print(list(map(itemgetter(0), LEITURAS)))

REGISTROS = [
    {"onu": "onu-03", "nivel": -31.2, "olt": "norte"},
    {"onu": "onu-01", "nivel": -19.5, "olt": "sul"},
    {"onu": "onu-05", "nivel": -8.0, "olt": "norte"},
]
print(sorted(REGISTROS, key=itemgetter("olt", "nivel")))
```

```
[('onu-03', -31.2), ('onu-01', -19.5), ('onu-05', -8.0)]
['onu-03', 'onu-01', 'onu-05']
[{'onu': 'onu-03', 'nivel': -31.2, 'olt': 'norte'}, {'onu': 'onu-05', 'nivel':
↪ -8.0, 'olt': 'norte'}, {'onu': 'onu-01', 'nivel': -19.5, 'olt': 'sul'}]
```

`itemgetter("olt", "nivel")` é a chave composta sem escrever tupla. Os irmãos dele são `attrgetter` (para atributos) e `methodcaller` (para chamar um método), e os três aparecem muito em código que usa `sorted`, `max`, `groupby` e `sort`.

21.6. functools.partial

`partial` fixa argumentos e devolve uma função nova:

```
from functools import partial

def escalar(fator, valor):
    return valor * fator

mw = partial(escalar, 0.001)
print(mw(1500), mw.func.__name__, mw.args)

def conectar(host, porta=22, timeout=5):
    return f"{host}:{porta} (t={timeout})"

netconf = partial(conectar, porta=830)
print(netconf("olt-norte"))
print(netconf("olt-sul", timeout=30))
```

```
1.5 escalar (0.001,)
olt-norte:830 (t=5)
olt-sul:830 (t=30)
```

É o `lambda v: escalar(0.001, v)` escrito de forma que preserve a função original (`mw.func`), os argumentos fixados (`mw.args`) e a possibilidade de sobrescrever o que ficou. Foi ele que resolveu a captura tardia do Capítulo 20.

21.7. *functools.reduce*, e por que quase nunca

```
from functools import reduce
import operator

niveis = [-19.5, -31.2, -8.0]
print(reduce(operator.add, niveis))
print(sum(niveis))
print(reduce(lambda a, b: a if a < b else b, niveis), min(niveis))
```

```
-58.7
-58.7
-31.2 -31.2
```

As duas linhas de cada par fazem a mesma coisa, e a segunda de cada par é legível. `reduce` saiu dos embutidos no Python 3 justamente por isso: quase toda redução real já tem nome (`sum`, `min`, `max`, `any`, `all`, `math.prod`, `"".join`), e as que não têm costumam ficar mais claras num laço de três linhas.

21.8. Um pipeline montado com funções

Com tudo junto, dá para montar um encadeamento em que as etapas são dados:

```
from functools import partial
from operator import itemgetter

LEITURAS = [
    {"onu": "onu-01", "nivel": -19.5, "olt": "norte"},
    {"onu": "onu-02", "nivel": -31.2, "olt": "norte"},
    {"onu": "onu-03", "nivel": None, "olt": "sul"},
    {"onu": "onu-04", "nivel": -26.4, "olt": "sul"},
    {"onu": "onu-05", "nivel": -8.0, "olt": "norte"},
]

def tem_leitura(registro):
    return registro["nivel"] is not None
```

21. Funções como objetos: lambda, map, filter e ordem superior

```
def abaixo_de(limite, registro):
    return registro["nivel"] < limite

def em_mw(registro):
    """Acrescenta a potencia em miliwatts a partir do nivel em dBm."""
    return registro | {"mw": round(10 ** (registro["nivel"] / 10), 5)}

def aplicar(dados, *etapas):
    """Passa os dados por cada etapa, na ordem."""
    for etapa in etapas:
        dados = etapa(dados)
    return list(dados)

criticas = aplicar(
    LEITURAS,
    partial(filter, tem_leitura),
    partial(filter, partial(abaixo_de, -25.0)),
    partial(map, em_mw),
    partial(sorted, key=itemgetter("nivel")),
)

for r in criticas:
    print(f"{r['onu']} {r['olt']:<6} {r['nivel']:>7.1f} dBm {r['mw']:.5f}
    ↪ mW")
```

```
onu-02  norte    -31.2 dBm  0.00076 mW
onu-04  sul      -26.4 dBm  0.00229 mW
```

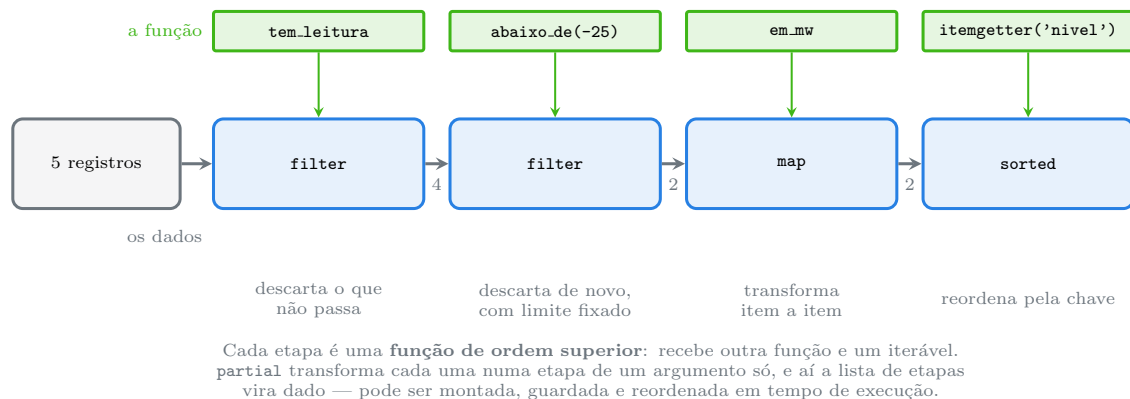


Figura 21.2.: Cada etapa recebe uma função e um iterável; com **partial**, a lista de etapas vira dado.

Vale dizer com todas as letras: em Python, esse pipeline geralmente fica **mais legível como um laço** ou como uma sequência de compreensões. A montagem acima existe quando as etapas precisam ser configuráveis — vindas de um arquivo de configuração, de um plugin, de uma escolha do usuário. Aí o fato de a função ser um objeto passa a ser a única forma razoável de fazer.

21.9. Jeito Pythônico

Sem parênteses para referir, com parênteses para chamar. Tabela de despacho, `key=`, `callback`, argumento de `map` — todos recebem o nome `nu`.

Compreensão no lugar de `map/filter` com `lambda`. Mais legível e mais rápida.

```
# antipadrão
maiusculas = list(map(lambda s: s.upper(), nomes))
ativos = list(filter(lambda u: u.ativo, usuarios))
```

```
# pythônico
maiusculas = [s.upper() for s in nomes]
ativos = [u for u in usuarios if u.ativo]
```

`map` quando a função já tem nome e é embutida. `map(int, campos)`, `map(str.strip, linhas)` — aí não há `lambda` no caminho e o laço roda em C.

Nunca `nome = lambda ...`. Use `def`. O `lambda` nomeado perde a docstring, aparece como `<lambda>` no traceback e não ganha nada.

`operator.itemgetter` e `attrgetter` no lugar de `lambda` em `key=`. Mais rápido, e diz o que faz.

```
# comum
sorted(registros, key=lambda r: (r["olt"], r["nivel"]))
```

```
# melhor
from operator import itemgetter

sorted(registros, key=itemgetter("olt", "nivel"))
```

`functools.partial` no lugar de `lambda` que só fixa argumento. Preserva a função de origem e evita a captura tardia.

`reduce` só quando não houver nome pronto. `sum`, `min`, `max`, `any`, `all`, `math.prod` e `str.join` cobrem quase tudo.

Função devolvida por função é normal. Fábrica de funções, decorador e *callback* com contexto são o uso legítimo dos fechamentos do Capítulo 20 — e é sobre isso que o Volume 8 vai se debruçar.

Sobre versões:

21. Funções como objetos: *lambda*, *map*, *filter* e ordem superior

- **Python 3.0** — `map` e `filter` passaram a devolver iteradores, e `reduce` saiu dos embutidos para o `functools`.
- **Python 3.8** — `functools.cached_property` e `singledispatchmethod`, ambos baseados em função como objeto.
- **Python 3.9** — `functools.cache`, o `lru_cache` sem argumentos.
- **Python 3.10** — `itertools.pairwise`, que substitui o `zip(seq, seq[1:])` do Capítulo 16.
- **Python 3.11** — `StrEnum` e melhorias de desempenho em chamadas de função (a especialização adaptativa reduz o custo de `map` com `lambda`, sem eliminá-lo).

21.10. Laboratório

Objetivo: quebrar uma tabela de despacho com parênteses a mais, medir `map` contra compreensão, trocar `lambda` por `itemgetter` e por `partial`, e terminar montando um pipeline em que as etapas são dados.

1. Ambiente.

```
mkdir -p ~/lab21 && cd ~/lab21
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. A tabela que guardou resultados. Crie `despacho_ruim.py`:

```
def reiniciar(onu):
    return f"{onu}: reiniciada"

def desligar(onu):
    return f"{onu}: desligada"

ACOES = {
    "reiniciar": reiniciar("onu-01"),
    "desligar": desligar("onu-01"),
}

print("tabela montada:", ACOES)
print(ACOES["reiniciar"]("onu-07"))
```

```
python despacho_ruim.py
```

```
tabela montada: {'reiniciar': 'onu-01: reiniciada', 'desligar': 'onu-01:
↵ desligada'}
Traceback (most recent call last):
  File "/home/vitor/lab21/despacho_ruim.py", line 15, in <module>
```

```
print(ACOES["reiniciar"]("onu-07"))
~~~~~^~~~~~
TypeError: 'str' object is not callable
```

A primeira linha já denuncia: a tabela tem strings, não funções — e a `onu-01` foi “reiniciada” no `import`.

3. A tabela certa. Crie `despacho.py`:

```
def reiniciar(onu):
    return f"{onu}: reiniciada"

def desligar(onu):
    return f"{onu}: desligada"

def diagnosticar(onu):
    return f"{onu}: diagnostico em andamento"

ACOES = {
    "reiniciar": reiniciar,
    "desligar": desligar,
    "diagnosticar": diagnosticar,
}

print("tabela montada:", {k: v.__name__ for k, v in ACOES.items()})
for comando in ("reiniciar", "diagnosticar", "voar"):
    acao = ACOES.get(comando)
    if acao is None:
        print(f"{comando}: comando desconhecido")
        continue
    print(acao("onu-07"))
```

```
python despacho.py
```

```
tabela montada: {'reiniciar': 'reiniciar', 'desligar': 'desligar',
↵ 'diagnosticar': 'diagnosticar'}
onu-07: reiniciada
onu-07: diagnostico em andamento
voar: comando desconhecido
```

4. Referência e chamada.

```
python -c "
def classificar(nivel):
    return 'critico' if nivel < -28 else 'normal'
print(classificar)
```

21. Funções como objetos: lambda, map, filter e ordem superior

```
print(classificar(-31.0))
print(callable(classificar), callable(classificar(-31.0)))
apelido = classificar
print(apelido(-31.0), apelido is classificar, apelido.__name__)
"
```

```
<function classificar at 0x730205c2fd80>
critico
True False
critico True classificar
```

5. key= em sorted, min e max.

```
python -c "
LEITURAS = [('onu-03', -31.2), ('onu-01', -19.5), ('onu-05', -8.0), ('onu-02',
↪ -26.4)]
print(sorted(LEITURAS))
print(sorted(LEITURAS, key=lambda par: par[1]))
print(sorted(LEITURAS, key=lambda par: par[1], reverse=True))
print(min(LEITURAS, key=lambda par: par[1]))
print(max(LEITURAS, key=lambda par: par[1]))
"
```

```
[('onu-01', -19.5), ('onu-02', -26.4), ('onu-03', -31.2), ('onu-05', -8.0)]
[('onu-03', -31.2), ('onu-02', -26.4), ('onu-01', -19.5), ('onu-05', -8.0)]
[('onu-05', -8.0), ('onu-01', -19.5), ('onu-02', -26.4), ('onu-03', -31.2)]
('onu-03', -31.2)
('onu-05', -8.0)
```

A primeira linha ordena por **nome**; as outras, por nível. A diferença é uma função.

6. itemgetter.

```
python -c "
from operator import itemgetter
LEITURAS = [('onu-03', -31.2), ('onu-01', -19.5), ('onu-05', -8.0)]
print(sorted(LEITURAS, key=itemgetter(1)))
print(list(map(itemgetter(0), LEITURAS)))
REGISTROS = [
    {'onu': 'onu-03', 'nivel': -31.2, 'olt': 'norte'},
    {'onu': 'onu-01', 'nivel': -19.5, 'olt': 'sul'},
    {'onu': 'onu-05', 'nivel': -8.0, 'olt': 'norte'},
]
print(sorted(REGISTROS, key=itemgetter('olt', 'nivel')))
"
```

```
[('onu-03', -31.2), ('onu-01', -19.5), ('onu-05', -8.0)]
['onu-03', 'onu-01', 'onu-05']
[{'onu': 'onu-03', 'nivel': -31.2, 'olt': 'norte'}, {'onu': 'onu-05', 'nivel':
↪ -8.0, 'olt': 'norte'}, {'onu': 'onu-01', 'nivel': -19.5, 'olt': 'sul'}]
```


7. map, filter e a compreensão equivalente.

```
python -c "
niveis = [-19.5, -31.2, -8.0, -26.4]
dobrados = map(lambda n: n * 2, niveis)
print(dobrados)
print(list(dobrados))
print(list(dobrados))
print(list(filter(lambda n: n < -20, niveis)))
print([n * 2 for n in niveis])
print([n for n in niveis if n < -20])
"
```

```
<map object at 0x782d3463bcd0>
[-39.0, -62.4, -16.0, -52.8]
[]
[-31.2, -26.4]
[-39.0, -62.4, -16.0, -52.8]
[-31.2, -26.4]
```

8. Meça.

```
python -c "
import timeit
setup = 'niveis = [-19.5, -31.2, -8.0, -26.4] * 250'
a = timeit.timeit('[n * 2 for n in niveis]', setup=setup, number=2000)
b = timeit.timeit('list(map(lambda n: n * 2, niveis))', setup=setup,
    ↪ number=2000)
c = timeit.timeit('list(map(float.__neg__, niveis))', setup=setup, number=2000)
print(f'compreensao      {a:.3f}s')
print(f'map com lambda   {b:.3f}s')
print(f'map com builtin   {c:.3f}s')
"
```

```
compreensao      0.043s
map com lambda    0.084s
map com builtin   0.051s
```

Os tempos absolutos dependem da máquina. A ordem — compreensão, depois `map` com função em C, depois `map` com `lambda` — é a mesma em qualquer lugar.

9. reduce contra o nome pronto.

```
python -c "
from functools import reduce
import operator
niveis = [-19.5, -31.2, -8.0]
print(reduce(operator.add, niveis))
print(sum(niveis))
print(reduce(lambda a, b: a if a < b else b, niveis), min(niveis))
"
```

21. Funções como objetos: lambda, map, filter e ordem superior

```
-58.7  
-58.7  
-31.2 -31.2
```

10. partial.

```
python -c "  
from functools import partial  
def escalar(fator, valor):  
    return valor * fator  
mw = partial(escalar, 0.001)  
print(mw(1500), mw.func.__name__, mw.args)  
def conectar(host, porta=22, timeout=5):  
    return f'{host}:{porta} (t={timeout})'  
netconf = partial(conectar, porta=830)  
print(netconf('olt-norte'))  
print(netconf('olt-sul', timeout=30))  
"
```

```
1.5 escalar (0.001,)  
olt-norte:830 (t=5)  
olt-sul:830 (t=30)
```

11. lambda com nome, e por que não. Crie lambda_ruim.py:

```
# antipadrao: lambda com nome  
classificar = lambda n: "critico" if n < -28 else "normal"  
  
def classificar_def(n):  
    return "critico" if n < -28 else "normal"  
  
print(classificar(-31.0), classificar_def(-31.0))  
print("nome do lambda:", classificar.__name__)  
print("nome do def:   ", classificar_def.__name__)  
try:  
    classificar("texto")  
except TypeError as e:  
    print("TypeError:", e)
```

```
python lambda_ruim.py
```

```
critico critico  
nome do lambda: <lambda>  
nome do def:   classificar_def  
TypeError: '<' not supported between instances of 'str' and 'int'
```

<lambda> é o que apareceria no traceback de produção, em vez do nome da função.

12. Chave composta e os agregadores.

```
python -c "
LEITURAS = [('onu-03', -31.2, 'norte'), ('onu-01', -19.5, 'sul'), ('onu-05',
↪ -8.0, 'norte'), ('onu-02', -26.4, 'sul')]
print(sorted(LEITURAS, key=lambda r: (r[2], r[1])))
print([r[0] for r in sorted(LEITURAS, key=lambda r: -r[1])])
print(any(n < -30 for _, n, _ in LEITURAS), all(n < 0 for _, n, _ in LEITURAS))
"
```

```
[('onu-03', -31.2, 'norte'), ('onu-05', -8.0, 'norte'), ('onu-02', -26.4,
↪ 'sul'), ('onu-01', -19.5, 'sul')]
['onu-05', 'onu-01', 'onu-02', 'onu-03']
True True
```

13. O pipeline configurável. Crie pipeline.py:

```
"""Pipeline de leituras usando funcoes como valores."""

from functools import partial
from operator import itemgetter

LEITURAS = [
    {"onu": "onu-01", "nivel": -19.5, "olt": "norte"},
    {"onu": "onu-02", "nivel": -31.2, "olt": "norte"},
    {"onu": "onu-03", "nivel": None, "olt": "sul"},
    {"onu": "onu-04", "nivel": -26.4, "olt": "sul"},
    {"onu": "onu-05", "nivel": -8.0, "olt": "norte"},
]

def tem_leitura(registro):
    return registro["nivel"] is not None

def abaixo_de(limite, registro):
    return registro["nivel"] < limite

def em_mw(registro):
    """Acrescenta a potencia em miliwatts a partir do nivel em dBm."""
    return registro | {"mw": round(10 ** (registro["nivel"] / 10), 5)}

def aplicar(dados, *etapas):
    """Passa os dados por cada etapa, na ordem."""
    for etapa in etapas:
        dados = etapa(dados)
    return list(dados)
```

```
criticas = aplicar(
    LEITURAS,
    partial(filter, tem_leitura),
    partial(filter, partial(abaixo_de, -25.0)),
    partial(map, em_mw),
    partial(sorted, key=itemgetter("nivel")),
)

for r in criticas:
    print(f"{r['onu']} {r['olt']:<6} {r['nivel']:>7.1f} dBm {r['mw']:.5f}
    ↪ mW")

print("total:", len(criticas))
print("pior:", min(criticas, key=itemgetter("nivel"))["onu"])
```

```
python pipeline.py
```

```
onu-02  norte      -31.2 dBm  0.00076 mW
onu-04  sul        -26.4 dBm  0.00229 mW
total: 2
pior: onu-02
```

Repare no `aplicar`: ele não sabe nada sobre leituras ópticas. Recebe dados e uma lista de funções, e passa um pelo outro. Trocar a ordem das etapas, tirar uma ou ler a lista de um arquivo de configuração não exige mudar uma linha dele — e é isso que “função é objeto” compra.

E repare também no `partial(filter, partial(abaixo_de, -25.0))`: um `partial` dentro do outro, o de dentro fixando o limite e o de fora fixando o predicado. Quando o encadeamento chega a esse nível, um laço explícito costuma ser mais honesto — o exercício aqui é entender o mecanismo, não recomendar o estilo.

14. Feche com `deactivate`.

21.11. Resumo

- Função é objeto de primeira classe: pode ser atribuída, guardada em lista ou dicionário, passada como argumento e devolvida. O nome refere; os parênteses chamam.
- A tabela de despacho guarda o nome nu. Com parênteses, ela guarda resultados — e os efeitos acontecem no `import`.
- Função de ordem superior recebe ou devolve função. `sorted`, `min` e `max` aceitam `key=`, que decide a ordem sem mudar os itens; chave composta é uma tupla, e `sorted` é estável.
- `lambda` é uma expressão só, sem nome; use apenas como argumento, e nunca atribua a um nome (E731).
- `map` e `filter` devolvem iteradores de uma passada. Com `lambda`, a compreensão é mais legível e mais rápida; `map` vale quando a função já tem nome e roda em C.
- `operator.itemgetter/attrgetter/methodcaller` substituem `lambda` em `key=` com mais clareza e menos custo.

- `functools.partial` fixa argumentos preservando a função original; `functools.reduce` só quando não existir um agregador com nome.
- Com funções como dados, um pipeline de etapas vira configuração — mas, quando as etapas são fixas, um laço ou uma compreensão continuam sendo a resposta pythônica.

Parte IV.

**Volume 4 — Estruturas de Dados
Nativas**

22. Listas: operações, mutabilidade e cópia

(em elaboracao)

23. Tuplas, empacotamento e desempacotamento

(em elaboracao)

24. Dicionários: chaves, ordem e os métodos essenciais

(em elaboracao)

25. Conjuntos e operações de conjunto

(em elaboracao)

26. Compreensões de lista, dicionário e conjunto

(em elaboracao)

27. Ordenação, chaves de ordenação e comparação de objetos

(em elaboracao)

28. Escolhendo a estrutura certa: custo das operações

(em elaboracao)

Parte V.

**Volume 5 — Módulos, Pacotes e o
Ambiente**

29. Módulos, import e o caminho de busca

(em elaboracao)

30. Pacotes, `__init__.py` e a organização de um projeto

(em elaboracao)

31. Ambientes virtuais: venv, pip e o isolamento de dependências

(em elaboracao)

32. Empacotamento moderno: pyproject.toml, build e publicação

(em elaboracao)

33. uv, pipx e Poetry: o ecossistema de gerenciamento

(em elaboracao)

34. Ponto de entrada: `__main__`, scripts de console e a linha de comando

(em elaboracao)

Parte VI.

**Volume 6 — Arquivos, Erros e
Contextos**

35. Leitura e escrita de arquivos de texto

(em elaboracao)

36. Arquivos binários, buffers e o módulo io

(em elaboracao)

37. Caminhos e o sistema de arquivos com pathlib

(em elaboracao)

38. Exceções: hierarquia e o bloco try, except, else e finally

(em elaboracao)

39. Levantando exceções e criando exceções próprias

(em elaboracao)

40. Gerenciadores de contexto e a instrução with

(em elaboracao)

Parte VII.

**Volume 7 — Programação Orientada a
Objetos**

41. Classes, instâncias e o self

(em elaboracao)

42. Atributos, propriedades e encapsulamento

(em elaboracao)

43. Herança, MRO e a função super

(em elaboracao)

44. Composição, delegação e quando não herdar

(em elaboracao)

45. Métodos estáticos, métodos de classe e o desenho da API

(em elaboracao)

46. Dataclasses, NamedTuple e enumerações

(em elaboracao)

47. Classes abstratas, protocolos e duck typing

(em elaboracao)

Parte VIII.

**Volume 8 — O Modelo de Dados de
Python**

48. Métodos especiais: o protocolo por trás da sintaxe

(em elaboracao)

49. Iteradores e o protocolo de iteração

(em elaboracao)

50. Geradores, yield e a avaliação preguiçosa

(em elaboracao)

51. Decoradores de função

(em elaboracao)

52. Decoradores de classe, functools e metadados preservados

(em elaboracao)

53. Descritores, `__slots__` e o acesso a atributos

(em elaboracao)

54. Metaclasses e `__init_subclass__`: quando (não) usar

(em elaboracao)

Parte IX.

**Volume 9 — A Biblioteca Padrão
Essencial**

55. os, sys e a interação com o sistema

(em elaboracao)

56. datetime, zoneinfo e o tratamento de tempo

(em elaboracao)

57. collections: deque, Counter, defaultdict e namedtuple

(em elaboracao)

58. itertools e functools: a caixa de ferramentas funcional

(em elaboracao)

59. math, statistics e random

(em elaboracao)

60. logging: registrando o que o programa faz

(em elaboracao)

Parte X.

**Volume 10 — Texto, Dados e
Serialização**

61. Expressões regulares com o módulo re

(em elaboracao)

62. JSON, CSV e formatos tabulares

(em elaboracao)

63. YAML, TOML e arquivos de configuração

(em elaboracao)

64. Serialização binária: pickle, struct e os riscos envolvidos

(em elaboracao)

65. SQLite e a DB-API: banco de dados sem servidor

(em elaboracao)

66. hashlib, secrets e o tratamento de dados sensíveis

(em elaboracao)

Parte XI.

**Volume 11 — Qualidade: Testes,
Tipagem e Ferramentas**

67. Estilo, PEP 8 e formatação automática com Ruff e Black

(em elaboracao)

68. Anotações de tipo: fundamentos e o módulo typing

(em elaboracao)

69. Tipagem avançada: genéricos, protocolos e mypy

(em elaboracao)

70. Testes com pytest: fundamentos, asserções e fixtures

(em elaboracao)

71. Parametrização, duplões de teste e cobertura

(em elaboracao)

72. Depuração: pdb, breakpoint e o traceback difícil

(em elaboracao)

73. Documentação: docstrings, doctest e Sphinx

(em elaboracao)

Parte XII.

**Volume 12 — Automação, Sistema e
Redes**

74. Scripts de automação: arquivos, planilhas e tarefas repetitivas

(em elaboracao)

75. subprocess: chamando programas externos com segurança

(em elaboracao)

76. Interfaces de linha de comando com argparse, Click e Typer

(em elaboracao)

77. Redes com sockets: TCP, UDP e o modelo cliente-servidor

(em elaboracao)

78. Requisições HTTP com requests e httpx

(em elaboracao)

79. Raspagem de dados responsável: HTML, seletores e limites éticos

(em elaboracao)

Parte XIII.

**Volume 13 — Dados, Análise e
Visualização**

80. NumPy: arrays, vetorização e broadcasting

(em elaboracao)

81. pandas: Series, DataFrame e a carga de dados

(em elaboracao)

82. Limpeza, transformação e agregação com pandas

(em elaboracao)

83. Visualização de dados com Matplotlib

(em elaboracao)

84. SciPy e a computação científica aplicada

(em elaboracao)

85. Introdução ao aprendizado de máquina com scikit-learn

(em elaboracao)

Parte XIV.

**Volume 14 — Web, APIs e Bancos de
Dados**

86. Como funciona uma aplicação web: HTTP, WSGI e ASGI

(em elaboracao)

87. Validação de dados com Pydantic

(em elaboracao)

88. APIs REST com FastAPI

(em elaboracao)

89. Flask e Django: quando usar cada um

(em elaboracao)

90. Bancos de dados relacionais com SQLAlchemy

(em elaboracao)

91. Autenticação, autorização e segurança em aplicações Python

(em elaboracao)

92. Implantação: contêineres, servidores de aplicação e produção

(em elaboracao)

Parte XV.

**Volume 15 — Concorrência,
Desempenho e Internos**

93. Concorrência, paralelismo e o GIL

(em elaboracao)

94. Threads e o módulo threading

(em elaboracao)

95. Multiprocessing e o paralelismo real

(em elaboracao)

96. Programação assíncrona: `async`, `await` e `asyncio`

(em elaboracao)

97. Medindo desempenho: timeit, profiling e otimização

(em elaboracao)

98. Memória, contagem de referências e o coletor de lixo

(em elaboracao)

Parte XVI.

**Volume 16 — Fronteira e Prática
Contínua**

99. Estendendo Python: C, Cython e ligações nativas

(em elaboracao)

100. Python acelerado: PyPy, Numba e o interpretador sem GIL

(em elaboracao)

101. Arquitetura de projetos e código sustentável

(em elaboracao)

102. Git, pre-commit e integração contínua para projetos Python

(em elaboracao)

103. Python e inteligência artificial: LLMs, APIs e agentes

(em elaboracao)

104. Para onde ir depois: PEPs, comunidade e prática contínua

(em elaboracao)

Referências

